



# SPRING

---

## COURSE MATERIAL

---



# NARESH <sup>TM</sup> *i*

Opp. Satyam Theatre, Ameerpet, Hyderabad - 16  
Ph: 040-23746666, 23734842 Cell: 9000994007 / 08

*technologies*

B.O. Kaba Plaza, 3rd Floor, LB Road, Adyar, Chennai - 20. Phone : 044-6555 5557, 6555 5558



info@nareshit.com



www.nareshit.com



nareshit



nareshitech



## Introduction

- >C,C++, Java and etc.. are programming languages.
- >JDBC, Servlet, EJB,JSP,JMS,Java mail,JTA,JPA and etc are java technologies.
- >Struts, Spring, Hibernate and etc are Java Frameworks.

### Programming Language:

- It is directly installable software having basic features to develop software applications. It defines syntaxes (Rules), semantics (Structure) of programming.
  - These are base to create software technologies, frameworks, Tools, DB softwares, operating systems and etc
  - >These are raw materials of software programming.
- eg: C,C++,C#,Java and etc...

### Software Technology:

- It is software specification having set of rules and guidelines to develop softwares by using programming languages support.
  - Software technology is not installable, but the software created based on software technology is installable software. Working with this software is nothing but working with software technologies.
  - In Java, technology rules are nothing but interfaces and guidelines are nothing but classes. Abstract classes represent both rules and guidelines.
- eg: JDBC,JSP, Servlet and etc....
- JDBC technology gives rules and guidelines to develop **JDBC driver software** using java, language. Working with **JDBC Driver software** is nothing but working with **Jdbc technology**.
  - There are 2 types of software technologies.

#### 1. Open Technologies

#### 2. Proprietary Technologies.

### Open Technologies:

- Here the rules and guidelines of technology are open to all software vendor companies to develop softwares.
- eg: All Java Technologies (JDBC, Servlet, JSP and etc..)

### Proprietary Technologies.

- Here the vendor company who has given technology is **only allowed** to develop software based on technology.
- eg: All Microsoft Technologies (Like asp.net)

### Framework:

- Framework is a installable software that uses existing technologies internally to simplify the application development having ability to generate common logics of application internally and dynamically.
  - Framework is installable software that provides abstraction layer on existing technologies to simplify the application development process.
- eg: : Struts, Spring, Hibernate,...
- While working with technologies the programmer has to develop both common logics and application specific logics of the application.
  - While working with frameworks the programmer has to develop only application specific logics because the common logics will be generated by the framework dynamically.
  - Every framework uses technology internally but it never makes the programmer to know about it. This is nothing but providing **abstraction layer (hiding implementations/internals)**

**Plain JDBC App:**

- >Register JDBC Driver (load Driver class)
- >Establish the Connection
- >Create Statement Object
- >Send and execute SQL query in DB software
- >Gather results and process results.
- >Close Jdbc Objects

While working with technology, we need to write same common logics in every application of that technology. This is called **boiler plate code** problem. The code that repeats across the multiple applications either with no changes or with changes is called **boiler-plate code** problem.

**Spring jdbc App:**

- >Get JdbcTemplate class Object.
- >Send and executes SQLQuery in DB software
- >Gather results and process the results.

→ While working with frameworks we just need to develop application specific logics that means it solves boiler plate code problem.

→ Based on the kinds of applications and logics we develop, there are 3 types of java Frameworks.

**Web Application Frameworks:**

- Provides Abstraction layer on servlets, jsp technology.
- Simplifies MVC Architecture based web application development.

**examples:**

- JSF -> From Sun MS (Oracle)
- Struts -> From Apache
- Webwork -> from open symphony.
- ADF -> from Oracle
- Spring MVC -> From interface

**ORM Frameworks:**

- Provides Abstraction layer on Jdbc technology.
- Simplifies Object based O-R mapping persistence logic development.
- Allows to develop portable/DB independent persistence logic.

**examples:**

- Hibernate -> from Softtraa (RedHat)
- TOPLink -> Oracle Cooperation
- OJB -> Apache
- iBatis -> from Apache
- JDO -> from adobe

**Application Frameworks:**

- Provides abstraction layer on multiple technologies like jdbc, servlets, jsp, jndi, jms, jta, JAAS, EJB, Rmi and etc..
  - Allows to develop all kinds of logics like business logic, persistence logic, presentation logic, integration logic and etc..
  - Allows to develop all kinds of applications like standalone applications, web applications distributed applications and etc..
- eg: Spring -> from interfaces



Based on mode of development they allow, there are 2 kinds of frameworks.

#### Invasive Framework:

→ Here the classes of application should extend from framework api class or should implement framework api interfaces. This process makes application classes as tightly bounded classes of frameworks. We cannot change framework of the application/ project by changing the libraries (jar files).

#### Non-Invasive Framework:

→ Here the classes of application need not to extend from framework api class or need not to implement framework api interfaces. So our application classes are not tightly bound with framework apis. We can change framework of application/ project by changing the libraries (jar files) dynamically at runtime.

eg: Struts 2.x, Spring, Hibernate, and etc..

#### Evaluation of spring

→ **Applets** -> For Gaming (1995)

→ **JavaBean** -> for B.logic/service logic development (1996)

(Disadvantage: Can't apply services, can't be accessed from remote clients)

-> Security, transactions, logging. Connection pooling and etc are called services/ middleware services.

→ **EJB** -> for Business logic and service logic development (1998)

#### **Advantages:**

→ Allows both local and remote clients

→ Allows to apply middleware services.

#### **Disadvantages:**

--> Heavy weight and complex.

→ EJB is suitable for banking projects.

#### Spring (2003)

→ Initially for business logic development with ordinary classes allowing to apply middleware services. Now spring can be used to develop all kinds of logics and applications in light weight environment.

#### **Before Frameworks:**

→ Programmer directly uses technologies to develop both common and application specific logics of application.

#### With Frameworks:

→ Programmers use frameworks to develop only application specific logics because the common logics will be generated by frameworks internally/dynamically by using **technologies** support.

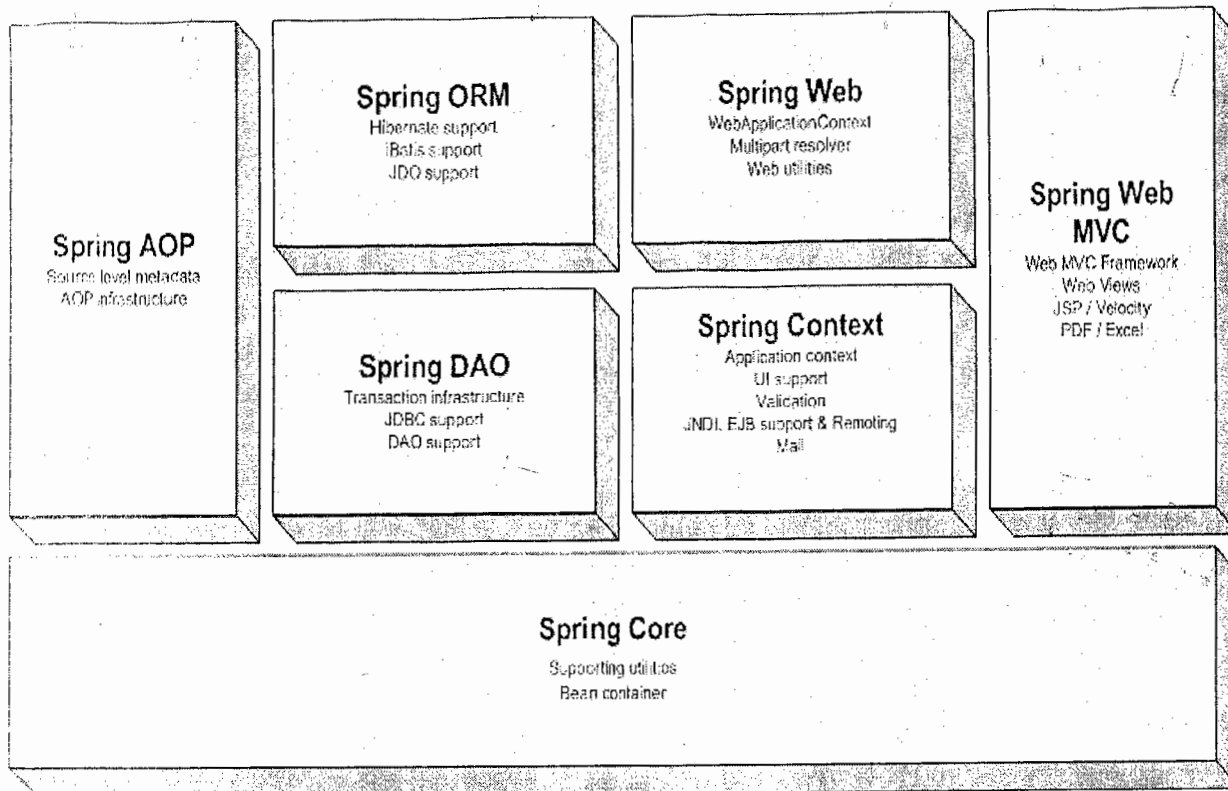
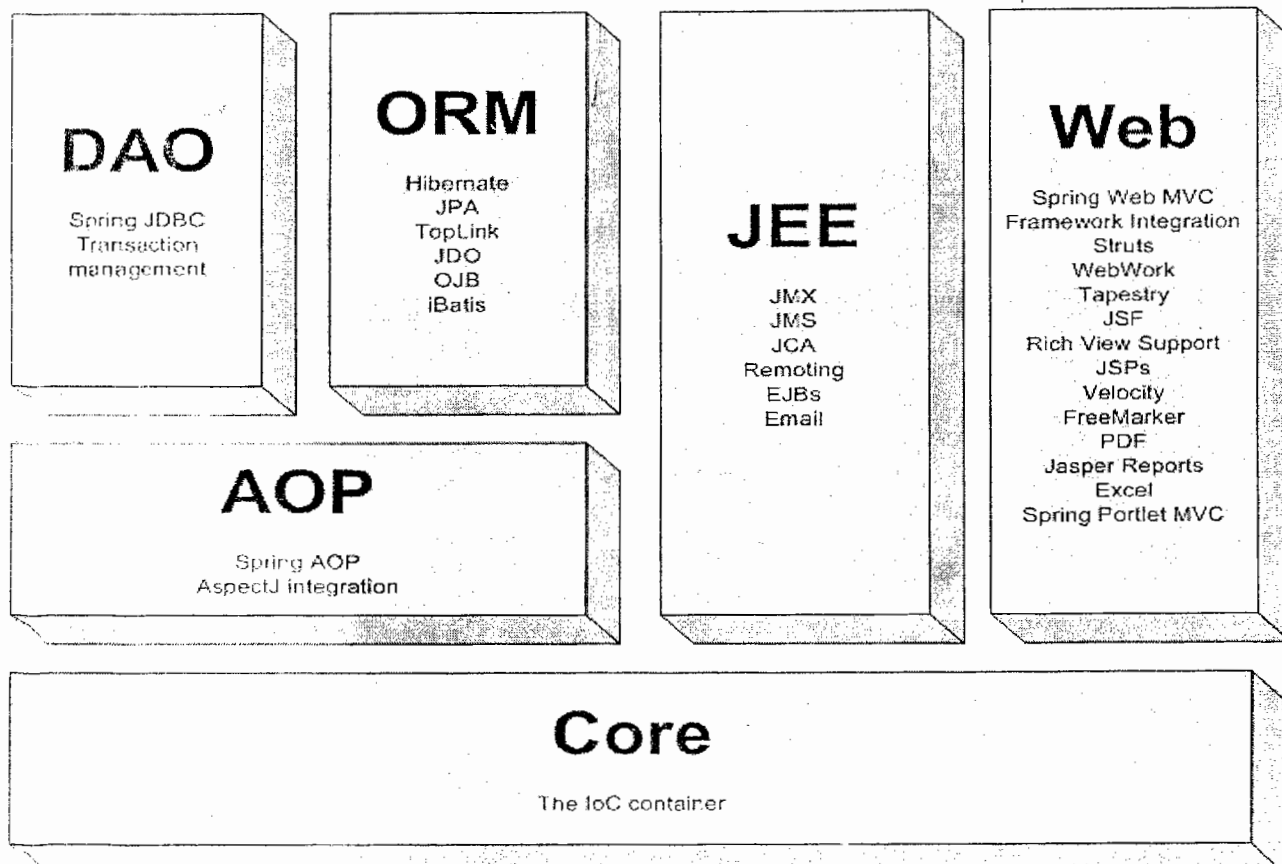
#### **Is the spring replacement for struts?**

Ans: Using struts, we can develop only web applications. There is no provision in struts to develop and use middleware services / aspects (security, logging and etc..)

→ Using spring we can develop all kinds of applications and all kinds of logics including web applications. It also provides environment to develop aspects/middleware services. Spring is more powerful than struts.

#### **Is the Spring replacement for JEE/JSE technologies?**

Ans: No, Spring complements JEE/JSE technologies by using them internally in the application development process.

Spring 1.X Modules (7 Modules) :Spring 2.x Overview Diagram:

->Spring 2.x web module = Spring 1.x web + spring 1.x web mvc module.

->Spring 2.x JEE modules = Spring 1.x context module

#### Spring Core:

→This module is base for other modules, provides containers that are required to manage various spring components/resources. Useful to develop standalone applications, Designed supporting dependency injection/management (injecting values to Objects dynamically)

#### Spring DAO/JDBC:

→Provides abstraction layer on plain JDBC to simplify the JDBC style persistence logic and Exception

Re-Throwing concept is used to design and throw **DAO Exception hierarchy**.

#### Spring ORM:

→Provides abstraction layer on multiple ORM frameworks (like Hibernate) and simplifies Object Based O-R mapping persistence logic development.

#### Spring Web:

→Provides plug-ins to make spring applications communicatable from other web framework applications like **Struts, JSF** and etc ...applications.

#### Spring Web MVC:

→Spring's own web framework to develop MVC architecture based web application with plan classes..

#### Spring Context:

→Provides abstraction layer on multiple JSE, JEE module technologies to simplify the development of the the following applications and logics.

→Mailing applications, Distributed applications, Messaging applications, Scheduling applications, and etc.....

#### Spring AOP:

→Provides a new methodology of programming to develop aspect/ middleware service logics and to apply them on spring applications.

#### Before AOP:

```
class Bank {  
    public void withdraw (int acno, float amt){  
        Security logic  
        Logging logic  
        Transaction logic  
        withdraw logic (bal=bal-amt)  
    }  
}  
  
public void deposit(int acno, float amt){  
    Security logic  
    Logging logic  
    Transaction logic  
    Deposit logic (bal=bal+amt)  
}  
}
```

->Here we are mixing up both primary logics and secondary logics..

With AOP:

```
class Bank{
    public void withdraw (int acno,float amt){
        Withdraw logic(bal=bal-amt);
    }
    public void deposit(int acno,float amt){
        Deposit logic (bal=bal+amt);
    }
}
```

```
class Security{
```

```
    ////
```

```
    .....
```

```
}
```

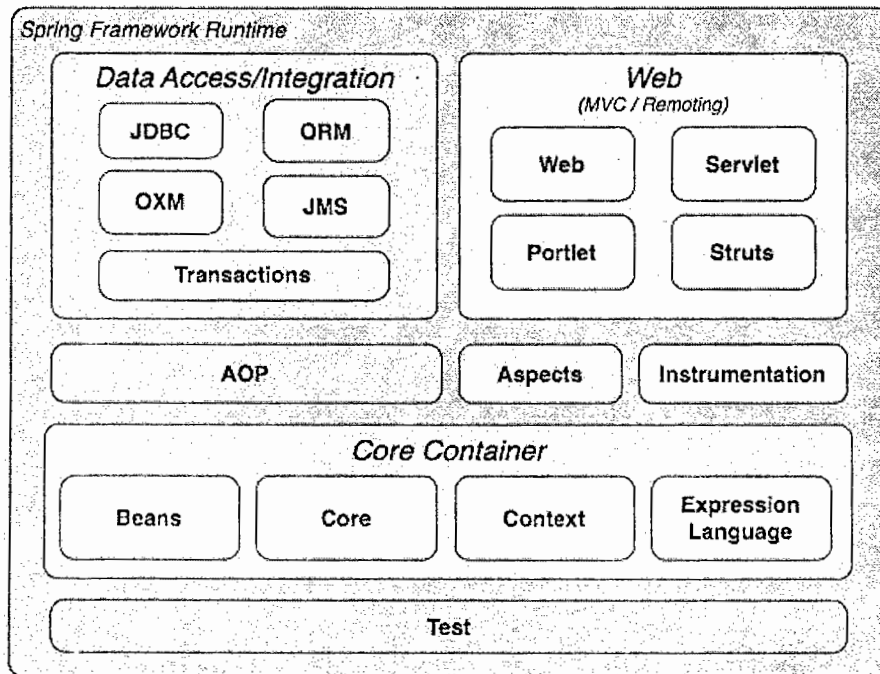
```
class Logging{
```

```
    .....
```

```
    .....
```

```
}
```

### Spring 3.x Overview Diagram:



→Spring 3.x is having about 20 modules ,that are grouped into **Core container**, **AOP**, **instrumentation**, **Test**, **DataAccess/integration**, **MVC/ remoting** groups.

→Test module allows to perform unit testing (programmers testing on his own piece of code)

→junit does not allow to create mock objects (dummy objs), where spring '**Test**' module allows us to work with mock objects.

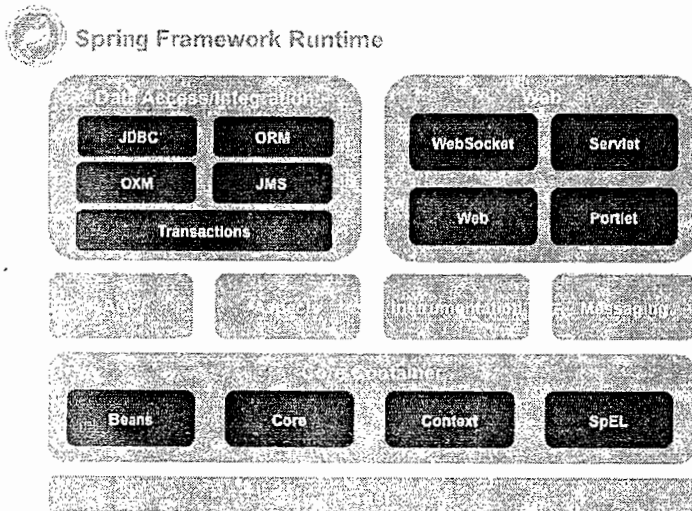
→ **Spring OXM** stands for **Spring Object XML Mappers** and it is a module available in Spring to ease the mapping between java objects and XML documents. The module is extensible and hence it provides integration with various popular frameworks like Castor, JAXB, XmlBeans and XStream.

→ Spring 2.x JEE services are placed in **DataAccess/integration, MVC/remoting groups**.

→ Portlet supports allows us to develop "portal" applications having multiple windows (sub web pages) in a single webpage. eg: iGoogle,AOL and etc...

→ **spring Instrumentation** is the ability to monitor the level of product's performance, to diagnose errors and to write trace information. Instrumentation is one of the key features from Spring framework to review application performance. Spring supports instrumentation through **AOP and Logging**.

### Spring 4.x overview diagram:



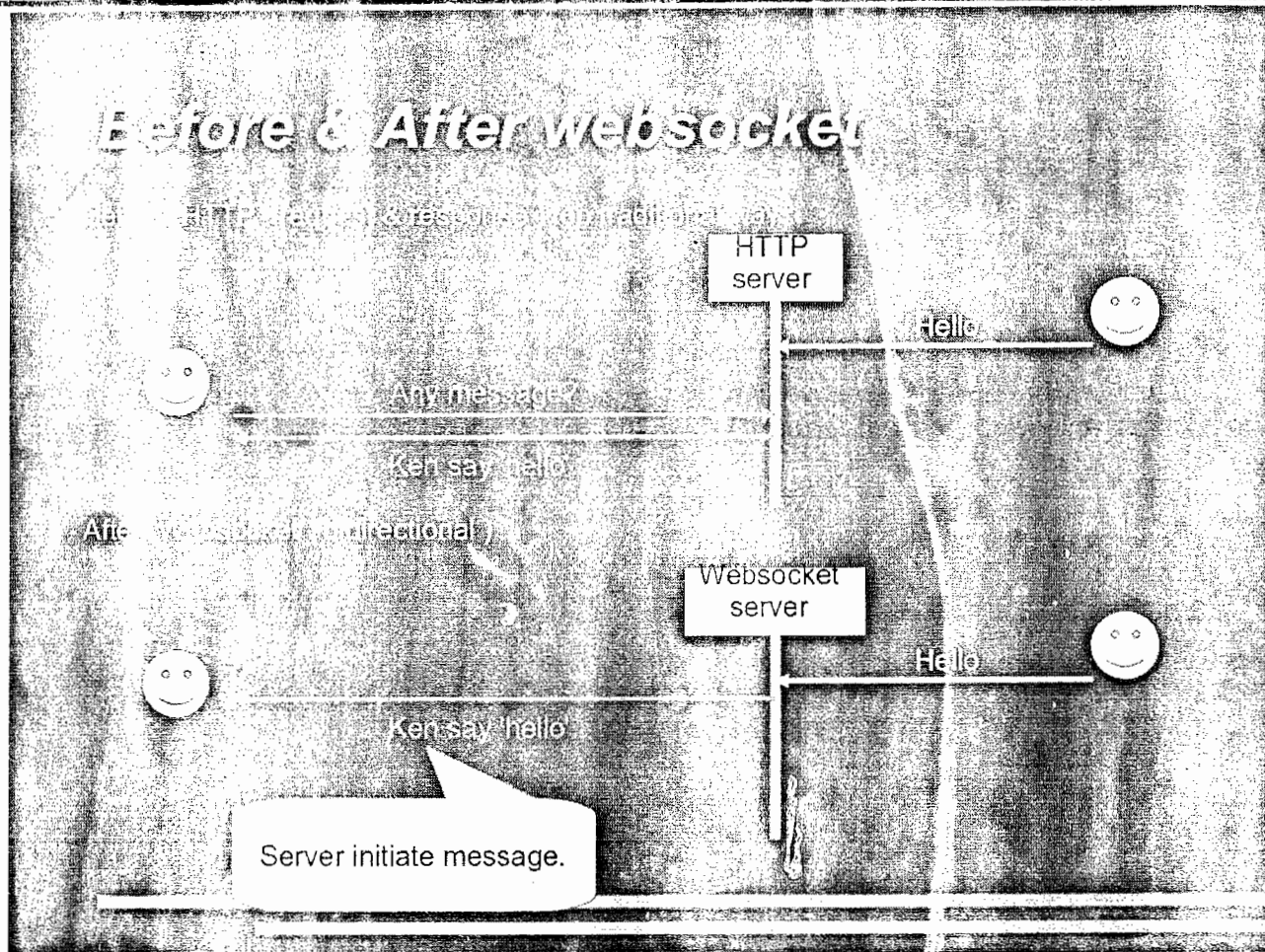
→ Spring 3.x is compatible with jdk 1.5 but supports all the features upto jdk 1.7

→ Spring 4.x is compatible with jdk1.6 and support all features upto jdk1.8. In this version all deprecated classes, methods are removed.

→ In spring 4.x messaging is a sub framework in spring that is given on the top of 'jms' to get messages based asynchronous communication between components.

→ Asynchronous communication allows the client to give next request or to perform client side operations without waiting for given request related response from server.

→ Websocket is a web level protocol that is designed on the top of protocol TCP for full 2-way communication. It is alternate to half duplex protocol http.



→ Industry uses "MVC" as defacto architecture to develop java based medium/large scale web applications.

**M-> Model ->Data + Business Logic+ persistence logic (like accounts officer)**

**V->View ->presentation logic (like Beautician)**

**C->Controller ->Integration logic (like Traffic police/supervisor)**

Note: Integration logic controls and monitors all the activities of application execution. It is responsible to get communication between view layer and model layer components.

Jsp----->servlets-----> java class/java Bean----->DB software  
(V) (C) (M)

Jsp----->servlets-----> EJB Session Bean----->Entity Bean----->DB software  
(V) (C) (M) (M)

Jsp----->servlets-----> EJB Session Bean----->DAO class (jdbc)----->DB software.  
(V) (C) (M) (M)

Struts----->EJB Session Bean----->DAO Class (jdbc)----->DB Software.  
(V) (C) (M)

**DAO: Data Access Object:**

→ The java class that separates persistence logic from other logics and makes the persistence logic

as reusable logic and flexible logic to modify is called **DAO**.

→Jdbc is light weight but does not allows to develop Object based persistence logic. EJB entity Beans are heavy weight, but allows to develop Objects based persistence logic. To overcome both problems we can use "hibernate"

**Struts----->EJB session Bean-----> hibernate----->DB software**

**Struts----->Spring----->Hibernate----->DB Software.**

**Struts----->WebServices----->Hibernate----->DB Software.**

**Struts----->Spring JEE/Bean----->Spring ORM+AOP/DAO----->DB Software.**

**Struts----->Spring JEE/Bean----->Spring ORM/DAO+AOP----->DB Software.**

**Struts----->Spring WebServices/Web services----->Spring ORM+AOP/DAO----->DB Software.**

**Spring:**

|----->Type: Java application framework

|-->Version:4.x (Compactable with jdk 1.8)

3.x (Compactable with jdk 1.5)

|-->vendor : interface 21

|-->Creator: Mr.Rod Jonshon

|-->Springsoftware comes as set of jars

!-->website: <http://spring.io>

!-->To downloadsoftware: Download as zip file from spring.io

spring-framework-4.2.2.RELEASE-dist.zip

Spring-framework-3.2.Release-with-docs.zip.

|-->To installsoftware: Extract zip file.

Spring 4.1.6/3.2.6 software installation gives

**libs-->having jar files.**

**docs-->apidocs, reference docs (PDF)**

**Schemas-->xsd file having schema rules.**

→Every DTD and XSD file contains rules to construct xml documents.

DTD->document type definition

XSD->xml schema definition.

**Spring Definition:**

**Spring is an opensource, light weight, loosely coupled, aspect oriented, dependency injection based java application framework to develop various java applications.**

→Along with spring software installation we get its source code. This makes spring as open source.

→Open source software are not only free software, they also expose their source code along with installation.

**Spring is lightweight because**

->Spring framework comes as zip file having less size

->Spring gives 2 lightweight containers which can be activated anywhere in the application.

→To work with spring container we donot need any application server/webserver software.



→We can use specific modules of spring without having any link with other modules.

eg: We can use spring core module alone or spring core and spring DAO module together without any link with other modules.

→If degree of dependency is less between components, then they are called loosely coupled components.

eg: Tv and remote.

→If degree of dependency is more between modules then they are called tightly coupled components.

eg: Keyboard and system.

→In spring framework core module is base module for all modules i.e we should use every module along with core module. But remaining modules of spring can be used alone without having any dependency with other modules. The degree of dependency is very less between various modules of spring. This makes spring as **loosely coupled framework**.

→AOP is a new methodology of programming that complements OOP to separate secondary middleware services from primary business logics and allows to link them at runtime dynamically through some configurations. This gives advantages of reusing middleware service logics and flexibility of enabling or disabling middleware service logics on primary logics (like security, transaction and etc....)

#### Dependency Lookup:

If the resource/ class/ Object is searching and getting is dependent values from other resources of project then it is called dependency lookup. **Here the resource pulls the dependent values.**

->The way we get Objects from Jndi registry is called "Dependency lookup"

->The way we get Jdbc connection Object from jdbc Connection pool is called "Dependency Lookup"

->If student is getting course material only after putting request for it is called dependency lookup.

#### Dependency injection (Inversion of control/IOC):

If underlying server/container/ framework/runtime (like JRE) assigns/injects values to resource/class/object dynamically at runtime then it is called **dependency injection**.

->Here dependent values will be pushed to the resource.

->eg: JVM calling constructor automatically to assign initial values of object when object is created.

->eg:Servlet container injects ServletConfig obj to our servlet class Object by calling **init(ServletConfig config) method** when servlet Container creates our Servlet class obj.

->Student gets course material the moment he register for that course.

**What is pojo class, POJI, Bean/Component class, Java bean & Spring Bean?**

#### **POJO class (Plain old java Object class)**

→The java class without any specialities i.e it is an ordinary java class.

→While developing this class we need not to follow any serious rules.

→The java class that is not extending from technology or framework api class and not implementing technology/ framework api interfaces is called **pojo class**.



eg:

```
class Demo
```

```
{.....
```

```
.....
```

```
}
```

→It is pojo class

```
class Test implements Serializable{
```

```
.....
```

```
.....
```

```
}
```

→Pojo class

```
class Test extends Demo
```

```
{
```

```
.....
```

```
.....
```

```
}
```

```
class Demo{
```

```
.....
```

```
.....
```

```
}
```

→Test,Demo classes are POJO classes

eg:

```
class Test extends Thread{
```

```
.....
```

```
.....
```

```
}
```

→Test is a POJO class

eg:

```
class Test extends HttpServlet{
```

```
.....
```

```
.....
```

```
}
```

“Test” is not a pojo class.

eg:

```
class Test implements java.sql.Connection{
```

```
.....
```

```
.....
```

```
} “Test” is non-pojo class
```

Eg:

```
class Test {
```

```
public void m1(Statement st){
```

```
.....
```

```
.....
```

```
}
```

```
}
```

**"Test" is pojo class.**

Eg:

```
class Test extends Demo{
```

```
.....
```

```
.....
```

```
}
```

```
class Demo implements java.rmi.Remote{
```

```
.....
```

```
.....
```

```
}
```

**"Demo", "Test" are not pojo class**

Eg:

```
@Entity
```

```
class Test{
```

```
.....
```

```
.....
```

```
}
```

**Test is a pojo class.**

-->Don't consider annotations and api's used inside the methods of a class to check wheather the class is pojo class or not.

**POJI (Plain Old Java Interafce)**

→It indicates an interface without specialities i.e an ordinary interface.

→The interface that is not extending from technology/ framework api interface is called POJI.

Eg:

```
interface Test{
```

```
.....
```

```
..... "Test" is pojo
```

```
}
```

Eg:

```
interface Test extends Serializable{
```

```
.....
```

```
.....
```

```
}
```

**"Test" is poji**

```
interface Test extends java.rmi.Remote{
```

```
.....
```

```
.....
```

```
}
```

**"Test" is not poji**

Eg:

```
interface Test extends Demo
{
.....
.....
}
interface Demo {
.....
.....
}
```

**Test, Demo are POJIs**

→The frameworks that support **pojo/ poji model programming** are called **non invasive frameworks or light weight frameworks**.

→ Hibernate, Spring, Struts 2.x etc

**Java Bean:**

It is a java class that is developed with some standards. The standards are

->Must be public class and should not be final or abstract class.

->Can implement Serializable() if needed.

->Must have direct or indirect 0-parameter constructor.

->Member variables are called bean properties and they must be taken as private and non-static.

->Every bean property should have setter and getter methods (public)

Eg:

```
public class StudentBean{
```

```
//bean properties
private int sno,
private String name;
//setter and getter
.....
.....
}
```

→In real time java bean is used as helper class to represent data with multiple values in the form of object and useful to send that object from one layer to another layer.

->Java Bean object is acceptable format to send and receive data across the multiple layers irrespective of technologies or that are used in the layers to develop the logics.

->The Java Bean class whose object holds input values given by client/enduser is called **VO class**.

->The Java Bean class whose object holds data needed for persistence operations (DAO Class) is called as (Business class) **BO class**.

->The java bean whose data is shippable over the network is called DTO class. This class generally implements Serializable().

**BO class: Business Object Class**

**VO class: Value Object class.**

**DTO class: DataTransferObject class.**

**Bean Class/Component Class:**

It is a java class that contains member variables and methods having business logic manipulating the data of member variables.

eg:

```
class CardProcessor{
private int cardNo;
public int processcard (int cardNo){
.....
..... //b.logic
.....
}
public boolean blockcard (int cardNo)
{
.....
..... //b.logic
.....
}
}
```

→The Bean/component class can extend or implement any class or interface.

#### **Spring Bean:**

→The java class whose is object created and managed by spring container is called springBean.

→SpringBean can be pojo class, java bean class, Bean/component class, but its object must be created and managed by spring container.

→Spring Bean can be user- defined class/pre-defined class /third party supplied class.

→Generally We can't take abstract class/interface as springBean because we can't create object for abstract class and interface.

→To make java class as spring Bean it must be configured in spring bean configuration file (xml) to make spring container to create and manage that spring Bean class object.

#### **Important points:**

→Spring Bean can be a pojo class? (Yes)

→Every java bean is a pojo class? (Yes)

→Every pojo class is a java Bean? (No)

→Every spring bean is a java Bean? (No)

→Spring Bean can be a java Bean? (Yes)

→Spring Bean can not implement spring api interface? (No)

→Any component/bean class can be taken spring as Bean? (Yes)

→Spring gives two light weight containers which can be started in any application by just creating object for pre-defined classes.

→Any java class or application that can manage the life cycle of given resource (class) is called container.

#### **Spring containers are :**

**1.BeanFactory (Basic container)**

(performs spring bean management and dependency injection)

**2.ApplicationContext (Advanced Container)**

(internally uses BeanFactory container so can perform advanced operations along with Bean management and dependency injection)

→To activate/ start BeanFactory container, we should create object of a class that implements "BeanFactory".

eg:

Beanfactory Factory= new XmlBeanFactory (Resource obj)

| →Points to spring bean config file (xml file)

→Spring containers (light weight) are not replacement/ alternate for servlets, jsp containers (heavy weight).

## Spring Core Module

→Base module for other spring module.Gives two IOC containers/spring containers.

1)BeanFactory

2)ApplicationContext.

→When it is used alone we can develop standard applications.When it is used along other technologies and modules we can develop model layer business logic by taking advantage of dependency injection.Talks about spring bean life cycle management and dependency injection.

### **Dependency Injection:**

→If the underlying server or container dynamically assigns the dependent values to our resource (class/object) then it is called dependency injection. There are multiple ways of performing dependency injection.

a)Setter injection (Container calls setXxx(-) to inject dependent values).

b)Constructor injection (container uses parameterized constructor to create bean class object and to inject value to that object)

c) interface injection (Aware injection)

d)Method injection

e)Lookup Method injection

and etc....

### **Setter injection sample code:**

```
//WishGenerator.java (POJO class)
package com.nt.beans;
import java.util.Date;
public class WishGenerator {
    //bean properties
    private String name;
    private Date date;
    public WishGenerator(){
        System.out.println("WishGenerator:0-param constructor");
    }
}
```

//setter methods for setter injection

```
public void setName(String name) {
    System.out.println("WishGenerator:setName(-)");
    this.name = name;
}
public void setDate(Date date) {
    System.out.println("WishGenerator:setDate(-)");
    this.date = date;
}
// B.method
public String generateWishMsg(){
    return "Good Morning:"+ name+"--->" +date;
}
}
} //class
```

**applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.3.0.xsd">

<bean id="dt" class="java.util.Date"/>
<bean id="wsg" class="com.nt.beans.WishGenerator" >
  <property name="name" value="raja"/>
  <property name="date" ref="dt"/>
</bean>
</beans>
```

→ Any <file-name>.xml can be taken as spring configuration file. But it is recommended to take "applicationContext.xml" in standalone spring applications.

In springBean configuration file we configure

a) classes as springBeans

b) Dependency injection configuration

c) Spring bean lifecycle Configuration and etc..

→ We must supply springBean configuration file while activating spring container.

→ Every spring Bean is identified with its bean id. <property> (tag) is used to configure bean property for setter injection.

→ In the above configuration, spring container creates "Date", WishGenerator" class objects using 0-param constructor and calls setName(-) and setDate (-) methods to perform setter injection.

Constructor injection Sample Code:

```
//WishGenerator.java
package com.nt.beans;
public class WishGenerator {
//bean properties
private String name;
private Date date;
//param constructor for constructor injection
public WishGenerator (String name, Date date){
this.name=name;
this.date=date;
}
//B.method
.....
}
}
```

**applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
<bean id="dt" class="java.util.Date"/>
<bean id="wsg" class="com.nt.beans.WishGenerator"
    <constructor-arg value="Raja"/>
    <constructor-arg value="dt"/>
</bean>
</beans>
```

→Based on above configuration, the spring container uses 0-param constructor to create Date class object and uses 2-param constructor to create WishGenerator class object. (because the <constructor-arg> tag is placed 2 times in the xml file). In this process injection will also be completed.

**Procedure to develop spring core module app that performs setter injection:**

**Step 1: keep eclipse IDE ready**

**Step 2: Design the application**

- ▲  IOC proj1 (Setter injection)
  - ▲  src
    - ▲  com.nt.beans
      - ▶  WishGenerator.java
    - ▲  com.nt.cfgs
      - ▶  applicationContext.xml
    - ▲  com.nt.test
      - ▶  ClientApp.java
  - ▶  JRE System Library [JavaSE-1.7]
  - ▲  IOCLib
    - ▶  spring-beans-4.1.6.RELEASE.jar - C:\Users\dell\
    - ▶  spring-context-4.1.6.RELEASE.jar - C:\Users\dell\
    - ▶  spring-context-support-4.1.6.RELEASE.jar - C:\
    - ▶  spring-core-4.1.6.RELEASE.jar - C:\Users\dell\
    - ▶  spring-expression-4.1.6.RELEASE.jar - C:\Users\dell\
    - ▶  commons-logging.jar - C:\Users\dell\Desktop

**Step3: Develop the above resources**



(Workspace : NtSp57)

**WishGenerator.java (pojo class):**

```
package com.nt.beans;
import java.util.Date;
public class WishGenerator{
//bean properties
private String name;
private Date date;
//Getter methods for setter injection
public void setName( String name){
    this.name=name;
}
public void setDate(Date date){
    this.date=date;
}
//B.logic
public String generatorWishMsg (){
    return "GoodMorning" +name+ "--->" +date;
}
}
```

**applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
<bean id="dt" class="java.util.Date"/>
<bean id="wsg" class="com.nt.beans.WishGenerator">
    <property name="name" value="Raja"/>
    <property name="date" ref="dt"/>
</bean>
</beans>
```

**Note:**

XSD,DTD are the two methodologies that can be used to define building blocks or rules for constructing xml documents.....

XSD-->Xml Schema Definition.

DTD->Document Type Defination.

**ClientApp.java:**

```
package com.nt.test;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
import com.nt.beans.WishGenerator;
public class ClientApp {
    public static void main(String[] args) {
        System.out.println("ClientApp:main(-)");
    }
}
```

```
//Locate Spring Bean configuration file
FileSystemResource res=new
FileSystemResource("src/com/nt/cfgs/applicationContext.xml");
//Activate BeanFactory container
XmlBeanFactory factory=new XmlBeanFactory(res);
// Get Bean class obj
//WishGenerator bean=(WishGenerator)factory.getBean("wsg");
WishGenerator bean=factory.getBean("wsg",WishGenerator.class);
// Call B.method
String result=bean.generateWishMsg();
System.out.println(result);
} //main
} //class
```

#### Step 4: Add the following jar files to Build path of project.

Right click on project-->Buildpath-->Config buildpath-->libraries--->Addexternal jars-->

Add external jars-->

Commons-logging.jar (collect from spring 2.5)

Spring-beans-<version>.jar

Spring-context-<vers>.jar

Spring-context-support-<version>.jar

Spring-core-<vers>.jar

Spring-expression-<version>.jar

#### Step 5: Run the client App

##### Understanding the Code

FileSystemResource res=new FileSystemResource("src/com/nt/cfgs/applicationContext.xml");

→ Gives resource object by using java.io.File .Internally pointing to the path and filename of spring bean configuration file.

XmlBeanFactory factory=new XmlBeanFactory (res);

\_\_\_\_(or)

BeanFactory factory=new XmlBeanFactory(res);

→ Based on the given "res" obj, it locates the spring bean configuration file (xml) from specified path of file system.

→ Checks wheather that xml file is well formed and valid documents or not.

→ Note1: If Xml file satisfies syntax rules then it is called **well-formed** document.

→ Note2: If xml file satisfies the **XSD and DTD** rules then it is called valid document.

→ Creates an empty logical IOC container in jvm memory where application is running.

→ Uses the SAXpraser or some other Parser (xml parser) to read xml file (Bean configuration file) data and maintains it as **inmemory metadata or inmemory configuration**.

→ Returns XmlBeanFactory class obj ref representing IOC container.

WishGenerator bean=(WishGenerator)factory.getBean("wsg");

a) Loads "com.nt.beans.WishGenerator" class from inmemory metadata based on the given "wsg" and creates object for it.

```
Class.forName("com.nt.beans.Wish").newInstance();
```

b) Observes "name" with <property> tag and calls setName("Raja") method on WishGenerator class obj.

c) Observes "date" with <property> tag and loads "java.util.Date" class to create the Object.

```
Class.forName("java.util.Date").newInstance();
```

Also calls setDate(Date class obj) on WishGenerator obj to perform setter injection.

d) Keeps Date class object and WishGenerator class obj in IOC container as elements of HashMap.

| Key       | Value                         |
|-----------|-------------------------------|
| dt        | Date Class object (dt)        |
| wsg       | WishGenerator class obj (wsg) |
| (bean id) | (Bean class object)           |

e) Returns WishGenerator class object reference back to ClientApp

→ If we configure both setter injection and constructor injection on bean property the values given by setter injection will remain as final values of injection because setter(-) methods execute after constructor execution and **overrides** the values injected by the constructor execution or injection.

→ If spring bean class is configured without any injections or only with setter injection (one or more properties) then container uses **O-param constructor** to create beans class object. If any property is configured for constructor injection then it uses **parameterized constructor** to create bean class object.

→ In spring 3.x bean properties and its setter methods can be taken as static. But in spring 2.x we cannot take setter methods as static.

→ **WishGenerator bean=(WishGenerator)factory.getBean("wsg");**  
(Typecasting mandatory)

→ **WishGenerator bean=factory.getBean("wsg", WishGenerator.class)**  
(we can avoid type casting)

**Prototype:**

```
public Object getBean (String id);  
public <T> getBean(String id, <T> clazz)
```

**What is the difference between FileSystemResource and ClassPathResource?**

→ FileSystemResource can locate given spring bean configuration file from the specified path of file system. Here we can pass either absolute path or relative path.

```
FileSystemResource res=new FileSystemResource ("src/con/nt/cfgs/applicationContext.xml")
```

→ ClasspathResource can locate given spring bean configuration file from directories or jarfiles that are added to classpath/buildpath.

```
ClassPathResource res=new ClassPathResource("applicationContext.xml");
```

(But add com.nt.cfgs package to build path)

→Right click on project->buildpath-->library-->Add class folder---->.....

### Understanding "Collaboration" and tight coupling, loose coupling:

→In real project development we can't develop entire logic in one class. We need to take multiple classes and these classes will maintain dependency with each other.

→The Class that uses other class logics/methods is called **target class** and the class that acts as helper class is called **dependent class** and we can say both classes are in **collaboration**.

→Flipkart and DTDC classes must be there in collaboration. i.e Flipkart class is target class/main class and DTDC class is dependent class.

→To make target class/main class using logics of dependent class we can use the following approaches.

#### **Approach 1:**

Create Dependent class object in targetclass (Composition)

#### **Approach 2:**

Create dependent class obj in Factory class and make target class using that factory class.

#### **Approach3:**

Target class gets Dependent class obj from jndi registry.

#### **Approach 4:**

Target class extends from dependent class (Inheritance)

#### **Approach 5:**

Make underlying container injecting dependent class object to target class.

#### **Approach 1:**

→Here target class creates the object of dependent class uses that object.

This is nothing but composition (classes containg HAS -A relation)

#### **Target class**

```
class Flipkart
{
    DTDC dtdc=new DTDC ();
    public void purchase(){
        dtdc.deliver();
    }
}
```

#### **Dependent Class**

```
class DTDC
{
    deliver(){
        .....
    }
}
```

#### **Limitations :**

- 1) Instead of DTDC courier, if Flipkart want to use BuleDart/FirstFlight courier, we need to modify target class (Flipkart).
- 2) If deliver() is changed to supply() method in DTDC class then we need to modify the Flipkart class.

The above limitation indicates target and dependent classes are tightly coupled.

#### **Approach 2:**

The class that can create and return one or more objs is called Factory class.

```

public class DTDCFactory{
    public static DTDC getInstance()
    {
        return new DTDC();
    }
}

class DTDC{
    public void deliver()
    {
        .....
        .....
    }
}

class Filpkart
{
    public void purchase()
    {
        DTDC dtdc=DTDCFactory.getInstance();
        dtdc.deliver();
    }
}

```

#### Limitations:

- 1) If Filpkart wants to use BuleDart then it needs BuleDartFactory class instead of DTDCFactory class
- 2) Same as approach 1

**Note:** Here tight coupling is observed.

#### Approach 3:

→ If you want to provide global visibility to object or object reference, then it should be placed in jndi registry.

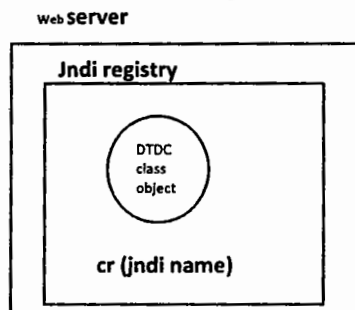
eg: rmi registry, cos registry and etc.....

**Note:** Every webserver/application server gives one built-in jndi registry.

```

class Filpkart{
    public void purchase(){
        InitialContext ic=new InitialContext();
        DTDC dtdc=(DTDC)ic.lookup("cr");
        dtdc.deliver();
    }
}

```



#### Advantages:

- 1) Because of jndi registry, DTDC obj gets global visibility.
- 2) Filpkart class can use already available DTDC class obj.

#### Limitations:

- 1) If Filpkart wants BuleDart obj instead of DTDC obj, then we need to place that obj in jndi registry.
- 2) Same as approach 1.

**Note:** It also gives tight coupling.

#### Approach 4:

Make target class extending from dependent class.

|                                                  |                                                                                    |
|--------------------------------------------------|------------------------------------------------------------------------------------|
| <pre> class DTDC { deliver () { ..... } } </pre> | <pre> class Filpkart extends DTDC { public void purchase() { deliver(); } } </pre> |
|--------------------------------------------------|------------------------------------------------------------------------------------|

**Limitations:**

- a)Filpkart class cannot extends more than one dependent class.
- b)In order to change dependency from DTDC to BuleDart, we need to make Filpkart extending from BuleDart class.
- c)Same as approach 1

**Note:**shows "tight coupling".

**Approach 5:**

→Make underlying container injecting dependent object to target object.**Use dependency injection (setter injection/constructor injection)**

|                                                                                                                                                                         |                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <pre> class Filpkart { private DTDC dtdc; //setter method public void setDtdc(DTDC dtdc) { this.dtdc=dtdc; } public void purchase () { ..... dtdc.deliver(); } } </pre> | <pre> class DTDC { public void deliver() { ..... } } </pre> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|

→This approach 5 is much similar to approach 1 or approach 2. only difference is the underlying container is managing dependency in approach 5 where as programmer is managing dependency in approach 1 and 2.

**Limitations:**

Same as approach 1

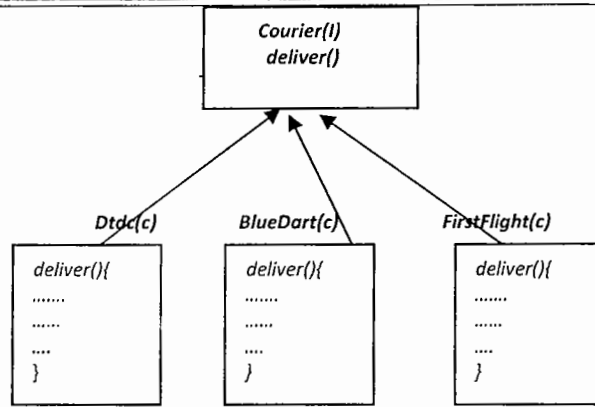
**Note:** shows tightly coupling.....

Approach 5 is good towards managing dependency. But in order to achieve loose coupling we shouldn't directly work with dependent classes in target class. It is recommended to make all dependent classes implementing common interface having common method declaration. So that we can start using this common interface in the target class.

```

class Flipkart
{
    private Courier courier;
    public void setCourier(Courier courier)
    {
        this.courier=courier;
    }
    public void purchase(){
        courier.delivery();
        .....
    }
}

```



→ The above “Flipkart(c)” class is not working with implementation class names directly. It is working with common interface name Courier, So we can make underlying container injecting any implementation class object to FlipKart (target class).

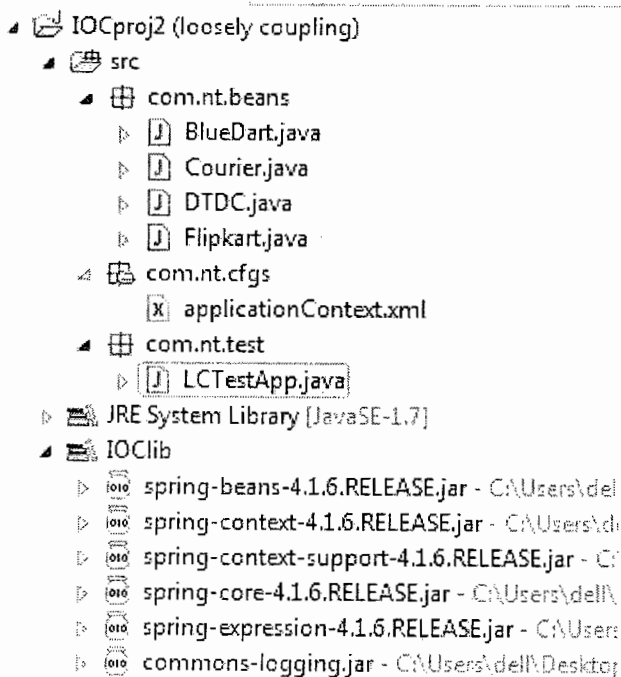
→ All dependent classes are implementing common interface “Courier(I)”, so there is no possibility of changing method name “deleiver()”

Note: The above code shows loose coupling between target class (Flipkart) and dependent classes.

To achieve loosely coupling in spring use.

a) Dependency injection

b) Interface model programming (POJI model)



**Courier.java:**

```
package com.nt.beans;
public interface Courier {
    public String deliver(int orderid);
}
```

**BlueDart.java:**

```
package com.nt.beans;
public class BlueDart implements Courier {
    public BlueDart(){
        System.out.println("BlueDart:0-param constructor");
    }
    @Override
    public final String deliver(int orderid) {
        return "BlueDart is ready to deliver products of "+orderid;
    }
}
} //class
```

**DTDC.java:**

```
package com.nt.beans;
public class DTDC implements Courier {
    public DTDC(){
        System.out.println("DTDC:0-param constructor");
    }
    public final String deliver(int orderid) {
        return "DTDC is ready to deliver products of "+orderid;
    }
}
} //class
```

**Flipkart.java:**

```
package com.nt.beans;
import java.util.Random;
public class FlipKart {
    private Courier courier;
    public FlipKart(){
        System.out.println("FlipKart:0-param constructor");
    }
    //setter method for setter injection
    public void setCourier(Courier courier){
        System.out.println("FlipKart:setCourier(-)");
        this.courier=courier;
    }
    public String purchase(String items[]){
        //generate order id
        Random rad=new Random();
        int orderid=rad.nextInt(700000);
        // deliver order
        String status=courier.deliver(orderid);
    }
}
```



```

        return status+ "Bill Amt for order id"+orderid+" is 7000";
    } //method
} //class

```

#### applicationContext.xml:

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="dtcd" class="com.nt.beans.DTDC" />
<bean id="blueDart" class="com.nt.beans.BlueDart" />
<bean id="fpk" class="com.nt.beans.FlipKart" >
    <property name="courier"><ref bean="dtcd"/></property>
</bean>
</beans>

```

#### LCTestApp.java:

```

package com.nt.test;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
import com.nt.beans.FlipKart;
public class LCTestApp {
    public static void main(String[] args) {
        // Create BeanFactory obj to create IOC container
        BeanFactory factory=new XmlBeanFactory
            (new FileSystemResource("src/com/nt/cfgs/applicationContext.xml"));
        // Get Bean obj
        FlipKart bean=factory.getBean("fpk",FlipKart.class);
        String billmsg=bean.purchase(new String[]{"shirt","mobile","books"});
        System.out.println(billmsg);
    } //main
} //class

```

#### In spring bean config file:

```

<property name="msg" value="Hello"/>Can be written as
<property name="msg"><value>Hello</value></property>

<property name="date" ref="dt"/>Can be written as
<property name="date"><ref bean="dt"/> or <ref local="dt"/></property>

```

#### Attaching source code and api docs (Spring) with eclipse ide:

Right click on project --->buildpath--->configurebuildpath  
 Select any jar file that contains the classes that you are using

#### Source attachment

```

edit--->external          location---->          path:--->external          file-->select
<spring_home>/libs/spring_beans_<x.x>
Release_sources jar.file

```

### Java doc location

Edit---->javadoc in archive-->external file---->archive path (Select spring software zip file)

Path in archive : select..docs/javadoc\_api folder in the above zip file --->validate-->ok

### Creating user-defined library (SpringIOC) in eclipse IDE:

Right click project-->build path---->configure build path---->library---->Add library-->user library-----  
-->next---->userlibrary--->new-->IOCLibrary-->Add external jars--->select required jar files.

### Design Patterns

Design patterns are set of rules which come as best solutions for reoccurring problems of application development. Design patterns are best practices for utilizing software languages, technologies and Frameworks effectively in application development.

eg: Singleton, factory and etc....

→ BeanFactory (IOC container) is designed based on the factory pattern....

→ It is always recommended to develop spring apps based on strategy pattern.

#### **Stratgrty pattern:**

While developing multiple classes having dependency it is recommended to follow this design pattern. This pattern is not spring pattern, it can be used anywhere since spring manages dependency between classes, it is recommended to use this pattern while developing spring application. Strategy pattern is all about implementing 3 principles.

a) Prefer composition(HAS-A) over inheritance.

b) Always code to interface and never code with implementations(classes)

c) Code should be open to extension and must be closed for modification

#### **Detailed Explanation:**

a) Prefer composition over inheritance:

→ If one class creates the objects of other class to use its logics then it is called **composition**  
(Has -A Relationship)

```
class A{  
.....  
.....  
}  
class B  
{  
    A a=new A();  
}
```

**If one class extends from another class then it is called inheritance (Is-A Relationship)**

```
class A  
{  
.....  
.....  
}  
class B extends A  
{  
.....
```

```
.....  
}
```

**Inheritance is having following limitations in java.**

- a) It doesn't support multiple inheritance
- b) Code becomes easily breakable

*class C extends A,B -> not possible*

```
{  
.....  
.....  
}
```

*class C { -> possible through composition*

```
A a=new A();  
B b=new B();  
}
```

**Code becomes easily breakable:**

Eg:

```
class Test{  
    public int x()  
    {  
        .....  
        .....  
        return 100;  
    }  
}
```

```
class Demo extends Test {  
    public int x(){  
        .....  
        .....  
        return 1000;  
    }  
}
```

If we change return type of 'x()' in "Test" we can not compile "Demo" class, because of overriding rules (let us assume "Test", "Demo" classes are given by two different developers)

Solution:

```
class A{  
    public int x(){  
        .....  
        return 100;  
    }  
}  
class B  
{  
    A a=new A();
```

```
public int y(){  
    int res=a.x();  
}  
}
```

- If 'x()' method return type is changed in "A", then there is no need of doing major modifications in 'B'.
- If x() return type is changed to float then we need to write y() method in 'B' as shown below

```
public int y(){  
    float res=a.x();  
    .....  
    .....  
}
```

\*The other classes that are using B class need not to have any modifications i.e the modification is restricted to only to **B class**

**b)Always code to interface and never code to implementation:**

```
class A{  
    .....  
}  
class B{  
    .....  
    .....  
}  
class Test{  
    A a=new new A();  
    .....  
    .....  
}
```

- Assigning class "B" object instead of class "A" object in Test class is not possible with minimum modification. Make B,A classes implementing common Interface as shown below

Eg:

```
interface X{  
    .....  
    .....  
}  
class A{  
    .....  
    .....  
}  
class B implements X{  
    .....  
    .....  
}
```

```
}  
class Test{  
X x=new A();  
(Or)  
X x=new B();  
.....  
.....  
}
```

- When we code with interfaces, we can achieve loose coupling between target/main class and dependent class.

```
interface X{  
.....  
}  
class A implements X{  
.....  
}  
class B implements X{  
.....  
}  
class Test{  
X x;  
public void setX(X x){  
this.x=x;  
}  
}
```

```
Test t=new Test();  
t.setX (new A());  
t.setX (new B());
```

Note:

Without modifying the code of "Test" class we can assign either the object of class A or class B. This gives **loose coupling**.

#### Code must be open for extension and must be closed for modification.

If we follow second principle perfectly by taking the methods of implementation classes A, B as final methods then our code becomes **closed for modification**. Similarly it allows to more implementation classes for **interface X** having new logics. This makes our code **open for extension**.

#### Observing strategy pattern in Application no:2 :

a) Flipkart class wants the logics of DTDC or BlueDart classes. But it is not inheriting from those classes, but getting object of those classes through composition. (principle 1)

b) BlueDart, DTDC classes are implementing "Courier(I)" or their objects are assigned to Flipkart class through Courier(I). (principle 2)

c) By making the deliver() method of BlueDart, DTDC classes as final methods, we can keep that code of classes closed for modification and we can develop new implementation classes for "Courier(I)". if new Courier service (like FirstFlight) is required for Flipkart class (principle-3)

**Understanding Realtime Dependency Injection**

DAO class→ Java class that contains persistence logic

Service class→ java class that contains B.logic

Controller class→ Java class that receives request and delegates to Service class..

**StudentVO.java**

```
package com.nt.vo;
public class StudentVO {
    private String sno,sname,m1,m2,m3;

    public String getSno() {
        return sno;
    }

    public void setSno(String sno) {
        this.sno = sno;
    }

    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }

    public String getM1() {
        return m1;
    }

    public void setM1(String m1) {
        this.m1 = m1;
    }

    public String getM2() {
        return m2;
    }

    public void setM2(String m2) {
        this.m2 = m2;
    }

    public String getM3() {
        return m3;
    }
    public void setM3(String m3) {
        this.m3 = m3;
    }
}
```

**StudentBO.java**

```
package com.nt.bo;

public class StudentBO {
    private int sno;
    private String sname;
    private int total;
    private float avg;
    private String result;

    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public int getTotal() {
        return total;
    }
    public void setTotal(int total) {
        this.total = total;
    }
    public float getAvg() {
        return avg;
    }
    public void setAvg(float avg) {
        this.avg = avg;
    }
    public String getResult() {
        return result;
    }
    public void setResult(String result) {
        this.result = result;
    }
}
```

**StudentDTO.java**

```
package com.nt.dto;
import java.io.Serializable;
public class StudentDTO implements Serializable {
    private int sno;
    private String sname;
```

```
private int m1,m2,m3;
public int getSno() {
    return sno;
}
public void setSno(int sno) {
    this.sno = sno;
}
public String getSname() {
    return sname;
}
public void setSname(String sname) {
    this.sname = sname;
}
public int getM1() {
    return m1;
}
public void setM1(int m1) {
    this.m1 = m1;
}
public int getM2() {
    return m2;
}
public void setM2(int m2) {
    this.m2 = m2;
}
public int getM3() {
    return m3;
}
public void setM3(int m3) {
    this.m3 = m3;
}
}
```

#### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

<bean id="drds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
    <property name="username" value="scott"/>
    <property name="password" value="tiger"/>
</property>
</bean>
<bean id="stDao" class="com.nt.dao.StudentDAOImpl">
```



```
<constructor-arg ref="drds"/>
</bean>
<bean id="stService" class="com.nt.service.StudentServiceImpl">
  <constructor-arg ref="stDao"/>
</bean>

<bean id="stController" class="com.nt.controller.StudentController">
  <constructor-arg ref="stService"/>
</bean>
</beans>
```

**StudentController.java**

```
package com.nt.controller;

import com.nt.dto.StudentDTO;
import com.nt.service.StudentService;
import com.nt.vo.StudentVO;

public class StudentController {
    private StudentService service;

    public StudentController(StudentService service) {
        this.service = service;
    }

    public String process(String sno,String sname,String m1,String m2,String m3){
        //prepare VO obj
        StudentVO vo=new StudentVO();
        vo.setSno(sno);vo.setSname(sname);
        vo.setM1(m1);vo.setM2(m2);vo.setM3(m3);
        //Convert to VO obj to DTO obj
        StudentDTO dto=new StudentDTO();
        dto.setSno(Integer.parseInt(vo.getSno()));
        dto.setSname(vo.getSname());
        dto.setM1(Integer.parseInt(vo.getM1()));
        dto.setM2(Integer.parseInt(vo.getM2()));
        dto.setM3(Integer.parseInt(vo.getM3()));
        //use Service class
        String result=service.generateResult(dto);
        return result;
    }
}
```

**StudentService.java**

```
package com.nt.service;

import com.nt.dto.StudentDTO;

public interface StudentService {
    public String generateResult(StudentDTO dto);
}
```

**StudentServiceImpl.java**

```
package com.nt.service;
import com.nt.bo.StudentBO;
import com.nt.dao.StudentDAO;
import com.nt.dto.StudentDTO;
public class StudentServiceImpl implements StudentService {
    private StudentDAO dao;

    public StudentServiceImpl(StudentDAO dao) {
        this.dao = dao;
    }

    @Override
    public String generateResult(StudentDTO dto) {
        //write b.logic
        int total=dto.getM1()+dto.getM2()+dto.getM3();
        float avg=(float)total/3.0f;
        String result=null;
        if(avg<35)
            result="Fail";
        else
            result="Pass";
        //create BO class obj
        StudentBO bo=new StudentBO();
        bo.setSno(dto.getSno());
        bo.setName(dto.getName());
        bo.setTotal(total);
        bo.setAvg(avg);
        bo.setResult(result);
        //use DAO
        int cnt=dao.insert(bo);

        if (cnt==0)
            return "Result:"+bo.getResult()+" "+bo.getSno()+" Registration Failed";
        else
            return "Result:"+bo.getResult()+" "+bo.getSno()+" Registration Succeeded";
    }
}
//method
//class
```

**StudentDAO.java**

```
package com.nt.dao;

import com.nt.bo.StudentBO;

public interface StudentDAO {
    public int insert(StudentBO bo);
}
```

**StudentDAOImpl.java**

```
package com.nt.dao;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import javax.sql.DataSource;
import com.nt.bo.StudentBO;

public class StudentDAOImpl implements StudentDAO{
    private static final String STUDENT_INSERT_QUERY="INSERT INTO SP_STUDENT
VALUES(?,?,?,?)";

    private DataSource ds;
    public StudentDAOImpl(DataSource ds) {
        this.ds = ds;
    }

    @Override
    public int insert(StudentBO bo) {
        //write jdbc code to insert record
        try{
            //get jdbc con
            Connection con=ds.getConnection();
            //create PreparedStatement obj
            PreparedStatement ps=con.prepareStatement(STUDENT_INSERT_QUERY);
            // set Query param values
            ps.setInt(1,bo.getSno());
            ps.setString(2,bo.getSname());
            ps.setInt(3,bo.getTotal());
            ps.setFloat(4,bo.getAvg());
            ps.setString(5,bo.getResult());
            // execute the Query
            int result=ps.executeUpdate();
            return result;
        }
        catch(SQLException se){
            return 0;
        }
        catch(Exception e){
            return 0;
        }
    }
    //method
}
//class
```

**ClientApp.java**

```
package test;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;

import com.nt.controller.StudentController;

public class ClientApp {

    public static void main(String[] args) {
        //Create IOC Container
        BeanFactory factory = new XmlBeanFactory(new
        FileSystemResource("src/com/nt/cfgs/applicationContext.xml"));
        // Get Bean
        StudentController controller = factory.getBean("stController", StudentController.class);
        //call method
        System.out.println(controller.process("1013", "raja", "56", "67", "78"));
    }
}
```

**Resolving/identifying params in constructor injection:**

We generally configure the dependent values of constructor injection in the order the params of constructor are available. But we can mismatch this order during configuration if we can identify the same params by using 'index or type or name'.

Eg::

**Resolving based on parameter type:**

- ▶  IOCProj3(Constructor Injection)
  - ▶  src
    - ▶  com.nt.beans
      - ▶  Faculty.java
      - ▶  Machine.java
      - ▶  Marks.java
      - ▶  Student.java
    - ▶  com.nt.cfgs
      - ▶  applicationContext.xml
    - ▶  com.nt.test
      - ▶  ClientApp.java
  - ▶  JRE System Library [JavaSE-1.7]
  - ▶  IOCLib
    - ▶  spring-beans-4.1.6.RELEASE.jar - C:\Users\c
    - ▶  spring-context-4.1.6.RELEASE.jar - C:\Users\c
    - ▶  spring-context-support-4.1.6.RELEASE.jar -
    - ▶  spring-core-4.1.6.RELEASE.jar - C:\Users\de
    - ▶  spring-expression-4.1.6.RELEASE.jar - C:\Us
    - ▶  commons-logging.jar - C:\Users\del\Desk

**Student.java:**

```
package com.nt.beans;
public class Student {
    private int sno;
    private String sname;
    private float avg;
    public Student(int sno, String sname, float avg) {
        System.out.println("Student:3-param constructor");
        this.sno = sno;
        this.sname = sname;
        this.avg = avg;
    }
    @Override
    public String toString() {
        return "Student [sno=" + sno + ", sname=" + sname + ", avg=" + avg + "]";
    }
}
```

**applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

<bean id="St" class="com.nt.beans.Student">
    <constructor-arg value="1001" type="int"/>
    <constructor-arg value="89.45F" type="float"/>
    <constructor-arg value="raja" type="java.lang.String"/>
</bean>
</beans>
```

**Limitation:**

If multiple params are having same type then this approach can't be used.

Eg2: Resolving/identifying based on index:

**Marks.java**

```
package com.nt.beans;
public class Marks{
    private int m1,m2,m3;
    public Marks(int m1,int m2,int m3){
        this.m1=m1;
        this.m2=m2;
        this.m3=m3;
    }
    //Place toString()
    @Override
    public String toString() {
        return "Marks [m1=" + m1 + ", m2=" + m2 + ", m3=" + m3 + "]";
    }
}
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="mk" class="com.nt.beans.Marks">
    <constructor-arg value="30" index="2"/>
    <constructor-arg value="20" index="1"/>
    <constructor-arg value="10" index="0"/>
</bean>
</beans>
```

**Ex3: Resolving constructor params through name:**Faculty.java:

```
package com.nt.beans;
import java.beans.ConstructorProperties;
public class Faculty {
    private int id;
    private String name,subject;
    @ConstructorProperties({"no","fname","sub"})//optional in spring 4.x
    public Faculty(int no, String fname, String sub) {
        this.id = no;
        this.name = fname;
        this.subject = sub;
    }
    public String toString() {
        return "Faculty [id=" + id + ", name=" + name + ", subject=" + subject + "];"
    }
}
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="st" class="com.nt.beans.Student">
    <constructor-arg value="1001" name="no"/>
    <constructor-arg value="java" name="sub"/>
    <constructor-arg value="raja" name="fname"/>
</bean>
</beans>
```

- In Spring 3.x the IOC container can't get the parameter names of constructor directly until the parameter names are configured by using "@ConstructorProperties(-)".
- In spring 4.x the IOC container can recognise constructor parameters directly.

**Cyclic/Circular dependency injection:**

If two beans are dependent to each other then we can say they are in cyclic dependency. Spring container can't perform cyclic dependency injection through constructor injection but can perform through setter injection.

**Problem:**

- ▲ IOCProj4(Cyclic Injection)
  - ▲ src
    - ▲ com.nt.beans
      - A.java
      - B.java
    - ▲ com.nt.cfgs
      - applicationContext.xml
    - ▲ com.nt.test
      - ClientApp.java
  - JRE System Library [JavaSE-1.7]
  - ▲ IOCLib
    - spring-beans-4.1.6.RELEASE.jar - C:\Users\del
    - spring-context-4.1.6.RELEASE.jar - C:\Users\del
    - spring-context-support-4.1.6.RELEASE.jar - C:\
    - spring-core-4.1.6.RELEASE.jar - C:\Users\del\A
    - spring-expression-4.1.6.RELEASE.jar - C:\Users
    - commons-logging.jar - C:\Users\del\Desktop

```

class A{
    B b;
    public A(B b){
        this.b=b;
    }
}
class B{
    A a;
    public B(A a){
        this.a=a;
    }
}

```

**Problem:****applicationContext.xml:**

```

<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="a" class="A">
        <constructor-arg ref="b"/>
    </bean>
    <bean id="b" class="B">
        <constructor-arg ref="a"/>
    </bean>
</beans>

```

**Solution: (Using Setter injection)**

```

class A{
    B b;
    public void setB(B b){
        this.b=b;
    }
}
class B {
    A a;
    public void setA(A a){
        this.a=a;
    }
}

```

**applicationContext.xml (Supports)**

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="a" class="A">
        <property name="b" ref="b"/>
    </bean>
    <bean id="b" class="B">
        <property name="a" ref="a"/>
    </bean>
</beans>

```

**When should we go for setter injection and when should we go for constructor injection?**

Ans: If bean class contains only one property or if all the properties of bean class should participate in dependency injection then go for **constructor injection**, because **construct injection is very faster than setter injection**.

→ If Bean class contains more than one property and there is no need of making all properties participating in dependency injection, then we need to go for setter injection.

Eg: If bean class contains four properties and there is no need of making all properties participating in dependency injection, then we need to write max of four setter methods to perform setter injection. But we need to write four factorial (24) constructors to support constructor injection in all angles.

**Difference between setter injection and constructor injection:**

| Setter injection                                                                             | Constructor injection                                                                        |
|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Performs injection after creating Bean class obj so it is bit delayed injection.             | Performs injection while creating Bean class object so no delay in injection.                |
| Supports cyclic dependency injection                                                         | Doesn't supports cyclic injection                                                            |
| <property> tag is required.                                                                  | <constructor-arg> tag is required.                                                           |
| To perform setter injection on "n-properties" In all angles "n-setter" methods are required. | To perform constructor injection on n properties in all angles n! constructors are required. |



If all properties are configured for setter Injection then container creates bean class object using O-param constructor.

If any property is configured for constructor injection container bean class object using parameterized constructor.

### Inner Beans:

This is no way related with inner classes of java. It is all about configuring one bean configuration as inner bean of another bean configuratuib by placing **<bean>** tag inside the **<property>** or **<constructor-arg>** tag.

EX:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="fjk" class="com.nt.beans.Flipkart">
    <property name="courier">
      <bean id="dtdc" class="com.nt.beans.DTDC"/>
    </property>
  </bean>
</beans>
```

Use inner bean configuration if we want to make bean as depedent bean to another bean only for one time.

### Limitations with inner bean:

- Inner bean cannot be used as dependent bean of multiple other beans.
- Inner bean object cannot be accessed from client applications.

Note: Bean id is optional for inner beans.

Can we configure spring bean class without bean id?

Ans: Yes possible, because the container generates default bean 'id' in the following notation.

<pkg>.<BeanClass>#n

#n-->0,1,2,3,.....

eg: <bean class="com.nt.beans.FlipKart">(bean id is "com.nt.beans.Flipkart#0")

.....

.....

</bean>

<bean class="com.nt.beans.Flipkart">(bean id is "com.nt.beans.Flipkart#1)

.....

.....

</bean>

### Spring supports dependency injection on the following properties

- Simple properties (primitive data types, String)
- Reference/Object type properties
- Collection properties (Array, List, Set, Map, Properties, and etc..... )

| Property type        | Tag for dependency injection configuration |
|----------------------|--------------------------------------------|
| Simple property      | value attribute or <value> tag             |
| Reference property   | ref attribute of <ref> tag                 |
| Array                | <list> tag                                 |
| java.util.List       | <list>                                     |
| java.util.Set        | <set>                                      |
| java.util.Map        | <map>                                      |
| java.util.Properties | <props>                                    |

#### IOCProj5(Collection Injection)

- src
  - com.nt.beans
    - College.java
    - EmployeeProfile.java
    - Student.java
    - User.java
    - User1.java
  - com.nt.cfgs
    - applicationContext.xml
  - com.nt.test
    - CollectionTest.java
- JRE System Library (JavaSE-1.7)
- IOCLib
  - spring-beans-4.1.6.RELEASE.jar - C:\Users\...
  - spring-context-4.1.6.RELEASE.jar - C:\Users\...
  - spring-context-support-4.1.6.RELEASE.jar - C:\Users\...
  - spring-core-4.1.6.RELEASE.jar - C:\Users\de...
  - spring-expression-4.1.6.RELEASE.jar - C:\Us...
  - commons-logging.jar - C:\Users\deli\Desk...

#### Student.java:

```

package com.nt.beans;
import java.util.Arrays;
import java.util.List;
public class Student {
    private int marks[];
    private List<String> names;
    public void setNames(List<String> names){
        this.names=names;    }
    public void setMarks(int[] marks) {
        this.marks = marks;
    }
    @Override
    public String toString() {
        return "Student [marks=" + Arrays.toString(marks) + ", names=" + names
            + " ]";
    }
}

```

**applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="st" class="com.nt.Student">
    <property name="colors">
      <list>
        <value>red</value>
        <value>green</value>
      </list>
    </property>
  </bean>
</beans>
```

**Internal Code:**

```
Student st=(Student)Class.forName("Student").newInstance();
String colors[]=new String[]{"red","green"};
st.setColors(colors);
```

**Injecting List:**

```
public class Student{
  private List<String> names;
  public void setNames (List<String> names){
    this.names=name;
  }
  //toString
  .....
  .....
}
```

**applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="St" class="com.nt.beans.Student">
    <property name="names">
      <list>
        <value>Raja</value>
        <value>Ravi</value>
        <value>Raju</value>
      </list>
    </property>
  </bean>
</beans>
```

Note: java.util.List allows duplicates. So we can inject duplicates values.

**Internal code:**

```
Student st=(Student)Class.forName("com.nt.beans.Student").newInstance();
List <String>names=new ArrayList <String>();
names.add("Raja");
names.add("Ravi");
names.add(Raju);
st.setName(names);
```

**Injecting Set:**

Java.util.Set type collection doesn't allows duplicate vales.

Eg:

```
package com.nt.beans;
import java.util.Set;
public class User{
    private Set<String> phones;
    public void setPhones(Set<String> phones){
        this.phones=phones;
    }
    // toString()
    @Override
    public String toString() {
        return "User [phones=" + phones + "]";
    }
}
```

**applicationContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="user" class="com.nt.beans.User">
    <property name="phones">
        <set>
            <value>+91-9859922331</value>
            <value>+91-892149322</value>
        </set>
    </property>
</bean>
</beans>
```

**Injecting Map:**

Each element of map allows one key, one value, use <map> to configure injection.

Eg:

```
package com.nt.beans;
import java.util.Map;
public class College{
    private Map<String,String> facultySubjects;
    private Map<?,?> facultyPhones;
```

```
public void setFacultySubjects(Map<String,String> facultySubjects){
    this.facultySubjects=facultySubjects;
}
public void setFacultyPhones(Map<?, ?> facultyPhones) {
    this.facultyPhones = facultyPhones;
}

@Override
public String toString() {
    return "College [facultySubjects=" + facultySubjects
        + ", facurtyPhones=" + facultyPhones + "];"
}
}
```

**applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="clg" class="com.nt.beans.College">
<property name="facultySubjects">
<map>
<entry key="Raja" value="java"/>

<entry>
<key><value>Ravi</value></key>
<value>.NET</value>
</entry>

<entry key="Ramesh">
<value>SAP</value>
</entry>

</map>
</property>
</bean>
</beans>
```

- Map collection elements allow any objects as keys and any objects as values, but keys cant be duplicated and values can be duplicated.

**Injecting java.util.Properties:**

- Java.util.Properties allows only String as the key-value pairs i.e key must be String and value must be String.

eg:

**EmployeeProfile.java**

```
package com.nt.beans;
import java.util.Properties;
public class EmployeeProfile{
```

```
private Properties empDesgs;
public void setEmpDesgs(Properties empDesgs){
    this.empDesgs=empDesgs;
}
// toString()
@Override
public String toString() {
    return "EmployeeProfile [empDesgs=" + empDesgs + "]";
}
}
```

#### applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="emp" class="com.nt.beans.employeeProfile">
<property name="empDesgs">
<props>
<prop key="Anil">Clerk</prop>
<prop key="Jani">SE</prop>
</props>
</property>
</bean>
</beans>
```

#### Injecting Null (<null>)

- Given to injection null value to reference type bean property.
- Useful in constructor injection to inject null value to certain reference type parameter when we cannot supply value.
- In constructor injection we need to supply values to all the params. If we cannot supply values to certain parameters then we can **<null /> tag to inject null value.**

EX:

```
public class User{
private int id;
private String name;
private Date doj;
public user(int id, String name, Date dob){
    this.id=id;
    this.name=name;
    this.dob=dob;
}
//toString()
.....
.....
}
```

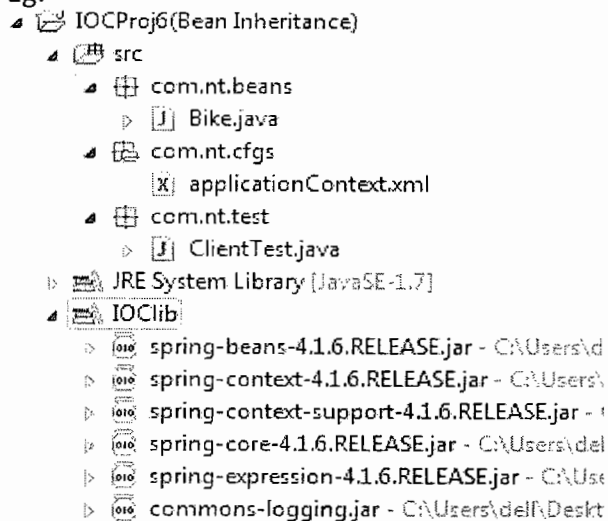
**applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="user" class="pkg.user">
<constructor-arg value="100"/>
<constructor-arg value="Raja"/>
<constructor-arg><null/></constructor-arg>
<!--null injection-->
</bean>
</beans>
```

### Bean Inheritance:

We use inheritance for reusability and extensibility. Bean inheritance is no way related with class level physical inheritance. It is all about about xml file level bean configurations inheritance. When we want configure same class for multiple times as multiple spring beans we can use reuse of bean properties configuration across the multiple bean configurations by using bean inheritance.

Eg:-



```
IOCLib
├── spring-beans-4.1.6.RELEASE.jar - C:\Users\d
├── spring-context-4.1.6.RELEASE.jar - C:\Users\d
├── spring-context-support-4.1.6.RELEASE.jar - C:\Users\d
├── spring-core-4.1.6.RELEASE.jar - C:\Users\d
├── spring-expression-4.1.6.RELEASE.jar - C:\Users\d
└── commons-logging.jar - C:\Users\d\de\l\Desk
```

**Bike.java:**

```
package com.nt.beans;
public class Bike{
    private String id;
    private String make;
    private String engineCC;
    //write seTxx(-) methods
    public void setId(String id) {
        this.id = id;
    }
    public void setMake(String make) {
        this.make = make;
    }
    public void setEngineCC(String engineCC) {
```

```
this.engineCC = engineCC;
}
//write toString()
@Override
public String toString() {
    return "Bike [id=" + id + ", make=" + make + ", engineCC=" + engineCC + "];"
}
}
```

**applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="pulsor1" class="com.nt.beans.Bike">
    <property name="id" value="TS081111"/>
    <property name="make" value="Bajaj"/>
    <property name="engineCC" value="150"/>
</bean>
<bean id="pulsor2" class="com.nt.beans.Bike">
    <property name="id" value="TS082222"/>
    <property name="make" value="Bajaj"/>
    <property name="engineCC" value="150"/>
</bean>
</beans>
```

**Problem:**

Make, engineCC property values configuration is not reusable even though they are same in multiple bean configurations.

**Solution: (Bean inheritance)****applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="pulsor1" class="com.nt.beans.Bike">
    <property name="id" value="TS081111"/>
    <property name="make" value="Bajaj"/>
    <property name="engineCC" value="150"/>
</bean>
<bean id="pulsor2" class="com.nt.beans.Bike" parent="pulsor1">
    <property name="id" value="TS082222"/>
</bean>
</beans>
```



**Solution2: (improved Solution)**

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="basepulsor" class="com.nt.beans.Bike" abstract="true">
    <property name="maxe" value="Bajaj"/>
    <property name="engineCC" value="150"/>
  </bean>
  <bean id="pulsor1" class="com.nt.beans.Bike" parent="basepulsor">
    <property name="id" value="TS021111"/>
  </bean>
  <bean id="pulsor2" class="com.nt.beans.Bike" parent="basepulsor">
    <property name="id" value="Ts022222"/>
  </bean>
</beans>
```

**Important Points:**

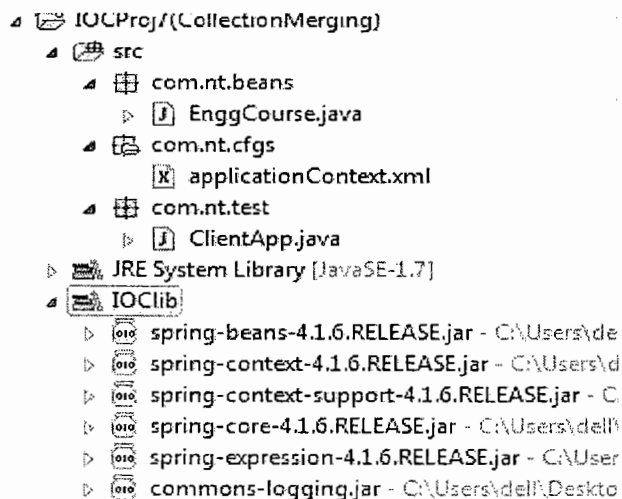
- It is not class level physical inheritance.
- Parent bean configuration and child bean configuration can refer same class or different classes.

**Note:**

When abstract ="true" it doesn't make bean class as abstract class, but it makes bean configuration as abstract bean configuration i.e we can't call `factory.getBean(-)` having that abstract bean configuration bean id (throws Exception)

**Collection Merging**

It allows to merge the values set to "Base bean" configuration collection property with the values set to "Child bean" configuration collection property by using `merge="true"` attribute.



**EnggCourse.java:**

```
package com.nt.beans;
import java.util.List;
public class EnggCourse{
    private List <String> subjects;
    public void setSubjects(List <String>subjects){
        this.subjects=subjects;
    }
    // toString()
    @Override
    public String toString() {
        return "EnggCourse [subjects=" + subjects + "]";
    }
}
```

**applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="base1styear" class="com.nt.beans.EnggCourse" abstract="true">
        <property name="subjects">
            <list>
                <value>C</value>
                <value>Math</value>
            </list>
        </property>
    </bean>

    <bean id="ECE1styear" class="com.nt.beans.EnggCourse" parent="base1styear">
        <property name="subjects">
            <list>
                <value>digital elect</value>
                <value>C</value>
                <value>Math</value>
            </list>
        </property>
    </bean>
</beans>
```

**Problem:** Writing common Subjects (C,Maths) again in child bean configuratuib indicates duplication.

**Approach2 :****applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<bean id="base1styear" class="com.nt.beans.EnggCourse" parent="base1styear">
  <property name="Subjetcs">
    <list>
      <value>digital Eletronics</value>
    </list>
  </property>
</bean>
</beans>
```

**Problem:**

Child bean configuration overrides the values of parent bean configuration.

**Approach3:****applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="ECE1styear" class="EnggCourse" parent="Base1styear">
    <property name="Subjecst">
      <list merge="true">
        <value>Dig EC</value>
      </list>
    </property>
  </bean>
</beans>
```

**Important points:**

- i) Merging is possible only on collection type property.
- ii) Base bean and Child bean configuration types must match to perform merging.

**Client App:**

```
EnggCourse EcEengg=factory.getBean("ECE1styear", EnggCourse.class);
System.out.println (ECEengg);
```

### Bean Alias

Alias name is nick name or petname. It is all about providing more alternative names/id's to use bean. Prior to spring 2.0 "name" attribute is given to provide "aliasname" from spring 2.0 <alias> tag is introduced for the same. We can use both name attribute, <alias> tags to provide alias names from spring 2.0.

Eg:

```
<bean id="system" name="pc, desktop" alias="com.nt.beans.computer">
.....
.....
</bean>
<alias name="system" alias="machine"/>
<alias name="system" alias="box"/>
```

Note1: <alias> tag is recommended to use because it allows to provide alias names to multiple bean in single place.

Note2: Aliasing is useful in the maintenance mode of bean configurations, where we can write

short alias name for lengthy beans ids.

### //real time use of Bean Alias

Generally bean class names will be taken as bean id's, some times they will be quite lengthy to provide shorter version of lengthy bean id's we can take the support of bean alias name.

### In ClientApp

```
factory.getBean("box");  
factory.getBean("pc");  
factory.getBean("system");
```

### Performing dependency lookup by using IOC container:

If target class/main class writes the code to search and gather dependent values/objects is called **dependency lookup or dependency pull**. By creating IOC container in target class or main class we can perform dependency lookup/pull operation to get dependent objects.

If target class wants to use dependent object in all the methods of target class then go for **dependency Injection**. If target class wants to use dependent class object only in specific method, then go for **dependency lookup or pull**.

Spring supports both dependency injection and dependency lookup and we can use both concepts in single application.

If target bean class of spring application wants to use external services components gathered from JNDI registry (like gathering **score component reference**, **weather report component reference** and etc then go for **dependency lookup**).

• IOCProj8(Dependency Lookup-IDREF)

```
src  
├── com.nt.beans  
│   ├── Engine.java  
│   └── Viechle.java  
├── com.nt.cfgs  
│   └── applicationContext.xml  
└── com.nt.test  
    └── ClientApp.java  
JRE System Library [JavaSE-1.7]  
IOCLib  
├── spring-beans-4.1.6.RELEASE.jar - C:\Users\del  
├── spring-context-4.1.6.RELEASE.jar - C:\Users\del  
├── spring-context-support-4.1.6.RELEASE.jar - C:  
├── spring-core-4.1.6.RELEASE.jar - C:\Users\del\A  
├── spring-expression-4.1.6.RELEASE.jar - C:\User  
└── commons-logging.jar - C:\Users\del\Desko
```

### Engine.java:

```
package com.nt.beans;  
public class Engine{  
    public Engine(){  
        System.out.println("Engine:0-param constructor");  
    }  
    public void start(){  
        System.out.println("Engg started");  
    }  
}
```

**Viechle.java:**

```
package com.nt.beans;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.cglib.proxy.Factory;
import org.springframework.core.io.FileSystemResource;
public class Viechle{
    private String beanId;
    public Viechle(){
        System.out.println("Viechle:0-param constructor");
    }
    public void setBeanId(String beanId){
        this.beanId=beanId;
    }
    public void move(){
        BeanFactory factory=new XmlBeanFactory(new
            FileSystemResource("src/com/nt/cfgs/applicationContext.xml"));
        Engine engine=factory.getBean("eng1",Engine.class);
        engine.start();
        System.out.println("Viechle moved for journey");
    }
}
```

**applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="eng1" class="com.nt.beans.Engine"/>
<bean id="viechle" class="com.nt.beans.Viechle">
    <property name="engId" value="eng1">
</bean>
</beans>
```

**Client Application:**

```
package test;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
import com.nt.beans.Viechle;
public class ClientApp {
    public static void main(String[] args) {
        BeanFactory factory=new XmlBeanFactory(new
            FileSystemResource("src/com/nt/cfgs/applicationContext.xml"));
        Viechle viechle=factory.getBean("viechle",Viechle.class);
        viechle.move();
    }
}
```

In the above code:

<property name="engId" value="eng1"/> tag injects dependent class bean id (engine class) to

target class object (vehicle class) as an ordinary 'String' value without checking wheather the bean class is configured or not **with that bean id**. To overcome this problem and to inject bean-id only when bean class is configured with that bean-id then we need to use '**idref**' tag.

```
EX:<bean id="engg" class="com.nt.beans.Engine"/>
```

```
<bean id="Vehicle" class="com.nt.beans.vehicle">
```

```
  <property name="engId">
```

```
    <idref bean="engg1"/>=>
```

```
  </property>
```

```
</bean>
```

Here the bean id "engg1" will be injected to the property engId only as beanId not as String value.

Eg: String name="test" ---->"test" class is maintained are String content.

Class name=Test.class-->"Test" class is maintained as class in the object of java.lang.Class

### **FactoryMethod**

The method that's capable of creating either same class object or different class object is called FactoryMethod.

Two types of factory methods

#### **1) Instance factory method**

Eg1:String S1=new String ("Hello");

String S2=S1.concat("123");//hello 123 (return same class object)

Eg2:StringBuffer sb=new StringBuffer("Hello how are you?");

String S1=sb.substring(0,4);//given hello (return different class object)

#### **2) Static Factory method:**

Eg1:Class e=Class.forName("java.util.Date"); (returns same class object)

Eg2:Console cons=System.console();(returns different class object)

We can make IOC container to create spring bean class Object in 4 ways

- 1) Using 0-param constructor.
- 2) Using parameterized constructor.
- 3) Using instance factory method.
- 4) Using static factory method.

Note points:

>Servlet container ,jsp container can create their servlet class object by using 0-param constructor.

>Spring IOC container can create bean class object by using above 4 ways.

>In spring we can configure abstract class as **spring bean** only when we enable

a)Factory method bean instantiation

b)Lookup method injection

Eg:

- ▲ IOCProj9(Factory methods)
  - ▲ src
    - ▲ com.nt.cfgs
      - applicationContext.xml
    - ▲ com.nt.test
      - ClientApp.java
  - ▶ JRE System Library (JavaSE-1.7)
  - ▲ IOCLib
    - ▶ spring-beans-4.1.6.RELEASE.jar - C:\Users\del
    - ▶ spring-context-4.1.6.RELEASE.jar - C:\Users\del
    - ▶ spring-context-support-4.1.6.RELEASE.jar - C:\Users\del
    - ▶ spring-core-4.1.6.RELEASE.jar - C:\Users\del
    - ▶ spring-expression-4.1.6.RELEASE.jar - C:\Users\del
    - ▶ commons-logging.jar - C:\Users\del\Desktop

**applicationContext.xml:**

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<!-- Using static factory methods -->
<bean id="c1" class="java.lang.Class" factory-method="forName">
  <constructor-arg value="java.util.Date"/>
</bean>
<bean id="c2" class="java.util.Calendar" factory-method="getInstance"/>
<!-- Using instance factory methods-->
<bean id="s1" class="java.lang.String">
  <constructor-arg value="hello"/>
</bean>
<bean id="s2" factory-bean="s1" factory-method="concat">
  <constructor-arg value="how are u?"/>
</bean>
<bean id="sb" class="java.lang.StringBuffer" >
  <constructor-arg value="hello how are u?"/>
</bean>
<bean id="s3" factory-bean="sb" factory-method="subString">
  <constructor-arg value="0"/>
  <constructor-arg value="5"/>
</bean>
</beans>

```

**ClientApp.java**

```

package test;
import java.util.Calendar;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
public class ClientApp {

```

```
public static void main(String[] args) {  
    // Create IOC Container  
    BeanFactory factory=new XmlBeanFactory(new  
    FileSystemResource("src/com/nt/cfgs/applicationContext.xml"));  
    // get Bean  
    Class c1=factory.getBean("c1",Class.class);  
    System.out.println("c1 data"+c1+" c1 class name"+c1.getClass());  
    //get Bean  
    Calendar c2=factory.getBean("c2",Calendar.class);  
    System.out.println("c2 data"+c2+" c2 class name"+c2.getClass());  
    // get Bean  
    String s2=factory.getBean("s2",String.class);  
    System.out.println("s2 data"+s2+" s2 class name"+s2.getClass());  
    //get Bean  
    String s3=factory.getBean("s3",String.class);  
    System.out.println("s3 data"+s3+" s3 class name"+s3.getClass());  
}
```

### Singleton java class

- The java class that allows us to create only one object per jvm is called singletonjava class.
- Take singleton java class in the following situations
  - a) If class is not having any state
  - b) If class is having only sharable read only state (final variable)
  - c) If class is having huge amount of state and performing write operations on that state in synchronized manner (1 thread at a time)
- Instead of creating multiple objects with same data or no data, create one object and access it for multiple times for this take the class as singleton java class.
- If we just create one object for java class even though class allows us to create multiple objects then that class is not called Singleton java class.

Eg: our servlet class is not singleton class because it allows us to create multiple objects but most of the times servlet container creates only 1 object.

### Bean Scopes

We can make IOC container keeping our spring bean class objects in different scopes. They are Spring 1.x

- a) singleton
- b) prototype

Spring2.x

- a) Singleton
- b) prototype
- c) session
- d) request
- e) globalSession

Note:globalSessoins is removed in spring 3.x and 4.x version

Default scope is "singleton".



**singleton:**

Returns same bean class object for all `factory.getBean(-)` method calls with same bean id.

**prototype:**

Returns separate bean class object for every `factory.getBean(-)` method call.

**request:**

Bean class object is specific to each request.

**session:**

Bean class object is visible through out session

**globalSession:**

Specific to portlets environment .Use "scope" attribute of `<bean>` tag to specify bean scope.

**Note:** request, session scopes can be used only in spring MVC applications.

**Eg:** `applicationContext.xml`

```
<bean id="v1" class="com.nt.beans.vehicle" scope="prototype">
```

```
.....
```

```
.....
```

```
</bean>
```

```
Vehicle v1=factory.getBean("v1", vehicle.class);
```

```
Vehicle v2=factory.getBean("v1", vehicle.class);
```

```
System.out.println (v1.hashCode()+" "+v2.hashCode());
```

```
System.out.println (v1==v2);
```

**Cache:**

Cache is a temporary memory that holds data for temporary period (buffer).if Dao class uses cache support it means database data like countries and uses the across multiple same request and reduces no of hits to db s/w.

We generally configure spring bean that is acting as DAO class as singleton scope spring bean because it contains SQL queries as the read only state(final static variables).The recommendation is to take java class as singleton java class.

**Note:**

The Bean scope 'singleton' never makes our bean class as singleton java class. But gives singleton behavior while creating object.

- If Bean scope is singleton then IOC container creates the bean class object and keeps in HashMap element as value by having bean 'id' as key and uses that object across the multiple "`factory.getBean(-)`" method calls.
- If scope is prototype then IOC contains doesn't keep the created bean class object in HashMap so it returns new object for every "`factory.getBean(-)`" call
- If scope is "request" then bean object will be maintained as request attribute.
- If scope is "Session" then the bean object will be maintained as a session attribute.

-----0-----

## Bean Wiring

Configuring beans and their properties for dependency injection in spring bean configuration file is called beab wiring.

- 1) **Explicit wiring:** Here we use <property>, <constructor-arg> tags for dependency injection configuration.
- 2) **Auto Wiring:** Here IOC container automatically detects dependents and injects then to target objects. Here no <property>, <constructor-arg> are required.

### **Autowiring/implicit wiring limitations:**

- Can be used only to inject objects but not the simple values.
- If IOC container have multiple dependencies to inject then **ambiguity problem** may raise.
- Kills the readability of xml file, so the bug fixing becomes very complex.

**Note:** Use autowiring only in POC projects or PILOT project for faster/Rapid application development.

By using "autowire" attribute of <bean> tag are can enable autowiring.

There are 4 modes of performing autowiring

- 1) **byName**
- 2) **byType**
- 3) **constructor**
- 4) **autodetect** (removed from spring 3.x)

### **autowire="byName"**

- Performs setter injection
- For this, all target class property names and dependent class bean ids must match.
- Here, there is no possibility of getting ambiguity problem



### **//TravelAgent.java**

```
package com.nt.beans;

public class TravelAgent {

    private TourPlan tp;

    public TravelAgent(){
        System.out.println("TravelAgent:0-param constructor");
    }

    public TravelAgent(TourPlan tp){
        System.out.println("TravelAgent:1-param constructor");
        this.tp=tp;
    }
}
```

```
    }  
    public void setTp(TourPlan tp){  
        System.out.println("TravelAgent:setTp(-)");  
        this.tp=tp;  
    }  
    // toString()  
    @Override  
    public String toString() {  
        return "TravelAgent [tp=" + tp + "]";  
    }  
}
```

**TourPlan.java (dependent class)**

```
package com.nt.beans;  
import java.util.Arrays;  
public class TourPlan {  
    private String[] places;  
    //setter method  
    public void setPlaces(String[] places) {  
        this.places = places;  
    }  
    // toString()  
    @Override  
    public String toString() {  
        return "TourPlan [places=" + Arrays.toString(places) + "]";  
    }  
}
```

**applicationContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xsi:schemaLocation="http://www.springframework.org/schema/beans  
http://www.springframework.org/schema/beans/spring-beans.xsd">  
    <bean id="tp" class="com.nt.beans.TourPlan">  
        <property name="places"><!-- explicit wiring -->  
            <list>  
                <value>Delhi</value>  
                <value>simla</value>  
            </list>  
        </property>  
    </bean>  
  
    <bean id="tp2" class="com.nt.beans.TourPlan">  
        <property name="places"><!-- explicit wiring -->  
            <list>  
                <value>NewYork</value>  
                <value>Goa</value>  
            </list>  
        </property>  
    </bean>  
</beans>
```

```

</bean>
<bean id="tagent" class="com.nt.beans.TravelAgent" autowire="constructor">
  <constructor-arg ref="tp2"/>
</bean>
</beans>

```

### ClientApp.java

```

package test;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
import com.nt.beans.TravelAgent;
public class ClientApp {
    public static void main(String[] args) {
        //Create IOC Container
        BeanFactory factory = new XmlBeanFactory(new
            FileSystemResource("src/com/nt/cfgs/applicationContext.xml"));
        // Get Bean
        TravelAgent agent = (TravelAgent)factory.getBean("tagent");
        System.out.println(agent);
    } //main
} //class

```

### autowire="byType":

- perform setter injection
- For this target class property type and dependent class must be match or must be compatible with each other.
- There is possibility of getting ambiguity problem.

Eg:

```

<bean id="tplan" class="com.nt.beans.Tourplan">
.....
.....
</bean>
<bean id="target" class="com.nt.beans.TravelAgent" autowiring="byType"/>

```

Note: If we configure "TourPlan" class one more time in spring bean configuration file with different bean id then we get **ambiguity problem**.

### atuowire="constructor"

- uses parameterized constructor to perform constructor injection.
- There is a possibility of getting ambiguity problem. If multiple dependencies are found, but it **doesn't throw exception if target class contains 0-param constructor otherwise exception will be thrown.**

Eg: <bean id="tp1" class="com.nt.beans.TravelAgent" autowire="constructor">

### autowire="autodetect" (Try in spring 2.x)

- If target class contains **0-param constructor** then it performs "byType" mode autowiring.
- If target class contains **parameterized constructor** then it performs "constructor" mode auto wiring.
- Removed from spring 3.0 onwards

```
Eg:<bean id="tp1" class="com.nt.beans.TourPlan">
.....
.....
</bean>
<bean id="target" class="com.nt.beans.TravelAgent" autowire="autodetect">
```

#### autowire="no"

Disable autowiring, even though default autowire is specified in the <bean> tag using default-autowire attribute.

**What happens if we enable both auto wiring and explicit wiring on same bean property?**

Ans: If both wirings are performing the same constructor injection or same setter injection then the explicit wiring value will be injected as final values. If both wirings are performing two different injections then setter injection values will be injected as final values.

### Dependency-Check

While performing constructor injection by using certain parameterized constructor, all the params of that constructor must be configured for wiring, if not configured, exception will be raised. The above restriction is not there for setter injection configuration. To enable such restriction while working with setter injection, go for dependency check by adding "dependency-check" attribute in <bean> tag.

#### dependency-check="simple":

Checks whether all simple properties are configured for setter injection or not, if not configured then exception will be raised.

#### dependency-check="objects":

Checks whether all reference type properties are configured for setter injection or not.

#### dependency-check="all":

Checks whether both simple and object properties are configured for setter injection or not.

#### dependency-check="none":

Disables dependency check. If no value is given default "none" will taken. We must use Spring-beans-2.0 xsd in applicationContext.xml.

```
IOCProj11(Dependency-check)
├── src
│   ├── com.nt.beans
│   │   ├── Engine.java
│   │   └── Viechle.java
│   ├── com.nt.cfgs
│   │   └── applicationContext.xml
│   └── com.nt.test
│       └── ClientApp.java
├── JRE System Library [JavaSE-1.7]
└── IOCLib
    ├── spring-beans-4.1.6.RELEASE.jar - C:\Users\deli\De
    ├── spring-context-4.1.6.RELEASE.jar - C:\Users\
    ├── spring-context-support-4.1.6.RELEASE.jar - C
    ├── spring-core-4.1.6.RELEASE.jar - C:\Users\del
    ├── spring-expression-4.1.6.RELEASE.jar - C:\Use
    └── commons-logging.jar - C:\Users\deli\Deskto
```

Eg:

**Vehicle.java**

```
package com.nt.beans;
public class Viechle {
    private int id;
    private Engine engine;
    public void setId(int id) {
        this.id = id;
    }
    public void setEngine(Engine engine) {
        this.engine = engine;
    }
    @Override
    public String toString() {
        return "Viechle [id=" + id + ", engine=" + engine + "];"
    }
}
```

**Engine.java**

```
package com.nt.beans;
public class Engine {
    private String type;
    public void setType(String type) {
        this.type = type;
    }
    @Override
    public String toString() {
        return "Engine [type=" + type + "];"
    }
}
```

**applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.2.0.xsd">
    <bean id="eng" class="com.nt.beans.Engine">
        <property name="type" value="diesel"/>
    </bean>
    <bean id="viechle" class="com.nt.beans.Viechle" dependency-check="all">
        <!-- <property name="id" value="1010"/> -->
        <property name="engine" ref="eng"/>
    </bean>
</beans>
```

**ClientApp.java:**

```
package test;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
import com.nt.beans.Viechle;
public class ClientApp {
    public static void main(String[] args) {
        //Create IOC Container
        BeanFactory factory=new XmlBeanFactory(new
FileSystemResource("src/com/nt/cfgs/applicationContext.xml"));
        //Get Bean
        Viechle v=factory.getBean("viechle",Viechle.class);
        System.out.println(v);
    } //main
} //class
```

**Note:**“dependency-check” is removed from spring 2.5 because it allow to apply restriction on group of properties not on specified properties. “@required” is given as an alternate.

**Nested BeanFactories**

In real time project contains multiple layers. Instead of configuring all the beans of these multiple layers in single IOC container, it is recommended to take multiple IOC containers on “one-per-layer/tier” basis. To use the beans of one IOC container in another IOC container it is recommended to make them as Nested containers like parent container and child containers. The beans of parent container can be used in child container beans but reverse is not possible. Generally service layer beans want to use the beans of persistence layer.

Note :

- 1) The components that takes inputs, displaying outputs by handling req, res are called **presentation tier components(Controller,View)**.
- 2) The components that deals with business logic, persistence logic are called **business tier components (Service, DAO classes)**.
- 3) It is always recommended to take two different IOC containers as nested/parent, child container to manage persistence tier, business tier components.

**Parent IOC container/ (Parent factory)**

(Manages business tier comps like Service class, DAO class and etc. )

**Child container/Nestead container/child factory**

(Manages presentation tier comps like Controller, View)

4) Since presentation tier components use business tier components, so manage business tier components in parent IOC container and manage presentation tier components in child IOC container.

```

  ▾ IOCProj12(NestedBeanFactories)
    ▾ src
      ▾ com.nt.beans
        ▾ BankLoanApprover.java
          ◉ BankLoanApprover
        ▾ LoanDetails.java
      ▾ com.nt.cfgs
        ▾ approval-beans.xml
        ▾ loan-beans.xml
      ▾ com.nt.test
        ▾ ClientApp.java
    ▾ JRE System Library [JavaSE-1.7]
    ▾ IOCLib
      ▾ spring-beans-4.1.6.RELEASE.jar - C:\Users\de
      ▾ spring-context-4.1.6.RELEASE.jar - C:\Users\c
      ▾ spring-context-support-4.1.6.RELEASE.jar - C
      ▾ spring-core-4.1.6.RELEASE.jar - C:\Users\delP
      ▾ spring-expression-4.1.6.RELEASE.jar - C:\User
      ▾ commons-logging.jar - C:\Users\delH\Desкто
  
```

#### **LoanDetails.java :**

```

package com.nt.beans;
public class LoanDetails {
    private int loanId;
    private String loanType;
    private String customer;
    public LoanDetails(){
        System.out.println("LoanDetails:0-param constructor");
    }
    //setters and getters
    public void setLoanId(int loanId) {
        this.loanId = loanId;
    }
    public void setLoanType(String loanType) {
        this.loanType = loanType;
    }
    public void setCustomer(String customer) {
        this.customer = customer;
    }
    public int getLoanId() {
        return loanId;
    }
    public String getLoanType() {
        return loanType;
    }
    public String getCustomer() {
        return customer;
    }
    //toString()
    @Override
    public String toString() {
        return "LoanDetails [loanId=" + loanId + ", loanType=" + loanType
  
```



```
+ ", customer=" + customer + "];
```

```
}
```

```
}
```

**BankLoanApprover.java**

```
package com.nt.beans;
```

```
public class BankLoanApprover {
```

```
    private LoanDetails details;
```

```
    public BankLoanApprover(){
```

```
        System.out.println("BankLoanApprover:0-param constructor");
```

```
    }
```

```
    //setters
```

```
    public void setDetails(LoanDetails details) {
```

```
        this.details = details;
```

```
    }
```

```
    // toString()
```

```
    @Override
```

```
    public String toString() {
```

```
        return "BankLoanApprover [details=" + details + "];
```

```
    }
```

```
    public String approveLoan(){
```

```
        if(details.getLoanType().equals("two-wheeler")){
```

```
            return "Loan is approved";
```

```
        }
```

```
        else{
```

```
            return "Loan in rejected";
```

```
        }
```

```
    }}
```

**loan-beans.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<bean id="ld" class="com.nt.beans.LoanDetails">
```

```
    <property name="loanId" value="11"/>
```

```
    <property name="loanType" value="two-wheeler"/>
```

```
    <property name="customer" value="raja"/>
```

```
</bean>
```

```
</beans>
```

**approval-beans.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<bean id="ld" class="com.nt.beans.LoanDetails">
```

```
    <property name="loanId" value="11"/>
```

```
    <property name="loanType" value="3-wheeler"/>
```

```
    <property name="customer" value="raja"/>
```

```
</bean>
<bean id="bla" class="com.nt.beans.BankLoanApprover">
  <property name="details" ref="ld"/>
</bean>
</beans>
```

**ClientApp.java:**

```
package test;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
import com.nt.beans.BankLoanApprover;
public class ClientApp {
    public static void main(String[] args) {
        //Create parent IOC container (Parent BeanFactory)
        BeanFactory pfactory=new XmlBeanFactory(new
        FileSystemResource("src/com/nt/cfgs/loan-beans.xml"));
        //create child IOC container (child BeanFactory)
        BeanFactory cfactory=new XmlBeanFactory(new
        FileSystemResource("src/com/nt/cfgs/approval-beans.xml"),pfactory);
        // get Bean
        BankLoanApprover approver=cfactory.getBean("bla",BankLoanApprover.class);
        System.out.println(approver.approveLoan());
    }
}
//main
}
//class
```

→While working with nested BeanFactory/NestedContainers the springBean configuration file associated with child container is called **child spring bean configuration file** and the spring bean configuration file associated with parent container is called **parent spring bean configuration file**.

**Can you explain various attributes of <ref>tag?**

Ans:

**1) <ref local="<bean id>">**

Locates given bean id based spring bean configuration from current/local spring bean configuration (Child configuration File)

eg:<property name="details">  
<ref local="id"/>  
</property>

**2)<ref parent="<bean id>">**

Locates the given bean id based spring bean configuration from parent spring bean configuration file.

Eg:<property name="details">  
<ref parent="id"/>  
</property>

**3)<ref bean="<beanid>">**

Frist tries to locate from current/local bean configuration file, if not available, then it tries to locate from parent bean configuration file

Eg: <property name="details">  
 <ref bean="id"/>  
 </property>  
 <property name="details" ref="id"/> is equal to <ref bean="id"/>

### P-Name space and C-Name space

→ In xml schema "namespace" is a library that contains set of xml tags. Every name space is identified with its name space url. In spring 3.0 p-name space is introduced to perform setter injection configuration without using lengthy <property> tags. It allows to place attributes/properties directly in <bean> tag having the following syntax.

p:<property name>="Value"

p:<property name>-ref="<bean id>"

p-name space url is: <http://www.springframework.org/schema/p>

Similarly c-namespace introduced from spring 3.1.1 version for constructor injection configuration by avoiding the lengthy <constructor-arg> tags. It allows to place attributes directly in <bean> tag having following syntax.

c:<property name>="value"

c:<property name>-ref="<bean id>"

c-name space uri is: <http://www.springframework.org/schema/c> (Collect from spring reference)

### Note:

p-namespace and c-namespace allows to write shorter configurations written in <bean> tag itself to perform setter, constructor injection configurations.



### Emp.java

```
package com.nt.beans;

public class Emp {
    private int eno;
    private String ename;
    private Dept dept;
    public int getEno() {
        return eno;
    }
    public void setEno(int eno) {
        this.eno = eno;
    }
    public String getEname() {
```

```
        return ename;
    }
    public void setName(String ename) {
        this.ename = ename;
    }
    public Dept getDept() {
        return dept;
    }
    public void setDept(Department dept) {
        this.dept = dept;
    }
    @Override
    public String toString() {
        return "Emp [eno=" + eno + ", ename=" + ename + ", dept=" + dept + "]";
    }
}
```

#### Dept.java

```
package com.nt.beans;
import java.util.Date;
public class Dept {
    private int deptno;
    private String name;
    private Date dop;
    public Dept(int deptno, String name, Date dop) {
        System.out.println("Dept:3-param constructor");
        this.deptno = deptno;
        this.name = name;
        this.dop = dop;
    }
    @Override
    public String toString() {
        return "Dept [deptno=" + deptno + ", name=" + name + ", dop=" + dop
            + "]";
    }
}
```

#### applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:c="http://www.springframework.org/schema/c"
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="dt" class="java.util.Date" p:year="115"
        p:month="10"
        p:date="20"/>
    <bean id="dept" class="com.nt.beans.Dept"
```

```
        c:deptno="8001"
        c:name="Accounts"
        c:dop-ref="dt"/>
<bean id="emp" class="com.nt.beans.Emp"
    p:eno="101"
    p:ename="raja"
    p:dept-ref="dept"/>

</beans>
```

### ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.Emp;
public class ClientApp {
    public static void main(String[] args) {
        // create IOC container
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //BeanFactory factory=new XmlBeanFactory(new
        FileSystemResource("src/com/nt/cfgs/applicationContext.xml"));
        //Get Bean
        Emp emp=ctx.getBean("emp",Emp.class);
        System.out.println(emp);
    } //main
} //class
```

### Working with ApplicationContext container (IOC container)

- BeanFactory Container means it is the IOC container that is created by creting BeanFactory object
- ApplicationContext Container means it is the IOC container that is created by creating ApplicationContext object.
- ApplicationContext object is the object of java class that implements org.springframework.context.ApplicationContext(I)(It is the sub interface of BeanFactory(I))
- There are three popular implementation classes for ApplicationContext(I) using which we can create ApplicationContext container (IOC container)

#### 1)FileSystemXmlApplicationContext:

Creates ApplicationContext container by locating spring bean configuration file from the specified path of FileSystem.

Eg: 

```
ApplicationContext ctx =new
FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
```

#### 2)ClassPathXmlApplication Context:

Create ApplicationContext container by locating spring bean configuration file from the directories and jar files added to classpath.

Eg: `ApplicationContext ctx =new ClassPathXmlApplicationContext ("applicationContext.xml");`

### 3) XmlWebApplicationContext:

Creates ApplicationContext container in web applications.

ApplicationContext container can perform all the operations of BeanFactory container and also perform these additional operations:

- a) **Pre-instantiation** of singleton spring Beans.
- b) Ability to work with **properties file**.
- c) Support for **l18n (internationalization)**.
- d) Support for event handling /processing.
- e) Ability to start,stop and close container.
- f) Automatic registration of **BeanPostProcessor** classes.
- g) Automatic registration of **BeanFactoryProcessor** classes and etc.....

While working with BeanFactory container the container instantiates spring bean only when `factory.getBean(-)` method is called (for any scope bean). While working with ApplicationContext container the singleton scope beans will be instantiated the moment the container is started. This is called **Pre-instantiation of singleton Beans (Also called Eager Instantiation)**.

If prototype scope bean is dependent to singleton scope, then prototype scope bean will also be pre-instantiated to support dependency injection on singleton scope bean. But it doesn't mean prototype bean scope has changed scope to "Singleton". **Bean pre-instantiation is very much similar to enabling <load-on-startup> on servlet.**

**Q)We have spring bean configuration file with 10 beans. How can we enable pre-instantiation only on 6 beans?**

Ans: Take 6 beans as singleton scope beans and other 4 beans as "prototype" scope beans and don't make 'prototype' scope beans as dependent to singleton scope beans.

**Q)Why there is no pre-instantiation on prototype scope beans while working with ApplicationContext container?**

Ans: IOC container creates bean class object for every `factory.getBean (-,-)` method call. So even though object is created through pre-instantiation process that will be wasted because it has to create new object when `factory.getBean (-,-)` method is called. So there is no pre-instantiation on prototype scope beans.

→The java class that contains persistence logic is called **DAO class**. The java class that contains business logic is called service/Business class. Generally service class needs DAO class object.

→JDBC DataSource object represents jdbc connections pool and we can use this jdbc DataSource object to get connection object from jdbc connection pool.

→Spring api gives **org.springframework.jdbc.datasource.DriverManagerDataSource** class to give dataSource object pointing to the IOC container managed jdbc connection pool. It creates the connection object in connection pool based on the bean property values configuration. **They are driverClassName, url, username, password.**

## Working with properties file

The text file that maintains the entries as key value pairs is called properties file. We generally use properties file to maintain jdbc properties like driverClassName, url, username and password and to make them as flexible to modify without touching the source code.

Spring application can achieve this flexibility through xml file (spring bean configuration file). If spring application is added as new app to the existing project and if those existing applications of project are getting jdbc properties from properties file then we should make sure the new spring application also gets jdbc properties from **properties file**.

In ApplicationContext container environment we need to configure "org.springframework.beans.factory.config.PropertyPlaceholderConfigurer" class as spring bean to make container to locate given properties file and to recognize place holders in spring bean configuration file.

**Place holders---->\${-----}**

→Each placeholder represents one key of the property file to get respective value **from properties file**.

### DBDetails.properties(com/nt/commons)

```
Jdbc.driver = oracle.jdbc.driver.OracleDriver
Jdbc.url =jdbc:oracle:thin:@localhost:1521:xe
db.user = system
db.pwd = manager
```

### applicationContext.xml

```
<bean id="ppc"
class="org.springframework.beans.factory.config.PropertyPlaceholderConfigurer">
    <property name="location" value="src/com/nt/commons/DBDetails.properties"/>
</bean>
<bean id="drds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
    <property name="driverClassName" value="${jdbc.driver}"/>
    <property name="url" value="${jdbc.url}"/>
    <property name="username" value="${jdbc.user}"/>
    <property name="password" value="${jdbc.pwd}"/>
</bean>
```

- ▲  IOCProj14(DataSource injection + DAO)
  - ▲  src
    - ▲  com.nt.bo
      - ▶  CustomerBO.java
    - ▲  com.nt.cfgs
      - ▶  applicationContext.xml
    - ▲  com.nt.commans
      - ▶  DBDetails.properties
    - ▲  com.nt.dao
      - ▶  LoanDAO.java
    - ▲  com.nt.jdbc
      - ▶  RecordsCountJdbcApp.java
    - ▲  com.nt.service
      - ▶  LoanService.java
    - ▲  com.nt.test
      - ▶  ClientApp.java
  - ▶  JRE System Library [JavaSE-1.7]
  - ▲  IOCLib
    - ▶  spring-beans-4.1.6.RELEASE.jar - C:\Users\
    - ▶  spring-context-4.1.6.RELEASE.jar - C:\User
    - ▶  spring-context-support-4.1.6.RELEASE.jar
    - ▶  spring-core-4.1.6.RELEASE.jar - C:\Users\c
    - ▶  spring-expression-4.1.6.RELEASE.jar - C\A
    - ▶  commons-logging.jar - C:\Users\dell\Des
  - ▲  Referenced Libraries
    - ▶  spring-jdbc-4.1.6.RELEASE.jar - C:\Users\c
    - ▶  spring-tx-4.1.6.RELEASE.jar - C:\Users\dell

### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:c="http://www.springframework.org/schema/c"
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="ppc"
    class="org.springframework.beans.factory.config.PropertyPlaceholderConfigurer">
    <property name="location" value="com/nt/commons/DBDetails.properties"/>
  </bean>
  <bean id="drds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
    <property name="driverClassName" value="${jdbc.driver}"/>
    <property name="url" value="${jdbc.url}"/>
    <property name="username" value="${db.user}"/>
    <property name="password" value="${db.pwd}"/>
  </bean>
  <bean id="dao" class="com.nt.dao.LoanDAO">
    <property name="ds" ref="drds"/>
  </bean>
  <bean id="service" class="com.nt.service.LoanService">
    <property name="dao" ref="dao"/>
  </bean>
</beans>
```



Customer.java

```
package com.nt.bo;
public class CustomerBO {
    private int cno;
    private String cname;
    private float pamt,intramt;

    public int getCno() {
        return cno;
    }
    public void setCno(int cno) {
        this.cno = cno;
    }
    public String getCname() {
        return cname;
    }
    public void setCname(String cname) {
        this.cname = cname;
    }
    public float getPamt() {
        return pamt;
    }
    public void setPamt(float pamt) {
        this.pamt = pamt;
    }
    public float getIntramt() {
        return intramt;
    }
    public void setIntramt(float intramt) {
        this.intramt = intramt;
    }
}
```

DBDetails.properties

```
#jdbc properties
db.user=scott
db.pwd=tiger
jdbc.driver=oracle.jdbc.driver.OracleDriver
jdbc.url =jdbc:oracle:thin:@localhost:1521:xe
```

LoanDAO.java

```
package com.nt.dao;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import javax.sql.DataSource;
import com.nt.bo.CustomerBO;
public class LoanDAO {
```

```
private static final String INSERT_QRY="INSERT INTO CUSTOMER_LOANINFO
VALUES(?,?,?,?)";
private DataSource ds;
public void setDs(DataSource ds) {
    this.ds = ds;
}
public int insert(CustomerBO cbo){
    Connection con=null;
    PreparedStatement ps=null;
    int result=0;
    try{
        // get Connection obj from jdbc con pool
        con=ds.getConnection();
        //create PreparedStatement obj
        ps=con.prepareStatement(INSERT_QRY);
        //set values to Query param
        ps.setInt(1,cbo.getCno());
        ps.setString(2,cbo.getCname());
        ps.setFloat(3,cbo.getPamt());
        ps.setFloat(4,cbo.getIntramt());
        //execute the Query
        result=ps.executeUpdate();
    }
    catch(SQLException se){
        se.printStackTrace();
    }
    catch(Exception e){
        e.printStackTrace();
    }
    finally{
        //jdbc objs
        try{
            if(ps!=null)
                ps.close();
        }
        catch(SQLException se){
            se.printStackTrace();
        }
        try{
            if(con!=null)
                con.close();
        }
        catch(SQLException se){
            se.printStackTrace();
        }
    }
    }finally{
        return result;
    }
}
}
}
```

RecordsCountJdbcApp.java

```
package com.nt.jdbc;
import java.io.FileInputStream;
import java.io.InputStream;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
import java.util.Properties;
public class RecordsCountJdbcApp {
    public static void main(String[] args) throws Exception{
        // locate properties file
        InputStream is=new FileInputStream("src/com/nt/commons/DBDetails.properties");
        // Load properties info to java.util.Properties class obj
        Properties props=new Properties();
        props.load(is);
        Class.forName(props.getProperty("jdbc.driver"));
        Connection con=DriverManager.getConnection(props.getProperty("jdbc.url"),
            props.getProperty("db.user"),
            props.getProperty("db.pwd"));
        Statement st=con.createStatement();
        ResultSet rs=st.executeQuery("select count(*) from student");
        int cnt=0;
        if(rs.next()){
            cnt=rs.getInt(1);
        }
        System.out.println("Records count"+cnt);

        //close jdbc objs
        rs.close();
        st.close();
        con.close();
    } //main
} //class
```

LoanService.java

```
package com.nt.service;
import com.nt.bo.CustomerBO;
import com.nt.dao.LoanDAO;
public class LoanService {
    private LoanDAO dao;
    public void setDao(LoanDAO dao) {
        this.dao = dao;
    }
    public float calcIntrAmt(int cno,String cname,float pamt, float rate,float time){
        //write b.logic
        float intramt=pamt*rate*time/100.0f;
        //use DAO class persistence logic
    }
}
```

```
CustomerBO cbo=new CustomerBO();
cbo.setPamt(pamt);
cbo.setCno(cno);
cbo.setCname(cname);
cbo.setIntramt(intramt);
int result=dao.insert(cbo);
return intramt;
```

```
//method
} //class
```

#### ClientApp.java

```
package test;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.config.BeanFactoryPostProcessor;
import org.springframework.beans.factory.config.ConfigurableListableBeanFactory;
import org.springframework.beans.factory.config.PropertyPlaceholderConfigurer;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
import com.nt.service.LoanService;
public class ClientApp {
    public static void main(String[] args) {
        // Create IOC container
        //ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        BeanFactory factory=new XmlBeanFactory(new
        FileSystemResource("src/com/nt/cfgs/applicationContext.xml"));
        //get PropertyPlaceholderConfigurer obj (pphc)
        BeanFactoryPostProcessor
        bfpp=factory.getBean("ppc",PropertyPlaceholderConfigurer.class);
        bfpp.postProcessBeanFactory((ConfigurableListableBeanFactory) factory);
        //get Bean
        LoanService service=ctx.getBean("service",LoanService.class);
        // call b.method
        float intramt=service.calcIntrAmt(1009,"Raja",9000,2,20);
        System.out.println("Intr Amount"+intramt);
    } //class
```

→The java class with standards (minimum getter & setter methods) is called javaBean.

→In realtime the data between 2 components/resources/layers will not be sent as individual values. they will be placed in java bean class object and will be sent to from source layer to destination layer.

→The javaBean whose object holds the end user supplied input values (form data) is called VO class (**Value Object**).

→The javaBean whose object holds the data required for persistence logic or the data generated by persistence logic is called **BO class(Business Object)**.

→The javaBean whose object holds data to transferring from one layer to another layer is called **DTO class**. This class generally implements "Serializable" interface.

**VO class** ---> value object

**BO class**---->Data Transfer Object

**DTO class**--->DataTransfer object

→ Generally the DAO Class contains 2 parts

a) Query Part

b) Code Part

Note: By using "locations" property we can configure multiple properties files in spring applicationcontext containers environment. In this process if all multiple properties files contains same keys with different values then the lastly configured file values will be taken by **overriding other file values**.

eg:

```
<bean id="ppc"
class="org.springframework.beans.factory.config.PropertyPlaceholderConfigurer">
  <property name="locations">
    <list>
      <value>src/com/nt/commons/DBDetails1.properties</value>
      <value>src/com/nt/commons/DBDetails2.properties</value>
    </list>
  </property>
</bean>
```

### I18n (Internationalization)

- **Locale** means language+ country

eg:en-US

en-Br (English as it speaks in Britan)

fr-FR

hi-IN

- Making our application working for different "locales" is called enabling I18n (internalization). It deals with displaying presentation labels, displaying numbers, currency symbols, displaying Date values and etc..
- In core java, JEE application we use ResourceBundle, Locale classes of java.util package to work with I18n.
- To apply I18n, we need to take multiple properties file having presentation labels on one per "locale" basis. All these files must have same keys and different values, we can get these values of different languages by using **Google translator**.

Eg: App.properties (Base file)

App\_de\_DE.properties (for German)

and etc....

- If no matching file is found then base properties file will be used automatically.

→ We can use google translator labels in other language based on the given English word. We can also place Unicode numbers in the properties files to render non English character labels. For that we need to use third party tools like Unicode editor and jdk's "native2ascii" tool.

**Procedure:**

- 1) Download Unicode editor from [www.higopi.com](http://www.higopi.com) and type hindi words (use index.html)
- 2) Copy hindi words from Unicode editors and place them in notepad.  
(save the file by taking "unicode" as encoding type)

**Input.txt:**

Write hindi words.

- 3) Use "native2Ascii" tool to get Unicode numbers.

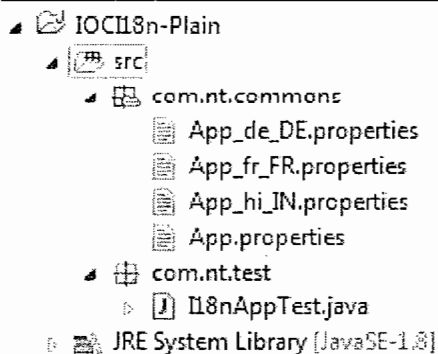
cmd>native2Ascii encoding=Unicode inputs.txt output.txt (contains Unicode numbers)

**Limitations of JEE/JSE environment while working with I18n:**

In a web application if multiple Servlet/jsp programs wants to display "labels" collected from properties file then we need to create "ResourceBundle" class object in multiple servlet/jsp programs or we need to place "ResourceBundle" object as request/session attribute value.

**Note:** Some plan has to be decided to use "ResourceBundle" object.

If we want to display values of same key belonging to multiple locale properties files then we need to create multiple "ResourceBundle" class objects. We can overcome these problems in spring environment by using "ApplicationContext" container with the support of "ResourceBundleMessageSource" class.

**Directory structure to create plain I18n Application:****App.properties****#Base properties file (english)**

label1=save  
label2=delete  
label3=modify  
label4=view

**App de DE.properties****#German properties file (english)**

label1=sparen  
label2=löschen  
label3=ändern  
label4=Aussicht

**App fr FR.properties****#French properties file (english)**

label1=enregistrer  
label2=effacer  
label3=modifier

label4=vue

### App hi IN.properties

label1=\u0938\u0941\u0930\u094d\u0938\u094d\u0915\u094d\u0930\u093f\u0924\u094d

label2=\u092e\u093f\u0924\u094d\u0924\u093e\u0928\u093e

label3=\u092a\u0930\u093f\u0935\u0930\u094d\u0935\u0928\u094d

label4=\u0926\u0946\u0916\u094a

### I18nTest.java

```
package com.nt.test;

import java.awt.FlowLayout;
import java.util.Locale;
import java.util.ResourceBundle;

import javax.swing.JButton;
import javax.swing.JFrame;

public class I18nTest {
    public static void main(String[] args) {
        Locale locale=null;
        ResourceBundle bundle=null;
        JFrame frame=null;
        JButton btn1=null,btn2=null,btn3=null,btn4=null;
        //create Local object
        locale=new Locale(args[0],args[1] );
        //create ResourceBundle object
        bundle=
ResourceBundle.getBundle("com/nt/commons/App",locale);
        //create JFrame
        frame=new JFrame("I18n App");
        frame.setLayout(new FlowLayout());
        //create Buttons
        btn1=new JButton(bundle.getString("label1"));
        btn2=new JButton(bundle.getString("label2"));
        btn3=new JButton(bundle.getString("label3"));
        btn4=new JButton(bundle.getString("label4"));

        frame.add(btn1);frame.add(btn2);frame.add(btn3);frame.add(btn4);
        frame.setVisible(true);
        frame.pack();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    } //main
} //class
```

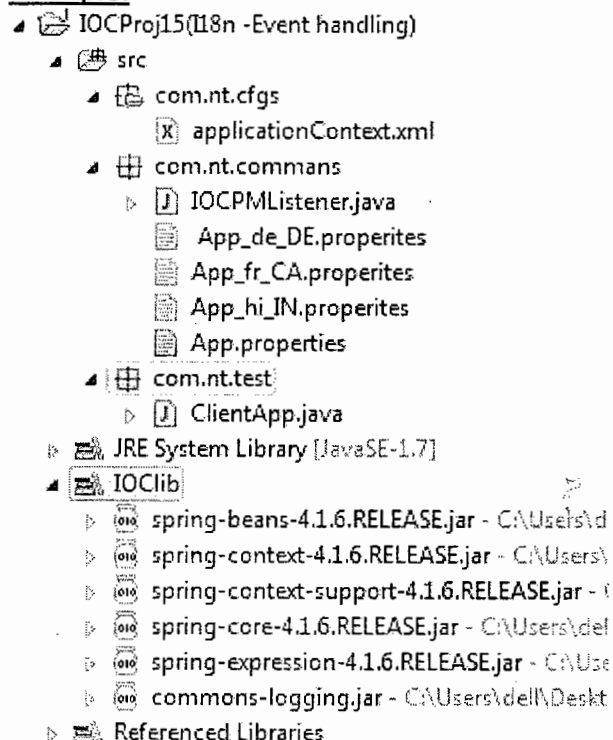
## I18n in spring

- Only "ApplicationContext" container supports I18n.
- We can use "ResourceBundleMessageSource" class as spring Bean by specifying base property file name as the value of "basename" property.
- Since we configure the above class as "singleton scope" SpringBean , the container will creates one object and uses it for multiple times. But the above bean must be configure with fixed bean id "messageSource".
- We can call "ctx.getMessage(-)" to read message from properties file by supplying appropriate "Locale" object.
- This "ctx.getMessage(-)" method makes container to locate properties file based on Locale object data with support of bean whose id is "messageSource".

### Advantages:

- Just by creating one object for "ResourceBundleMessageSource" we can use properties file in multiple locations.
- Allows to read same message in different languages with one object of above class.

### Example:



### App.properties

#(BaseFile-->English)

str1=delete

str2=save

str3=stop

str4=cancel



**App hi IN.properties**

```
str1=\u0928\u0916\u093e\u094a
str2=\u0938\u0941\u0930\u094d\u0915\u093f\u0925\u094d
str3=\u0930\u094a\u0915\u094a
str4=\u0930\u0926\u094d\u0926\u094d
```

**App de DE.properties**

#for german language

```
str1=LOSCHEN
str2=DENN
str3=ZUG
str4=ABSAGEN
```

**App fr CA.properties**

#for French language

```
str1=EFFACER {0} {1}
str2=SAUF
str3=ARRET
str4=ANNULER
```

**ClientApp.java**

```
package test;
import java.awt.Container;
import java.awt.FlowLayout;
import java.util.Locale;
import javax.swing.JButton;
import javax.swing.JFrame;
import org.springframework.context.support.FileSystemXmlApplicationContext;

public class ClientApp{
    public static void main(String[] args){
        //create IOC container
        FileSystemXmlApplicationContext ctx= new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get Messages
        Locale l = new Locale("fr", "CA");
        String s1=ctx.getMessage("str1",null,"msg1",l);
        String s2=ctx.getMessage("str2",null,"msg2",l);
        String s3=ctx.getMessage("str3",null,"msg3",l);
        String s4=ctx.getMessage("str4",null,"msg4",l);

        JFrame jf = new JFrame();
        //create btns by getting labels from activated properties file..
        JButton b1 = new JButton(s1);
        JButton b2 = new JButton(s2);
        JButton b3 = new JButton(s3);
        JButton b4 = new JButton(s4);

        setLayout(new FlowLayout());
        add(b1);
        add(b2);
```

```

add(b3);
add(b4);

jf.pack();
f.setVisible(true);
ctx.close();
}
}

```

#### applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:c="http://www.springframework.org/schema/c"
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

<bean id="messageSource"
class="org.springframework.context.support.ResourceBundleMessageSource">
    <property name="baseName" value="src/com/nt/commons/App"/>
</bean>
</beans>

```

**Note1:** ctx.getBean(-)/factory.getBean(-) gives Bean class object based on given bean id.

**Note2:** ctx.getMessage(-,-) gives "Locale" specific message collected from properties based on given "Key".

### Event Handling

→Event is an action that is raised on the component/object. Event handling means executing logic when event is raised. For this we need the support of "EventListeners". There are four actors in Event processing/handling.

- a) Source object (eg: Button)
- b) Event Class (eg: ActionEvent)
- c) Event Listener (eg: ActionListener)
- d) Event handling method (eg: actionPerformed(-))

We can perform "Event handling" on "ApplicationContext" container to keep track of when container is **started and stopped**. This allows programmer to evaluate the performance of spring code/logic.

#### In spring environment for event handling

Source object: ApplicationContext container

Event class: ApplicationEvent

Event Listener: ApplicationListener

Event handling methods: onApplicationEvent(-) //executes when container started or stopped.

#### Example-app:

- 1) Keep any application ready having the support of "ApplicationContext" container.

- 2) Develop the java class for event handling
  - 3) Configure the above "Listener" class in "applicationContext.xml"
- ```
<bean class="com.nt.event.IOCAMOListner"/>
```
- 4) Create "ApplicationContext" container and stop that in the client application.
- While executing this application also add spring-aop-<ver>.jar file.

**IOCPOMListner.java:**

```
package com.nt.event;
public class IOCPOMListner implements ApplicationListner{
    private Long end;
    private Long start;
    public void onApplicaitonEvent (ApplicationEvent ae){
        If (ae.toString().indexOf("Refreshed")!=-1){
            start=System.currentTimeMillis();
        }
        else{
            end=System.currentTimeMillis();
            System.out.println("Container acting time" +(end-start) +"ms");
        }
    }
    }//method
    }//class
```

**ClientApp.java:**

```
FileSystemXmlApplicationContext ctx=new FileSystemXmlApplicationContext
("src/com/nt/cfgs/applicationContext.xml");
.....
.....
ctx.close(); //close the container
```

**Event handling on IOC container is useful to evaluate and optimize the performance of IOC container.**

**Where to use BeanFactory? Where to use ApplicationContext?**

Ans: If we are developing small scale applications likes mobile applications, EmbeddedSystem Applications having the support of spring then use BeanFactory (Here few extra KBs are also matters towards application size)

If we are developing enterprise applications (like web applications, Distributed applications) then use ApplicationContext (always prefer working with ApplicationContext).

Similarities and difference between bean BeanFactory and ApplicationContext container?

| Operation          | BeanFactory<br>container | ApplicationContext<br>container |
|--------------------|--------------------------|---------------------------------|
| Bean instantiation | Yes                      | Yes                             |
| Bean wiring        | Yes                      | Yes                             |
| Auto Wiring        | Yes                      | Yes                             |

|                                                       |                        |     |
|-------------------------------------------------------|------------------------|-----|
| Properties File Support                               | Yes<br>(not directly). | Yes |
| l18n                                                  | no                     | Yes |
| Eventhandling<br>-Eventpublication.                   | no                     | Yes |
| Automatic Registration<br>of BeanPostProcessors       | no                     | Yes |
| Automatic Registration<br>of BeanFactoryPostProcessor | no                     | Yes |
| Stop/closing Container                                | no                     | Yes |

Note:

*Constructor injection, dependency-check or @required keeps the restriction of configuring properties for dependency injection but they don't verify correctness of that data. For this we need to take the support of spring bean life cycle.*

### Bean Life Cycle

→ Keeping tracking all the activities from birth (Object creation) to death (object destruction) is called Bean life cycle.

→ It talks about spring bean Class object creation to destruction. In servlet lifecycle we use following lifecycle methods

a) init(-)

b) service (-,-)

c) destroy ()

→ In ordinary class we use constructor, finalize() methods to place initialization and uninitialization logics. SpringBean allows two lifecycle methods.

#### a) init - method:

→ Executes after all dependent values are injected. So it can be used to verify whether injections are done properly or not (with appropriate values or not).

#### b) destroy- method:

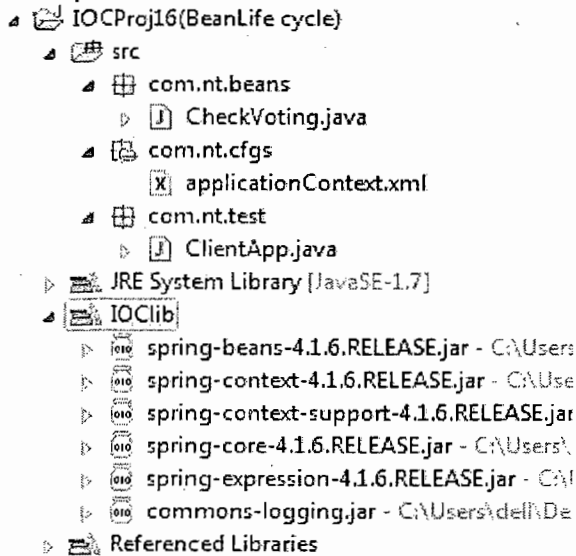
→ Executes when IOC container is about to destroy our bean class object. Useful to place uninitialization logics. (Where these methods can be custom names)

In spring we can perform spring lifecycle operations in three ways:

- 1) Declarative Approach
- 2) Programming Approach
- 3) Annotation Approach

#### Declarative Approach (xml):

Allows to configure two user-defined methods as bean lifecycle methods. These methods are called **custom init method**, **custom destroy method**. These methods return type should be "void" and must not contain "parameters". We must configure the above methods as lifecycle methods using "init-method", "destroy-method" attributes of <bean> tag.

**Example:****applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:c="http://www.springframework.org/schema/c"
xmlns:p="http://www.springframework.org/schema/p"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="cv" class="com.nt.beans.CheckVoting" init-method="check"
destroy-method="myDestroy">

<property name="name" value="ravi"/>
<property name="addrs" value="hyd"/>
<property name="age" value="30"/>
</bean>
</beans>
```

**CheckVoting.java:**

```
package com.nt.beans;
import org.springframework.beans.factory.DisposableBean;
import org.springframework.beans.factory.InitializingBean;
public class CheckVoting implements InitializingBean, DisposableBean {
    private String name, addrs;
    private int age;
    public void setName(String name) {
        System.out.println("setName(-)");
        this.name = name;
    }
    public void setAddrs(String addrs) {
        System.out.println("setAddrs(-)");
        this.addrs = addrs;
    }
    public void setAge(int age) {
        System.out.println("setAge(-)");
    }
}
```

```
        this.age = age;
    }
    //custom init-method
    public void check(){
        System.out.println("Bean:custom-init()");
        if(name==null || age<=0)
            throw new IllegalArgumentException("name, age must set with values");
    }
    public String checkVotingElgibility(){
        if(age>=18){
            return name+" u r elgible to vote ";
        }
        else{
            return name+" u r not elgible vote";
        }
    }
}

//custom destroy method
public void myDestroy(){
    System.out.println("Bean:custom Destroy()");
    name=null;
    age=0;
    addrs=null;
}
}
```

**ClientApp.java:**

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.CheckVoting;
public class ClientApp {
    public static void main(String[] args) {
        // Create IOC container
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get Bean
        CheckVoting cv=(CheckVoting)ctx.getBean("cv");
        System.out.println(cv.checkVotingElgibility());
        ((FileSystemXmlApplicationContext)ctx).close();

        /* BeanFactory factory=new XmlBeanFactory(new
        FileSystemResource("src/com/nt/cfgs/applicationContext.xml"));
        CheckVoting cv=(CheckVoting)factory.getBean("cv");
        System.out.println(cv.checkVotingElgibility());
        ((XmlBeanFactory)factory).destroySingletons(); */
    }
}
```

In custom "init()" method we generally don't write initialization logic because it won't execute after Bean instantiation it executes after completing all injections. While working with "ApplicationContext" container when we close the container by calling "ctx.close()". The container destroys bean class objects when this task is about to happen custom "destroy method" will execute. while working with "BeanFactory" container we can use "factory.destroySingletons()" method to destroy all singleton scope bean class objects.

#### Limitations of Declarative approach of bean Lifecycle:

- 1) If you forgot to configure lifecycle methods in xml file, they will not be executed.
- 2) Identifying lifecycle methods is very complex while dealing with predefined, third party supplied classes as spring beans.

#### 2) Bean LifeCycle programmatic approach:

- Here our spring bean class must implement "InitializationBean()" and "DisposableBean()" given by spring api (Bean class becomes api dependent ---->(invasive))
- InitializationBean() gives **afterPropertiesSet()**, DisposableBean() gives **destroy()** method.
- We use "afterPropertiesSet()" method as alternative to custom init method and **destroy()** as alternate for custom destroy method.

#### Sample Code:

```
public class checkVoting implements InitializingBean, DisposableBean
{
    private int age;
    private String name, address;
    //setters
    .....
    .....
    public void afterPropertiesSet()
    {
        If (age<=0 || name==null)
        {
            Throw new IllegalArgumentException("name age must be set");
        }
    }
    public void destroy () {
        age=0;
        name=null;
        address=null;
    }
    .....
    .....
}
```

Note: There is no need of configuring any life cycle methods in spring bean configuration file.

If we place Bean LifeCycle configuration in both programmatic approach and declarative approach then what happen?

**Ans: Programmatic approach configurations execute before declarative approach configurations.**

Note1: It is recommended to use one approach of bean lifecycle at a time.

Note2: Programmatic approach makes our springBean class as spring api dependent class, most of spring api supplied class are given based on programmatic approach of bean lifecycle.

Note3: Custom init() and InitializationBean's afterPropertiesSet() is useful to check wheather important bean properties are injected with values or not. Custom destroy method, DisposableBean's destroy() is useful to nullify bean properties or to releases non-java resources that are associated with Bean class.

### 3) **Bean lifecycle Annotation Approach:**

→will be discussed in annotations section of spring core module

### **Conclusion on Bean LifeCycle:**

- 1) While working with user-defined classes as spring beans prefer declarative or annotation approach.
- 2) While working with third party API supplied classes as spring beans use Declarative approach.
- 3) While working with spring api supplied classes as spring beans 1<sup>st</sup> check for programmatic approach implementation is done or not. if not found the go for declarative approach.

### **Aware Injection/Interface Injection/Contextual Dependency Lookup**

→If underlying container assigns or injects objects/values to target class only when target class implements/extends certain interface/class then it is called Contextual dependency lookup. Servlet container assigns ServletConfig object to our servlet class object only when our servlet class implements javax.servlet.Servlet() directly or indirectly (Contextual dependency lookup).

If one class needs another class then we go for either dependency lookup or dependency injection. If target class wants to use dependent objects in all the methods then we go for **dependency injection**. If target class wants to use dependent object in specific method then go for **dependency lookup**.

*If we perform dependency-lookup in regular fashion we need to create one more IOC container in target class to get dependent class object, so instead of creating separate IOC container in target class method we ask underlying IOC container ("Created in Client App") to inject "BeanFactory or ApplicationContext" object by implementing "BeanFactoryAware() or ApplicationContextAware()" on target class and we can use that injected objects to get dependent class objects.*

Here underlying container is injecting object/value only when target class implements "xxxAware" interface(s). So it is called **Aware Injection or interface injection or Contextual Dependency lookup (CDL)**.

Note: Aware injection also supports to perform dependency lookup to get some dependency class



objects in specific methods of target class, but it doesn't use separate IOC container in these methods. It uses the underlying IOC container where target bean is running.

To perform setter and constructor injections the IOC container doesn't check for any contract/condition supported by the beans class, but to perform aware injection the IOC container checks for contract/condition supported by the bean class that is implementation of **xxxAware** interfaces.

#### xxxAware interfaces are:

*BeanNameAware----->To inject currentbeanid*  
*BeanFactoryAware----->To inject BeanFactory object*  
*ApplicationContextAware----->To inject ApplicationContext object.*

- IOProj17(Aware Injection or InterfaceInjection)
  - src
    - com.nt.beans
      - Engine.java
      - Viechle.java
    - com.nt.cfgs
      - applicationContext.xml
    - com.nt.test
      - ClientApp.java
  - JRE System Library [JavaSE-1.7]
    - IOClib
      - spring-beans-4.1.6.RELEASE.jar - C:\Users\deli\...
      - spring-context-4.1.6.RELEASE.jar - C:\Users\deli\...
      - spring-context-support-4.1.6.RELEASE.jar - C:\Users\deli\...
      - spring-core-4.1.6.RELEASE.jar - C:\Users\deli\...
      - spring-expression-4.1.6.RELEASE.jar - C:\Users\deli\...
      - commons-logging.jar - C:\Users\deli\Desktop\...

#### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="engg1" class="com.nt.beans.Engine"/>
<bean id="viechle" class="com.nt.beans.Viechle" >
  <property name="beanId">
    <idref bean="engg1"/>
  </property>
</bean>
</beans>
```

#### Viechle.java

```
package com.nt.beans;
import org.springframework.beans.BeansException;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.BeanFactoryAware;
import org.springframework.beans.factory.BeanNameAware;
import org.springframework.context.ApplicationContext;
import org.springframework.context.ApplicationContextAware;
```

```
public class Viechle implements ApplicationContextAware{
    private String beanId;
    private ApplicationContext ctx;
    public void setApplicationContext(ApplicationContext ctx)
        throws BeansException {
        System.out.println("Viechle:setApplicationContext(-)");
        this.ctx =ctx ;
    }
    public Viechle(){
        System.out.println("Viechle:0-param constructor");
    }
    public void setEnggId(String enggId){
        System.out.println("Viechle:setEnggId");
        this.enggId=enggId;
    }
    public void move(){
        Engine engine=ctx.getBean(enggId,Engine.class);
        engine.start();
        System.out.println("Viechle moved for journey");
    }
}
```

#### Engine.java

```
package com.nt.beans;
public class Engine{
    public Engine(){
        System.out.println("Engine:0-param constructor");
    }
    public void start(){
        System.out.println("Engg started");
    }
}
```

#### ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.Viechle;
public class ClientApp {
    public static void main(String[] args) {
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        Viechle viechle=ctx.getBean("viechle",Viechle.class);
        viechle.move();
    }
}
```

Note:

- 1)The IOC container that is created by creating "BeanFactory" object internally maintains bean ids and BeanFactory objects.
- 2)The IOC container that created by creating ApplicationContext object internally maintains bean ids, BeanFactory object, ApplicationContext object and etc.. All these objects can be injected to any class through **Aware injection**. This injection can't be used to inject regular values to spring bean. ->It is useful to inject underlying IOC container maintained objects to spring bean.

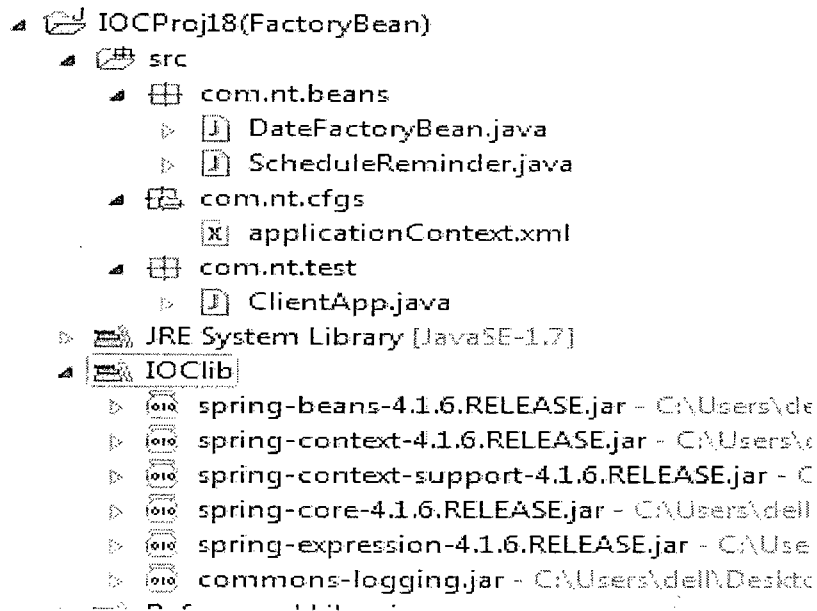
## FactoryBean

->The class that is capable of creating and returning objects is called "Factory" class.

->In spring FactoryBean is "**selfless**" bean i.e it doesn't gives its own class object, it always gives one **Resultant object**. To develop class as "FactoryBean" class, the class must implement the spring API's "FactoryBean()" and should provide definition for three methods.

- a) **getObject()**---->creates and returns resultant object.
- b) **getObjectType()**----->returns java.lang.Class object holding resultant object class name.
- c) **isSingleton()**----->Specifies wheather resultant object should be taken as singleton(true) object or prototype object (false)

->The Bean that gets objects from jndi registry based on given jndi name will be developed as Factory Bean. FactoryBean is very important to gather and inject external services objects to target bean class properties. The external services are those services which are not part of our project but should be gathered from outside of the project by doing some **lookup operations over the network**.



### DateFactoryBean.java

```
package com.nt.beans;
import java.util.Date;
import org.springframework.beans.factory.FactoryBean;
public class DateFactoryBean implements FactoryBean<Date>{
    private int year;
    private int month;
    private int date;
    public DateFactoryBean(int year, int month, int date) {
```

```
System.out.println("DateFactoryBean:3-param constructor");
    this.year = year;
    this.month = month;
    this.date = date;
}
//@Override
public Date getObject() throws Exception {
    System.out.println("DFB:getObject()");
    return new Date(year,month,date);
}

//@Override
public Class<?> getObjectType() {
    System.out.println("DFB:getObjectType()");
    return Date.class;
}

//@Override
public boolean isSingleton() {
    System.out.println("DFB:isSingleton()");
    return true;
}}
```

#### ScheduleReminder.java

```
package com.nt.beans;
import java.util.Date;
public class ScheduleReminder {
    private Date date;
    public void setDate(Date date) {
        this.date = date;
    }
    @Override
    public String toString() {
        return "ScheduleReminder [date=" + date + "]";
    }
}
```

#### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="dfb" class="com.nt.beans.DateFactoryBean">
    <constructor-arg value="110"/>
    <constructor-arg value="10"/>
    <constructor-arg value="20"/>
</bean>
```

```
</bean>
<bean id="reminder" class="com.nt.beans.ScheduleReminder">
  <property name="date" ref="dfb"/>
</bean>
</beans>
```

#### ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.ScheduleReminder;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC Container
        ApplicationContext ctx =new
            FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        // get Bean
        ScheduleReminder reminder=ctx.getBean("reminder",ScheduleReminder.class);
        System.out.println(reminder);
    }
}
```

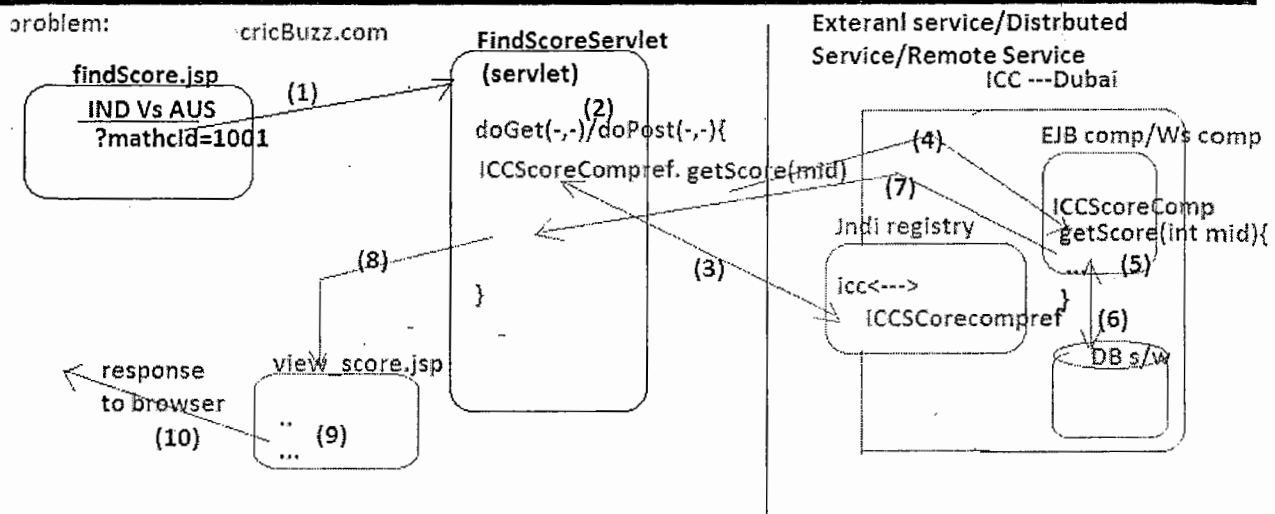
#### **How FactoryBean is able to give another object (Resultant Object)?**

IOC Container creates "FactoryBean" class object normally based on configurations done in spring bean configuration file, but before keeping that object in a scope, IOC container checks whether the class of the object is implementing "FactoryBean() / not" to get Resultant object and calls "isSingleton()" to keep Resultant object as "singleton scope or prototype scope" object with the bean id given for FactoryBean class in configuration file(Xmlfile).

#### **ServiceLocator(use case of FactoryBean):**

The services that are there with in the project are called **local services**. The services that are external to project but used in the project are called **external services**. Crickbuzz.com uses **ICCscore card** component as external service. ShareKhan.com uses **BSE or NSE stocktrade component** as external service. These external services generally developed by using distributed technologies like EJB, RMI, WebSevices and etc... they will be registered in JNDI registry to expose them for out siders.

The local service class/controller components maintaining JNDI lookup code to get external service component reference from the JNDI registry. But placing JNDI lookup code directly in **local service/controller components** is a bad practice and having following limitations.



**Limitations:**

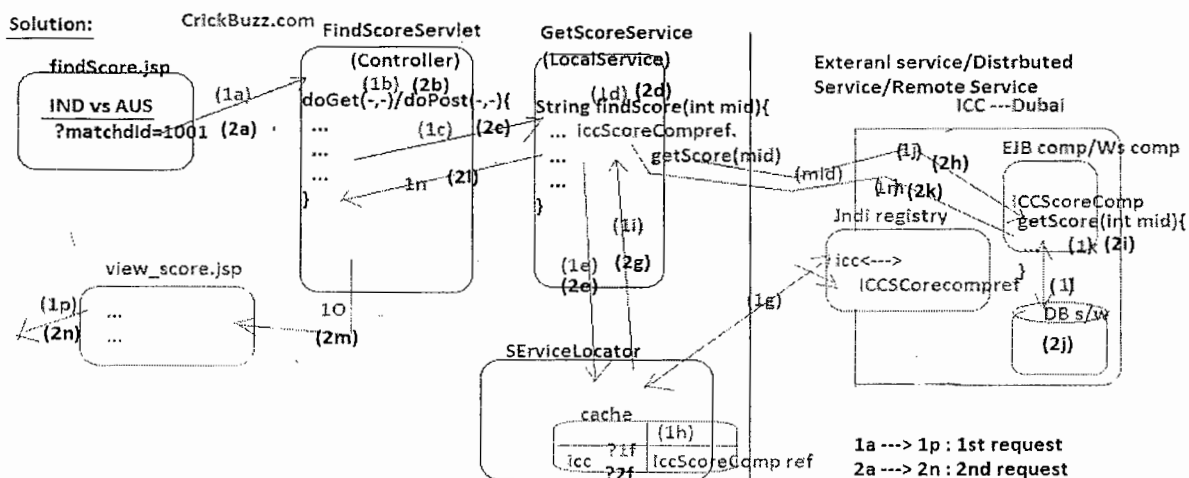
1)Jndi lookup code will not be reusable code, if want you write this code in multiple services components.

2) Any change in external service technology or location or JNDI registry details we need to modify the JNDI lookup code in multiple places (service classes).

→To overcome this problems it is recommended to implement a design pattern called **ServiceLocator**.

ServiceLocator is a java class having JNDI lookup code to get **external service component reference**, **giving reusability for JNDI lookup code** and **caching support** to reduce network round trips between java application and JNDI registry of external service and also gives flexibility to modify JNDI lookup code without disturbing other components of the application when external service related technology/location/JNDI registry details are changed.

If ServiceLocator developed as spring bean it recommended to develop it as **FactoryBean** because though it is dependent to other controller/service components of the application those components are not interested in **ServiceLocator class object**, they are interested to get the **external service components reference** that is gathered through JNDI lookup as resultant object by ServiceLocator class..



**Note:** If web application uses spring libraries they must be added to build path of the project and WEB-INF/lib folder.

→ When we use spring in web application we can't do dependent injection of servlet component because servlet container manages servlet component and the spring beans are managed by IOC container can't be injected.

→ If you want to perform dependency management between two classes then both the classes must **be managed by IOC container**.

In the above situation servlet component can get spring bean through dependency lookup(`ctx.getBean(-,-)`).

#### Example Application:

##### ExternalICCScoreCompService.java

```
package com.nt.ext.icc;  
public interface ExternalICCScoreCompService {  
    public String getScore(int matchId);  
}
```

##### ExternalICCScoreCompServiceImpl.java

```
package com.nt.ext.icc;  
public class ExernalICCScoreCompServiceImpl implements ExternalICCScoreCompService{  
    @Override  
    public String getScore(int matchId) {  
        if(matchId==3456)  
            return "456/9";  
        else if(matchId==1234)  
            return "345/10";  
        else  
            return "No Match found";  
    }  
}
```

ICCScoreCompServiceLocator.java

```
package com.nt.serviceLocator;
import java.util.HashMap;
import java.util.Map;
import org.springframework.beans.factory.FactoryBean;
import com.nt.ext.icc.ExternalICCScoreCompServiceImpl;
import com.nt.ext.icc.ExternalICCScoreCompService;

public class ICCScoreCompServiceLocator implements
FactoryBean<ExternalICCScoreCompService>{
    private Map<String,ExternalICCScoreCompService> cache=new HashMap();
    private String jndi;

    public ICCScoreCompServiceLocator(String jndi) {
        this.jndi = jndi;
    }

    @Override
    public ExternalICCScoreCompService getObject() throws Exception {
        ExternalICCScoreCompService extComp=null;
        if(!cache.containsKey(jndi)){
            //write jndi lookup code /web service consumer code to get External ref and place in
            cache
            extComp=new ExternalICCScoreCompServiceImpl();
            cache.put(jndi, extComp);
        }
        return cache.get(jndi);
    } //method

    @Override
    public Class getObjectType() {
        return ExternalICCScoreCompService.class;
    }

    @Override
    public boolean isSingleton() {
        return true;
    }
}
```



CrickBuzzFindScoreServiceImpl.javaCrickBuzzFindScoreService.java

```
package com.nt.service;

import com.nt.ext.icc.ExternalICCScoreCompService;

public class CrickBuzzFindScoreServiceImpl implements CrickBuzzFindScoreService {
    private ExternalICCScoreCompService extComp;

    public void setExtComp(ExternalICCScoreCompService extComp) {
        this.extComp = extComp;
    }

    @Override
    public String findScore(int matchId) {
        //use extenal service
        return extComp.getScore(matchId);
    }
}
```

```
package com.nt.service;

public interface CrickBuzzFindScoreService {
    public String findScore(int matchId);
}
```

**applicationContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<!--Factory Bean -->
<bean id="serviceLocator" class="com.nt.serviceLocator.ICCScoreCompServiceLocator">
    <constructor-arg value="iccJndi"/>
</bean>
<bean id="crickBuzzService" class="com.nt.service.CrickBuzzFindScoreServiceImpl">
    <property name="extComp" ref="serviceLocator"/>
</bean>

</beans>
```

**FindScoreControllerServlet.java**

```
package com.nt.controller;
import java.io.IOException;
import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.nt.service.CrickBuzzFindScoreService;

public class FindScoreControllerServlet extends HttpServlet {

    ApplicationContext ctx=null;
    public void init(){
        //create IOCContainer
        ctx=new ClassPathXmlApplicationContext("com/nt/cfgs/applicationContext.xml");
    }

    @Override
    public void doGet(HttpServletRequest req, HttpServletResponse res) throws ServletException,
    IOException {
        int matchid;
        CrickBuzzFindScoreService localService=null;
        String scoreBoard=null;
        RequestDispatcher rd=null;
        //read matchid from hyperlinks
        matchid=Integer.parseInt(req.getParameter("mid"));
        //get LocalService class object
```

```

localService=ctx.getBean("crickBuzzService",CrickBuzzFindScoreService.class);
//use Local Service
scoreBoard=localService.findScore(matchid);
//keep ScoreBoard in request scope
req.setAttribute("score",scoreBoard);
//forward to view_score.jsp to display results
} //doGet(-,-)
@Override
public void doPost(HttpServletRequest req, HttpServletResponse res) throws ServletException,
IOException {
    doGet(req,res);
}
}

```

find\_score.jsp

```

h1 style="color:red;text-align:center"> Welcome to CrickBuzz.com</h1>
<a href="controller?mid=3456">IND Vs PAK</a> <br><br>
<a href="controller?mid=1234">ENG Vs AUS</a> <br><br>

```

view\_score.jsp

```

<h1 style="color:red;text-align:center">Score Board</h1>
<b><i>Score:::</i></b> ${score} <br>
<a href="find_score.jsp">home</a>

```

web.xml

```

<web-app>
<display-name>Archetype Created Web Application</display-name>
<welcome-file-list>
<welcome-file>find_score.jsp</welcome-file>
</welcome-file-list>
<servlet>
<servlet-name>findScore</servlet-name>
<servlet-class>com.nt.controller.FindScoreControllerServlet</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>findScore</servlet-name>
<url-pattern>/controller</url-pattern>
</servlet-mapping>
</web-app>

```

pom.xml

```

<project>
<modelVersion>4.0.0</modelVersion>
<groupId>com.nt</groupId>
<artifactId>crickbuzz</artifactId>
<packaging>war</packaging>
<version>1.0.0</version>
<dependencies>
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-web</artifactId>
<version>4.0.0</version>
</dependency>
</dependencies>
</project>

```

```
<modelVersion>4.0.0</modelVersion>
<groupId>nit</groupId>
<artifactId>IOCProj10-InnerBean</artifactId>
<version>0.0.1-SNAPSHOT</version>
<packaging>jar</packaging>

<name>IOCProj10-InnerBean</name>
<url>http://maven.apache.org</url>
<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
    <scope>test</scope>
  </dependency>
  <!-- https://mvnrepository.com/artifact/org.springframework/spring-context -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>4.2.2.RELEASE</version>
  </dependency>
  <!-- https://mvnrepository.com/artifact/org.springframework/spring-context-support -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context-support</artifactId>
    <version>4.2.2.RELEASE</version>
  </dependency>

  <!-- https://mvnrepository.com/artifact/javax.servlet/javax.servlet-api -->
  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>3.1.0</version>
  </dependency>
  <!-- https://mvnrepository.com/artifact/org.springframework/spring-jdbc -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jdbc</artifactId>
    <version>4.2.2.RELEASE</version>
  </dependency>
  <!-- https://mvnrepository.com/artifact/com.hynnet/oracle-driver-odbc6 -->
  </dependencies>
</project>
```

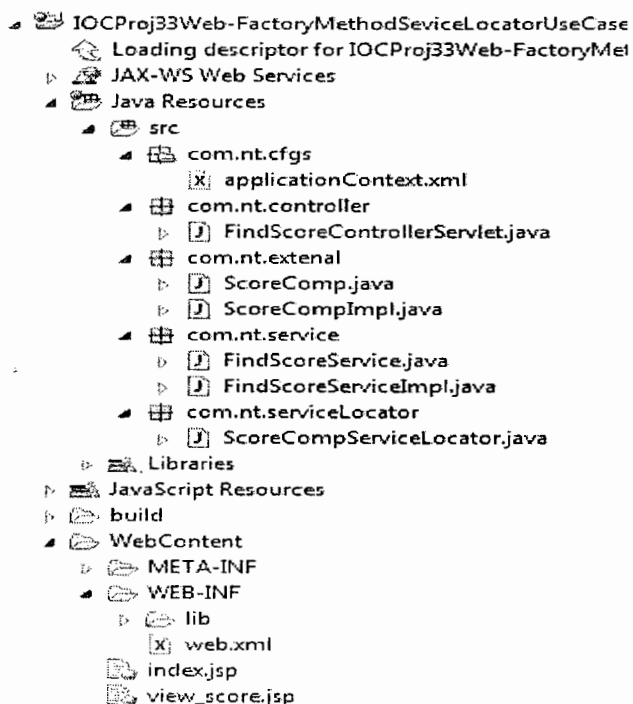
Note: When we take ServiceLocator as FactoryBean it actually fulfills the requirement but kills non-invasive principle of spring because we need to implement FactoryBean (I) on ServiceLocator class directly. To overcome this problem we can go for FactoryMethod based bean instantiation as shown below.

#### ScoreCompServiceLocator.java

```
public class ScoreCompServiceLocator{
    private static Map<String,ScoreComp> cache=new HashMap<String,ScoreComp>();
    public static ScoreComp getService(){
        //actually we need JNDI lookup code here to gather external component reference
        if(!cache.containsKey("icc")){
            cache.put("icc", new ScoreCompImpl());
        }
        return cache.get("icc");
    }
}
//class
```

#### applicationContext.xml:

```
<bean id="serviceLocator" class='com.nt.servicelocator.ScoreCompServiceLocator'
      factory-method="getService"/>
<bean id="scoreService" class='com.nt.service.FindScoreServiceImpl">
    <property name="externalComp" ref="serviceLocator"/>
</bean>
```



**applicationContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- static factory method bean instantiation
    <bean id="serviceLocator" class="com.nt.serviceLocator.ScoreCompServiceLocator"
                                factory-method="getService"/>

    <bean id="scoreService" class="com.nt.service.FindScoreServiceImpl">
        <property name="extenalComp" ref="serviceLocator"/>
    </bean> -->
    (or)
    <!-- instance factory method bean instantiation -->
    <bean id="serviceLocator" class="com.nt.serviceLocator.ScoreCompServiceLocator"/>

    <bean id="extenalService" factory-bean="serviceLocator" factory-method="getService"/>

    <bean id="scoreService" class="com.nt.service.FindScoreServiceImpl">
        <property name="extenalComp" ref="extenalService"/>
    </bean>
</beans>
```

**FindScoreControllerServlet.java**

```
package com.nt.controller;

import java.io.IOException;

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.nt.service.FindScoreService;

public class FindScoreControllerServlet extends HttpServlet {
    ApplicationContext ctx=null;

    public void init(){
        //create IOC container
        ctx=new ClassPathXmlApplicationContext("com/nt/cfgs/applicationContext.xml");
    }

    @Override
    public void doGet(HttpServletRequest req, HttpServletResponse res) throws ServletException,
    IOException {
```

```
int mid;
FindScoreService service=null;
String scoreCard=null;
RequestDispatcher rd=null;
//read matchId
mid=Integer.parseInt(req.getParameter("matchId"));
//get Local Service class objref
service=ctx.getBean("scoreService",FindScoreService.class);
//use Service class
scoreCard=service.getScore(mid);
//keep the results in requestScope
req.setAttribute("scoreCard",scoreCard);
//forward request to result.jsp
rd=req.getRequestDispatcher("/view_score.jsp");
rd.forward(req,res);

} //doGet(-,-)
```

#### ScoreComp.java

```
package com.nt.extenal;

public interface ScoreComp {

    public String findScore(int matchId);

}
```

#### ScoreCompImpl.java

```
package com.nt.extenal;

public class ScoreCompImpl implements ScoreComp {

    @Override
    public String findScore(int matchId) {
        if(matchId==345)
            return "TestMatch:345/6";
        elseif(matchId==567)
            return "OneDay: 234/9";
        else
            return "Wrong MatchId";
    } //method
} //class
```

**FindScoreService.java**

```
package com.nt.service;

public interface FindScoreService {
    public String getScore(int matchId);
}
```

**FindScoreServiceImpl.java**

```
package com.nt.service;

import com.nt.extenal.ScoreComp;

public class FindScoreServiceImpl implements FindScoreService {
    private ScoreComp extenalComp;

    public void setExtenalComp(ScoreComp extenalComp) {
        this.extenalComp = extenalComp;
    }

    @Override
    public String getScore(int matchId) {
        String scoreCard = null;
        scoreCard = extenalComp.findScore(matchId);
        return scoreCard;
    }
}
```

**ScoreCompServiceLocator.java (Service Locator with static /instance factory method)**

```
package com.nt.serviceLocator;

import java.util.HashMap;
import java.util.Map;
import com.nt.extenal.ScoreComp;
import com.nt.extenal.ScoreCompImpl;

public class ScoreCompServiceLocator {
    private static Map<String, ScoreComp> cache = new HashMap<String, ScoreComp>();

    /*public static ScoreComp getService(){
        //actually we need indi lookup code here to gather extenalcompref
        if(!cache.containsKey("icc")){
            cache.put("icc", new ScoreCompImpl());
        }
        return cache.get("icc");
    }*/

    public ScoreComp getService(){
```



```
//actually we need indi lookup code here to gather extenalcompref
if(!cache.containsKey("icc")){
    cache.put("icc",new ScoreComplImpl());
}
return cache.get("icc");
}
}
```

#### web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.org/xml/ns/javaee/web-
app_3_1.xsd" id="WebApp_ID" version="3.1">
<display-name>IOCProj32Web-FactoryBeanSeviceLocatorUseCase</display-name>
<welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
</welcome-file-list>
<servlet>
    <servlet-name>controller</servlet-name>
    <servlet-class>com.nt.controller.FindScoreControllerServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>controller</servlet-name>
    <url-pattern>/controller</url-pattern>
</servlet-mapping>
</web-app>
```

#### index.jsp

```
<a href="controller?matchId=345"> IND Vs WI </a><br>
<a href="controller?matchId=567"> AUS Vs PAK </a><br>
```

#### view\_score.jsp

```
<%@ page language="java" isELIgnored="false" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">

<h1><center> Score Card</center></h1>

Total Score ::: ${scoreCard}
<br>
<a href="index.jsp">home</a>
```

## Method Replacement/Method Injection

→In normal classes, if you want to replace existing logic of business method with new logic, we have two options.

a)Comment the existing logic and write the new logic in the same business method.

b)Develop the sub class and override business method with new logics.

→In the above model we need to modify source code to revert back to original logics. if we write new logics on experimental basis or if new logics execution is failed in production environment at client side. The another problem with above approach is reverting back to original business logic is complex when source code is not **exposed** (.java code).

→In retail industry **offers to customers** will be placed for certain period, once the offer period is over standard prices will be charged. To execute new logics during offer period and to revert back to original logics once the offer period is over without touching the source code, we can't use the above models.

→To overcome the above problems use "**method replacer/replacement**" concept of spring. For this we need to develop separate class implementing "**MethodReplacer(I)**" and we need to write new logics in "**reimplement(-,-,-)**" method.

```
public object reimplement (Object obj, Method method, Object [] args)
```

*Object obj---->gives the access to original bean class object.*

*Method method---->gives the access to original method that we want to replace.*

*Object [] args---->gives access original method args.*

→We need to configure "MethodReplacer" class as spring bean and we need to tell the IOC container to execute "reimplement(-,-,-)" when original method is called, for this we need to use `<replaced-method name="....." (Original Method) replacer="....." (bean id method replacer class)/>`

Note:Method replacement is also called as method injection.

**Note:**

Every project goes through 2 phases:

1. Development and Testing phase.
2. Production phase (after release).

**Example:**

- ▲  IOCProj19(Method Replacer)
  - ▲  src
    - ▲  com.nt.beans
      - ▷  Bank.java
      - ▷  IntrRateReplacer.java
    - ▲  com.nt.cfgs
      - ▷  applicationContext.xml
    - ▲  com.nt.test
      - ▷  ClientApp.java
  - ▷  JRE System Library [JavaSE-1.7]
  - ▲  IOCLib
    - ▷  spring-beans-4.1.6.RELEASE.jar - C:\Users\
    - ▷  spring-context-4.1.6.RELEASE.jar - C:\User
    - ▷  spring-context-support-4.1.6.RELEASE.jar
    - ▷  spring-core-4.1.6.RELEASE.jar - C:\Users\d
    - ▷  spring-expression-4.1.6.RELEASE.jar - C:\U
    - ▷  commons-logging.jar - C:\Users\dell\Des

**Bank.java**

```
package com.nt.beans;
public class Bank {
    public float calcIntrAmt(float pamt,float time){
        System.out.println("Bank:calcIntrAmt(-,-)");
        return pamt*time*2.0f/100;
    }
}
```

**IntrRateReplacer.java**

```
package com.nt.beans;
import java.lang.reflect.Method;
import java.util.Arrays;
import org.springframework.beans.factory.support.MethodReplacer;
public class IntrRateReplacer implements MethodReplacer {
    private float rate;

    public IntrRateReplacer() {
        System.out.println("IntrRateReplacer:0-param constructor");
    }
    public void setRate(float rate) {
        this.rate = rate;
    }
    @Override
    public Object reimplement(Object bean, Method method, Object[] args)
        throws Throwable {
        System.out.println("IntrRateReplacer:reimplement(-,-,-)");
        System.out.println("bean class"+bean.getClass());
        System.out.println("Method name"+method.getName());
        System.out.println("Method args"+Arrays.toString(args));
    }
}
```

```
        if(method.getName().equals("calcIntrAmt")){  
            float pamt=(Float)args[0];  
            float time =(Float)args[1];  
            return pamt*time*rate/100.0f;  
        }  
        else{  
            return 0.0f;  
        }  
    }  
} //reimplement  
} //class
```

## applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="bank" class="com.nt.beans.Bank">
        <replaced-method name="calcIntrAmt" replacer="mr">
            <arg-type>float</arg-type>
            <arg-type>float</arg-type>
        </replaced-method>
    </bean>

    <bean id="mr" class="com.nt.beans.IntrRateReplacer">
        <property name="rate" value="1.2f"/>
    </bean>
</beans>
```

### ClientApp.java

```
package test;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;

import com.nt.beans.Bank;

public class ClientApp {
    public static void main(String[] args) {
        //create IOC Container
        ApplicationContext ctx=new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get Bean
        Bank bank=ctx.getBean("bank",Bank.class);
        System.out.println("Intr Amt"+bank.calcIntrAmt(10000,20));
    }
}
```

When `ctx.getBean(-,-)` is called having `beanId`, the container checks whether `<replaced-method>` is configured or not in the inmemory meta data of xml file. If configured container generates inmemory proxy class and returns that object. If not configured it is creates bean

**Internals:**

IOC container creates sub class for main bean class (Bank) like "Bank\$\$CGLIB....." By using CGLib libraries internally, In this class business method (calcIntrAmt(-,-)) will be overridden. In this method definition, the object of **MethodReplacer** class (**IntrRateReplacer**) will be accessed and reimplement(-,-) method will be called. Due to this new logics will be executed when client application calls **bank. calcIntrAmt(10000,20)** the bellow generated subclass object will be returned.

BankService.java

```
public class BankService {
    public float calcIntrAmount(float pAmt, float time) {
        System.out.println("BankService:calcIntrAmount(-,-)");
        return (pAmt*time*2.0f)/100;
    } //method
} //class
```

ClientApp

```
//get Bean
service=ctx.getBean("service", BankService.class);
//call b.methods (a)
intrAmt=service.calcIntrAmount(3000,10);
S.o.p("Intrest Amount ::: "+intrAmt);
```

```
public class IntrRateReplacer implements MethodReplacer {
```

```
@Override (d)
```

```
public Object reimplement(Object bean, Method method, Object[] args) throws Throwable {
```

```
    System.out.println("IntrRateReplacer:reimplement(-,-,-) method");
```

```
    float pAmt=0.0f;
```

```
    float time=0.0f;
```

```
    System.out.println("target Bean:"+bean.getClass()+" target method"+method.getName()+"
```

```
target method args"+Arrays.toString(args));
```

```
    //get target method args
```

```
    pAmt=(Float)args[0];
```

```
    time=(Float)args[1];
```

```
    //calc Intr Amount with new details
```

```
    return pAmt*time*rate/100.0f; (e)
```

```
}
```

```
}
```

**Proxy class (InMemory class)**

```
public class BankService$Proxy extends BankService implements ApplicationContextAware
{
    ApplicationContext ctx;
    public void setApplicationContext(ApplicationContext ctx){
        this.ctx=ctx;
    }
    public float calcIntrAmt(float pAmt,float time){
        //get Beans
        BankService service=ctx.getBean("service",BankService.class);
        InterestRateReplacer imr=ctx.getBean("imr",InterestRateReplacer.class);
        Method method=BankService.class.getDeclaredMethods()[0];
        Object args[]=new Object[2];
        args[0]=pAmt; args[1]=time;
        // invoke method
        float result=(Float)imr.reimplement(service,method,args);
        return result;
    }
}
```

**Important points :**

- 1)The method that you want to replace should not be "final, static" (these methods can't be overridden in subclass)
- 2)Eventhough you can write replacement logics for multiple method in single "reimplement(-,-,-)" method by using if conditions, but it is recommended to take separate **MethodReplacer** for every class.
- 3) We can't take bean class as final class while working with **MethodReplacer** concepts.

**Explain various injections supported by spring?**

- a)Setter injection
- b)Constructor injection
- c)Aware injection/context dependency lookup/ Interface injection
- d)Method injection (method replacer)
- e)Lookup method injection

## Lookup method injection

### Problem with dependency injection:

When prototype scope bean is dependent to singleton scope bean some how the prototype scope bean also acts like singleton scope bean.

Eg: All request coming to webApplication will be handled by single **Servlet container/ web container (Singleton)**. This container uses separate "RequestHandler" to process each request, so "ServletContainer" class is target class and "RequestHandler" is dependent class, more over ServletContainer should be taken as singleton scope bean and "RequestHandler" should taken as prototype scope, but because of above problem "RequestHandler" also acts as the singleton scope bean.

```
<bean id="container" class="com.nt.beans.MyServletContainer" scope="singleton">
    <property name="handler" ref="rh"/>
</bean>
<bean id="rh" class="com.nt.beans.RequestHandler" scope="prototype"/>
```

| <u>Target Bean Scope</u> | <u>Dependent Bean Scope</u> | <u>Resultant Scope of Dependent Bean</u>                              |
|--------------------------|-----------------------------|-----------------------------------------------------------------------|
| Prototype                | Singleton                   | Singleton (OK)                                                        |
| Singleton                | Singleton                   | Singleton (OK)                                                        |
| Prototype                | Prototype                   | Prototype (OK)                                                        |
| Singleton                | Prototype                   | Singleton (Not OK) (use dependency lookup or-Lookup method injection) |

- ▲  IOCProj20(LMInjection-problem)
  - ▲  src
    - ▲  com.nt.beans
      - ▶  MyServletContainer.java
      - ▶  RequestHandler.java
    - ▲  com.nt.cfgs
      - ▶  applicationContext.xml
    - ▲  com.nt.test
      - ▶  ClientApp.java
  - ▶  JRE System Library [JavaSE-1.7]
  - ▲  IOCLib
    - ▶  spring-beans-4.1.6.RELEASE.jar - C:\Users\c
    - ▶  spring-context-4.1.6.RELEASE.jar - C:\Users\c
    - ▶  spring-context-support-4.1.6.RELEASE.jar -
    - ▶  spring-core-4.1.6.RELEASE.jar - C:\Users\de
    - ▶  spring-expression-4.1.6.RELEASE.jar - C:\Us
    - ▶  commons-logging.jar - C:\Users\delin\Desktop

**MyServletContainer.java**

```
package com.nt.beans;
public class MyServletContainer {
    private RequestHandler handler;
    public MyServletContainer() {
        System.out.println("MyServletContainer:0-param constructor");
    }
    public void setHandler(RequestHandler handler) {
        this.handler = handler;
    }
    public void processRequest(String data){
        handler.handle(data);
    }
}
```

**RequestHandler.java**

```
package com.nt.beans;
public class RequestHandler {
    private static int count;
    public RequestHandler() {
        count++;
        System.out.println("RequestHandler: 0-param constructor");
    }
    public void handle(String data){
        System.out.println("Handling Request "+count+" with data "+data);
    }
}
```

**applicationContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="rh" class="com.nt.beans.RequestHandler" scope="prototype"/>
<bean id="container" class="com.nt.beans.MyServletContainer" scope="singleton">
<property name="handler" ref="rh"/>
</bean>
</beans>
```

**ClientApp.java**

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.MyServletContainer;
public class ClientApp {
    public static void main(String[] args) {
        // IOC Container
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
```



```
//get Bean
MyServletContainer container=ctx.getBean("container",MyServletContainer.class);
container.processRequest("hello");
container.processRequest("hai");
}
}
```

### IOCProj21(LMInjection-solution1)

#### RequestHandler.java

```
package com.nt.beans;
publicclass RequestHandler {
    private static int count;
    public RequestHandler() {
        count++;
        System.out.println("RequestHandler: 0-param constructor");
    }
    publicvoid handle(String data){
        System.out.println("Handling Request"+count+"with data"+data);
    }
}
```

#### MyServletContainer.java

```
package com.nt.beans;
import org.springframework.beans.BeansException;
import org.springframework.context.ApplicationContext;
import org.springframework.context.ApplicationContextAware;
publicclass MyServletContainer implements ApplicationContextAware {
    ApplicationContext ctx;
    private String beanId;
    publicvoid setBeanId(String beanId) {
        this.beanId = beanId;
    }
    public MyServletContainer() {
        System.out.println("MyServletContainer:0-param constructor");
    }
    publicvoid processRequest(String data){
        //get Dependent Bean
        RequestHandler handler = ctx.getBean(beanId,RequestHandler.class);
        handler.handle(data);
    }
    publicvoid setApplicationContext(Applicationcontext ctx)
        throws BeansException {
        this.ctx = ctx;
    }
}
```

**applicationContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="rh1" class="com.nt.beans.RequestHandler" scope="prototype"/>
  <bean id="container" class="com.nt.beans.MyServletcontainer" scope="singleton">
    <property name="beanId">
      <idref bean="rh1"/>
    </property>
  </bean>
</beans>
```

**ClientApp.java**

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.MyServletContainer;
public class ClientApp {
    public static void main(String[] args) {
        //IOC Container
        ApplicationContext ctx = new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationcontext.xml");
        //get Bean
        MyServletContainer container = ctx.getBean("container", MyServletContainer.class);
        container.processRequest("hello");
        container.processRequest("hai");
    }
}
```

→ We should not think about dependency injection (constructor/setter) in the following situations.

- a) If dependent bean class object is required in TargetBean only in specific methods
  - b) If target Bean scope is singleton and Dependent Bean scope is prototype (refer above table)
- In the above situation prefer using Dependency lookup .

**Note1:** If we go for traditional dependency lookup code we need to create extra IOC container in the specific method of target class (which is bad practice)

**Note2:** By using AwareInjection, we can avoid that extra IOC container towards dependency lookup operation. But it kills the non-invasive behavior of spring Application because target bean class should implement "xxxAware" interfaces.

**Note3:** To overcome the above two problems we can use "Lookup method Injection (LMI)" which says perform "dependency lookup" but container should write code for it (not the programmer).

To perform **Lookup Method Injection** we need to consider the following points:

a) Target bean class should have **abstract method** returning Dependent Bean class (for this target bean class must be taken as **abstract class**)

b) We must configure the above abstract method with target Bean class configuring by using **<lookup-method>** tag

IOCProj22(Lookup Mmethod Injection – Final Solution)RequestHandler.java

```
package com.nt.beans;
public class RequestHandler {
    private static int count;
    public RequestHandler() {
        count++;
        System.out.println("RequestHandler: 0-param constructor");
    }
    public void handle(String data){
        System.out.println("Handling Request"+count+"with data"+data);
    }
}
```

MyServletContainer.java

```
package com.nt.beans;
public abstract class MyServletContainer {
    public MyServletContainer() {
        System.out.println("MyServletContainer:0-param constructor");
    }
    public void processRequest(String data){
        //get Dependent Bean
        RequestHandler handler = getRequestHandler();
        handler.handle(data);
    }
    public abstract RequestHandler getRequestHandler();
}
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="rh" class="com.nt.beans.RequestHandler" scope="prototype"/>
    <bean id="container" class="com.nt.beans.MyServletcontainer" scope="singleton">
        <lookup-method name="getRequestHandler" ref="rh"/>
    </bean>
</beans>
```

ClientApp.java:

```
package com.nt.beans;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
public class ClientApp {
    public static void main(String[] args) {
        //IOC Container
        ApplicationContext ctx = new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationcontext.xml");
        //get Bean
        MyServletContainer container=ctx.getBean("container",MyServletContainer.class);
    }
}
```

```

        container.processRequest("hello");
        container.processRequest("hai");
        System.out.println(container.getClass());
    }
}

```

**Note:**

We can configure abstract class as spring bean only when **<lookup-method>** tag is placed for lookup method injection otherwise not possible. But while working with **factory method bean instantiation** we also can take abstract class as spring bean.

Spring IOC container generates subclass for targetBean class and implements the abstract method of the target class to get and Return dependent bean class object. For this it internally uses **CGLIB Libraries**. In spring 2.x,3.x we need to supply CGLIB Libraries separately, where as in spring 4.x, same thing is not required.

**Lookup method injection use cases**

**ServletContainer (Singleton)<----->Request Handler (prototype)**

**Manager(Singleton)<----->Customer (prototype)**

**LoanApprover (Singleton)<----->LoanDetails (prototype).**

**Internal code for the above final solution example (Lookup Method Injection):**

```

public abstract class MyServletContainer {

    public MyServletContainer() {
        System.out.println("MyServletContainer:0-param constructor");
    }

    (c)
    public void processRequest(String data){
        RequestHandler handler=null;
        System.out.println("ServletContainer is processing the request");
        // get Request Handler
        handler=getRequestHandler(); (d)
        System.out.println(handler.handleRequest(data));
    } //method (g)

    //mandatory method in Lookupmethod injection
    public abstract RequestHandler getRequestHandler();

} //class

```

Proxy class: (Container created in Memory class)

```

public class MyServletContainer$Proxy extends MyServletContainer
    implements ApplicationContextAware{
    private ApplicationContext ctx;
    public void setApplicationContext(ApplicationContext ctx){
        this.ctx=ctx;
    }

    (e)
    public RequestHandler getRequestHandler(){
        RequestHandler handler=null;
        //get Dependent bean class object
        handler=ctx.getBean("rh",RequestHandler.class);
        return handler; (f)
    }
}

```

ClientApp

(a) (givesn Dynamically generated Proxy class obj{ Sub class obj for taget class)

container=ctx.getBean("container",MyServletContainer.class);

//invoke methods

container.processRequest("Ind Vs Pak match"); (b)

(h)

BeanPostProcessors (Bpp)

Assigning some values to spring bean properties after injection is called Post processing. We can do this post processing inside the spring bean class and specific to each spring bean class by using **custom-init()** method or **InitializingBean's afterPropertiesSet()** of bean class.If we want to apply common post processing logics for multiple spring beans that to being from outside the bean class then we can use **BeanPostProcessor** class.

The "**BeanPostProcessor**" class executes for all beans that are configured in spring bean configuration file.To develop BeanPostProcessor we need to take a class that implements "**BeanPostProcessor()**" and should provide implementation for two methods.

a)postProcessorBeforeInitialization(Object bean, Spring Beanid)

Executes before afterPropertiesSet() for every beans class.

b)postProssorAfterInitialization (Object bean, Spring bean id)

Executes after custom init() for every bean class.

**Note:**The concept is useful to develop custom post process/initialization logics for all beans in one class.

 IOCProj23(BPP)

 src

 com.nt.bbp

 BOPostProcessor.java

 com.nt.bo

 BaseBO.java

 CustomerBO.java

 RequestHandler.java

 StudentBO.java

 com.nt.cfgs

 applicationContext.xml

 com.nt.dao

 CustStudDAO.java

 com.nt.service

 CustStudService.java

 com.nt.test

 ClientApp.java

 JRE System Library [JavaSE-1.7]

 IOCLib

 spring-beans-4.1.6.RELEASE.jar - C:\Users\del

 spring-context-4.1.6.RELEASE.jar - C:\Users\del

 spring-context-support-4.1.6.RELEASE.jar - C:\

 spring-core-4.1.6.RELEASE.jar - C:\Users\del\

 spring-expression-4.1.6.RELEASE.jar - C:\Users

 commons-logging.jar - C:\Users\del\Desktop

 Referenced Libraries

 spring-jdbc-4.1.6.RELEASE.jar - C:\Users\del\

 spring-tx-4.1.6.RELEASE.jar - C:\Users\del\De

**BaseBO.java**

```
package com.nt.bo;
import java.util.Date;
public abstract class BaseBO {
    private int id;
    private String name;
    private Date doj;
    public BaseBO() {
        System.out.println("BaseBO:0-param constructor");
    }
    public void setDoj(Date doj) {
        this.doj = doj;
    }
    public void setId(int id) {
        this.id = id;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getId() {
        return id;
    }
    public String getName() {
        return name;
    }
    public Date getDoj() {
        return doj;
    }}
}
```

**CustomerBO.java**

```
package com.nt.bo;
public class CustomerBO extends BaseBO {
    private float billAmt;
    public CustomerBO() {
        System.out.println("CustomerBO:0-param constructor");
    }
    public float getBillAmt() {
        return billAmt;
    }
    public void setBillAmt(float billAmt) {
        this.billAmt = billAmt;
    }
}
```

**StudentBO.java**

```
package com.nt.bo;
public class StudentBO extends BaseBO {
```

```
private String course;
public StudentBO() {
    System.out.println("StudentBO:0-param constructor");
}
public String getCourse() {
    return course;
}
public void setCourse(String course) {
    this.course = course;
}
}
```

#### **BOPostProcessor.java**

```
package com.nt.bpp;
import java.util.Date;
import org.springframework.beans.BeansException;
import org.springframework.beans.factory.config.BeanPostProcessor;
import com.nt.bo.BaseBO;
public class BOPostProcessor implements BeanPostProcessor {
    public BOPostProcessor() {
        System.out.println("BPP: 0-param constructor");
    }
    @Override
    public Object postProcessBeforeInitialization(Object bean, String bid)
        throws BeansException {
        System.out.println("BPP:BeforeInitialization(-,-)");
        if(bean instanceof BaseBO){
            ((BaseBO) bean).setDoj(new Date());
        }
        return bean;
    }
    @Override
    public Object postProcessAfterInitialization(Object bean, String bid)
        throws BeansException {
        System.out.println("BPP:AfterInitialization(-,-)");
        return bean;
    }
}
```

#### **CustStudDAO.java**

```
package com.nt.dao;
import com.nt.bo.CustomerBO;
import com.nt.bo.StudentBO;
public class CustStudDAO {
    public CustStudDAO() {
        System.out.println("DAO:0-param constructor");
    }
}
```

```
}  
    public void insertData(StudentBO stbo){  
        System.out.println("Student Data is");  
        System.out.println(stbo.getId()+" "+stbo.getName()+" "+stbo.getCourse()+"  
"+stbo.getDoj());  
    }  
    public void insertData(CustomerBO custbo){  
        System.out.println("Customer Data is");  
        System.out.println(custbo.getId()+" "+custbo.getName()+" "+custbo.getBillAmt()+"  
"+custbo.getDoj());  
    }  
}
```

### CustStudService.java

```
package com.nt.service;  
import com.nt.bo.CustomerBO;  
import com.nt.bo.StudentBO;  
import com.nt.dao.CustStudDAO;  
public abstract class CustStudService {  
    private CustStudDAO dao;  
  
    public CustStudService() {  
        System.out.println("Service : 0-param constructor");  
    }  
    public abstract StudentBO getStudentBO();  
    public abstract CustomerBO getCustomerBO();  
    public void setDao(CustStudDAO dao) {  
        this.dao = dao;  
    }  
    public void processStudent(int id,String name,String course){  
        System.out.println("Procesing Student Info");  
        StudentBO stbo=getStudentBO();  
        stbo.setId(id); stbo.setName(name); stbo.setCourse(course);  
        //use dao  
        dao.insertData(stbo);  
    }  
    public void processCustomer(int id,String name,float billAmt){  
        System.out.println("Procesing Customer Info");  
        CustomerBO custbo=getCustomerBO();  
        custbo.setId(id); custbo.setName(name); custbo.setBillAmt(billAmt);  
        //use dao  
        dao.insertData(custbo);  
    }  
}
```



ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.CustStudService;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");

        /*      BeanFactory factory=new XmlBeanFactory(new
        FileSystemResource("src/com/nt/cfgs/applicationContext.xml"));
        //register BeanPostProcessor
        BOPostProcessor bpp=new BOPostProcessor();
        ((ConfigurableListableBeanFactory)factory).addBeanPostProcessor(bpp); */

        //get Bean
        CustStudService service=ctx.getBean("service",CustStudService.class);
        //call b.methods
        service.processCustomer(1001,"raja",9000);
        service.processStudent(101,"karan","java");
        service.processCustomer(1002,"raja",9000);
        service.processStudent(102,"karan","java");
    }
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="stBO" class="com.nt.bo.StudentBO" scope="prototype"/>
<bean id="custBO" class="com.nt.bo.CustomerBO" scope="prototype"/>
<bean id="dao" class="com.nt.dao.CustStudDAO" scope="singleton"/>

<bean id="service" class="com.nt.service.CustStudService" scope="singleton">
    <property name="dao" ref="dao"/>
    <lookup-method name="getStudentBO" bean="stBO"/>
    <lookup-method name="getCustomerBO" bean="custBO"/>
</bean>
<bean id="bpp" class="com.nt.bpp.BOPostProcessor"/>
</beans>
```

→ In real time BeanPostProcessor will be used to place common post process initialization logic that is required for multiple Beans like assigning **system date as date of joining or last modification date or date of updation and etc.....**This concept is very useful towards auditing activity.

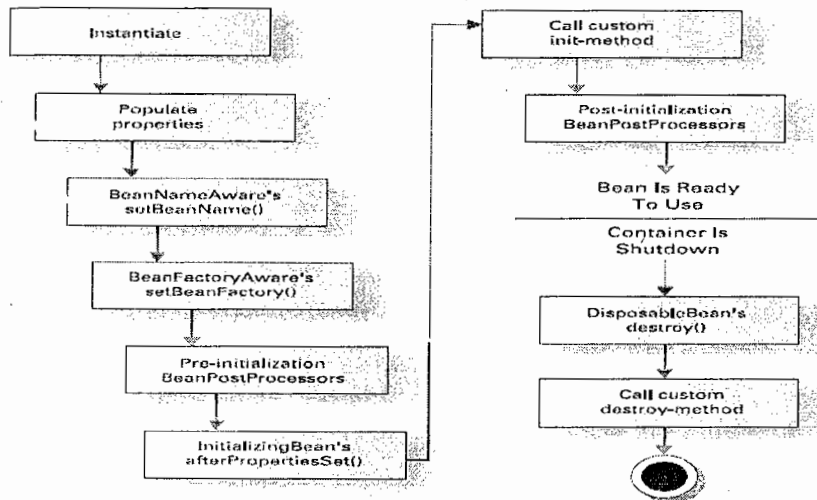
→ While working with ApplicationContext container if we just configure BeanPostProcessor

class in Xml file as spring Bean the container automatically recognize and use that BeanPostProcessor. While working with BeanFactory container we must have to register BeanPostProcessor explicitly as shown below

```

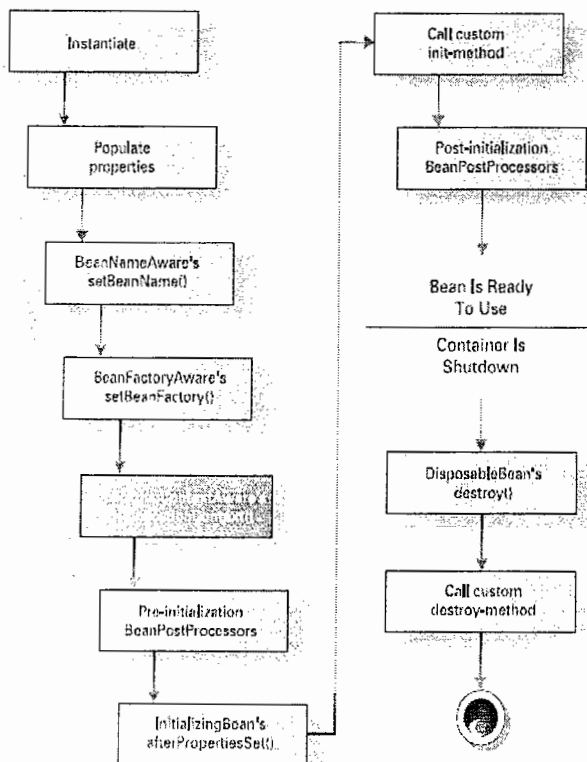
BOPostProcessor bpp=new BOPostProcessor (); //Bpp Lass
((ConfigurableListableBeanFactory)factory).addBeanPostProcessorc(bpp);
  
```

**Bean Life Cycle diagram with respect to BeanFactoryContainer:**



If the underlying container is ApplicationContext container it can inject both BeanFactory, ApplicationContext objects to spring bean through Awareinjection.

**Bean Life Cycle diagram with respect to ApplicationContext container:**



**Complete Internal flow of core module application:**

- a) Programmers create BeanFactory object.
- b) Given SpringBean configuration file will be located and it will be verified whether document is well formed document or valid document.
- c) Creates Inmemory logical IOC container in the jvm's memory.
- d) Reads springBean configuration file entries and creates an inmemory metadata having all the configuration's.
- e) ClientApp calls factory.getBean("id") method.
- f) Loads Bean class from inmemory meta data based on given bean id and instantiates that bean based on constructor injection configuration.
- g) Keep Bean class objects in specified scope (if cyclic dependency is found Bean Instantiation fails)
- h) Performs setter injection based on configurations by performing dependency-check.
- i) Performs Aware injection based on the xxxAware interfaces that are implemented.
- j) Calls postProcessBeforeInitialization(-) if BeanPostProcessor is added.
- k) Calls afterPropertiesSet(-) if InitializationBean(I) is implemented.
- l) Calls custom init method if "init-method" attribute is specified.
- m) Calls postProcessAfterInitialization()
- n) Returns Bean class object to client application.

**Note: If ApplicationContext object is created (f) to (n) also takes place without calling getBean(-) for singleton scope beans.**

Note: if <lookup-method> or <replaced-method> tags are configured inside <bean> tag then container does not create object for given bean class. It creates object for the sub class of the bean class which is **generated by IOC container**.

Note: If Bean class FactoryBean (I) or if the **factory-method attribute is configured in the <bean> tag**, then container doesn't create given bean class object. Container creates / gets resultant class object (generated proxy class) and keeps that resultant object with the **configured beanId**.

## BeanFactoryPostProcessors

→ BeanPostProcessor is useful to write postProcess initialization logics with respect to multiple beans.

→ BeanFactoryPostProcessor is useful to add extra configurations by modifying in-memory metadata of bean configuration file, after the creation of IOC container and before accessing bean object by calling `getBean(-)`

→ It acts as extension hook/anchor for IOC container to add extra configurations without touching the xml file (Springbean configuration file) i.e it modifies the in-memory metadata of IOC container dynamically at runtime, after IOC container creation and before bean class object is accessed.

To develop custom BeanFactoryProcessor we need to take a class implementing "BeanFactoryPostProcessor(I)". Spring has given lots of pre-defined "BeanFactoryPostProcessor(I)" like 'PropertyPlaceholderConfigurer'. This class modifies in memory metadata by replacing placeholders (\${-}) with the values collected from properties File.

→ "ApplicationContext" object based IOC container automatically recognizes the BeanFactoryPostProcessor that is configured in xml file, But for BeanFactory IOC container we need to register it explicitly.

By default the IOC container of "BeanFactory" can't work with properties file and placeholders. But we can make it working by registering "PropertyPlaceholderConfigurer" as BeanFactoryPostprocessor

### Example:

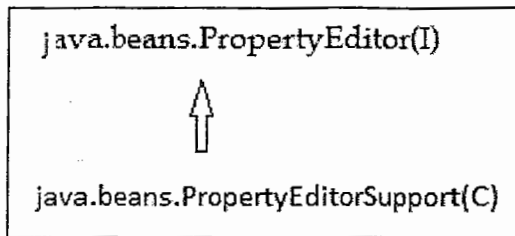
```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="pcc" class="org.gpr.prk.bean.Factory.config.PlaceholderConfigurer">
<property name="location" value="com/nt/commons/DBdetails.properties"/>
</bean>
</beans>
```

### ClientApp:

```
BeanFactory factory=new XmlBeanFactory (new FilesystemResource
("src/com/nt/cfgs/applicationContext.xml"));
//get propertyplaceholder configure object (pphc)
BeanFactoryPostProcessor bfpp=factory.getBean ("PPC" PropertyPlaceholderConfigurer.class);
//register BFPP
bfpp.postProcessBeanFactory((ConfigurableListableBeanFactory)factory);
```

### PropertyEditors

In spring we use setter or constructor injection to assign values to bean properties. In spring Bean configuration file we configure these dependent values as String values, but they will be converted into int, long, float, Date and etc. values as required for bean properties automatically. For this spring internally uses pre-defined PropertyEditors to convert **String values to appropriate values**. All property editors are the classes implementing **java.beans.PropertyEditor(I)**. Spring has given multiple built-in PropertyEditors like StringArrayPropertyEditors, ByteArrayPropertyEditor, DateFormatterPropertyEditor and etc.... (FileEditor, CharacterEditor)



PropertyEditor hierarchy

#### Note:

Registrar maintains registry and we can register one (or) more property editor objects with registry.

#### Understanding Built-in property editors.

- ▶ IOCProj25(PropertyEditor1)
  - ▶ src
    - ▶ com.nt.beans
      - ▶ AadharDetails.java
    - ▶ com.nt.cfgs
      - ▶ applicationContext.xml
    - ▶ com.nt.test
      - ▶ ClientApp.java
  - ▶ JRE System Library [JavaSE-1.7]
  - ▶ IOCLib
    - ▶ spring-beans-4.1.6.RELEASE.jar - C:\
    - ▶ spring-context-4.1.6.RELEASE.jar - C:\
    - ▶ spring-context-support-4.1.6.RELEASE.jar - C:\
    - ▶ spring-core-4.1.6.RELEASE.jar - C:\
    - ▶ spring-expression-4.1.6.RELEASE.jar - C:\
    - ▶ commons-logging.jar - C:\Users\di\

#### AadharDetails.java

```
package com.nt.beans;
import java.io.File;
import java.util.Arrays;
import java.util.Date;
public class AadharDetails {
    private int uid;
    private String name;
    private Date dob;
    private File photo;
    private String[] verifiers;
```

```
public void setUid(int uid) {
    this.uid = uid;
}
public void setName(String name) {
    this.name = name;
}
public void setDob(Date dob) {
    this.dob = dob;
}
public void setPhoto(File photo) {
    this.photo = photo;
}
public void setVerifiers(String[] verifiers) {
    this.verifiers = verifiers;
}
@Override
public String toString() {
    return "AadharDetails [uid=" + uid + ", name=" + name + ", dob=" + dob
        + ", photo=" + photo + ", verifiers="
        + Arrays.toString(verifiers) + "]";
}
}
```

#### applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="aadhar" class="com.nt.beans.AadharDetails">
    <property name="uid" value="101"/>
    <property name="name" value="Raja"/>
    <property name="dob" value="12/11/1990"/>
    <property name="Photo" value="C:/abd/picd.gif"/>
    <property name="verifiers" value="sachin,Ramesh,Ravi"/>
</bean>
</beans>
```

Note: In both containers(BeanFactory,ApplicationContext) the Built-in property editors are automatically registered. Spring Internally uses "DateFormatPropertyEditor" to convert given date value(String) to java.util.Date class object.Spring allows to create custom property editor class either by implementing **java.beans.PropertyEditor(I)** or by extending from **PropertyEditorSupport(c)**.

#### Sample code

##### IntrAmtDetails.java

```
public class IntrAmtDetails
{
    private float p,t,r;
```

```
//setters
}
IntrAmtCalculator.java
public class IntrAmtCalculator
{
private IntrAmtDetails details;
//setters
.....
}
```

#### In applicationContext.xml

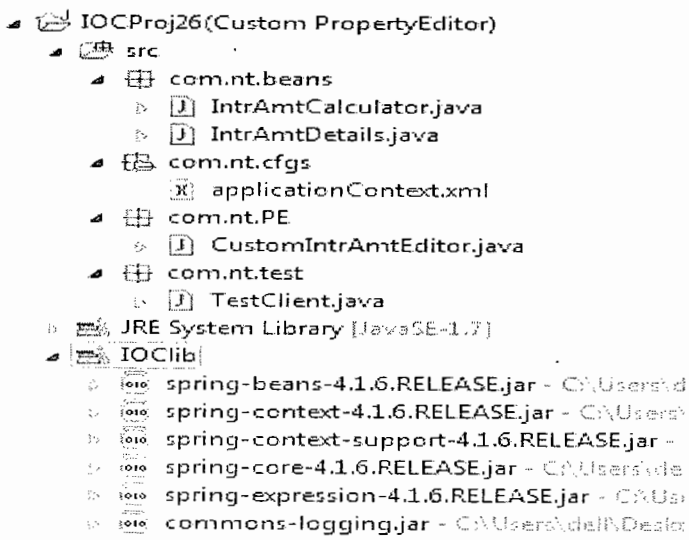
```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="amtdetails" class="pkg.IntrAmtDetails"/>
<property name="P" value="1000"/>
<property name="t" value="10"/>
<property name="r" value="2"/>
</bean>
<bean id="intrCalculator" class="pkg.IntrAmtCalculator">
<property name="details" ref="amtdetails"/>
</bean>
</beans>
```

**But we want like this**

**applicationContext.xml:**

```
<bean id="intrCalculator" class="pkg.IntrAmtCalculator">
<property name="details" values="1000,10,2"/>
</bean>
```

→ Here "details" is reference type property but we want to perform simple values injection to same property which is not possible directly. So must go for **custom property editor development**.



**IntrAmtCalculator.java**

```
package com.nt.beans;
public class IntrAmtCalculator {
    private IntrAmtDetails details;
    public void setDetails(IntrAmtDetails details) {
        this.details = details;
    }
    public float calcIntrAmt(){
        return (details.getPrinciple()*details.getRate()*details.getTime())/100.0f;
    }
}
```

**IntrAmtDetails.java**

```
package com.nt.beans;
public class IntrAmtDetails {
    private int principle;
    private int time;
    private int rate;
    public void setPrinciple(int principle) {
        this.principle = principle;
    }
    public void setTime(int time) {
        this.time = time;
    }
    public void setRate(int rate) {
        this.rate = rate;
    }
    public int getPrinciple() {
        return principle;
    }
    public int getTime() {
        return time;
    }
    public int getRate() {
        return rate;
    }
}
```

**applicationContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">

<!-- <bean id="amtDetails" class="com.nt.beans.IntrAmtDetails">
    <property name="principle" value="1000"/>
    <property name="time" value="10"/>
```



```
<property name="rate" value="2"/>
</bean>
<bean id="intrCalculator" class="com.nt.beans.IntrAmtCalculator">
  <property name="details" ref="amtDetails"/>
</bean> -->
<bean id="intrCalculator" class="com.nt.beans.IntrAmtCalculator" scope="prototype">
  <property name="details" value="1000,10,2"/>
</bean>
</beans>
```

#### CustomIntrAmtEditor.java

```
package com.nt.PE;
import java.beans.PropertyEditorSupport;
import com.nt.beans.IntrAmtDetails;
public class CustomIntrAmtEditor extends PropertyEditorSupport {
    public CustomIntrAmtEditor() {
        System.out.println("CustomIntrAmtEditor:0-param constructor");
    }
    @Override
    public void setAsText(String text) throws IllegalArgumentException {
        System.out.println("CustomIntrAmtEditor:setAsText(-)");
        // spilt the given value
        int principle=Integer.parseInt(text.substring(0,text.indexOf(", ")));
        int time=Integer.parseInt(text.substring(text.indexOf(",")+1,text.lastIndexOf(", ")));
        int rate=Integer.parseInt(text.substring(text.lastIndexOf(",")+1,text.length()));
        IntrAmtDetails details=new IntrAmtDetails();
        details.setPrinciple(principle); details.setTime(time); details.setRate(rate);
        setValue(details);
    } //method
} //class
```

#### TestClient.java

```
package test;
import org.springframework.beans.PropertyEditorRegistrar;
import org.springframework.beans.PropertyEditorRegistry;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.config.ConfigurableListableBeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.FileSystemResource;
import com.nt.PE.CustomIntrAmtEditor;
import com.nt.beans.IntrAmtCalculator;
import com.nt.beans.IntrAmtDetails;
public class TestClient {
    public static void main(String[] args) {
        //Create IOC container
        BeanFactory factory=new XmlBeanFactory(new
        FileSystemResource("src/com/nt/cfgs/applicationContext.xml"));
        ((ConfigurableListableBeanFactory)factory).
```

```
        addPropertyEditorRegistrar(new MyCustomPER());  
  
    //get Bean  
    IntrAmtCalculator iCalc=factory.getBean("intrCalculator",IntrAmtCalculator.class);  
    System.out.println("Intr Amt"+iCalc.calcIntrAmt());  
    }  
    }  
} //main  
}  
//class  
class MyCustomPER implements PropertyEditorRegistrar{  
    @Override  
    public void registerCustomEditors(PropertyEditorRegistry registry) {  
        registry.registerCustomEditor(IntrAmtDetails.class,new CustomIntrAmtEditor());  
    }  
}
```

→ Every IOC container internally maintains one built-in object called “**PropertyEditorRegistry**” and IOC container expose that object to a class that implements “**PropertyEditorRegistrar**” interface as the argument of ‘**registerCustomEditors()**’ method, so that we can use that object to register CustomPropertyEditor by specifying property type and the object of CustomPropertyEditor class. **PropertyEditorRegistrar** is an object maintained by IOC container and IOC container exposes that object to internally maintained PropertyEditorRegistry implementation class registerCustomEditors (method).

### Spring Core Module with Annotations:

**MetaData:** Data about data and code about code is called MetaData. Annotations are java statements to perform metadata operations in “.java” files. Annotations are alternate for xml files to perform metadata, resource configuration operations. Xml files give flexibility of modification but gives bad performance because the xml parsers are heavy to process the xml documents. Annotations do not provide flexibility of modification but provide **good performance**.

**Syntax:** @<Annotation name> (param1=val1,...)

Annotations for documentation are introduced from jdk 1.0 and annotations for programming are introduced from jdk 1.5. We can apply annotations at different level like class level, Method level, field level, Param level and etc...The annotations will be used by compilers, containers, AnnotationProcessors, JRE and etc...

Some annotations provide instructions in the generation of api documents.

*eg: @Author, @Since and etc...*

Some annotations will provide instruction to compiler but no effect will be there at runtime.

*Eg: @Override (Tells compiler that this method is inherited method of super class)*

Some annotations are useful to mark the classes/interfaces as special items.

*Eg: @Entity- makes ordinary JavaBean as Hibernate domain class.  
@WebServlet - makes class as servlet comp and etc...*

Some annotations are useful to generate new classes using tools dynamically based on given class code.

*Eg: @WebService*

All the java technologies, Frameworks that are designed based on jdk1.5+ are supporting annotations.

*Eg: Servlet 3.X+, hibernate 3.2.X+, Spring2.X+, EJB3.0+, and etc...*

→ If we perform certain settings by using annotations and xml file then the settings done in xml file will override the settings done through annotations.

### Spring annotations: (For core module)

Spring 2.0 onwards there is a support for annotation and more annotations are added incrementally in next versions. In spring programming we need to use both xml and annotations together (most of the cases) because we can instruct IOC container to recognize the used annotations only through xml file. We cannot use annotations to make pre-defined classes as the spring beans because we can't add annotations by opening the source code in those classes. To configure such classes as spring beans we need to work with **xml support**.

#### Spring 2.0

*@Configuration*  
*@Required*  
*@Repository*

#### Spring 2.5

*@Autowired*  
*@Qualifier*  
*@Scope*

#### Stereotype annotations:

*@Component*  
*@Service*  
*@Controller*

#### Spring 3.x

*@Bean*                      *@ComponentScan*  
*@DependsOn*              *@PropertySource*  
*@Lazy*                      *@PropertySources*  
*@Value*  
*@Import*  
*@ImportResource*

Java config annotations (can be used only with spring3.x):

```

@PostConstruct
@PreDestruct
@Resource
@Inject
@Named

```

**@Required:**

To make dependency injection (setter injection) mandatory on properties we can use dependency-check activity while working with xml configurations. But this is removed from spring2.5 because it does not allow the programmer to enable the restriction on specific properties. → It allows the programmer to enable restriction on group of properties like "simple", "objects", "all". To overcome this problem we can use **@Required**.



Spring-aop-RELEASE.jar( also add this jar to IOCLib)

**Robot.java**

```

package com.nt.beans;
import org.springframework.beans.factory.annotation.Required;
public class Robot{
    private int id;
    private String type;
    public void setId(int id){
        this.id=id;
    }
    @Required
    public void setType(String type){
        this.type=type;
    }
    //toString()
    @Override
    public String toString() {
        return "Robot [id=" + id + ", type=" + type + "]";
    }
}

```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<bean id="robot" class="com.nt.beans.Robot">
    <property name="id" value="1001"/>
    <!-- <property name="type" value="Army"/> -->
</bean>
<context:annotation-config/>
// (or)
<bean
class="org.springframework.beans.factory.annotation.RequiredAnnotationBeanPostProcessor"/
>
</beans>
```

**<context:annotation-config>:->** The above tag makes IOC container to recognize the annotation and to provide functionality for the **@Required** annotation.

ClientApp.java:

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.Robot;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC Container
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.
xml");
        //getBean
        Robot robo=ctx.getBean("robot",Robot.class);
        System.out.println(robo);
    }
}
```

The "BeanPostProcessor" of every annotation contains logic to recognize the annotation and to provide functionality for the annotation. But configuring the BeanPostProcessor class of every annotation is complex process. To overcome this problem use **<context:annotation-config/>** tag. To work with this tag import **context** schema namespace in the xml.

@Autowired:

Given to perform byType, byName, constructor mode of autowiring. This annotation can be applied at field level, setter method level, constructor level and arbitrary method level.

-  IOCProj28(@AutoWired ,@Qualifier)
  -  src
    -  com.nt.beans
      - ▷  Engine.java
      - ▷  Viechle.java
    -  com.nt.cfgs
      - ▷  applicationContext.xml
    -  com.nt.test
      - ▷  TestClient.java
  - ▷  JRE System Library [JavaSE-1.7]
  -  IOCLib
    - ▷  spring-beans-4.1.6.RELEASE.jar - CAU
    - ▷  spring-context-4.1.6.RELEASE.jar - CA
    - ▷  spring-context-support-4.1.6.RELEASE
    - ▷  spring-core-4.1.6.RELEASE.jar - CAUse
    - ▷  spring-expression-4.1.6.RELEASE.jar -

**Engine.java:**

```
package com.nt.beans;
public class Engine{
    private int id;
    private String type;
    //setters
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getType() {
        return type;
    }
    public void setType(String type) {
        this.type = type;
    }
    //toString()
    @Override
    public String toString() {
        return "Engine [id=" + id + ", type=" + type + "]";
    }
}
```

**Vehicle.java**

```
package com.nt.beans;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
public class Viechle{
    @Autowired
    @Qualifier("e")
    private Engine engg;

    /* @Autowired(required=true)
    public Viechle(Engine engg){
        System.out.println("1-param constructor");
    }
}
```

```
        this.engg=engg;
    }*/

    /* @Autowired(required=true)
    public void setEngg(Engine engg){
        System.out.println("setEngg");
        this.engg=engg;
    }

    @Autowired(required=true)
    public void assign(Engine engg){
        System.out.println("assign");
        this.engg=engg;
    }*/
}*/
}*/
@Override
public String toString() {
    return "Viechle [engg=" + engg + "]";
}
}
```

**applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-context.xsd">

    <bean id="eng" class="com.nt.beans.Engine">
        <property name="id" value="11"/>
        <property name="type" value="CRDI"/>
        <qualifier value="e"/>
    </bean>
    <bean id="eng1" class="com.nt.beans.Engine">
        <property name="id" value="12"/>
        <property name="type" value="DDIS"/>
        <qualifier value="e1"/>
    </bean>
    <bean id="viechle" class="com.nt.beans.Viechle"/>
    <context:annotation-config/>
</beans>
```

**ClientApp.java:**

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.Viechle;
public class ClientApp {
    public static void main(String[] args) {
```

```
//create IOC Container
ApplicationContext ctx =new
FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
//getBean
Viechle viechle=ctx.getBean("viechle",Viechle.class);
System.out.println(viechle);
}
}
```

While working with <context:annotation-cnfig> tag we need to add "spring-aop<version>"-Release .jar file as an extra jar file to build path.

#### Vehicle.java (at settermethod level)

```
public class vehicle
{
    private Engine engg;
    @Autowired (required=true)
    public void setEngg (Engine engg)
    {
        this.engg=engg;
    }
    .....
    .....
}
```

#### **@Autowired (required=true)**

If dependent bean is not available to perform dependency injection through autowiring then exception will be raised when required is **true**. Default value is "true". Null will be assigned if required="false" and dependent bean configuration is not available. (i.e no exception will be raised)

#### Autowiring by using arbitrary method:

```
public class vehicle
{
    private Engine engg;
    @Autowired (required=true)
    public void assign (Engg engg) //arbitrary method (your choice method)
    {
        this.engg=engg;
    }
    //toString()
    .....
    .....
}
```



**Autowiring by using constructor:**

```
public class Vehicle
{
    private Engine engg;
    @Autowired (required=true)
    public Vehicle (Engine engg)
    {
        this.engg=engg;
    }
    //toString()
    .....
    .....
}
```

**Note:**

We can place @Autowired annotation on the top of only one parameterized constructor at a time. But the same annotation can be applied on multiple fields, setter methods, arbitrary methods at a time.

**Note:** We cannot apply @Autowired on the top of 0-param constructor

→ At max only one constructor can have @Autowired annotation. When @Autowired is placed on the top of field or setter method or arbitrary method then **"byType" mode autowiring** takes place. When the same annotation is placed on top of constructor then **constructor mode autowiring** will take place.

**@Qualifier:**

While performing "byType" mode autowiring there is a possibility of getting ambiguity if multiple dependents are found to inject to target class. In order to overcome this problem we need to use **"@Qualifier" annotation** which makes IOC container to perform "byName" mode of autowiring.

**@Component, @Service, @Repository, @Controller (Stereotype annotations)**

The above annotations are called stereotype annotations, because all the 4 types annotations are having same purpose with minor changes. We can use all the 4 annotations to configure java classes as the spring beans.

**@Component:**

Generic purpose stereotype annotation to configure any java class as spring bean. i.e. exposes given java class as spring bean to IOC container.

**@Service:**

Specialization of @Component, useful to configure the java classes of service layer. As of now no special behavior, so it is same as @Component

**@Repository:**

Specialization of @Component, useful to configure the DAO classes as the Spring bean. Capable of translating one form of exception to another form of exceptions (useful to translate persistence technology exception to another exception).

**@Controller:**

Specialization of @Component, useful to configure classes as the controller of spring mvc app to process request given by clients.

**Note:1)** If annotation parameter name is "value" we can set value/data to that parameter without specifying the parameter name (value).

2)<context:component-scan base-package="-----"/> searches in the given packages to take the classes that are annotated with stereo type annotations as spring beans and also uses some **BeanfactoryPostProcessor** to update the inmemory meta data that is having these loaded classes.It internally contains the behavior of <context:annotation-config/>( So this tag is not required while working with<context:component-scan> tag).

Eg:

- ▲ IOCProj29(@Component,@Value)
  - ▲ src
    - ▲ com.nt.beans
      - ▶ Engine.java
      - ▶ Viechle.java
    - ▲ com.nt.cfgs
      - ▶ applicationContext.xml
    - ▲ com.nt.test
      - ▶ TestClient.java
  - ▶ JRE System Library [JavaSE-1.7]
  - ▲ IOCLib
    - ▶ spring-beans-4.1.6.RELEASE.jar - C:\Use
    - ▶ spring-context-4.1.6.RELEASE.jar - CAU
    - ▶ spring-context-support-4.1.6.RELEASE.j
    - ▶ spring-core-4.1.6.RELEASE.jar - CAUser
    - ▶ spring-expression-4.1.6.RELEASE.jar - C
    - ▶ commons-logging.jar - C:\Users\delinc

#### Vehicle.java:

```
package com.nt.beans;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
@Component
public class Viechle{
    @Autowired
    private Engine engg;
    //toString
    @Override
    public String toString() {
        return "Viechle [engg="+ engg + "]";
    }
}
```

#### Engine.java:

```
package com.nt.beans;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
@Component("eng")
public class Engine{
    @Value("1001")
    private int id;
    @Value("DDIS")
    private String type;
    //setters
```

```
public void setId(int id) {
    this.id = id;
}
public void setType(String type) {
    this.type = type;
}
//toString()
@Override
public String toString() {
    return "Engine [id=" + id + ", type=" + type + "]";
}
}
```

**applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<context:component-scan base-package="com.nt.beans"/>
</beans>
```

**ClientApp.java:**

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.Viechle;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC Container
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //getBean
        Viechle viechle=ctx.getBean("viechle",Viechle.class);
        System.out.println(viechle);
    }
}
```

**@Value:**

It is given to configure dependent values for simple properties of dependency injection. Also useful to inject the values collected from the properties file

### Java config annotations

**@Componet**, **@Autowired** and etc. are spring supplied annotations, if we use those annotations the code of spring bean becomes tightly coupled code to spring framework. This kills non-invasive feature of spring. To overcome this problem use java config annotations given by JSE and JEE modules. For these annotations functionality will be given by the containers and frameworks that we are using. For Example For **@Inject** annotation spring provides it's functionalities and EJB provides it's functionalities. So if our code moves from one framework to another framework we can still continue with the **same old java config annotations**.

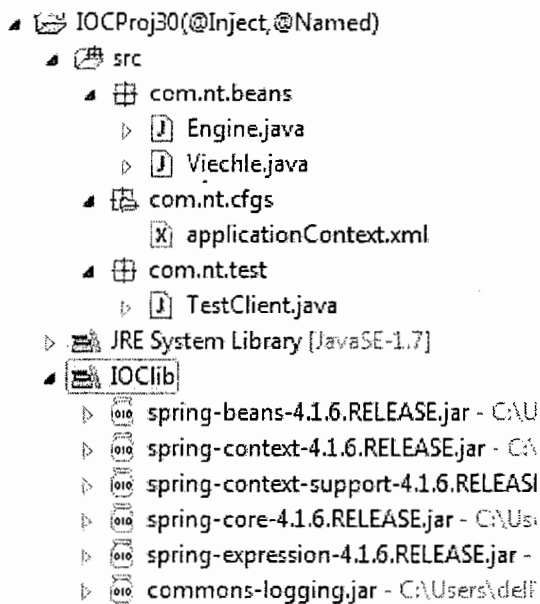
These annotations are part of JSE and JEE specifications but implementation will be provided by different servers and frameworks. Even though we change runtime libraries of Application we can still use same annotations in the classes.

Ex: **@Inject**, **@Named**, **@Resource**, **@PostConstruct**, **@PostDestory** and etc..

#### @Inject:

It is given in JEE specification in javax.inject package having ability of performing byname/byType/constructor mode of autowiring. This is similar to **@Autowire** but the differences are →

- a) No "required" parameter in **@Inject**
- b) **@Inject** is java config annotation, so non-invasive spring application can be developed.



- Add Spring-aop-<version>.jar+javcx.inject.jar (collect from internet).

**Vehicle.java:**

```
package com.nt.beans;
import javax.annotation.Resource;
import javax.inject.Inject;
import javax.inject.Named;
@Named("viechle")
public class Viechle{
    private Engine engg;
    /* @Inject
    public Viechle(Engine engg){
        this.engg=engg;
    }*/
    @Inject
    @Named("eng1")
    public void assign(Engine engg){
        this.engg=engg;
    }
    /* @Inject
    public void setEngg(Engine engg){
        this.engg=engg;
    }*/
    //toString
    @Override
    public String toString() {
        return "Viechle [engg=" + engg + "]";
    }
}
```

**Engine.java**

```
package com.nt.beans;
import javax.inject.Named;
import org.springframework.beans.factory.annotation.Value;
@Named("eng")
public class Engine{
    @Value("1001")
    private int id;
    @Value("DDIS")
    private String type;
    //setters
    public void setId(int id) {
        this.id = id;
    }
    public void setType(String type) {
        this.type = type;
    }
    //toString()
    @Override
    public String toString() {
        return "Engine [id=" + id + ", type=" + type + "]";
    }
}
```

applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<!-- <bean id="eng1" class="com.nt.beans.Engine">
    <property name="id" value="1001"/>
    <property name="type" value="DDIS"/>
</bean>
<bean id="eng2" class="com.nt.beans.Engine">
    <property name="id" value="1002"/>
    <property name="type" value="CRDI"/>
</bean> -->
<context:component-scan base-package="com.nt.beans"/>
</beans>
```

ClientApp.java:

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.Viechle;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC Container
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //getBean
        Viechle viechle=ctx.getBean("viechle",Viechle.class);
        System.out.println(viechle);
    }
}
```

@Inject can be applied at field/method (setter or arbitrary) method/constructor level. To work with @Inject, @Named kind of java config annotations we need to add 'javax.inject.jar' files.

@Named:

→ It is a java config annotation of 'javax.inject' package (JEE specification). This annotation can be used in 2 ways

a) To resolve ambiguity problems that raised while performing byType mode of autowiring (alternate to @Qualifier) can be used along with @Inject, @Resource, @Autowired.

b) To configure/mark java classes as spring beans as alternate to spring specific annotations like @Component and etc...

Eg:

• Vehicle.java:

```

package com.nt.beans;
@Named("vehicle")//bean configuration
public class Vehicle{
@Inject
@Named("eng1")//to perform byName mode autowiring
private Engine engg;
//override toString()
.....
.....
}

```

**Engine.java:**

```

package com.nt.beans;
public class Engine{
private int id;
private String type;
//setters
.....
.....
//toString
}

```

**applicationContext.xml:**

```

<bean id="eng1" class="com.nt.beans.Engine">
.....
</bean>
<bean id="eng2" class="com.nt.beans.Engine">
.....
</bean>
<context:component-scan base-package="com.nt.beans"/>

```

**@Resource**

- It is jdk supplied java config annotations of javax.annotations package
- Can be used to perform byType, byName mode of Autowiring.
- Can be applied at field/method (setter/arbitrary) level.
- By using "name" param of this annotation we can perform byName mode of autowiring.

```

IOCProj31(@Resource, @Scope)
├── src
│   ├── com.nt.beans
│   │   ├── Engine.java
│   │   └── Vehicle.java
│   ├── com.nt.cogs
│   │   └── applicationContext.xml
│   └── com.nt.test
│       └── TestClient.java
├── JRE System Library [JavaSE-1.7]
└── IOCLib
    ├── spring-beans-4.1.6.RELEASE.jar - C:\Users\...
    ├── spring-context-4.1.6.RELEASE.jar - C:\Users\...
    ├── spring-context-support-4.1.6.RELEASE.jar - C:\Users\...
    ├── spring-core-4.1.6.RELEASE.jar - C:\Users\...
    ├── spring-expression-4.1.6.RELEASE.jar - C:\Users\...
    └── commons-logging.jar - C:\Users\...

```

**Vehicle.java:**

```
package com.nt.beans;
import javax.annotation.Resource;
import javax.inject.Named;
import org.springframework.context.annotation.Scope;
@Named("viechle")
@Scope("prototype")
public class Viechle{
    @Resource
    private Engine engg;
    public Viechle() {
        System.out.println("Viechle:0-param constructor");
    }
    /* @Resource
    public void assign(Engine engg){
        this.engg=engg;
    }*/
    //@Resource(name="eng1")
    public void setEngg(Engine engg){
        this.engg=engg;
    }
    //toString
    @Override
    public String toString() {
        return "Viechle [engg=" + engg + "]";
    }
}
```

**Engine.java:**

```
package com.nt.beans;
import javax.inject.Named;
import org.springframework.beans.factory.annotation.Value;
public class Engine{
    private int id;
    private String type;
    //setters
    public void setId(int id) {
        this.id = id;
    }
    public void setType(String type) {
        this.type = type;
    }
    //toString()
    @Override
    public String toString() {
        return "Engine [id=" + id + ", type=" + type + "]";
    }
}
```



applicationContext.xml:

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<bean id="eng1" class="com.nt.beans.Engine">
  <property name="id" value="1001"/>
  <property name="type" value="DDIS"/>
</bean>
<context:component-scan base-package="com.nt.beans"/>
</beans>
```

ClientApp.java:

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.Viechle;
public class ClientApp {
  public static void main(String[] args) {
    //create IOC Container
    ApplicationContext ctx =new
    FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
    //getBean
    Viechle viechle=ctx.getBean("viechle",Viechle.class);
    System.out.println(viechle);
    ctx.getBean("viechle");
    ctx.getBean("viechle");

  }
}
```

Differences between @Autowired, @Inject and @Resource

| <u>@Autowired</u>                                | <u>@Inject</u>         | <u>@Resource</u>                               |
|--------------------------------------------------|------------------------|------------------------------------------------|
| ->Spring Annotation                              | JEE java configuration | JDK java config annotation                     |
| ->Supports byType,<br>byName,constructor         | same                   | Supports byType, byname<br>mode of autowiring. |
| → Can be applied on field,<br>Method,constructor | same                   | can be applied on field,<br>method.            |

→ Makes spring class as  
invasive

non-invasive class

non-invasive class

→ param "required" is  
available

no-param

"param" name is available.

Use @Qualifier or @Name to  
resolve ambiguity problem.

same

We can use name param resolve ambiguity  
problem.

### @Scope

This is spring annotation which can be applied on class level to specify spring bean scope.

@Named("vehicle")

@Scope("Singleton")

public class vehicle

{

.....

.....

}

If the annotation param name is "value" then we can place the content of that param without specifying param name.

→ @Named(Value="Vehicle") is equal to @Named ("vehicle")

Note: As of now spring is providing functionality only to few java config annotations.

### @PostConstruct, @PreDestroy (Java Config annotations)

Bean lifecycle configuration can be done in 3 ways:

a) Declarative Approach

(By using init-method, destroy-method attributes of <bean> tag)

b) Programmatic Approach

(By implementing InitializationBean, DisposableBean interfaces on Spring Bean class)

c) Annotation Approach

(By using javaconfig annotations @PostConstruct, @PreDestroy)

#### @PostConstruct:

To configure normal method as custom init method

#### @PreDestroy:

To configure normal method as custom destroy method

Note: Both method should have the following signature

```

public void <method> (){
    -----
    -----
}

```

IOCProj32(@PostConstruct,@PreDestroy)

src

com.nt.beans

Voter.java

com.nt.cfgs

applicationContext.xml

com.nt.test

TestClient.java

JRE System Library [JavaSE-1.7]

IOCLib

spring-beans-4.1.6.RELEASE.jar - CAUser

spring-context-4.1.6.RELEASE.jar - CAU

spring-context-support-4.1.6.RELEASE,

spring-core-4.1.6.RELEASE.jar - CAUser

spring-expression-4.1.6.RELEASE.jar - C

commons-logging.jar - C:\Users\delhi

➤ Add javax.inject-<ver>.jar

➤ Add spring-aop-<ver>.jar

### Voter.java:

```

package com.nt.beans;
import javax.annotation.PostConstruct;
import javax.annotation.PreDestroy;
import javax.inject.Named;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Scope;
@Named("vt")
@Scope("singleton")
public class Voter {
    @Value("raja")
    private String name;
    @Value("30")
    private int age;
    @PostConstruct
    public void myInit(){
        System.out.println("Voter:myInit()");
        if(name==null || age<=0){
            throw new IllegalArgumentException(" name,age must be set");
        }
    }
    @PreDestroy
    public void myDestroy(){
        System.out.println("Voter:myDestroy()");
    }
}

```

```
    name=null;
    age=0;
}
public String checkVotingEligibility(){
    if(age<18)
        return name+" u r not eligible to vote";
    else
        return name+"u r eligible to vote";
}
}
```

**applicationContext.xml:**

```
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-context.xsd">
    <context:component-scan base-package="com.nt.beans"/>
</beans>
```

**ClientApp.java**

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.Voter;
public class ClientApp {
    public static void main(String[] args) {
        // create IOC Container
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get Bean
        Voter vt=ctx.getBean("vt",Voter.class);
        String status=vt.checkVotingEligibility();
        System.out.println("Result: "+status);
        ((FileSystemXmlApplicationContext)ctx).close();
    }
}
```

**Working with properties file in annotations environment**

- a) To detect the classes and to recognize them as spring beans through annotations we can use **<context:component-scan>**tag
- b) To detect the specified 'properties' file that is configured through annotation we can use **<context:property-placeholder>**tag
- c) To inject the values of 'properties' files to "bean properties" we need to specify the keys of

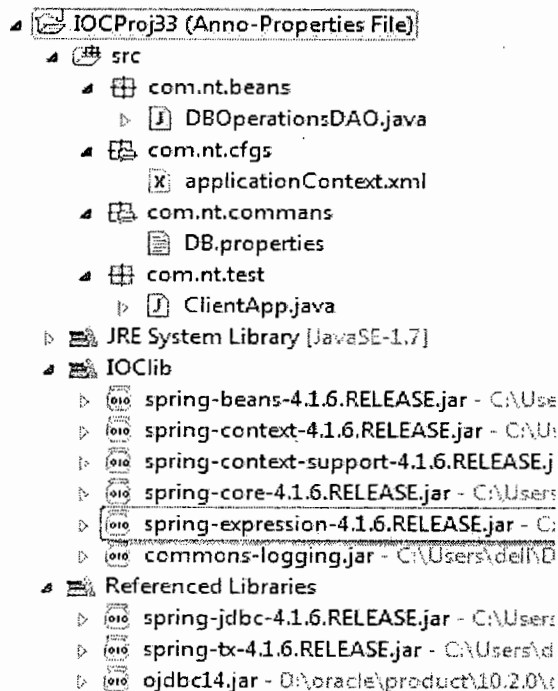
'properties' file through placeholder `${<key>}` with the support of `@Value`.

d) In `<context:property-placeholder location="-----"/>` tag we can specify multiple properties files locations as the (,) separated values in `location` attribute.

e) `<context:property-placeholder />` tag internally uses "PropertyPlaceholderConfigurer" class to locate the properties file and to recognize placeholders (`${---}`).

f) `PropertyPlaceholderConfigurer` class not only looks to get given property value from properties file and it can also get it from java system properties

```
@Value("${java.v4.specification.vendor}")
private String vendor;
```



#### DBOperations.java:

```
package com.nt.beans;
import javax.inject.Named;
import org.springframework.beans.factory.annotation.Value;
@Named("dao")
public class DBOperationsDAO {
    @Value("${jdbc.driver}")
    private String driver;
    @Value("${jdbc.url}")
    private String url;
    @Value("${jdbc.user}")
    private String username;
    @Value("${jdbc.pwd}")
    private String password;
    @Override
    public String toString() {
```

```

        return "DBOperationsDAO [driver=" + driver + ", url=" + url
            + ", username=" + username + ", password=" + password + "];"
    }
}

```

**applicationContext.xml:**

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
    <context:component-scan base-package="com.nt.beans"/>
    <context:property-placeholder location="src/com/nt/commons/DB.properties"/>
</beans>

```

**DB.properties**

```

jdbc.driver=oracle.jdbc.driver.OracleDriver
jdbc.url=jdbc:oracle:thin:@localhost:1521:xe
jdbc.user=scott
jdbc.pwd=tiger

```

**ClientApp.java:**

```

package test;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.beans.DBOperationsDAO;

public class ClientApp {
    public static void main(String[] args) {
        //Create IOC Container
        ApplicationContext ctx = new
            FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get Bean
        DBOperationsDAO dao = ctx.getBean("dao", DBOperationsDAO.class);
        System.out.println(dao);
    }
}

```

**Note:** working with <context: property-placeholder....> tag is equal to working with 'PropertyPlaceholderConfigurer' Bean class of Spring API.

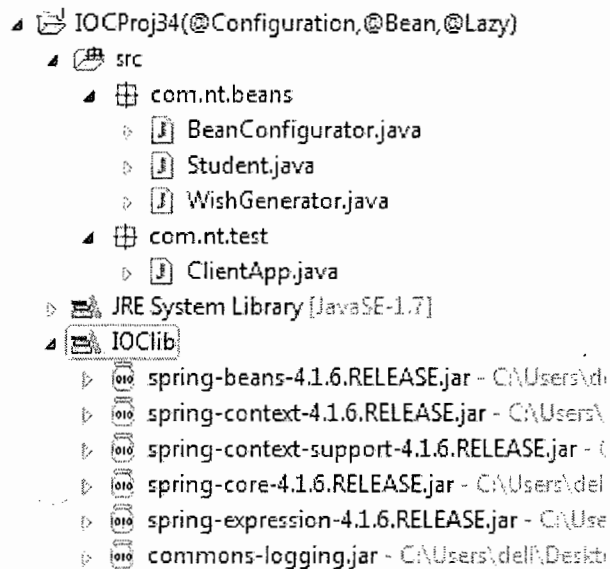
### **Developing spring core module application without xml file/(100% java code based spring core module app):**

For this we need to use "AnnotationConfigApplicationContext" to create core IOC container by specifying "Configuration" class. The java class that contains methods to create and return bean class objects is called "Configuration" class. We use @Configuration to configure ordinary class as configuration class and we use @Bean on the top the configuration class methods which produces

spring bean class objects (to configure these objects as bean objects use @Bean).

Note:

- 1) 100% java code approach improves the performance because of no XML file configuration and also improves the readability.
- 2) We can take configuration class that is annotated with @Configuration as alternate to spring bean configuration file.
- 3) We place @Bean annotated methods in the configuration class to create and return predefined class objects and to make them as spring beans. But we can configure user defined classes as spring beans by using @Named, @Component annotations. We link them to configuration class by using @ComponentScan annotation.



#### BeanConfigurator.java:

```
package com.nt.config;
import java.util.Calendar;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.DependsOn;
import org.springframework.context.annotation.Scope;

@Configuration
@ComponentScan(basePackages="com.nt.beans")
public class BeanConfigurator {
    static{
        System.out.println("BeanConfigurator:static block");
    }

    public BeanConfigurator() {
        System.out.println("BeanConfigurator:0-param constructor");
    }
}
```

```
@Bean(name="cal")
@Scope("singleton")
@DependsOn("wish")
public Calendar createCalendar(){
    System.out.println("BeanConfigurator:createCalendar()");
    return Calendar.getInstance();
}
}
} //class
```

**WishGenerator.java:**

```
package com.nt.beans;
import java.util.Calendar;
import javax.annotation.Resource;
import javax.inject.Named;

@Named("wish")
public class WishGenerator {
    @Resource
    private Calendar calendar;

    public String generateWishMsg(String user){
        int hour=0;
        //get current hour of the day
        hour=calendar.get(Calendar.HOUR_OF_DAY); // 24 hrs format
        //generate wish msg
        if(hour<=12)
            return " Good Morning::"+user;
        else if(hour<=16)
            return "Good After noon::"+user;
        else if(hour<=20)
            return "Good Evening::"+user;
        else
            return "Good night::"+user;
    }
} //method
}
```

**ClientApp.java:**

```
package com.nt.test;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import com.nt.beans.WishGenerator;
import com.nt.config.BeanConfigurator;

public class AnnotationConfigTest {

    public static void main(String[] args) {
        AnnotationConfigApplicationContext ctx=null;
        WishGenerator wish=null;
        //create IOC Container
```





Even though bean scope is singleton, if you want to delay bean instantiation until `getBean(-,-)` is called in `ApplicationContext` container use **@Lazy** annotation/**Lazy="true"** of `<bean>` in xml.

While working in 100% code based spring application we can take configurator class specifying **@ComponentScan** when all beans are user defined classes.

In 100% code based spring application development we can't use **PropertyPlaceholderConfigurer** to recognize and use placeholders in configuration class itself. But it will recognize and use placeholders specified in spring beans in the **@Value** annotation.

## Spring JDBC/DAO

**Where should we use jdbc and where should we use hibernate?**

Ans: While working with jdbc, if you want to process huge amount of records then we need to deal with single jdbc ResultSet object. While working with hibernate, if we want to process huge amount of records then we need to take multiple objects representing multiple records. This is a problem and it may crash the system, while creating huge amount of objects.

**Use jdbc generally for offline applications that deals with huge amount of data and comes to our App batch by batch and where we don't need database portability feature. (changing DB software without modifying the code).**

Eg: Storing and managing census details, political party membership details by performing data validation and also correction.

**While developing online applications (Internet applications) that deals with limited amount of personalized data then use Hibernate.**

Eg: Online shopping mobile recharge online etc...

**While developing internet application that performs batch processing of huge amount of data then use jdbc technology.**

Eg: Courier Application that takes all orders and process at a time.

Eg: mobile number activation application that activates set of numbers at a time.

Spring JDBC provides abstraction layer on plain JDBC and simplifies persistence logic development. Instead of using plain JDBC in offline applications that process huge amount of data batch by batch, Intranet applications that deals with huge amount of data through batch processing it is recommended to use **spring JDBC instead of Plain JDBC**.

***Irrespective of whether application is offline/online use JDBC/spring JDBC to process huge amount of data that comes batch by batch and similarly use hibernate to process limited amount of data.***

### Plain JDBC code:

- Register JDBC driver
- Establish the connection
- Create Statement
- Send and execute SQL query
- Gather Results and process Results
- Exception handling
- Close JDBC objects

Here we need to write both common logics and applications specific logics in the application, so it raises boiler plate code problem. The code repeats in multiple parts of the Application either with no changes or with minor changes is **called boiler-plate code**

**Limitations with plain JDBC:**

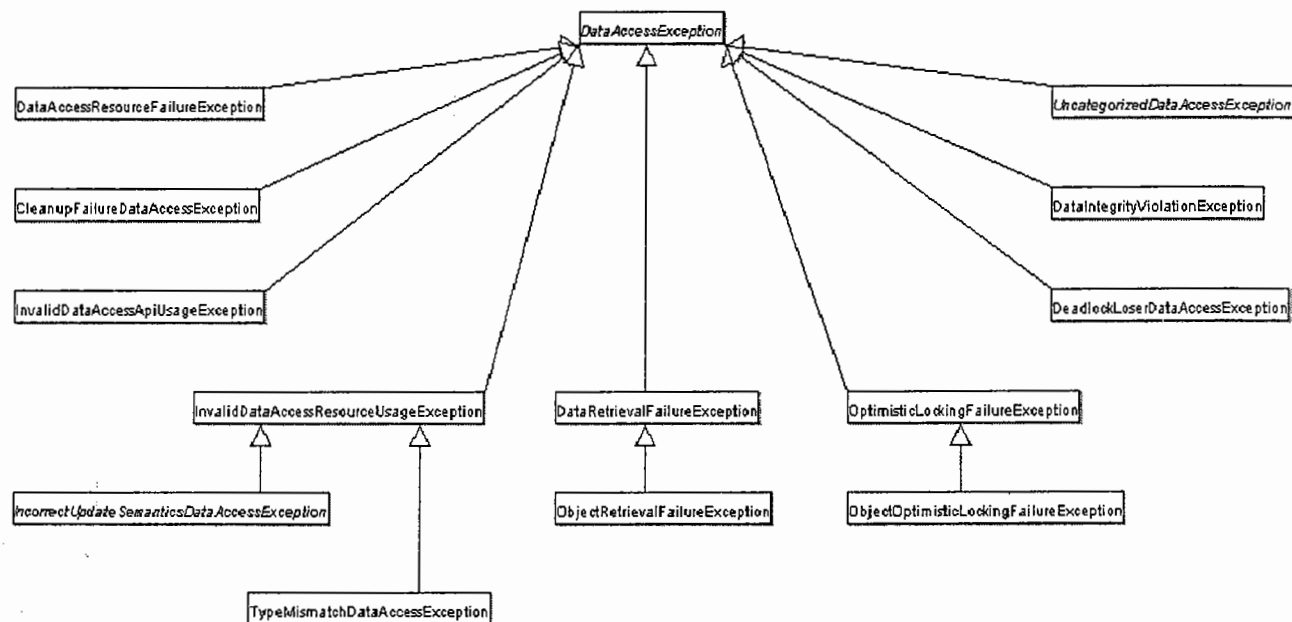
- Boiler plate code problem
- Exception handling is mandatory
- Processing Resultset to get result in desired type is complex process.
- No support for proper Transaction management.

**Spring JDBC App:**

- Configure and get `JdbcTemplate` class object
- Send and execute SQL query.
- Gather results and process results

**Note:**

Here we need to take care of only application specific logics of development. solves boiler plate code problem

**Spring JDBC/DAO Advanatges:**

- Solves Boiler plate code problem
- Makes programmer to take care of only application specific logics as shown above.
- Raises unchecked Exceptions (Sub classes of `DataAccessException` class ) so the exception is optional to catch and handle. It internally uses exception rethrowing concepts to convert `checkedException` (`SQLException`) into unchecked exception.
- Gives various methods to get the records of `ResultSet` in desired types.
- Takes care of opening connection and close connection activities.
- Support both Named and positional params in **SQL query**.

Allows to use callback interface to develop more customized persistence logic by using plain JDBC code.

| Action                                       | Programmer | Spring JDBC |
|----------------------------------------------|------------|-------------|
| Configuring DataSource                       | Yes        |             |
| Creating/getting connection                  |            | Yes         |
| Creating Statement                           |            | Yes         |
| Preparing SQL query with Params              | Yes        |             |
| Sending & executing SQL query in DB software | Yes        |             |
| Iterating through ResultSet (if needed)      |            | Yes         |
| Exception Handling                           |            | Yes         |
| Transaction management                       |            | Yes         |
| Closing JDBC objects                         |            | Yes         |

#### Spring JDBC api pkgs

→ *org.springframework.jdbc and its sub pkgs*  
 → *org.springframework.dao and its sub pkgs*  
 (this package contains only Exceptions)

Note: Spring JDBC/ORM module internally uses multiple other technologies/Frameworks to develop persistence logics, but translates the exceptions of underlying technology/framework to spring's **DataAccessException** class hierarchy.

There are 5 methodologies/approaches to develop persistence logic in spring JDBC.

- 1) Using JdbcTemplate
- 2) Using NamedParameterJdbcTemplate
- 3) Using SimpleJdbcTemplate (removed from spring 4.0 onwards)
- 4) SimpleJdbcInsert, SimpleJdbcCall
- 5) Modeling JDBC operations as Java objects (Mapping SQL operations as subclasses using SQLQuery, SqlUpdate classes. (JDO Style))

While working with any technique of spring JDBC we need to use JDBC connection to interact with database software. It is recommended to use JDBC connection pool based JDBC connection instead of direct JDBC connection. Do not use spring's built-in classes like **DriverManagerDataSource**, **SingletonConnectionDataSource** and etc... to use JDBC connection

pool in spring App because those classes are actually doesn't pool the connection objects, more over it creates JDBC connection objects in jdbc con pool on demand basis.

**What is the JDBC connection pool that should be used in our spring application?**

**Ans:**

- 1) If our spring app is standalone app running outside the servers then use third party JDBC connection pool like **c3p0** or **apacheDBCP** and etc..
- 2) If our spring app is deployable in server like web application then use **server managed JDBC connection pool**

### Using JdbcTemplate

This is central class of spring JDBC provides abstraction layer on plain JDBC and simplifies persistence logic development. This class takes care of common workflow activities of JDBC programming like opening connection, creating statement, closing connectin and etc..... but makes the programmer to just provide SQL query and to extract results. **This class translates JDBC generated SQLExceptions into spring specific unchecked exceptions like DataAccessException.**

This allows the programmer to avoid common errors in application development like forgetting to close JDBC objects. To create JdbcTemplate class object we need JDBC DataSource object. We generally configure JdbcTemplate in spring bean configuration file and use injection concept to inject that object to DAO class.

**persistence –beans.xml:**

```
<bean id="drds" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
.....
.....
</bean>
<bean id="template" class="org.springframework.jdbc.support.JdbcTemplate">
  <property name="dataSource" ref="drds"/>
</bean>
```

**There are 2 approaches of working with JdbcTemplate**

- 1) Using direct methods (No need of JDBC code)
- 2) Using callback interfaces (Need to write plain JDBC code to place customized persistence logic)

### Working with direct methods of JdbcTemplate class.

**Select-Operations:** (query(), queryForXxx(-) methods)

→ queryForInt(), queryForLong()

To execute query that gives single numeric value as result.

Eg: **Select count(\*) from student;**

queryForMap():

→ To execute the query that gives single record.

Eg: **Select \* from student where sno=1001;**

**queryForObject()**

→To execute the query and to get result into predefined or **BO class object**.

Eg: Select \* from student where sno=1001;

**queryForList()**

→To execute query that gives multiple records.

Eg: Select \* from student;

**queryForRowSet()**

→To execute query that gives JDBC RowSet object and etc...

**Note:**If the above methods are called with one argument value then they internally uses JDBC simple Statements object.

**Note:**If the above methods are called with 2 or 3 arguments then they internally uses JDBC PreparedStatement object.

**Non-Select operations**

update (-) methods

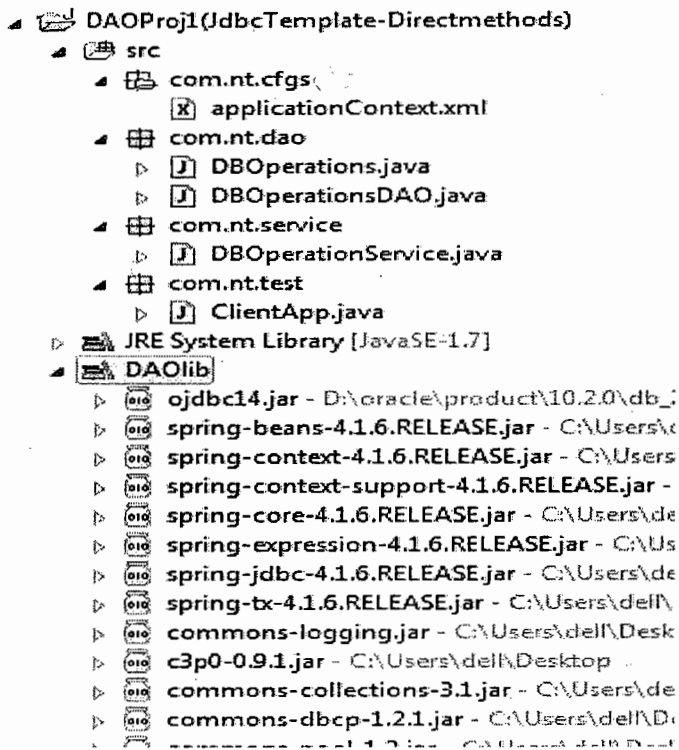
**batch updation**

batchUpdate (-) and many more

**Note:**Spring DAO internally uses Exception rethrowing concept to translate underlying Jdbc persistence technology generated **SQLException** to **DataAccessExceptions**.

Ex:-

```
public int update(String query, String ... args){
    try{
        -----//Jdbc code
        -----
    }catch(SQLIntegrityConstraintViolationException scve){
        throw new DuplicateKeyException();
    } catch(SQLException se){
        throw new DataAccessException();
    }
}
```



### DBOperations.java

```
package com.nt.dao;
import java.util.List;
import java.util.Map;
public interface DBOperations {
    public int insert(int no,String name,String job,long sal);
    public int getSalary(int no);
    public Map<String,Object> listEmpDetails(int no);
    public List<Map<String,Object>> listAllEmpDetails();
    public int UpdateEmpSalary(int no,int newsal);
    public int deleteEmp(int no);
}
```

### DBOperationsDAO.java

```
package com.nt.dao;
import java.util.List;
import java.util.Map;
import org.springframework.jdbc.core.JdbcTemplate;
public class DBOperationsDAO implements DBOperations {
    private static final String INSERT_QRY="INSERT INTO EMP(EMPNO,ENAME,JOB,SAL)
values(?,?,?,?)";
    private static final String GET_EMP_SALARY="SELECT SAL FROM EMP WHERE EMPNO=?";
    private static final String GET_EMP_DETAILS_BY_NO="SELECT EMPNO,ENAME,JOB,SAL
from EMP where EMPNO=?";
    private static final String GET_ALL_EMP_DETAILS="SELECT EMPNO,ENAME,JOB,SAL
FROM EMP";
    private static final String UPDATE_EMP_SALARY="UPDATE EMP SET SAL=? WHERE
```



```
EMPNO=?";
    private static final String DELETE_EMP="DELETE FROM EMP WHERE EMPNO=?";
    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    @Override
    public int insert(int no, String name, String job, long sal) {
        int result=jt.update(INSERT_QRY, no,name,job,sal);
        return result;
    }
    @Override
    public int getSalary(int no) {
        int salary=jt.queryForInt(GET_EMP_SALARY,no);
        return salary;
    }
    @Override
    public Map<String,Object> listEmpDetails(int no) {
        Map<String,Object> map=jt.queryForMap(GET_EMP_DETAILS_BY_NO,no);
        return map;
    }
    @Override
    public List<Map<String,Object>> listAllEmpDetails() {
        List<Map<String,Object>> list=jt.queryForList(GET_ALL_EMP_DETAILS);

        return list;
    }
    @Override
    public int updateEmpSalary(int no, int newsal) {
        int result=jt.update(UPDATE_EMP_SALARY,newsal,no);
        return result;
    }
    @Override
    public int deleteEmp(int no) {
        int result=jt.update(DELETE_EMP,no);
        return result; }}
}
```

#### DBOperationService.java

```
package com.nt.service;
import java.util.List;
import java.util.Map;
import com.nt.dao.DBOperationsDAO;
public class DBOperationService {
    private DBOperationsDAO dao;
    public void setDao(DBOperationsDAO dao) {
        this.dao = dao;
    }
    public String registerEmp(int no,String name,String job,int sal)
    {
    }
```

```
int result=dao.insert(no, name, job, sal);
if(result==0)
    return no+" Emp is not registered";
else
    return no+" Emp is registered";
}
public int showSalary(int no){
int salary=dao.getSalary(no);
return salary;
}
public String hikeSalary(int no){
//write B.logic
//get Salary
int salary=showSalary(no);
int newsal=0;
if(salary>=50000)
    newsal=Math.round(salary+(salary*0.1f));
else
    newsal=Math.round(salary+(salary*0.2f));
//use DAO
int result=dao.updateEmpSalary(no, newsal);
if(result==0)
    return no+" Emp Salary not hiked";
else
    return no+"Emp Salary hiked";
}
public Map<String,Object> getEmpDetails(int no)throws Exception{
Map<String,Object> map=dao.listEmpDetails(no);
if(map!=null)
    return map;
else
    throw new Exception(no+" Emp not found");
}
public List<Map<String,Object>> getAllEmpDetails()throws Exception{
List<Map<String,Object>> list=dao.listAllEmpDetails();
if(list!=null)
    return list;
else
    throw new Exception("Employees not found");
}
public String fireEmp(int eno){
int result=dao.deleteEmp(eno);
if(result==0)
    return eno+" Employee not found";
else
    return "Employee deleted";
}
}
```

**applicationContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.3.0.xsd">
<!-- <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
  <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
  <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
  <property name="username" value="scott"/>
  <property name="password" value="tiger"/>
</bean> -->

<bean id="dataSource" class="com.mchange.v2.c3p0.ComboPooledDataSource">
  <property name="driverClass" value="oracle.jdbc.driver.OracleDriver"/>
  <property name="jdbcUrl" value="jdbc:oracle:thin:@localhost:1521:xe"/>
  <property name="user" value="scott"/>
  <property name="password" value="tiger"/>
  <property name="initialPoolSize" value="10"/>
  <property name="maxPoolSize" value="20"/>
</bean>

<bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
  <property name="dataSource" ref="dataSource"/>
</bean>

<bean id="DBODao" class="com.nt.dao.DBOperationsDAO">
  <property name="jt" ref="template"/>
</bean>

<bean id="DBOService" class="com.nt.service.DBOperationService">
  <property name="dao" ref="DBODao"/>
</bean>
</beans>
```

**ClientApp.java**

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.DBOperationService;
public class ClientApp {
    public static void main(String[] args) throws Exception{
        //Create IOC Container
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get Service Bean
        DBOperationService service=(DBOperationService)ctx.getBean("DBOService");
        // Call B.methods
        System.out.println("7499 Emp Details"+service.getEmpDetails(7499));
        System.out.println("All Emp Details"+service.getAllEmpDetails());
        System.out.println("7499 Emp Salary(Before hike)+"service.showSalary(7499));
        System.out.println("7499 Emp Salary hiking..."service.hikeSalary(7499));
```

```

        System.out.println("7499 Emp Salary(After hike)+"service.showSalary(7499));
        //System.out.println(service.registerEmp(9001,"Raja","progammer",8999));
        //System.out.println("Fire Emp"+service.fireEmp(9001));
    } //main
} //class

```

Jar files: IOCLib jar files+object 14/6.jar+org.springframework.jdbc<ver>.jar

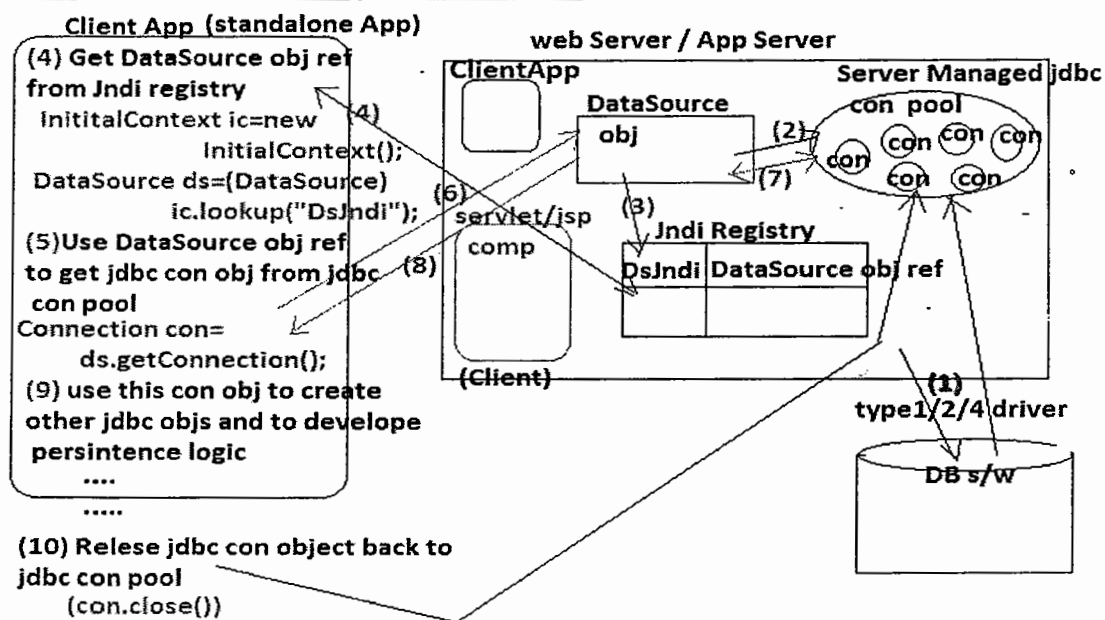
Commons-pool.jar+commons-dbcp.jar+org.springframework.+n<ver>.jar (from spring 2.5 software) or c3p0-0.9.1.2.jar (To work with c3p0 pool)

### JNDI Registry and ServerManaged Jdbc connection pool

To provide global visibility to object or object reference, it will be placed in **jndi registry** having nickname/jndi name. Every webserver/App server gives one built-in jndi registry. In server managed JDBC connection pool environment the JDBC con pool will be represented by DataSource object and its reference will be placed in jndi registry of server for global visibility having jndi name.

The apps deployed in server or running from outside the server can access, DataSource object reference from jndi registry to get JDBC connection Object from JDBC connection Pool. In one server we can create multiple JDBC connection pool belonging to same or different database softwares. All connection objects in connection pool represents connectivity with same database software. The client application that wants to use connection pool can be there outside the server as standalone application or can be there inside the server as deployable servlet component, jsp component, EJB component.

### Understanding Server Managed JDBC Con pool



### With respect to diagram:

- (1) TL or PL or Admin makes server to use type1/2/4 JDBC driver to interact with database software and to create servermanaged JDBC connection pool having connection objects.(2)&(3)
- (2) TL/PL/Admin creates DataSource object representing JDBC connection pool places that DataSource object reference in jndi registry having jndi name for global visibility.

→ In weblogic software we can create multiple domains and each domain acts as one application server. Multiple projects of a company use multiple domain servers of weblogic software on one per project basis i.e. the weblogic software will be installed only for one time in a common computer and in that software multiple domains will be created for multiple projects on one per project basis.

**Procedure to create user-defined domains server in weblogic 12c:**

Start--->Oracle----->oracle home----->weblogic server12c----->tools----->configurationwizard--->create new domain (domain location:..... sp27Domain)----->next----->next  
Name: javaboss  
Pass: javaboss1  
Confirm-pass:javaboss1  
next--->next--->select admin server--->next--->proj:7080----->create--->next----->finish.

**Procedure to create JDBC DataSource pointing JDBC con pool for oracle in "ntsp67Domain" server of weblogic:**

**a) start ntsp27Domain server**

D:\weblogic 12c\user\_projects\Domains\ntsp27Domain\startweblogic.cmd

**b) Open admin console of that domain server**

<http://localhost:7080/console>----->

username: javaboss

password: javaboss1 -> login

**(c) Create a JDBC DataSource**

Adminconsole--->services----->datasources--->new--->generic datasource

Name: ds1

Jndi name: Dsjndi

DatabaseType: Oracle

Driver: Oracle's Driver (thin) for service connection

Next--->next

Database name: xe

Hostname : localhost

Port: 1521

Db username: Scott

Password: tiger

Confirm pass: tiger ----->Test Configuration--->next

Select admin Server----->Finish

Launch

Connection pool tab--->Advanced---->

Min:3

Max: 30

Save

**Advanced:**

**Shrink frequency : 900** (specifies the idle connection objects of JDBC connection pool will be destroyed automatically for every 900sec)

----->save

→ Spring gives 1 built-in FactoryBean class to get object from jndi registry based on given jndi name.

This FactoryBean class name is **org.springframework.jndi.JndiObjectFactoryBean**

Eg: in bean Configuration file.

```

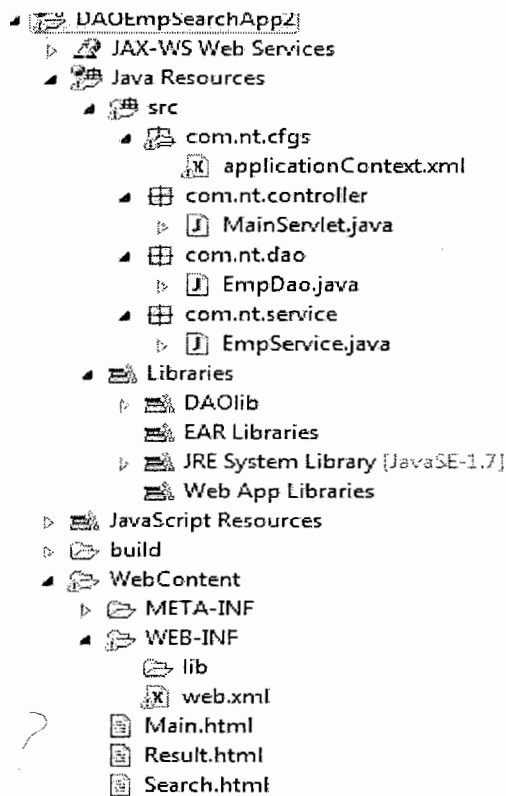
<bean id="jofb" class="org.springframework.jndi.JndiObjectFactoryBean">
  <property name="JndiName" value="DsJndi"/>
</bean>
<bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
  <property name="dataSource" ref="jofb"/>
</bean>

```

Since the above bean is FactoryBean and the gathered DataSource object will be injected here.

### Web Application by using mvc2,DAO design patters:

The below application is web application so it is deployed in server so use server managed JDBC con pool.



### EmpDAO.java

```

package com.nt.dao;
import java.util.List;
import java.util.Map;
import org.springframework.jdbc.core.JdbcTemplate;
public class EmpDao {
    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    public List getEmpDetails(String condition){
        List<Map<String,Object>> list=jt.queryForList("SELECT EMPNO,ENAME,JOB,SALE FROM
EMP WHERE JOB IN "+condition);

```

```
    return list;
}
}
```

### EmpService.java

```
package com.nt.service;
import java.util.List;
import java.util.Map;
import com.nt.dao.EmpDao;
public class EmpService {
    private EmpDao dao;

    public void setDao(EmpDao dao) {
        this.dao = dao;
    }

    public List search(String job[]){
        //prepare condition
        StringBuffer sb=new StringBuffer("");
        for(int i=0;i<job.length;++i){
            if(i==job.length-1){
                sb.append(""+job[i]+"");
            }
            else{
                sb.append(""+job[i]+",");
            }
        }
        //for
        sb.append("");
        String condition=sb.toString();
        //Use DAO class persistence logic
        List<Map<String,Object>> list=dao.getEmpDetails(condition);
        return list;
    }
}
//search
//class
```

### MainServlet.java

```
package com.nt.controller;
import java.io.IOException;
import java.util.List;
import java.util.Map;
import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.nt.service.EmpService;
```

```
public class MainServlet extends HttpServlet {
    ApplicationContext ctx;
    EmpService service;
    public void init(){
        //create IOC container
        ctx=new ClassPathXmlApplicationContext("com/nt/cfgs/applicationContext.xml");
        // get Service class bean
        service=(EmpService)ctx.getBean("service");
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        //read form data
        String jobs[]=req.getParameterValues("job");
        //use Service class
        List<Map<String, Object>> list=service.search(jobs);
        // keep results in request attribute
        req.setAttribute("result",list);
        //forward the request to result page
        RequestDispatcher rd=req.getRequestDispatcher("/Result.jsp");
        rd.forward(req,resp);
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        doGet(req,res);
    }
}

//doGet(-,-)
//doPost(-,-)
//class
```

#### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.3.0.xsd">

    <bean id="jofb" class="org.springframework.jndi.JndiObjectFactoryBean">
        <property name="jndiName" value="java:/comp/env/mypool"/>
    </bean>

    <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
        <property name="dataSource" ref="jofb"/>
    </bean>

    <bean id="dao" class="com.nt.dao.EmpDao">
        <property name="jt" ref="template"/>
    </bean>

    <bean id="service" class="com.nt.service.EmpService">
```



```
<property name="dao" ref="dao"/>
</bean>
</beans>
```

**main.jsp**

```
<frameset rows="40%,60%">
  <frame name="f1" src="search.jsp"/>
  <frame name="f2"/>
</frameset>
```

**result.jsp**

```
<%-- <@page import="java.util.*" %> --%>
<%-- <%//read request attribute
List<Map<String,Object>> list=(List<Map<String,Object>>)request.getAttribute("result");
if(list!=null){ %> --%>
<!-- <table> -->
<!-- <tr> -->
<!-- <th>NO</th><th>NAME</th><th>JOB</th><th>Salary</th> -->
<!-- </tr> -->
<%-- <% for(Map map:list){ %> --%>
<!-- <tr> -->
<%-- <td><%=map.get("EMPNO") %></td> --%>
<%-- <td><%=map.get("ENAME") %></td> --%>
<%-- <td><%=map.get("JOB") %></td> --%>
<%-- <td><%=map.get("SAL") %></td> --%>
<!-- </tr> -->
<%-- <% } //for --%>
// } //if
<%-- %> --%>
<!-- </table> -->
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<c:choose>
<c:when test="{not empty result}">
<table>
<tr>
<th>NO</th><th>NAME</th><th>JOB</th><th>Salary</th>
</tr>
<c:forEach var="map" items="{result}">
  <tr>
    <td><c:out value="{map.EMPNO}"/></td>
    <td><c:out value="{map.ENAME}"/></td>
    <td><c:out value="{map.JOB}"/></td>
    <td><c:out value="{map.SAL}"/></td>
  </tr>
</c:forEach>
</table>
</c:when>
<c:otherwise>
```

```
No Employee Found
</c:otherwise>
</c:choose>
```

**search.jsp**

```
<h1><center>Search EmpDetails </center></h1>
<form action="controller" target="f2">
  Select job(s) <select name="job" multiple>
    <option value="CLERK">CLERKS</option>
    <option value="MANAGER">MANAGERS</option>
    <option value="SALESMAN">SALESMEN</option>
    <option value="ANALYST">ANALYSTS</option>
    <option value="PRESIDENT">PRESIDENTS</option>
  </select>
  <input type="submit" value="search"/>
</form>
```

**web.xml**

```
<web-app>
  <servlet>
    <servlet-name>Main</servlet-name>
    <servlet-class>com.nt.controller.MainServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>Main</servlet-name>
    <url-pattern>/controller</url-pattern>
  </servlet-mapping>
</web-app>
```

Jar files: spJDBCLib jarfile+Ojdbc14.jar+JSTL.jar+Servlet-api.jar+standard.jar----->collect from tomcat installation (perform Deployment assembly)

→While developing eclipse based web application the jar files placed in build path will not be moved to WEB-INF/lib folder automatically. For that we need to perform deploymentAssembly separately.

Right click on project---->buildpath--->configure buildpath---->deployment assembly--->Add--->build path entries--->select jar files---->apply--->ok

To deploy Eclipse web project in Ntsp57 Domain of weblogic:

a)Start NTSp27 domain server

<weblogic\_name>\user-projects\domains\NtSp27domain\startweblogic.cmd

b)Export war file to NTSp27 Domain server

Right click project-->export-->warfile

Destination: D:\.... AutoDeploy/DAOEmpSearchApp2.war

**c)Test the application.**

<http://localhost:7080/DAOEmpSearchApp2/main.html>

**Procedure to create user defined domain in GlassFish server:**

<Glassfish\_home>\bin>asadmin create-domain --adminport=4343 --user=testuser Ntsp27

Enter password: testuser

Re-Enter password: testuser

.....

.....

Domain created

**→To install glassfish 4.x extract the zip file**

→Glassfish uses separate port number for http operations and separate port number for admin operations.

**Procedure to create jdbc DataSource pointing to JDBC con pool for oracle in GFNTsp57 Domain server of GlasFish:**

→Place Ojdbc14/6.jar file in <Glassfish\_home>/domains/GFNTSP57Domain/lib/ext folder

**→Start the domain server**

<Glassfish\_home>\bin>asadmin start-domain GFNTSp57Domain

→Open admin console of domain server

<http://localhost:4646>

username: testuser

password: testuser

login

**→For Stopping domain**

<Glassfish\_home>\bin>asadmin Stop-domain GFNTsp57Domain

**→Create JDBC conn pool for Oracle**

Admin console---->resources--->JDBC--->jdbcConnection pools--->new--->pool name: pool1

Resource type: javax.sql.DataSource

DB Driver vendor: Oracle----->Next

Initial and min pool size: 3

Max pool size:100

.....

.....

**Additional properties**

User:scott

DBName:Xp

Password:tiger

Servername:localhost

Url:jdbc:oracle:thin@localhost:1521.xe

Port no:1521

---->finish

Launch pool1--->ping

**→Create JDBC DataSource pointing to the above JDBC connection pool:**

Adminconsole---->resources--->JDBC-->jdbc resources--->new--->jnfi name:DsJndi→Pool name: pool1 ----->ok

To deploy application in Glassfish Ntsp27Domain server of glassfish-→ then place directory or war File in <Glassfish\_home>\domain\GFNtsp57Domain\AutoDeploy folder

Note: While working with spring JDBC in servlet then ApplicationContext container should be ClassPathXmlApplicationContext container.

### Creating JDBC connection pool for oracle in tomcat server:

**Step1:** make sure that tomcat-jdbc.jar file is available in <tomcat\_home>/lib folder

Note: This jar file is builtin file in tomcat but must be arranged separately for other versions of tomcat(tomcat 6,7( by extracting apache-tomcat-jdbc-1.1.0.1\_bin.zip file.

**Step2:** place ojdbc 14/6.jar file in <tomcat\_home>/lib folder

**Step 3:** place the following <Resource>tag in <tomcat\_home>/config/context.xml

File as the subtag of <context> tag

```
<Resource name="DsJndi" type="javax.sql.DataSource" authenticate="container"
driverClassName="Oracle.jdbc.driver.OracleDriver" url="jdbc:oracle:thin:@localhost:1521:xe"
username="Scott" password="tiger" maxActive="20" maxIdle="10" validationQuery="SELECT
SYSDATE FROM DUAL" />
```

**Step 4:** restart the server

Note: To use the above JDBC connection pool in web application we need to place jndi name as shown below:

```
java:/comp/env/DsJndi
prefix      (jndi name)
```

Note: If you deploy any web application in tomcat server through eclipse IDE by configuration tomcat server with eclipse IDE. It is recommended to place "Context.xml" file in servers folder of eclipse package explorer in order to use tomcat server managed JDBC connection pool in that web application.

### Callback Interfaces

The method that is called by underlying jvm/container/server/etc.... automatically and dynamically is **callback method**. The interface that contains such methods is called **callback interface**. Spring provides lots of callback interfaces along with lots of Templates classes. We can use these callback interfaces to write customized logics to process data and to get data in **customized manner**. For example the **queryForMap(-)** gives single record in the form of Map object having column names as keys and column values as the values in Map collection elements. But industry standard is getting that record as BO class object. For this spring gives **RowMapper** callback interface. Various callback interfaces of spring JDBC are

- >StatementCallback--->Allows to work with statement object
- >PreparedStatementCallback
- >CallableStatementCallback
- >PreparedStatementCreator-->provides con object to create PreparedStatement
- >PreparedStatementSetter-->provides preparedStatement object to set value to queryParams.
- >RowMapper---->To process one record of ResultSet at a time.
- >ResultSetExtractor---->To process multiple records of ResultSet at a time.

Note: The implementation class methods of callback interfaces will be called by JdbcTemplate internally.

Eg: Application to get single employee record as EmployeeBO class object and multiple

**Employee records as multiple EmployeeBO class objects stored in List Collection.**

- ▲ DAOProj3 (CallbakInterfaces)
  - ▲ src
    - ▲ com.nt.bo
      - ▷ EmployeeBO.java
    - ▲ com.nt.cfgs
      - ▷ applicationContext.xml
    - ▲ com.nt.dao
      - ▷ ListEmpDao.java
      - ▷ ListEmpDaoImpl.java
    - ▲ com.nt.dto
      - ▷ EmployeeDTO.java
    - ▲ com.nt.service
      - ▷ ListEmpService.java
    - ▲ com.nt.test
      - ▷ ClientApp.java
  - ▷ JRE System Library [JavaSE-1.7]
  - ▲ DAOlib
    - ▷ ojdbc14.jar - D:\oracle\product\10.2.0\db\_
    - ▷ spring-beans-4.1.6.RELEASE.jar - C:\Users\
    - ▷ spring-context-4.1.6.RELEASE.jar - C:\Users\
    - ▷ spring-context-support-4.1.6.RELEASE.jar -
    - ▷ spring-core-4.1.6.RELEASE.jar - C:\Users\di
    - ▷ spring-expression-4.1.6.RELEASE.jar - CAU
    - ▷ spring-jdbc-4.1.6.RELEASE.jar - C:\Users\di
    - ▷ spring-tx-4.1.6.RELEASE.jar - C:\Users\del\
    - ▷ commons-logging.jar - C:\Users\del\Desktop
    - ▷ c3p0-0.9.1.jar - C:\Users\del\Desktop
    - ▷ commons-collections-3.1.jar - C:\Users\de
    - ▷ commons-dbcp-1.2.1.jar - C:\Users\del\K
    - ▷ commons-pool-1.2.jar - C:\Users\del\Des

**EmployeeBO.java**

```
package com.nt.bo;
public class EmployeeBO {
    private int no;
    private String ename,job;
    private int sal;
    public EmployeeBO() {
    }
    public EmployeeBO(int no, String ename, String job, int sal) {
        this.no = no;
        this.ename = ename;
        this.job = job;
        this.sal=sal;
    }
    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }
}
```

```
}  
public String getEname() {  
    return ename;  
}  
public void setEname(String ename) {  
    this.ename = ename;  
}  
public String getJob() {  
    return job;  
}  
public void setJob(String job) {  
    this.job = job;  
}  
public int getSal() {  
    return sal;  
}  
public void setSal(int sal) {  
    this.sal = sal;  
}  
}
```

#### ListEmpDAO.java

```
package com.nt.dao;  
import java.util.List;  
import com.nt.bo.EmployeeBO;  
public interface ListEmpDao {  
    public EmployeeBO getEmpDetailsByNo(int eno);  
    public List<EmployeeBO> getEmpDetailsByDesg(String desg);  
}
```

#### ListEmpDAOImpl.java

```
package com.nt.dao;  
import java.sql.ResultSet;  
import java.sql.SQLException;  
import java.util.ArrayList;  
import java.util.List;  
import org.springframework.dao.DataAccessException;  
import org.springframework.jdbc.core.JdbcTemplate;  
import org.springframework.jdbc.core.ResultSetExtractor;  
import org.springframework.jdbc.core.RowMapper;  
import com.nt.bo.EmployeeBO;  
public class ListEmpDaoImpl implements ListEmpDao {  
    private static final String GET_EMP_DETAILS_BY_NO="SELECT EMPNO,ENAME,JOB,SAL FROM  
        EMP WHERE EMPNO=?";  
    private static final String GET_EMP_DETAILS_BY_DESG=" SELECT EMPNO,ENAME,JOB,SAL  
        FROM EMP WHERE JOB=?";  
    private JdbcTemplate jt;  
    public void setJt(JdbcTemplate jt) {
```

```
        this.jt = jt;
    }
    @Override
    public EmployeeBO getEmpDetailsByNo(int eno) {
        EmployeeBO empbo=jt.queryForObject(GET_EMP_DETAILS_BY_NO,
                                           new EmpRowMapper(),
                                           eno);

        return empbo;
    }
    private class EmpRowMapper implements RowMapper<EmployeeBO>{

        @Override
        public EmployeeBO mapRow(ResultSet rs, int pos) throws SQLException {
            System.out.println("mapRow(-,-)");
            // Keep the record of ResultSet into BO class obj
            EmployeeBO ebo=new EmployeeBO(rs.getInt(1),
                                           rs.getString(2),
                                           rs.getString(3),
                                           rs.getInt(4));

            return ebo;
        }
    }
    @Override
    public List<EmployeeBO> getEmpDetailsByDesg(String desg) {
        List<EmployeeBO> list=jt.query(GET_EMP_DETAILS_BY_DESG,
                                      new ResultSetExtractor<List<EmployeeBO>>(){

            @Override
            public List<EmployeeBO> extractData(ResultSet rs)
            throws SQLException,DataAccessException {
                List<EmployeeBO> list=new ArrayList<EmployeeBO>();
                while(rs.next()){
                    EmployeeBO ebo=new EmployeeBO(rs.getInt(1),
                                                    rs.getString(2),
                                                    rs.getString(3),
                                                    rs.getInt(4));

                    list.add(ebo);
                }
                return list;
            }
        },desg);
    }
}
```

**ListEmpService.java**

```
package com.nt.service;
import java.util.ArrayList;
```

```
import java.util.List;
import com.nt.bo.EmployeeBO;
import com.nt.dao.ListEmpDao;
import com.nt.dto.EmployeeDTO;
public class ListEmpService {
    private ListEmpDao dao;
    public void setDao(ListEmpDao dao) {
        this.dao = dao;
    }

    public EmployeeDTO searchEmpByNo(int no){
        //use DAO class
        EmployeeBO ebo=dao.getEmpDetailsByNo(no);
        // Transpringframeworker BO data to DTO data
        EmployeeDTO dto=new EmployeeDTO(ebo.getNo(),
                                         ebo.getEname(),
                                         ebo.getJob(),
                                         ebo.getSal());

        return dto;
    }

    public List<EmployeeDTO> searchEmpsByDesg(String desg){
        //use DAO class
        List<EmployeeBO> listbo=dao.getEmpDetailsByDesg(desg);
        // Copy List<BO> data to List<DTO>
        List<EmployeeDTO> listdto=new ArrayList<EmployeeDTO>();
        for(EmployeeBO bo:listbo){
            EmployeeDTO dto=new EmployeeDTO(bo.getNo(),
                                             bo.getEname(),
                                             bo.getJob(),
                                             bo.getSal());

            listdto.add(dto);
        }
        return listdto;
    }
}
//method
}
//class
```

**EmployeeDTO.java**

```
package com.nt.dto;
import java.io.Serializable;
public class EmployeeDTO implements Serializable {
    private int no;
    private String name,job;
    private int sal;
    public EmployeeDTO() {
        // TODO Auto-generated constructor stub
    }
    public int getNo() {
        return no;
    }
}
```



```
public void setNo(int no) {
    this.no = no;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getJob() {
    return job;
}

public void setJob(String job) {
    this.job = job;
}

public int getSal() {
    return sal;
}

public void setSal(int sal) {
    this.sal = sal;
}

public EmployeeDTO(int no, String name, String job, int sal) {
    super();
    this.no = no;
    this.name = name;
    this.job = job;
    this.sal = sal;
}

@Override
public String toString() {
    return "EmployeeDTO [no=" + no + ", name=" + name + ", job=" + job
        + ", sal=" + sal + "]";
}
}
```

#### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.3.0.xsd">
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
<property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
<property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
<property name="username" value="scott"/>
<property name="password" value="tiger"/>
</bean>
<bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
<property name="dataSource" ref="dataSource"/>
```

```
</bean>
<bean id="empDao" class="com.nt.dao.ListEmpDaoImpl">
    <property name="jt" ref="template"/>
</bean>
<bean id="empService" class="com.nt.service.ListEmpService">
    <property name="dao" ref="empDao"/>
</bean>
</beans>
```

### ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.ListEmpService;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC Container
        ApplicationContext ctx=new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get Bean
        ListEmpService service=(ListEmpService)ctx.getBean("empService");
        //call b.methods
        System.out.println("7499 Emp Details "+service.searchEmpByNo(7499));
        System.out.println("All CLERK Detials "+service.searchEmpsByDesg("CLERK"));
    } //main
} //class
```

Jarfiles:----->spjdbc6.jar+ojdbc6.jar

→ Two methods of JdbcTemplate to work with RowMapper, ResultSetExtractor callback interfaces for the above requirement.

- 1) public Object queryForObject (String sql, RowMapper<T> rowmapper, Object ...args)  
(useful to get Bo class object having single record)
- 2) public List<T> query(String sql, ResultSetExtractor<T> extractor, Object ... args);

In real time the enduser supplied data will be received by 'service' class in the form of VO or DTO class object data. After processing request the BO class object will be prepared to pass to DAO. DAO uses BO class object data for persistence operations and gives another BO class object back to 'service' class. This service class converts BO class object data to DTO class object data to send to client. VO, BO class object are not 'Serializable' to send over the network whereas DTO class objects are Serializable objects so we can send them to network.

### Batch processing/Updating

Note: Batch processing is very useful in bulk registrations like batch of student registrations, bulk SMS delivery, Senses details insertion and etc...

- 1) **Batch processing/ Batch updation is Simple Statement is adding multiple queries to the batch.**
- 2) While working with PreparedStatement we can't add multiple queries to the batch because this always confines to single SQL query. But we can add multiple sets of query parameter values to the batch and this reduces network roundtrips between Java application and DB software.

#### Sample code:

```
PreparedStatement ps =
    con.prepareStatement("insert into student values(?,?,?)");
//set values to query params as batch
ps.setInt(1,00);
ps.setString(2,"raja");
ps.setString(3,"hyd");
ps.addBatch(); // adds first set of values to batch
ps.setInt(1,01);
ps.setString(2,"ravi");
ps.setString(3,"vizag");
ps.addBatch(); // adds second set of values to batch
//execute
int result[]=ps.executeBatch();
```

In real time DAO gets batch of values in the form of ListBO, setting these values to query params of SQL query through batch is bit complex while working with plain JDBC, To overcome this problem use **BatchPreparedStatementSetter** callback interface that simplifies the whole process.

Instead of performing multiple insert, update, delete operations on DB table by sending same query to database software for multiple times it is recommended to take the support of batch processing which sends query to database software **for one time and sets batch of values to query** for one time to reduce network roundtrips between java application and database software. While working with **JdbcTemplate** we will use **batchUpdate()** method for this operation.

- 1) **batchUpdate(String sql, List<Object []>args)**
- 2) **batchUpdate(String sql, PreparedStatementBatchSetter psbs)**
- 3) **batchUpdate(String sql, List<object []> args, int [] args types) and etc.....**

**PreparedStatementBatchSetter** is a callback interface that can be used to set batch of values to PreparedStatement object SQL Query provided by JdbcTemplate. **It allows programmer to set batch of values to SQL query parameters in customized form by gathering values from java objects like BO objects, but not from object[].** This callback interface gives 2 methods

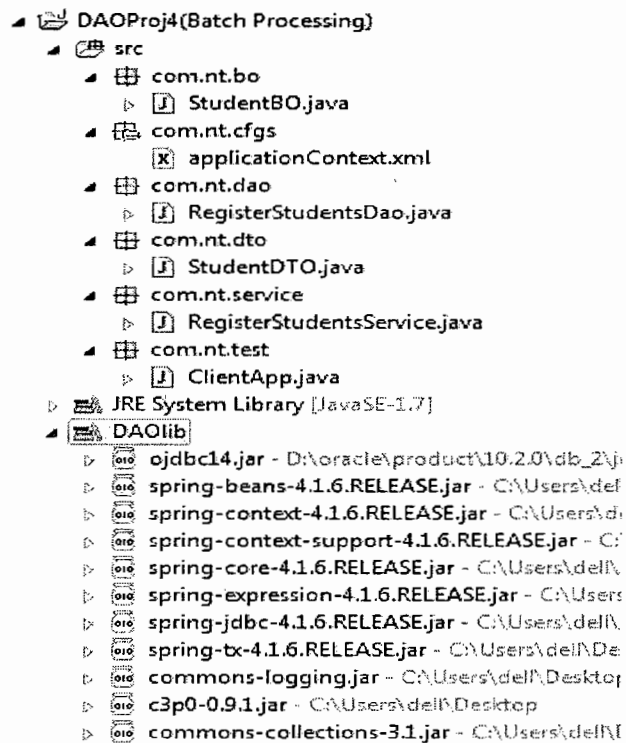
**a)setValues(int index,Object value)**

->Allows to set values to query params by specifying its index

**b)getBatchSize()**

→Returns numeric value to specify the number of times that setValues(-,-) method should execute.

eg: If getBatchSize() method returns 5 then setValues(-,-) method executes for 5 times to assign 5 set of values to batch. Later the query executes multiple times (5 times) in DB software having those batch of values.

**StudentBO.java**

```
package com.nt.bo;
public class StudentBO {
    private int sno;
    private String sname,sadd;
    public StudentBO() {
    }
    public StudentBO(int sno, String sname, String sadd) {
        this.sno = sno;
        this.sname = sname;
        this.sadd = sadd;
    }

    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
```

```
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}
```

### RegisterStudentsDao.java

```
package com.nt.dao;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.List;
import org.springframework.jdbc.core.BatchPreparedStatementSetter;
import org.springframework.jdbc.core.JdbcTemplate;
import com.nt.bo.StudentBO;
public class RegisterStudentsDao {
    private static final String STUD_INSERT_QRY="INSERT INTO STUDENT VALUES (?, ?, ?)";
    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    public int[] insert(final List<StudentBO> listbo){
        int result[]=jt.batchUpdate(STUD_INSERT_QRY, new BatchInsertion(listbo));
        return result;
    }
    private static final class BatchInsertion implements BatchPreparedStatementSetter{
        private List<StudentBO> listbo;
        public BatchInsertion(List<StudentBO> listbo){
            this.listbo=listbo;
        }
        @Override
        public int getBatchSize() {
            System.out.println("getBatchSize()");
            return listbo.size();
        }
        @Override
        public void setValues(PreparedStatement ps, int index)
            throws SQLException {
            System.out.println("setValues(-, -)");
            ps.setInt(1, listbo.get(index).getSno());
            ps.setString(2, listbo.get(index).getSname());
            ps.setString(3, listbo.get(index).getSadd());
        }
    }
}
```

```
    }//setValues(-,-)
  }//class
}
```

### RegisterStudentsService.java

```
package com.nt.service;
import java.util.ArrayList;
import java.util.List;
import com.nt.bo.StudentBO;
import com.nt.dao.RegisterStudentsDao;
import com.nt.dto.StudentDTO;

public class RegisterStudentsService {
    private RegisterStudentsDao dao;
    public void setDao(RegisterStudentsDao dao) {
        this.dao = dao;
    }
    public String registerStudents(List<StudentDTO> listdto){
        //convert ListDTO to ListBO obj
        List<StudentBO> listbo=new ArrayList<StudentBO>();
        for(StudentDTO dto:listdto){
            StudentBO bo=new StudentBO(dto.getSno(),
                                         dto.getSname(),
                                         dto.getSadd());
            listbo.add(bo);
        }
        // Use DAO class
        Int result[]=dao.insert(listbo);
        if(result!=null){
            return "Batch insertion success";
        }
        else{
            return "Batch insertion failed";
        }
    }
}
} //method
```

### StudentDTO.java

```
package com.nt.dto;
import java.io.Serializable;
public class StudentDTO implements Serializable {
    private int sno;
    private String sname,sadd;

    public StudentDTO() {
    }
    public StudentDTO(int sno, String sname, String sadd) {
        this.sno = sno;
    }
}
```

```
        this.sname = sname;
        this.sadd = sadd;
    }

    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}
```

#### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.3.0.xsd">
    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
        <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
        <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
        <property name="username" value="scott"/>
        <property name="password" value="tiger"/>
    </bean>
    <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
        <property name="dataSource" ref="dataSource"/>
    </bean>
    <bean id="stDao" class="com.nt.dao.RegisterStudentsDao">
        <property name="jt" ref="template"/>
    </bean>
    <bean id="stService" class="com.nt.service.RegisterStudentsService">
        <property name="dao" ref="stDao"/>
    </bean>
</beans>
```

**ClientApp.java**

```

package test;
import java.util.ArrayList;
import java.util.List;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.dto.StudentDTO;
import com.nt.service.RegisterStudentsService;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext ctx=new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext
        .xml");
        // get Bean
        RegisterStudentsService
        service=ctx.getBean("stService",RegisterStudentsService.class);
        // Prpare Batch of Studnets
        StudentDTO st1=new StudentDTO(101,"raja","hyd");
        StudentDTO st2=new StudentDTO(102,"ravi","hyd");
        List<StudentDTO> students=new ArrayList<StudentDTO>();
        students.add(st1); students.add(st2);
        //call B,method
        String result=service.registerStudents(students);
        System.out.println(result);
    } //main
} //class

```

**Student ( table)**

```

|---->Sno (n) (pk)
|---->Sname (VC2)
|----->Sadd (VC2)

```

➔ In order to use local variables or parameters of the outer class method definition inside the inner class we must take them as final variables.

**NamedParameterJdbcTemplate**

Spring JDBC supports 2 types of params in the SQL Query

- 1) Positional parameter (?)
- 2) Named Parameter (:<name>)

JdbcTemplate supports only positional parameters(?). we need to use var-args values or Object[] values to assign values to positional parameters based on their indexes. In this process if we get confusion towards identifying the indexes of the parameters and there is a possibility of getting problems. To overcome above problems the spring JDBC has given **NamedParameterJdbcTemplate** as a Wrapper around JdbcTemplate supporting Named parameters (:<varname>) in the query. This provides all the functionalities of JdbcTemplate but allows us to work with named parameters (:<varname>)



eg: `select * from emp where empno>=:min and empno<=:max;`

The underlying JDBC of spring JDBC converts each **named parameter** into **positional parameter** before executing the query. There are 2 ways to set values to Named Parameters

### 1) As Map<String, Object> obj

→ Specify the names of named parameters as keys and the objects as values in elements.

Eg: `Map<String, Object> map=new HashMap<String, Object>();`  
`map.put("min", 10);`  
`map.put("max", 100);`

### 2) By using the implementation class objects of SqlParameterSource(I)

#### a) MapSqlParameterSource(c)

→ Allows to specify the names, values of named parameters as the args of `addValue(-,-)`.

Eg: `SqlParameterSource params=new MapSqlParameterSource();`  
`params.addValue("min", 10);`  
`params.addValue("max", 100);`

#### b) BeanPropertySqlParameterSource

Note: Allows to specify bean object values as the named parameters values. For this Bean Property names(member variables) and the names of named parameters must match

- **NamedParameterJdbcTemplate** internally uses **JdbcTemplate** to send and execute SQLQuery once it translates named parameters to the **Jdbc style positional parameters**.
- To create **NamedParameterJdbcTemplate** class object we need **DataSource** object and it should be injected through constructor injection because there is no **setDataSource()** method.

DAOProj5(NamedParameterJdbcTemplate)

src

com.nt.bo

EmployeeBO.java

com.nt.cfgs

com.nt.dao

EmployeeDao.java

com.nt.dto

EmployeeDTO.java

com.nt.service

EmployeeService.java

com.nt.test

ClientApp.java

JRE System Library [JavaSE-1.7]

DAOLib

### EmployeeBO.java

```
package com.nt.bo;

public class EmployeeBO {
    private int eno,salary;
    private String ename,job;
    public EmployeeBO() {
    }
    public EmployeeBO(int eno, int salary, String ename, String job) {
        this.eno = eno;
        this.salary = salary;
        this.ename = ename;
    }
}
```

```
        this.job = job;
    }
    public int getEno() {
        return eno;
    }
    public void setEno(int eno) {
        this.eno = eno;
    }
    public int getSalary() {
        return salary;
    }
    public void setSalary(int salary) {
        this.salary = salary;
    }
    public String getEname() {
        return ename;
    }
    public void setEname(String ename) {
        this.ename = ename;
    }
    public String getJob() {
        return job;
    }
    public void setJob(String job) {
        this.job = job;
    }
}
```

### EmployeeDAO.java

```
package com.nt.dao;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.HashMap;
import java.util.Map;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.jdbc.core.namedparam.BeanPropertySqlParameterSource;
import org.springframework.jdbc.core.namedparam.MapSqlParameterSource;
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
import com.nt.bo.EmployeeBO;
public class EmployeeDao {
    private static final String GET_EMP_NAME_QRY="SELECT ENAME FROM EMP WHERE
EMPNO=:eno";
    private static final String GET_EMP_DETAILS_BY_NO_QRY="SELET EMPNO,ENAME
,JOB,SAL FROM EMP WHERE EMPNO=:eno";
    private static final String EMP_INSERT_QRY="INSERT INTO EMP(EMPNO,ENAME
,JOB,SAL) VALUES(:eno,:ename,:job,:salary)";
    private NamedParameterJdbcTemplate njt;

    public void setNjt(NamedParameterJdbcTemplate njt) {
```

```
        this.njt = njt;
    }
    public String findName(int no){
        //prepare param values
        Map <String,Object> map=new HashMap<String,Object>();
        map.put("eno",no);
        //excute query
        String name=njt.queryForObject(GET_EMP_NAME_QRY,
                                      map,
                                      String.class);

        return name;
    }
    public EmployeeBO findEmpDetails(int no){
        //parpare param values
        MapSqlParameterSource params=new MapSqlParameterSource();
        params.addValue("eno",no);
        //execute Qurey
        EmployeeBO ebo=njt.queryForObject(GET_EMP_DETAILS_BY_NO_QRY,
                                      params,
                                      new EmpRowMapper());

        return ebo;
    }
    private static final class EmpRowMapper implements RowMapper<EmployeeBO>{
        @Override
        public EmployeeBO mapRow(ResultSet rs, int pos) throws SQLException {
            // Store ResultSet record into BO obj
            EmployeeBO ebo=new EmployeeBO(rs.getInt(1),
                                      rs.getInt(4),
                                      rs.getString(2),
                                      rs.getString(3));

            return ebo;
        }
    }
    public int insertRecord(EmployeeBO ebo){
        //prepare param values
        BeanPropertySqlParameterSource params=
            new BeanPropertySqlParameterSource(ebo);
        int result=njt.update(EMP_INSERT_QRY, params);
        return result;
    }
}
```

### EmployeeService.java

```
package com.nt.service;
import com.nt.bo.EmployeeBO;
import com.nt.dao.EmployeeDao;
import com.nt.dto.EmployeeDTO;
```

```
public class EmployeeService {
    private EmployeeDao dao;
    public void setDao(EmployeeDao dao) {
        this.dao = dao;
    }
    public String searchNameByNo(int no){
        //use DAO
        String name=dao.findName(no);
        return name;
    }
    public EmployeeDTO searchEmpByNo(int no){
        //use DAO
        EmployeeBO bo=dao.findEmpDetails(no);
        // convert BO into DTO
        EmployeeDTO dto=new EmployeeDTO(bo.getEno(),
                                           bo.getSalary(),
                                           bo.getEname(),
                                           bo.getJob());
        return dto;
    }
    public String registerEmp(int no,String name,String job,int sal){
        //create DTO
        EmployeeDTO dto=new EmployeeDTO(no,sal,name,job);
        // convert DTO to BO
        EmployeeBO bo=new EmployeeBO(dto.getEno(),
                                       dto.getSalary(),
                                       dto.getEname(),
                                       dto.getJob());

        //use DAO
        int result=dao.insertRecord(bo);
        if(result==1)
            return "Regisration Success";
        else
            return "Registraion failed";
    }
}
```

### EmployeeDTO.java

```
package com.nt.dto;
import java.io.Serializable;
public class EmployeeDTO implements Serializable {
    private int eno,salary;
    private String ename,job;
    public EmployeeDTO() {
    }
    public EmployeeDTO(int eno, int salary, String ename, String job) {
        this.eno = eno;
        this.salary = salary;
    }
}
```

```
        this.ename = ename;
        this.job = job;
    }
    public int getEno() {
        return eno;
    }
    public void setEno(int eno) {
        this.eno = eno;
    }
    public int getSalary() {
        return salary;
    }
    public void setSalary(int salary) {
        this.salary = salary;
    }
    public String getEname() {
        return ename;
    }

    public void setEname(String ename) {
        this.ename = ename;
    }
    public String getJob() {
        return job;
    }
    public void setJob(String job) {
        this.job = job;
    }
    @Override
    public String toString() {
        return "EmployeeDTO [eno=" + eno + ", salary=" + salary + ", ename="
            + ename + ", job=" + job + "];"
    }
}
```

#### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.3.0.xsd">
    <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
        <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
        <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
        <property name="username" value="scott"/>
        <property name="password" value="tiger"/>
    </bean>
    <bean id="ntemplate"
class="org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate">
```

```

<constructor-arg ref="dataSource"/>
</bean>

<bean id="empDao" class="com.nt.dao.EmployeeDao">
  <property name="njt" ref="ntemplate"/>
</bean>
<bean id="empService" class="com.nt.service.EmployeeService">
  <property name="dao" ref="empDao"/>
</bean>
</beans>

```

### ClientApp.java

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.EmployeeService;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext ctx = new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get Bean
        EmployeeService service = ctx.getBean("empService", EmployeeService.class);
        //call b.methods
        System.out.println("7499 emp name"+service.searchNameByNo(7499));
        System.out.println("7499 emp details"+service.searchEmpByNo(7499));
        System.out.println("Emp    Registered?" + service.registerEmp(9999, "raja", "clerk", 8999));
    }
}

```

Note: Spring 2.5 is compatible with JDK 1.4 but JdbcTemplate class is designed without having support for **Generics, var args, AutoBoxing features**. To overcome this problem **SimpleJdbcTemplate** class is given as a Wrapper around JdbcTemplate supporting JDK 1.6 features. → In spring 3 JdbcTemplate class is upgraded to JDK 1.6 features so **SimpleJdbcTemplate** class is deprecated in spring 3.x (**SimpleJdbcTemplate removed in spring 4.x**)

### Working with SimplejdbcInsert, SimpleJdbcCall

→ Uses metadata concepts as simplifies environment to call PL/SQL procedures and to insert records.

#### SimpleJdbcInsert

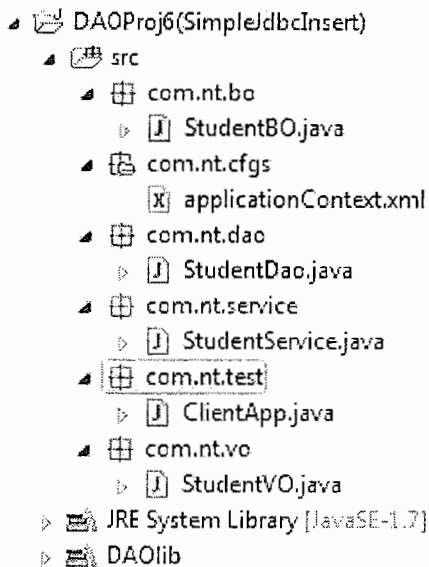
It uses Database table metadata concepts are used to insert records dynamically **without insert SQL query**. It just takes table name, column names, column values from the programmers and uses them to generate dynamic SQL insert query to insert record. We can give these column names and values either as **Map object** or as **SqlParameterSource(I) implementation class objects**.

The Map object contains column names as keys and column values as values. To create SimpleJdbcInsert object DataSource object is dependent object and we can use **setTableName(-)** to specify db table name and **execute(-)** method to specify column names, column values and also to execute the dynamically generated **Insert SQL query**. If our application deals with only registrations (**insert operation**) then we need to work with SimpleJdbcInsert class instead of JdbcTemplate.

SimpleJdbcInsert internally uses JdbcTemplate while completing the persistence operations. JdbcTemplate, SimpleJdbcTemplate, NamedParameterJdbcTemplate class objects are thread safe once they are configured i.e they allow one thread at a time to perform persistence operations which degrades the performance of the Application. To overcome this problem we can use SimpleJdbcInsert or SimpleJdbcCall whose objects are multi threaded and allows multiple threads simultaneously to perform persistence operations.

#### Use case:

When we ask visitors to register in timeframe (like online web counselling) we need to allow simultaneous record insertions for better performance, so prefer using SimpleJdbcInsert in that situation.



#### StudentBO.java

```
package com.nt.bo;

public class StudentBO {
    private int sno;
    private String sname;
    private String sadd;
    public StudentBO() {
    }
    public StudentBO(int sno, String sname, String sadd) {
        this.sno = sno;
        this.sname = sname;
        this.sadd = sadd;
    }
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
}
```

```
public void setName(String sname) {  
    this.sname = sname;  
}  
public String getSadd() {  
    return sadd;  
}  
public void setSadd(String sadd) {  
    this.sadd = sadd;  
}  
}
```

#### StudentDAO.java

```
package com.nt.dao;  
import java.util.HashMap;  
import java.util.Map;  
import org.springframework.jdbc.core.simple.SimpleJdbcInsert;  
import com.nt.bo.StudentBO;  
public class StudentDao {  
    private SimpleJdbcInsert sji;  
    public void setSji(SimpleJdbcInsert sji) {  
        this.sji = sji;  
    }  
    public int insert(StudentBO bo){  
        //prepare params for execute(-)  
        Map<String,Object> map=new HashMap<String,Object>();  
        map.put("sno",bo.getSno());  
        map.put("sname",bo.getSname());  
        map.put("sadd",bo.getSadd());  
        //specify table name  
        sji.setTableName("student");  
        //execute the logic  
        int result=sji.execute(map);  
        return result;  
    }  
    }  
}
```

#### StudentService.java

```
package com.nt.service;  
import com.nt.bo.StudentBO;  
import com.nt.dao.StudentDao;  
import com.nt.vo.StudentVO;  
public class StudentService {  
    private StudentDao dao;  
    public void setDao(StudentDao dao) {  
        this.dao = dao;  
    }  
    public String registerStudent(String sno,String sname,String add){  
        //prepare VO  
        StudentVO vo=new StudentVO(sno,sname,add);  
        //Convert VO obj to BO obj
```



```
StudentBO bo=new StudentBO();
bo.setSno(Integer.parseInt(vo.getNo()));
bo.setSname(vo.getSname());
bo.setSadd(vo.getSadd());
//use DAO
int result=dao.insert(bo);
if(result==0)
    return "Registration failed";
else
    return "Registractrion success";
} //method
} //class
```

### StudentVO.java

```
package com.nt.vo;
public class StudentVO {
    private String no,sname,sadd;
    public StudentVO(String no, String sname, String sadd) {
        this.no = no;
        this.sname = sname;
        this.sadd = sadd;
    }
    public StudentVO() {
    }
    public String getNo() {
        return no;
    }
    public void setNo(String no) {
        this.no = no;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}
```

### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
  <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
  <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
  <property name="username" value="scott"/>
  <property name="password" value="tiger"/>
</bean>
<bean id="simpleJdbcInsert" class="org.springframework.jdbc.core.simple.SimpleJdbcInsert">
  <constructor-arg ref="dataSource"/>
</bean>
<bean id="stDao" class="com.nt.dao.StudentDao">
  <property name="sj" ref="simpleJdbcInsert"/>
</bean>
<bean id="stService" class="com.nt.service.StudentService">
  <property name="dao" ref="stDao"/>
</bean>
</beans>
```

### ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.StudentService;
public class ClientApp {
    public static void main(String[] args) {
        //Create IOC container
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get ServiceBean
        StudentService service=ctx.getBean("stService",StudentService.class);
        //call B.method
        System.out.println("Result="+service.registerStudent("1010", "Ramesh","hyd"));
    }
}
```

### SimpleJdbcInsert with xml & Annotations:

#### EmployeeBO.java

```
package com.nt.bo;
public class EmployeeBO {
    private int empno;
    private String ename;
    private String job;
    private int sal;
    public int getEmpno() {
        return empno;
    }
    public void setEmpno(int empno) {
        this.empno = empno;
    }
}
```

```
}  
public String getEname() {  
    return ename;  
}  
public void setEname(String ename) {  
    this.ename = ename;  
}  
public String getJob() {  
    return job;  
}  
public void setJob(String job) {  
    this.job = job;  
}  
public int getSal() {  
    return sal;  
}  
public void setSal(int sal) {  
    this.sal = sal;  
}  
}
```

#### EmployeeDAO.java

```
package com.nt.dao;  
import com.nt.bo.EmployeeBO;  
public interface EmployeeDAO {  
    public int insert(EmployeeBO bo);  
}
```

#### EmployeeDAOImpl.java

```
package com.nt.dao;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.jdbc.core.namedparam.BeanPropertySqlParameterSource;  
import org.springframework.jdbc.core.simple.SimpleJdbcInsert;  
import org.springframework.stereotype.Repository;  
  
import com.nt.bo.EmployeeBO;  
@Repository("empDAO")  
public class EmployeeDAOImpl implements EmployeeDAO {  
    @Autowired  
    private SimpleJdbcInsert sji;  
  
    /*public int insert(EmployeeBO bo) {  
        //Using Map object  
        Map<String,Object> map=null;  
        int count=0;  
        //prepare Col names and col values as Map object  
        map=new HashMap<String,Object>();  
        map.put("empno", bo.getEmpno());  
    }  
}
```

```
        map.put("ename",bo.getEname());
        map.put("job",bo.getJob());
        map.put("sal", bo.getSal());
        //set Table name
        sji.setTableName("emp");
        //execute Logic giving col names and values
        count=sji.execute(map);
        return count;
    } //insert(-) */

    public int insert(EmployeeBO bo) {
        //Using BeanPropertySqlParameterSource object
        BeanPropertySqlParameterSource source=null;
        int count=0;
        //prepare Col names and col values.
        source=new BeanPropertySqlParameterSource(bo);
        //set Table name
        sji.setTableName("emp");
        //execute Logic giving col names and values
        count=sji.execute(source);
        return count;
    } //insert(-)

} //class
```

#### EmployeeService.java

```
package com.nt.service;
import com.nt.dto.EmployeeDTO;
public interface EmployeeService {
    public String registerEmployee(EmployeeDTO dto);
}
```

#### EmployeeServiceImpl.java

```
package com.nt.service;
import org.springframework.beans.BeanUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.nt.bo.EmployeeBO;
import com.nt.dao.EmployeeDAO;
import com.nt.dto.EmployeeDTO;

@Service("empService")
public class EmployeeServiceImpl implements EmployeeService {
    @Autowired
    private EmployeeDAO dao;
    public String registerEmployee(EmployeeDTO dto) {
        EmployeeBO bo=null;
        int count=0;
```

```
//Convert DTO to BO
bo=new EmployeeBO();
BeanUtils.copyProperties(dto,bo);
//use DAO
count=dao.insert(bo);
if(count==1)
    return "Registration Successful";
else
    return "Registration failed";
} //method
} //class
```

### EmployeeDTO.java

```
package com.nt.dto;
import java.io.Serializable;
public class EmployeeDTO implements Serializable {
    private int empno;
    private String ename;
    private String job;
    private int sal;
    public int getEmpno() {
        return empno;
    }
    public void setEmpno(int empno) {
        this.empno = empno;
    }
    public String getEname() {
        return ename;
    }
    public void setEname(String ename) {
        this.ename = ename;
    }
    public String getJob() {
        return job;
    }
    public void setJob(String job) {
        this.job = job;
    }
    public int getSal() {
        return sal;
    }
    public void setSal(int sal) {
        this.sal = sal;
    }
}
```

SimpleJdbcInsertTest.java

```
package com.nt.test;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.AbstractApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;

import com.nt.dto.EmployeeDTO;
import com.nt.service.EmployeeService;

public class SimpleJdbcInsertTest {

    public static void main(String[] args) {
        ApplicationContext ctx=null;
        EmployeeService service=null;
        EmployeeDTO dto=null;
        //Create Container
        ctx=new
        FileSystemXmlApplicationContext("src/main/java/com/nt/cfgs/applicationContext.xml");
        //get Bean
        service=ctx.getBean("empService",EmployeeService.class);
        //use Service class
        dto=new EmployeeDTO();
        dto.setEmpno(1034); dto.setEname("raja"); dto.setJob("clerk"); dto.setSal(9000);
        System.out.println(service.registerEmployee(dto));
        //close IOC container
        ((AbstractApplicationContext) ctx).close();
    } //main
} //class
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
    <import resource="persistence-beans.xml"/>
    <import resource="services-beans.xml"/>
</beans>
```

persistence-beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
```

```
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<bean id="bds" class="org.apache.commons.dbcp2.BasicDataSource">
  <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
  <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
  <property name="username" value="system"/>
  <property name="password" value="manager"/>
  <property name="initialSize" value="8"/>
</bean>
<bean id="sji" class="org.springframework.jdbc.core.simple.SimpleJdbcInsert">
  <constructor-arg ref="bds"/>
</bean>
<context:component-scan base-package="com.nt.dao"/>
</beans>
```

#### services-beans.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">

  <context:component-scan base-package="com.nt.service"/>
</beans>
```

### SimpleJdbcInsert Example with 100% Code Approach(JavaConfig Style)

#### EmployeeBO.java

```
package com.nt.bo;

public class EmployeeBO {
  private int empno;
  private String ename;
  private String job;
  private int sal;
  public int getEmpno() {
    return empno;
  }
  public void setEmpno(int empno) {
    this.empno = empno;
  }
  public String getEname() {
```

```
        return ename;
    }
    public void setName(String ename) {
        this.ename = ename;
    }
    public String getJob() {
        return job;
    }
    public void setJob(String job) {
        this.job = job;
    }
    public int getSal() {
        return sal;
    }
    public void setSal(int sal) {
        this.sal = sal;
    }
}
```

#### EmployeeDAO.java

```
package com.nt.dao;
import com.nt.bo.EmployeeBO;
public interface EmployeeDAO {
    public int insert(EmployeeBO bo);
}
```

#### EmployeeDAOImpl.java

```
package com.nt.dao;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.namedparam.BeanPropertySqlParameterSource;
import org.springframework.jdbc.core.simple.SimpleJdbcInsert;
import org.springframework.stereotype.Repository;
import com.nt.bo.EmployeeBO;
@Repository("empDAO")
public class EmployeeDAOImpl implements EmployeeDAO {
    @Autowired
    private SimpleJdbcInsert sji;

    /*public int insert(EmployeeBO bo) {
        //Using Map object
        Map<String, Object> map=null;
        int count=0;
        //prepare Col names and col values as Map object
        map=new HashMap<String, Object>();
        map.put("empno", bo.getEmpno());
        map.put("ename", bo.getEname());
        map.put("job", bo.getJob());
    }*/
}
```



```
map.put("sal", bo.getSal());
//set Table name
sj.setTableName("emp");
//execute Logic giving col names and values
count=sj.execute(map);
return count;
} //insert(-) */

public int insert(EmployeeBO bo) {
    //Using BeanPropertySqlParameterSource object
    BeanPropertySqlParameterSource source=null;
    int count=0;
    //prepare Col names and col values.
    source=new BeanPropertySqlParameterSource(bo);
    //set Table name
    sj.setTableName("emp");
    //execute Logic giving col names and values
    count=sj.execute(source);
    return count;
} //insert(-)

} //class
```

#### EmployeeService.java

```
package com.nt.service;

import com.nt.dto.EmployeeDTO;

public interface EmployeeService {
    public String registerEmployee(EmployeeDTO dto);
}
```

#### EmployeeServiceImpl.java

```
package com.nt.service;
import org.springframework.beans.BeanUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.nt.bo.EmployeeBO;
import com.nt.dao.EmployeeDAO;
import com.nt.dto.EmployeeDTO;

@Service("empService")
public class EmployeeServiceImpl implements EmployeeService {
    @Autowired
    private EmployeeDAO dao;
    public String registerEmployee(EmployeeDTO dto) {
        EmployeeBO bo=null;
        int count=0;
```

```
//Convert DTO to BO
bo=new EmployeeBO();
BeanUtils.copyProperties(dto,bo);
//use DAO
count=dao.insert(bo);
if(count==1)
    return "Registration Successful";
else
    return "Registration failed";
} //method
} //class
```

**EmployeeDTO.java**

```
package com.nt.dto;

import java.io.Serializable;

public class EmployeeDTO implements Serializable {
    private int empno;
    private String ename;
    private String job;
    private int sal;
    public int getEmpno() {
        return empno;
    }
    public void setEmpno(int empno) {
        this.empno = empno;
    }
    public String getEname() {
        return ename;
    }
    public void setEname(String ename) {
        this.ename = ename;
    }
    public String getJob() {
        return job;
    }
    public void setJob(String job) {
        this.job = job;
    }
    public int getSal() {
        return sal;
    }
    public void setSal(int sal) {
        this.sal = sal;
    }
}
```

**SimpleJdbcInsertTest.java**

```
package com.nt.test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.support.AbstractApplicationContext;
import com.nt.cfgs.BeanConfigurator;
import com.nt.dto.EmployeeDTO;
import com.nt.service.EmployeeService;

public class SimpleJdbcInsertTest {

    public static void main(String[] args) {
        ApplicationContext ctx=null;
        EmployeeService service=null;
        EmployeeDTO dto=null;
        //Create Container
        ctx=new AnnotationConfigApplicationContext(BeanConfigurator.class);
        //get Bean
        service=ctx.getBean("empService",EmployeeService.class);
        //use Service class
        dto=new EmployeeDTO();
        dto.setEmpno(1036); dto.setEname("raja"); dto.setJob("clerk"); dto.setSal(9000);
        System.out.println(service.registerEmployee(dto));
        //close IOC container
        ((AbstractApplicationContext) ctx).close();
    } //main
} //class
```

**BeanConfigurator.java**

```
package com.nt.cfgs;

import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Import;

@Configuration
@Import({PersistenceBeanConfigurator.class,ServiceBeanConfigurator.class})
public class BeanConfigurator {

}
```

**PersistenceBeanConfigurator.java**

```
package com.nt.cfgs;
import javax.sql.DataSource;

import org.apache.commons.dbcp2.BasicDataSource;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
```

```
import org.springframework.jdbc.core.simple.SimpleJdbcInsert;

@Configuration
@ComponentScan(basePackages="com.nt.dao")
public class PersistenceBeanConfigurator {

    @Bean
    public DataSource createDataSource(){
        BasicDataSource bds=null;
        bds=new BasicDataSource();
        bds.setDriverClassName("oracle.jdbc.driver.OracleDriver");
        bds.setUrl("jdbc:oracle:thin:@localhost:1521:xe");
        bds.setUsername("system");
        bds.setPassword("manager");
        return bds;
    }

    @Bean
    public SimpleJdbcInsert createSimpleJdbcInsert(){
        SimpleJdbcInsert sji=null;
        sji=new SimpleJdbcInsert(createDataSource());
        return sji;
    }
}
}
}
```

#### ServiceBeanConfigurator.java

```
package com.nt.cfgs;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages="com.nt.service")
public class ServiceBeanConfigurator {
}
```

### SimpleJdbcCall

It is a multithreaded, reusable object representing call to PL/SQL procedure or function. It runs on metadata concepts and simplifies the code that is required to call procedure or function. We just need to provide procedure or function name with Map of 'IN' parameters to call procedure or function and this returns a Map having 'OUT' parameter names and values. Instead of writing common persistence logic in every module as SQL queries, it is recommended to write only for one time in DB software as PL/SQL procedure or function.

Ex: Instead of writing authentication logic in every module separately, write it only once as PL/SQL procedure or function and use it in multiple modules of a project.

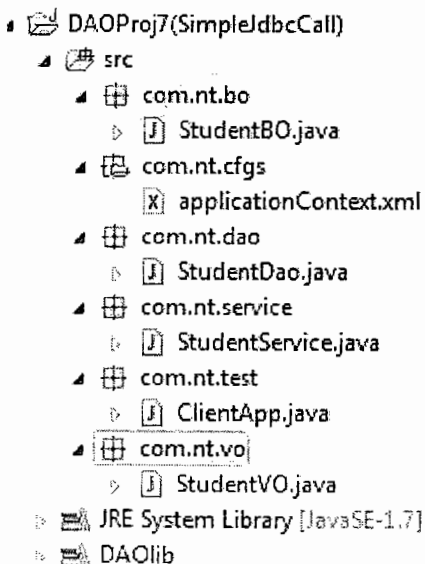
#### PL/SQL procedures in Oracle:

```
create or replace procedure getDetails (no in number, name out varchar2, address out varchar2)
as
begin
select sname, sad into name, address from Student where sno=no;
end;
```

#### SimpleJdbcCall

```
/--->DataSource object (dependent object--->constructor injection)
/--->execute(-)
    /--->Map as param (keys---> IN param names
                        Values---> IN param values)
    /--->Map as return value (keys---> OUT param names
                        Values---> OUT param values).
```

- In real time minimum complex PL/SQL procedure/functions must be developed by programmers and the complex PL/SQL procedure/functions will be developed by SQL developers.
- 50%--60% of persistence logic will be created in the form of PL/SQL procedures/functions for re-usability and for the centralization of the logic.



**Assignment**

Use SimpleJdbcCall support to call PL/SQL procedure or function that performs authentication.

**StudentBO.java**

```
package com.nt.bo;
public class StudentBO {
    private int sno;
    private String sname;
    private String sadd;
    public StudentBO() {
    }
    public StudentBO(int sno, String sname, String sadd) {
        this.sno = sno;
        this.sname = sname;
        this.sadd = sadd;
    }
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() { return sname; }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
}
```

**StudentDAO.java**

```
package com.nt.dao;
import java.util.HashMap;
import java.util.Map;
import org.springframework.jdbc.core.simple.SimpleJdbcCall;
import org.springframework.jdbc.core.simple.SimpleJdbcInsert;
import com.nt.bo.StudentBO;
public class StudentDao {
    private SimpleJdbcCall sjc;
    public void setSjc(SimpleJdbcCall sjc) {
        this.sjc = sjc;
    }
    public StudentBO getStudentDetails(int id){
        // set procedure name
        sjc.setProcedureName("getDetails");
        //Prepare IN params
    }
}
```

```
        Map <String,Object> inParams=new HashMap<String,Object>();
        inParams.put("no",id);
        // call Pl/sql procedue
        Map<String,Object>outParams=sjc.execute(inParams);
        // Place OutParam values in BO obj
        StudentBO bo=new StudentBO(id,
            (String)outParams.get("NAME"),
            (String)outParams.get("ADDRESS"));
        return bo;
    } //getStudentDetails(-)
} //class
```

#### StudentService.java

```
package com.nt.service;
import com.nt.bo.StudentBO;
import com.nt.dao.StudentDao;
import com.nt.dto.StudentDTO;
public class StudentService {
    private StudentDao dao;
    public void setDao(StudentDao dao) {
        this.dao = dao;
    }
    public StudentDTO seachStudentByNo(int no){
        //use DAO
        StudentBO bo=dao.getStudentDetails(no);
        //Convert BO obj to DTO obj
        StudentDTO dto=new StudentDTO(bo.getSno(),
            bo.getSname(),
            bo.getSadd());
        return dto;
    }
} //class
```

#### StudentDTO.java

```
package com.nt.dto;
public class StudentDTO {
    private int no;
    private String sname,sadd;
    public StudentDTO(int no, String sname, String sadd) {
        this.no = no;
        this.sname = sname;
        this.sadd = sadd;
    }
    public StudentDTO() {
    }
    public int getNo() {
        return no;
    }
    public void setNo(int no) {
```

```
        this.no = no;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
    @Override
    public String toString() {
        return "StudentDTO [no=" + no + ", sname=" + sname + ", sadd=" + sadd
            + "]";
    }
}
```

#### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
    <property name="username" value="scott"/>
    <property name="password" value="tiger"/>
</bean>
<bean id="simpleJdbcCall" class="org.springframework.jdbc.core.simple.SimpleJdbcCall">
    <constructor-arg ref="dataSource"/>
</bean>
<bean id="stDao" class="com.nt.dao.StudentDao">
    <property name="sjc" ref="simpleJdbcCall"/>
</bean>
<bean id="stService" class="com.nt.service.StudentService">
    <property name="dao" ref="stDao"/>
</bean>
</beans>
```

#### ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.StudentService;
```



```
public class ClientApp {  
    public static void main(String[] args) {  
        //Create IOC container  
        ApplicationContext ctx=new  
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");  
        //get ServiceBean  
        StudentService service=ctx.getBean("stService",StudentService.class);  
        //call B.method  
        System.out.println("1001 student details"+service.seachStudentByNo(1001));  
    }  
}
```

### Mapping SQL operations as Sub Classes

This is technique in spring jdbc to perform persistence operations. This technique makes programmer to develop sub classes where we set DataSource, declare parameters and compile query only for 1 time but we execute that query multiple times by reusing the object of subclass for multiple times. This concept is similar to JDO (Java Data Objects) programming. Spring jdbc supplies SqlQuery, SqlUpdate classes to develop these sub classes. Use SqlQuery for select operations, use SqlUpdate for non-select operations.

#### Reason for going to this Mapping SQL operations as subclasses model:

While working with JdbcTemplate, NamedParameterJdbcTemplate, SimpleJdbcTemplate classes if same query is executed for multiple times then it creates multiple Jdbc PreparedStatement objects internally on one PreparedStatement object for each execution basis. This is bad practice and also degrades the performance. To overcome this problem we can use "Mapping SQL Operations as sub classes" Technique.

#### Using SqlQuery or MappingSqlQuery:

We develop separate subclass extending from SqlQuery for each select query inside DAO class. This subclass supplies DataSource, query to superclass to execute query for multiple times by compiling the query only for one time. SqlQuery supplies execute(...) method to execute query that gives List of objects (multiple records) and also give finder(..) methods to execute query that gives only one object (one record). This class also gives mapRow(-,-) method which can be used to process each record of ResultSet object in to another object (Generally BO class object). The execute(), finder() methods internally it calls mapRow(-,-) method. Since SqlQuery is abstract class having more abstract methods we prefer using "MappingSqlQuery" which is subclass in the inheritance hierarchy of SqlQuery. While developing subclass for SqlQuery/MappingSqlQuery classes we use following methods in the subclasses.

- a) declareParameter (-)
- b) compile()
- c) executeXxx(-)
- d) findXxx(-) methods and etc.....

This technique says give every thing to super class from subclass for query compilation, setting datasource and reuse that subclass object for multiple times to execute that query for multiple times.

- In this technique, we need to take one separate inner class forever SQL query. inside the DAO class either extending from SqlQuery (or) SqlUpdate class. For example if DAO class want to execute 4 SQL queries then we need to take 4 sub classes as inner classes of DAO class and each object of sub class represents one SQL query that is compiled. **We can use that sub class/inner class object for multiple times in DAO class to execute the query for multiple times without doing compilation for multiple times.**

▲  DAOProj8(MappingSQLOperatios as Sub classes)

▲  src

▲  com.nt.bo

▷  StudentBO.java

▲  com.nt.cfgs

▷  applicationContext.xml

▲  com.nt.dao

▷  StudentDao.java

▲  com.nt.dto

▷  StudentDTO.java

▲  com.nt.service

▷  StudentService.java

▲  com.nt.test

▷  ClientApp.java

▷  JRE System Library [JavaSE-1.7]

▷  DAOlib

### StudentBO.java

```
package com.nt.bo;
public class StudentBO {
    private int sno;
    private String sname;
    private String sadd;
    public StudentBO() {
    }

    public StudentBO(int sno, String sname, String sadd) {
        this.sno = sno;
        this.sname = sname;
        this.sadd = sadd;
    }
    public int getSno() {
        return sno;
    }
    public void setSno(int sno) {
        this.sno = sno;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
}
```

```
}  
public String getSadd() {  
    return sadd;  
}  
public void setSadd(String sadd) {  
    this.sadd = sadd;  
}  
}}
```

### StudentDAO.java

```
package com.nt.dao;  
import java.sql.ResultSet;  
import java.sql.SQLException;  
import java.sql.Types;  
import java.util.List;  
import javax.sql.DataSource;  
import org.springframework.jdbc.core.SqlParameter;  
import org.springframework.jdbc.object.MappingSqlQuery;  
import org.springframework.jdbc.object.SqlUpdate;  
import com.nt.bo.StudentBO;  
public class StudentDao {  
    private static final String GET_STUDETNS_BY_ADDRESS="SELECT * FROM STUDENT  
WHERE SADD=?";  
    private DataSource ds;  
    StudentSelector selector;  
    public StudentDao(DataSource ds) {  
        this.ds=ds;  
        selector=new StudentSelector(ds,GET_STUDETNS_BY_ADDRESS);  
    }  
    //DAO class method  
    public List<StudentBO> findStudentsByAdderess(String addrs){  
        List<StudentBO> listbo=selector.findByAddress(addrs);  
        return listbo;  
    }  
    private static final class StudentSelector extends MappingSqlQuery<StudentBO>{  
        public StudentSelector(DataSource ds,String qry){  
            //supply dataSoruce,query details to super class  
            super(ds,qry);  
            super.declareParameter(new SqlParameter(Types.VARCHAR));  
            super.compile();  
        }  
        @Override  
        protected StudentBO mapRow(ResultSet rs, int index) throws SQLException {  
            System.out.println("mapRow(-)");  
            return new StudentBO(rs.getInt(1),  
                                rs.getString(2),  
                                rs.getString(3));  
        }  
        public List<StudentBO> findByAddress(String addrs){  
            List<StudentBO> listbo=super.execute(addrs);  
        }  
    }  
}
```

```
        return listbo;
    }
    }//inner class/sub class
} //class
```

### StudentService.java

```
package com.nt.service;
import java.util.ArrayList;
import java.util.List;
import com.nt.bo.StudentBO;
import com.nt.dao.StudentDao;
import com.nt.dto.StudentDTO;
public class StudentService {
    private StudentDao dao;
    public void setDao(StudentDao dao) {
        this.dao = dao;
    }
    public List<StudentDTO> searchStudentsByAddress(String addr){
        //use DAO
        List<StudentBO> listbo=dao.findStudentsByAdderess(addr);
        //convert listBO obj to listDTO obj
        List<StudentDTO> listdto=new ArrayList<StudentDTO>();
        for(StudentBO bo:listbo){
            StudentDTO dto=new StudentDTO(bo.getSno(),
                                            bo.getSname(),
                                            bo.getSadd());
            listdto.add(dto);
        }
        return listdto;
    } //method
} //class
```

### StudentDTO.java

```
package com.nt.dto;
public class StudentDTO {
    private int no;
    private String sname,sadd;
    public StudentDTO(int no, String sname, String sadd) {
        this.no = no;
        this.sname = sname;
        this.sadd = sadd;
    }
    public StudentDTO() {
    }
    public int getNo() {
        return no;
    }
    public void setNo(int no) {
```

```
        this.no = no;
    }
    public String getSname() {
        return sname;
    }
    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
    @Override
    public String toString() {
        return "StudentDTO [no=" + no + ", sname=" + sname + ", sadd=" + sadd
            + "];"
    }
}
```

#### ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.StudentService;
public class ClientApp {
    public static void main(String[] args) {
        //Create IOC container
        ApplicationContext ctx=new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get ServiceBean
        StudentService service=ctx.getBean("stService",StudentService.class);
        //call B.method
        System.out.println("Hyd student details"+service.searchStudentsByAddress("hyd"));
    }
}
```

#### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
    <property name="username" value="scott"/>
    <property name="password" value="tiger"/>
</bean>
```

```
<bean id="stDao" class="com.nt.dao.StudentDao">
    <constructor-arg ref="dataSource"/>
</bean>
<bean id="stService" class="com.nt.service.StudentService">
    <property name="dao" ref="stDao"/>
</bean>
</beans>
```

### SqlUpdate:

It is similar to SqlQuery class but given for non-select queries (update queries) execution. We can prepare subclass to SqlUpdate class inside the DAO class where we supply DataSource, query to SqlUpdate class only once but we use these subclass objects for multiple times to execute same non-select query with the support of **update(-, -) methods**. SqlUpdate is a concrete class and supplies lots of update(-) methods to execute non-select queries with same or different parameters for multiple times. For every non-select sql query we need to take separate sub class for **SqlUpdate** inside the DAO class.

**Why should we learn SimpleJdbcInsert, SimpleJdbcCall and MappingSQL operations as subclasses techniques even though JdbcTemplate class is already available?**

→ Ans: JdbcTemplate class object is single threaded object/ threadsafe object i.e it allows one thread at a time to perform the persistence operations. To overcome this problem use SimpleJdbcInsert, SimpleJdbcCall classes (which are multi threaded). When we use JdbcTemplate class object to execute same query for multiple times, it internally creates multiple PreparedStatement objects on **one object for each execution basis which is a bad practice**. To overcome this problem use MappingSql Operations as Subclasses technique.

- DAOPorj9 ( SqlUpdate operation us subclass
  - src
    - com.nt.bo
      - StudentBO.java
    - com.nt.cfgs
      - applicationContext.xml
    - com.nt.dao
      - StudentDao.java
    - com.nt.dto
      - StudentDTO.java
    - com.nt.service
      - StudentService.java
    - com.nt.test
      - ClientApp.java
  - JRE System Library [JavaSE-1.7]
  - DAOLib

### StudentBO.java

```
package com.nt.bo;
public class StudentBO {
    private int sno;
    private String sname;
    private String sadd;
    public StudentBO() {
```

```
}  
public StudentBO(int sno, String sname, String sadd) {  
    this.sno = sno;  
    this.sname = sname;  
    this.sadd = sadd;  
}  
public int getSno() {  
    return sno;  
}  
public void setSno(int sno) {  
    this.sno = sno;  
}  
public String getSname() {  
    return sname;  
}  
public void setSname(String sname) {  
    this.sname = sname;  
}  
public String getSadd() {  
    return sadd;  
}  
public void setSadd(String sadd) {  
    this.sadd = sadd;  
}  
}
```

#### StudentDAO.java

```
package com.nt.dao;  
import java.sql.ResultSet;  
import java.sql.SQLException;  
import java.sql.Types;  
import java.util.List;  
import javax.sql.DataSource;  
import org.springframework.jdbc.core.SqlParameter;  
import org.springframework.jdbc.object.MappingSqlQuery;  
import org.springframework.jdbc.object.SqlUpdate;  
import com.nt.bo.StudentBO;  
public class StudentDao {  
    private static final String UPDATE_STUDENT_ADDRS_BY_NO="UPDATE STUDENT SET SADD=?  
WHERE SNO=?";  
    private DataSource ds;  
    StudentUpdator updator;  
    public StudentDao(DataSource ds) {  
        this.ds=ds;  
        updator=new StudentUpdator(ds,UPDATE_STUDENT_ADDRS_BY_NO);  
    }  
    public int updateStudentDetails(int no,String newAddrs){  
        int cnt=updator.updateDetails(no, newAddrs);  
        return cnt;  
    }  
}
```

```
}  
//inner class  
private static final class StudentUpdator extends SqlUpdate{  
    public StudentUpdator(DataSource ds,String qry) {  
        //provide DataSource, Query to super class  
        super(ds,qry);  
        //Declare Parameters and given to super class  
        super.declareParameter(new SqlParameter(Types.VARCHAR));  
        super.declareParameter(new SqlParameter(Types.INTEGER));  
        //compile the Query  
        super.compile();  
    }  
    public int updateDetails(int no,String newAddrs){  
        int cnt=super.update(newAddrs,no);  
        return cnt;  
    }  
}  
}  
} //class
```

#### StudentService.java

```
package com.nt.service;  
import java.util.ArrayList;  
import java.util.List;  
import com.nt.bo.StudentBO;  
import com.nt.dao.StudentDao;  
import com.nt.dto.StudentDTO;  
public class StudentService {  
    private StudentDao dao;  
    public void setDao(StudentDao dao) {  
        this.dao = dao;  
    }  
    public String modifyStudentDetails(int no,String newAddrs){  
        int cnt=dao.updateStudentDetails(no, newAddrs);  
        if(cnt==0)  
            return no+" Student details are not updated";  
        else  
            return no+" student details are updated";  
    }  
} //method  
} //class
```

#### StudentDTO.java

```
package com.nt.dto;  
public class StudentDTO {  
    private int no;  
    private String sname,sadd;  
    public StudentDTO(int no, String sname, String sadd) {  
        super();  
        this.no = no;  
        this.sname = sname;  
    }  
}
```



```
        this.sadd = sadd;
    }
    public StudentDTO() {
    }
    public int getNo() {
        return no;
    }
    public void setNo(int no) {
        this.no = no;
    }
    public String getSname() {
        return sname;
    }

    public void setSname(String sname) {
        this.sname = sname;
    }
    public String getSadd() {
        return sadd;
    }
    public void setSadd(String sadd) {
        this.sadd = sadd;
    }
    @Override
    public String toString() {
        return "StudentDTO [no=" + no + ", sname=" + sname + ", sadd=" + sadd
            + "]\n";
    }
}
```

#### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
    <property name="username" value="scott"/>
    <property name="password" value="tiger"/>
</bean>
<bean id="stDao" class="com.nt.dao.StudentDao">
    <constructor-arg ref="dataSource"/>
</bean>
<bean id="stService" class="com.nt.service.StudentService">
    <property name="dao" ref="stDao"/>
</bean>
</beans>
```

**ClientApp.java**

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.StudentService;
public class ClientApp {
    public static void main(String[] args) {
        //Create IOC container
        ApplicationContext ctx=new FileSystemXmlApplicationContext
            ("src/com/nt/cfgs/applicationContext.xml");
        //get ServiceBean
        StudentService service=ctx.getBean("stService",StudentService.class);
        //call B.method
        System.out.println(service.modifyStudentDetails(1001, "hyd11"));
        }//main
    }//class
```

## **Spring Boot**

### **Introducing to Spring Boot**

Spring has been a great framework for years however it had few drawbacks. In this tutorial, An Introduction to Spring Boot we will see how Spring Boot has not only addressed the drawbacks but also supports modern software architecture. Spring Boot is around for sometime now.

Spring Team has released one of major innovation on the top of existing Spring Framework is Spring Boot. It is a completely new project from Pivotal Team (The Spring Team). Spring Boot is their latest innovation to keep up to date with the changing technology needs. The primary motivation behind developing Spring Boot is to simplify the process for configuring and deploying the spring applications.

Spring Boot offers a new paradigm for developing Spring applications with minimal friction. With Spring Boot, you'll be able to develop Spring applications with more agility and be able to focus on addressing your application's functionality needs with minimal (or possibly no) thought of configuring Spring itself. It uses completely new development model to make Java Development very easy by avoiding some tedious development steps and boilerplate code and configuration.

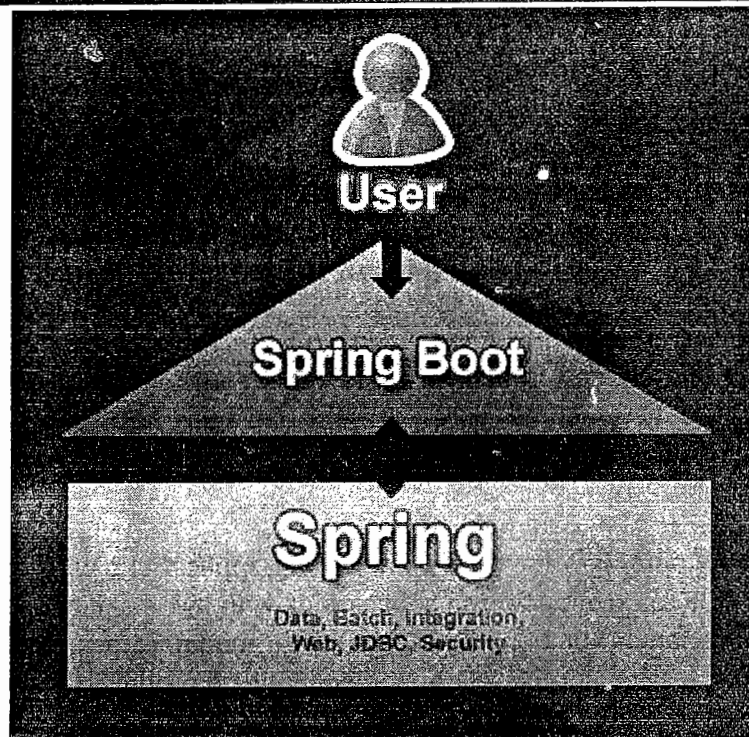
Spring Boot makes it easy to create stand-alone, production-grade Spring based Applications that you can "just run". We take an opinionated view of the spring platform and third-party libraries so you can get started with minimum fuss. Most Spring Boot applications need very little Spring configuration.

You can use Spring Boot to create Java applications that can be started using `java -jar` or more traditional war deployments. We also provide a command line tool that runs "spring scripts".

### **What is spring boot?**

First of all Spring Boot is not a framework, it is a way to ease to create stand-alone application with minimal or zero configurations. It is approach to develop spring based application with very less configuration. It provides defaults for code and annotation configuration to quick start new spring projects within no time. It leverages existing spring projects as well as Third party projects to develop production ready applications. It provides a set of Starter pom's or gradle build files which one can use to add required dependencies and also facilitate auto configuration.

Spring Boot automatically configures required classes depending on the libraries on its classpath. Suppose your application want to interact with DB, if there are Spring Data libraries on class path then it automatically sets up connection to DB along with the Data Source class.



## Spring Boot Primary Goals

Spring Boot primary goals are:

- Provide a radically faster and widely accessible getting started experience for all Spring development.
- Be opinionated out of the box, but get out of the way quickly as requirements start to diverge from the defaults.
- Provide a range of non-functional features that are common to large classes of projects (e.g. embedded servers, security, metrics, health checks, externalized configuration).
- Absolutely no code generation and no requirement for XML configuration, to avoid XML Configuration completely
- To avoid defining more Annotation Configuration(It combined some existing Spring Framework Annotations to a simple and single Annotation)
- To avoid writing lots of import statements
- To provide some defaults to quick start new projects within no time.

## Why New Project Need Spring Boot?

Suppose we are developing one of Hello World application in Spring Framework, for that there is only one item is specific to developing the Hello World functionality: the controller. The rest of it is generic boilerplate that you'd need for any web application developed with Spring. But if all Spring web applications need it, why should you have to provide it? So there are following things any new project need Spring Boot.

- To ease the Java-based applications Development, Unit Test and Integration Test Process.
- To reduce Development, Unit Test and Integration Test time by providing some defaults.
- To increase Productivity.
- When we talk about defaults, Spring Boot has its own opinions. If you are not specifying the details, it will use its own default configurations. If you want persistence, but don't specify anything else in your POM file, then Spring Boot configures Hibernate as a JPA provider with an HSQLDB database.

- To provide bunch of non-functional features/solutions that are very much common to large scale projects (e.g. embedded servers, security, metrics, health checks, externalized configuration).

## **What Spring Boot Isn't?**

As above mention Spring Boot is not a framework to write applications, it helps you to develop and build, package and deploy application with minimal configuration or zero configuration. It is not an application server. But it's the embedded servlet container that provides application server functionality, not Spring Boot itself.

Similarly, Spring Boot doesn't implement any enterprise Java specifications such as JPA or JMS. For example, Spring Boot doesn't implement JPA, but it does support JPA by auto-configuring the appropriate beans for a JPA implementation (such as Hibernate) finally, Spring Boot doesn't employ any form of code generation to accomplish its magic. Instead, it leverages conditional configuration features from Spring 4, along with transitive dependency resolution offered by Maven and Gradle, to automatically configure beans in the Spring application context.

In short, at its heart, Spring Boot is just Spring. Future Spring projects would not have any XML configurations as part of it, everything will be handled by the project Spring Boot.

## **Pros/Cons of Spring Boot**

### **Pros of Spring Boot:**

- It is very easy to develop Spring Based applications with Java or Groovy.
- It reduces lots of development time and increases productivity.
- It avoids writing lots of boilerplate Code, Annotations and XML Configuration.
- It is very easy to integrate Spring Boot Application with its Spring Ecosystem like Spring JDBC, Spring ORM, Spring Data, Spring Security etc.
- It follows "Opinionated Defaults Configuration" Approach to reduce Developer effort
- It provides Embedded HTTP servers like Tomcat, Jetty etc. to develop and test our web applications very easily.
- It provides CLI (Command Line Interface) tool to develop and test Spring Boot (Java or Groovy) Applications from command prompt very easily and quickly.
- It provides lots of plugins to develop and test Spring Boot Applications very easily using Build Tools like Maven and Gradle.
- It provides lots of plugins to work with embedded and in-memory Databases very easily.

### **Limitation of Spring Boot:**

It is very tough and time consuming process to convert existing or legacy Spring Framework projects into Spring Boot Applications. It is applicable only for brand new/Greenfield Spring Projects.

## **Spring boot releases**

Latest Releases: Spring Boot 1.4.x and 1.5.x available now. You require minimum Spring Framework 4.2.2 for this version.

Features Added in SpringBoot 1.4.x are:

- Executable JAR Layout
- Startup error improvements
- Hibernate 5
- Spring Framework 4.3

- Third Party Library
- Custom JSON Serializer and Deserializer
- New auto-configuration support
- Couchbase
- Neo4j
- Narayana transactional manager
- Caffeine Cache
- Actuator improvements
- Testing improvements

## **Getting started with Spring Boot**

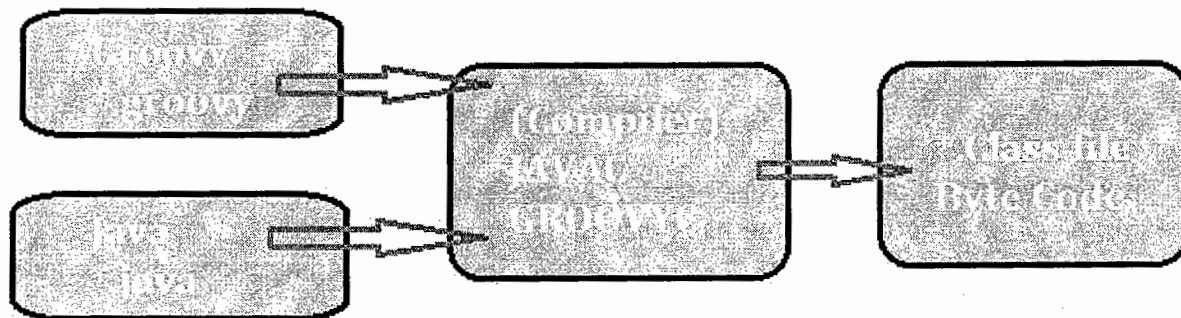
Ultimately, a Spring Boot project is just a regular Spring project that happens to leverage Spring Boot starters and auto-configuration. To Start Opinionated Approach to create Spring Boot Applications, The Spring Team (The Pivotal Team) has provided the following approaches.

- Using Spring Boot CLI Tool
- Using Spring STS IDE
- Using Spring Initializer (Website <http://start.spring.io/>)
- By configuring STS plugin in Eclipse IDE

We can develop two flavors of Spring-Based Applications using Spring Boot

- Java-Based Applications
- Groovy Applications

We can use Spring Boot CLI or Spring STS IDE or Spring Initializer Website to develop Spring Boot Groovy Applications. However, we can use Spring STS IDE or Spring Initializer Website to develop Spring Boot Java Applications.



Anyhow, Groovy is also JVM language almost similar to Java Language. We can combine both Groovy and Java into one Project. Because like Java files, Groovy files are finally compiled into \*.class files only. Both \*.groovy and \*.java files are converted to \*.class file (Same byte code format).

Spring Boot Framework Programming model is inspired by Groovy Programming model. Spring Boot internally uses some Groovy based techniques and tools to provide default imports and configuration.

Spring Boot Framework also combined existing Spring Framework annotations into some simple or single annotations. We will explore those annotations one by one in coming posts with some real-time examples.

Spring Boot Framework drastically changes Spring-Java Based Applications Programming model

into new Programming model. As of now, Spring Boot is at initial stage only but future is all about Spring Boot only.

### **Spring Boot CLI**

It is the easiest and quickest way to start using the Spring Boot. It is a command line tool used for executing the groovy scripts. In summary, you can install this tool by following these steps:

1. Download the binary distributions for this project from here. Spring Boot CLI requires Java JDK v1.6 or above in order to run. Groovy v2.1 is packaged as part of this distribution, and therefore does not need to be installed (any existing Groovy installation is ignored)
2. If you unpack the zip file, you will find spring.bat which will check the all the settings. This script can be found under the directory /bin.

## **Why Spring Boot ?**

Even though Spring is a great framework it has few pitfalls. Spring Boot was build not just to address them, it also provides direction to the future of software development. Spring's XML based configuration is a nightmare in the world of annotation. There was no clear leader in the java framework world to support Microservices. You really don't want different teams building Microservices to adapt different set of Java Libraries and look very different to each other.

## **What Spring Boot brings to the table ?**

### **Convention over configuration**

Spring Boot has taken away all the XML based configurations and provided Annotations for using the Spring Framework. You can start your application with a very minimum annotation in no time. This would be very helpful to the developers, the team productivity would greatly be impacted positively.

### **Standardization for Microservices**

One of the main objective of Spring Boot is to provide a unified ecosystem of libraries & standards to all the developers (teams) utilizing Microservices methodologies. Any project adapting Microservices would have multiple teams and we certainly don't want each of team to build the softwares in very different way. The teams will be also benefit from the all the annotation and tooling Spring Boot brings, however it also provides-

1. A common platform and library support
2. Reduced setup, configuration time in development environment.
3. Cloud Support

### **Integrated Server for Development**

Spring Boot attaches a Tomcat/Jetty server with the compiled Jar using Maven/Gradle. This helps the developers to expedite the development process by using the integrated server. In 2-3 mins you could build & test a RESTful web service from scratch.

## Cloud Support

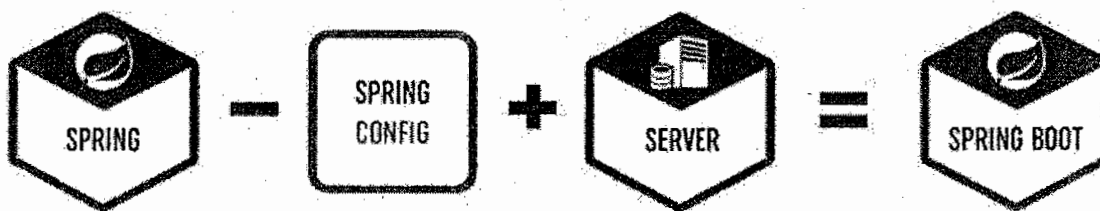
Spring Boot provides cloud support for configuration, tools and clients. It's also compatible with Cloud Native and works seamlessly with Cloud Foundry, Pivotal etc.

## Adapt & Support for 3rd Party Library

Spring Boot has taken a significant step and widen support for 3rd Party Open Source Library like Netflix OSS, No-SQL DB, Distributed Cache etc. You will find a full list in the Spring Boot home page, however the seamless integration using Annotation is very powerful.

## What is Spring Boot ?

In one sentence, Spring Boot is equal to ( Spring Framework – XML Configuration ) + Integrated Server.



## Spring Boot Components

### Spring Boot Auto Configure

Module to auto configure a wide range of Spring projects. It will detect availability of certain frameworks (Spring Batch, Spring Data JPA, Hibernate, JDBC). When detected it will try to auto configure that framework with some sensible defaults, which in general can be overridden by configuration in an application.properties/yml file.

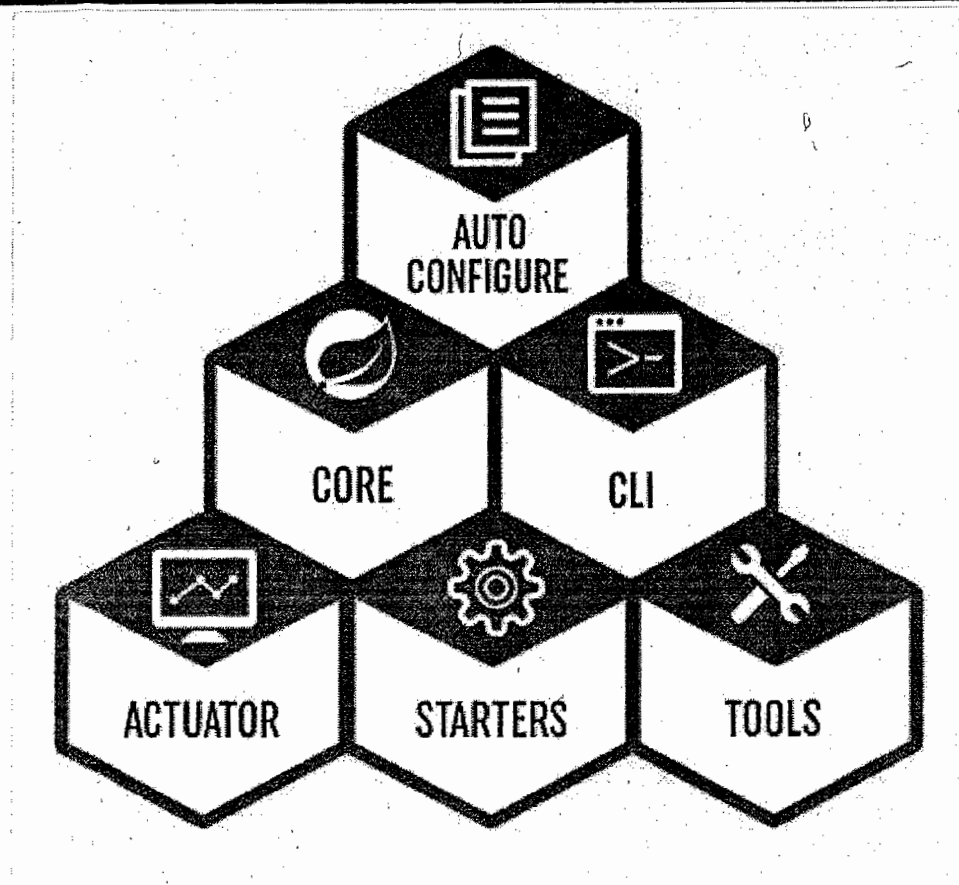
### Spring Boot Core

The base for other modules, but it also provides some functionality that can be used on its own, eg. using command line arguments and YAML files as Spring Environment property sources and automatically binding environment properties to Spring bean properties (with validation).

### Spring Boot CLI

A command line interface, based on ruby, to start/stop spring boot created applications.





## Spring Boot Actuator

This project, when added, will enable certain enterprise features (Security, Metrics, Default Error pages) to your application. As the auto configure module it uses auto detection to detect certain frameworks/features of your application. For an example, you can see all the REST Services defined in a web application using Actuator.

## Spring Boot Starters

Different quick start projects to include as a dependency in your maven or gradle build file. It will have the needed dependencies for that type of application. Currently there are many starter projects (We will learn about few of them in the next section) and many more are expected to be added.

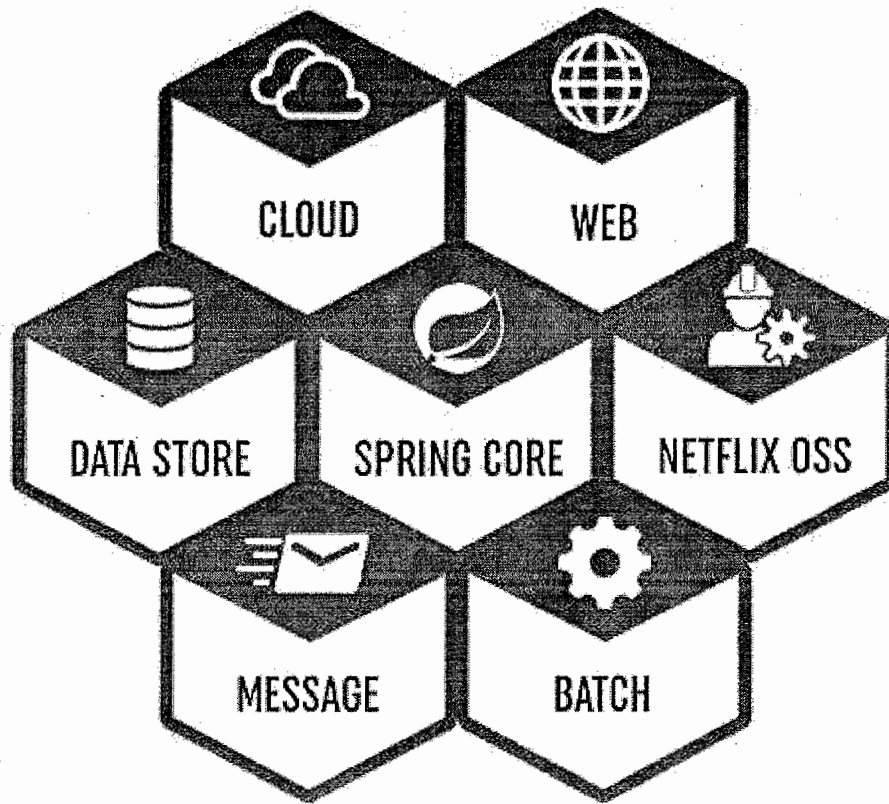
## Spring Boot Tools

The Maven and Gradle build tool as well as the custom Spring Boot Loader (used in the single executable jar/war) is included in this project.

## Spring Boot Starters

Spring Boot incorporates many starters packages (in Maven & Gradle) which you can include in order to add appropriate support in your project. At a high-level there are so far 6 types of starters

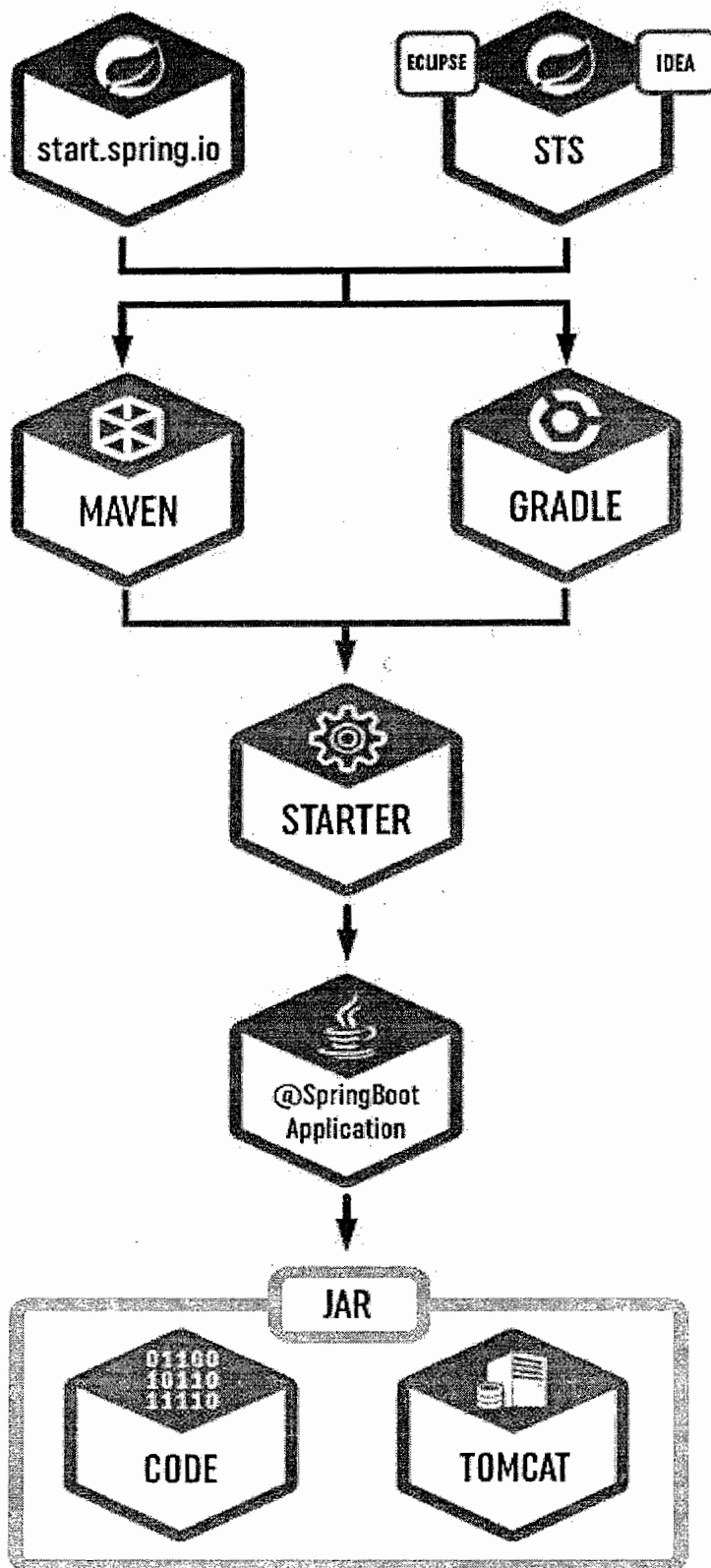
packages available. You can find all of them in Spring Boot official WebSite.



## How to use Spring Boot

In another post we will see how to use and run Spring Boot, however here are the steps you need to follow.

- You can use spring initialize to create the initial setup. You can visit either [start.spring.io](http://start.spring.io) or use STS (Spring Tool Suite) Support available in IDEA or Eclipse to choose all the Spring Boot Starters
- You need to also choose whether to use Maven or Gradle as the build tool.
- If you are using [start.spring.io](http://start.spring.io), you need to then download the zip and configure your workspace. Otherwise using your preferred IDE will automatically create the required file in the workspace.
- Add your code as required
- You can either use `mvn clean package` or use IDEA or Eclipse to build and create the jar file.
- By default the JAR would include integrated Tomcat server, so just by executing the JAR you should be able to use your program.



## Disadvantages of Spring Boot

### Migration Efforts

Migration from already existing spring project to spring boot is not straightforward. Spring Boot is mostly for new development project.

### Deployment to WebSphere/WebLogic Servers

Deploying Spring Boot application to WebSphere/WebLogic Servers are also not very simple. You need to make few changes like downgrading JPA Version, remove conflicting Jars etc in order to make it work in WebSphere/WebLogic Application Server.

### Microservices & Cloud Native

Spring Boot has been developed keeping Micro services & Cloud Native in mind. You may not see improvements in other areas.

## Applications

BootProj1-Core-DI:

### WishMessageGeneratorService.java

```
package com.nt.service;  
  
import java.util.Calendar;  
  
import org.springframework.stereotype.Service;  
  
@Service("wish")  
  
public class WishMessageGeneratorService {  
  
    public String generateWishMessage(String name){  
  
        Calendar calendar=null;  
  
        int hour=0;  
  
        //get Instance  
  
        calendar=Calendar.getInstance();  
  
        //get curent hour of the day  
  
        hour=calendar.get(Calendar.HOUR_OF_DAY);  
  
    }  
}
```

```
//generate wish Message

if(hour<=12)

    return "GM:"+name;

else if(hour<=16)

    return "GA:"+name;

else if(hour<=20)

    return "GE:"+name;

else

    return "GN:"+name;

} //method

} //class
```

#### **BootProj1CoreDiApplication.java**

```
package com.nt.test;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.ComponentScan;
import com.nt.service.WishMessageGeneratorService;

@SpringBootApplication
@ComponentScan(basePackages="com.nt.service")

public class BootProj1CoreDiApplication {

    public static void main(String[] args) {

        ApplicationContext ctx=null;

        WishMessageGeneratorService service=null;
```

```
//get IOC container

ctx=SpringApplication.run(BootProj1CoreDiApplication.class, args);

//get Bean

service=ctx.getBean("wish",WishMessageGeneratorService.class);

//invoke method

System.out.println(service.generateWishMessage("raja"));

}

}
```

**pom.xml:**

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>nit</groupId>

    <artifactId>BootProj1-Core-DI</artifactId>

    <version>0.0.1-SNAPSHOT</version>

    <packaging>jar</packaging>

    <name>BootProj1-Core-DI</name>

    <description>Demo project for Spring Boot</description>

    <parent>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter-parent</artifactId>
```

```
<version>1.5.1.RELEASE</version>

<relativePath/><!-- lookup parent from repository -->

</parent>

<properties>

    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

    <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>

    <java.version>1.8</java.version>

</properties>

<dependencies>

    <dependency>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter</artifactId>

    </dependency>

    <dependency>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter-test</artifactId>

        <scope>test</scope>

    </dependency>

</dependencies>

<build>

    <plugins>

        <plugin>

            <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-maven-plugin</artifactId>

    </plugin>

</plugins>

</build>

</project>
```

**BootProj2-Core-LC-StrategyPattern:****Courier.java:**

```
package com.nt.beans;

public interface Courier {

    public String deliver(int orderId);

}
```

**BlueDart.java:**

```
package com.nt.beans;

import javax.inject.Named;

@Named("blueDart")

public class BlueDart implements Courier {

    @Override

    public final String deliver(int orderId) {

        return "BluerDart:delevering order with OrderId:"+orderId;

    }

}
```

**DTDC.java:**

```
package com.nt.beans;

import javax.inject.Named;
```



```
@Named("dtdc")

public class DTDC implements Courier {

    @Override

    public final String deliver(int orderId) {

        return "DTDC:delevering order with OrderId:"+orderId;

    }

}
```

**Flipkart.java:**

```
package com.nt.beans;

import java.util.Arrays;

import java.util.Random;

import javax.annotation.Resource;

import javax.inject.Named;

public class Flipkart {

    @Resource(name="dtdc")

    private Courier courier;

    public String Shopping(String items[]){

        Random random=null;

        int orderId=0;

        //generate orderid

        random=new Random();

        orderId=random.nextInt(1999);

        //deliver products

    }

}
```

```
        System.out.println(courier.deliver(orderId));

        return Arrays.toString(items)+" are delivered ";

    }

}
```

**BootProj2CoreLcStrategyPatternApplication.java:**

```
package com.nt.test;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.ComponentScan;
import com.nt.beans.Flipkart;

@SpringBootApplication
@ComponentScan(basePackages="com.nt.beans")

public class BootProj2CoreLcStrategyPatternApplication {

    public static void main(String[] args) {

        ApplicationContext ctx=null;

        Flipkart flipkart=null;

        //get IOC container

        ctx=SpringApplication.run(BootProj2CoreLcStrategyPatternApplication.class,
args);

        //get bean

        flipkart=ctx.getBean("flipkart",Flipkart.class);

        System.out.println(flipkart.Shopping(new String[]{"shirt","pant"}));

    }

}
```

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>nit</groupId>

  <artifactId>BootProj2-Core-LC-StrategyPattern      </artifactId>

  <version>0.0.1-SNAPSHOT</version>

  <packaging>jar</packaging>

  <name>BootProj2-Core-LC-StrategyPattern</name>

  <description>Demo project for Spring Boot</description>

  <parent>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-parent</artifactId>

    <version>1.5.1.RELEASE</version>

    <relativePath/><!-- lookup parent from repository -->

  </parent>

  <properties>

    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

    <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>

    <java.version>1.8</java.version>

  </properties>

  <dependencies>
```

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>

<!-- https://mvnrepository.com/artifact/javax.inject/javax.inject -->
<dependency>
<groupId>javax.inject</groupId>
<artifactId>javax.inject</artifactId>
<version>1</version>
</dependency>

</dependencies>

<build>

    <plugins>

        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>

    </plugins>

</build>

</project>
```

**Spring Boot –Bean Life cycle****VoterChecking.java:**

```
package com.nt.beans;

import javax.annotation.PostConstruct;
import javax.annotation.PreDestroy;
import javax.inject.Named;
import org.springframework.beans.factory.annotation.Value;

@Named("voter")

public class VoterChecking {

    @Value("raja")

    private String name;

    @Value("30")

    private int age;

    @PostConstruct

    public void myInit()throws IllegalArgumentException{

        System.out.println("VoterChecking:myInit()");

        if(age<=0)

            throw new IllegalArgumentException("age must be +ve number");

    }

    public String checkVotingElgibility(){

        if(age<18)

            return name+" u r not elgible to vote";

        else

            return name+" u r elgible to vote";

    }

}
```

```

}

@PreDestroy

public void myDestroy(){

    System.out.println("VoterChecking:myDestroy()");

    name=null;

    age=0;

}

```

### BootProj2CoreLcStrategyPatternApplication.java:

```
package com.nt.test;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

import org.springframework.context.ApplicationContext;

import org.springframework.context.annotation.ComponentScan;

import com.nt.beans.VoterChecking;

@SpringBootApplication

@ComponentScan(basePackages="com.nt.beans")

public class BootProj2CoreLcStrategyPatternApplication {

    public static void main(String[] args) {

        ApplicationContext ctx=null;

        VoterChecking checking=null;

        //get IOC container

        ctx=SpringApplication.run(BootProj2CoreLcStrategyPatternApplication.class, args);

        //get bean
```

```
checking=ctx.getBean("voter",VoterChecking.class);

        System.out.println(checking.checkVotingElgibility());
    }
}
```

**pom.xml:**

Same as previous

**BootProj4-DAO-JdbcTemplate:**

In the production environment In Memory database is not good choice because of there are lots of limitations. For production environment we can use any database like **MySQL, DB2, Oracle, PostgreSQL** etc. Production database connections can also be auto-configured using a pooling **DataSource**. Here's the algorithm for choosing a specific implementation:

- First we prefer the **Tomcat pooling DataSource** for its performance and concurrency, so if that is available we always choose it.
- Otherwise, if **HikariCP** is available we will use it.
- If **Tomcat pooling datasource** and **HikariCP** are not available then we can choose **Commons DBCP**, but we don't recommend it in production.
- Lastly, if **Commons DBCP2** is available we will use it.

If you use the **spring-boot-starter-jdbc** or **spring-boot-starter-data-jpa** 'starters' you will automatically get a dependency to **tomcat-jdbc**.

We can configure the database properties in **application.properties** file so that **SpringBoot** will use those jdbc parameters to configure **DataSource** bean.

**Using another Connection Pooling library**

By default Spring Boot pulls in **tomcat-jdbc-{version}.jar** and uses **org.apache.tomcat.jdbc.pool.DataSource** to configure **DataSource** bean. Spring Boot check the availability of classes in the classpath in following order by default:

- **org.apache.tomcat.jdbc.pool.DataSource**
- **com.zaxxer.hikari.HikariDataSource**
- **org.apache.commons.dbcp.BasicDataSource**
- **org.apache.commons.dbcp2.BasicDataSource**

If you want to override default behavior suppose you want use **HikariDataSource** then you can exclude **tomcat-jdbc** and add **HikariCP** dependency as follows:

```
1. <dependency>
2.     <groupId>org.springframework.boot</groupId>
3.     <artifactId>spring-boot-starter-jdbc</artifactId>
4.     <exclusions>
5.         <exclusion>
```

```
6.      <groupId>org.apache.tomcat</groupId>
7.      <artifactId>tomcat-jdbc</artifactId>
8.      </exclusion>
9.  </exclusions>
10.     </dependency>
11.
12.     <dependency>
13.         <groupId>com.zaxxer</groupId>
14.         <artifactId>HikariCP</artifactId>
15.     </dependency>
```

With this dependency configuration **SpringBoot** will use **HikariCP** to configure **DataSource** bean.

### Connection to a JNDI DataSource

If you are deploying your Spring Boot application to an Application Server you might want to configure and manage your DataSource using your Application Servers built-in features and access it using JNDI.

The **spring.datasource.jndi-name** property can be used as an alternative to the **spring.datasource.url**, **spring.datasource.username** and **spring.datasource.password** properties to access the **DataSource** from a specific JNDI location.

For example, the following section in **application.properties** shows how you can access a Tomcat AS defined DataSource:

```
1. spring.datasource.jndi-name=java:tomcat/datasources/users
```

### applications.properties:

#### **#for mysql (Tomcat connection pooling)**

```
#spring.datasource.driver-class-name=com.mysql.jdbc.Driver
```

```
#spring.datasource.url=jdbc:mysql:///ntsp67DB
```

```
#spring.datasource.username=root
```

```
#spring.datasource.password=root
```

#### **#for oracle (Tomcat connection pooling)**

```
spring.datasource.driver-class-name=oracle.jdbc.driver.OracleDriver
```

```
spring.datasource.url=jdbc:oracle:thin:@localhost:1521:xe
```

```
spring.datasource.username=system
```

```
spring.datasource.password=manager
```



EmpDAO.java:

```
package com.nt.dao;  
  
import java.util.List;  
  
import com.nt.bo.EmployeeBO;  
  
public interface EmpDAO {  
  
    public int getEmpCount();  
  
    public List<EmployeeBO> getEmployeeDetailsByDesg(String desg);  
  
}
```

EmpDAOImpl.java:

```
package com.nt.dao;  
  
import java.sql.ResultSet;  
  
import java.sql.SQLException;  
  
import java.util.ArrayList;  
  
import java.util.List;  
  
import javax.annotation.Resource;  
  
import javax.inject.Named;  
  
import org.springframework.jdbc.core.JdbcTemplate;  
  
import org.springframework.jdbc.core.RowCallbackHandler;  
  
import com.nt.bo.EmployeeBO;  
  
@Named("empDAO")  
  
public class EmpDAOImpl implements EmpDAO {  
  
    private static final String GET_EMP_COUNT="SELECT COUNT(*) FROM EMP";  
  
    private static final String GET_EMP_DETAILS_BY_DESG="SELECT  
EMPNO,ENAME,JOB,SAL FROM EMP WHERE JOB=?";  
  
    @Resource
```

```
private JdbcTemplate jt;

@Override

public int getEmpCount() {

    int count=0;

    count=jt.queryForObject(GET_EMP_COUNT,Integer.class);

    return count;

}

@Override

public List<EmployeeBO> getEmployeeDetailsByDesg(String desg) {

    List<EmployeeBO> listBO=null;

    //execute Query

    listBO=new ArrayList<EmployeeBO>();

    jt.query(GET_EMP_DETAILS_BY_DESG,new EmpRowCallbackHandler(listBO),
desg);

    return listBO;

}

private static class EmpRowCallbackHandler implements RowCallbackHandler{

    private List<EmployeeBO> listBO=null;

    public EmpRowCallbackHandler(List<EmployeeBO> listBO) {

        this.listBO=listBO;

    }

    @Override

    public void processRow(ResultSet rs) throws SQLException {

        System.out.println("ProcessRow(-)");

        EmployeeBO bo=null;
```

```
//copy ResultSet obj record each record to BO  
  
bo=new EmployeeBO();  
  
bo.setEmpno(rs.getInt(1));  
  
bo.setEname(rs.getString(2));  
  
bo.setJob(rs.getString(3));  
  
bo.setSalary(rs.getInt(4));  
  
listBO.add(bo);  
  
} //method  
  
} //inner class  
  
} //outer class
```

**EmpService.java:**

```
package com.nt.service;  
  
import java.util.List;  
  
import com.nt.dto.EmployeeDTO;  
  
public interface EmpService {  
  
    public int findEmpCount();  
  
    public List<EmployeeDTO> findEmployeesByDesg(String desg);  
  
}
```

**EmpServiceImpl.java:**

```
package com.nt.service;  
  
import java.util.ArrayList;  
  
import java.util.List;  
  
import javax.annotation.Resource;  
  
import javax.inject.Named;
```

```
import org.springframework.beans.BeanUtils;

import com.nt.bo.EmployeeBO;

import com.nt.dao.EmpDAO;

import com.nt.dto.EmployeeDTO;

@Named("empService")

public class EmpServiceImpl implements EmpService {

    @Resource

    private EmpDAO dao;

    @Override

    public int findEmpCount() {

        int count=0;

        //use dAO

        count=dao.getEmpCount();

        return count;

    }

    @Override

    public List<EmployeeDTO> findEmployeesByDesg(String desg) {

        List<EmployeeDTO> listDTO=null;

        List<EmployeeBO> listBO=null;

        EmployeeDTO dto=null;

        //use DAO

        listBO=dao.getEmployeeDetailsByDesg(desg);

        //copy ListBO to ListDTO

        listDTO=new ArrayList();
```

```
        for(EmployeeBO bo:listBO){

            dto=new EmployeeDTO();

            BeanUtils.copyProperties(bo,dto);

            listDTO.add(dto);

        }

        return listDTO;

    } //method

} //class
```

**BootProj4DaoJdbcTemplateApplication.java:**

```
package com.nt.test;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.ConfigurableApplicationContext;
import org.springframework.context.annotation.ComponentScan;
import com.nt.service.EmpService;

@SpringBootApplication
@ComponentScan(basePackages={"com.nt.service","com.nt.dao"})

public class BootProj4DaoJdbcTemplateApplication {

    public static void main(String[] args) {

        ApplicationContext ctx=null;

        EmpService service=null;

        ctx=SpringApplication.run(BootProj4DaoJdbcTemplateApplication.class, args);

        //get Service bean
```

```
        service=ctx.getBean("empService",EmpService.class);

        System.out.println("Emp Count:::"+service.findEmpCount());

        System.out.println("CLERK Desg
Employees::"+service.findEmployeesByDesg("CLERK"));

        //close container

        ((ConfigurableApplicationContext) ctx).close();

    }

}
```

**EmployeeBO.java:**

```
package com.nt.bo;

public class EmployeeBO {

    private int empno;

    private String ename;

    private String job;

    private int salary;

    public int getEmpno() {

        return empno;

    }

    public void setEmpno(int empno) {

        this.empno = empno;

    }

    public String getEname() {

        return ename;

    }

    public void setEname(String ename) {
```

```
        this.ename = ename;

    }

    public String getJob() {

        return job;

    }

    public void setJob(String job) {

        this.job = job;

    }

    public int getSalary() {

        return salary;

    }

    public void setSalary(int salary) {

        this.salary = salary;

    }

}
```

**EmployeeDTO.java:**

```
Package com.nt.dto;

import java.io.Serializable;

public class EmployeeDTO implements Serializable {

    private int empno;

    private String ename;

    private String job;

    private int salary;
```

```
public int getEmpno() {  
    return empno;  
}  
  
public void setEmpno(int empno) {  
    this.empno = empno;  
}  
  
public String getEname() {  
    return ename;  
}  
  
public void setEname(String ename) {  
    this.ename = ename;  
}  
  
public String getJob() {  
    return job;  
}  
  
public void setJob(String job) {  
    this.job = job;  
}  
  
public int getSalary() {  
    return salary;  
}  
  
public void setSalary(int salary) {  
    this.salary = salary;  
}
```



```
@Override

public String toString() {

    return "EmployeeDTO [empno=" + empno + ", ename=" + ename + ", job=" + job
+ ", salary=" + salary + "];"

}

}
```

**pom.xml:**

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>nit</groupId>

    <artifactId>JdbcTemplateTest    </artifactId>

    <version>0.0.1-SNAPSHOT</version>

    <packaging>jar</packaging>

    <name>BootProj4-DAO-JdbcTemplate</name>

    <description>Demo project for Spring Boot</description>

    <parent>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter-parent</artifactId>

        <version>1.5.1.RELEASE</version>

        <relativePath /><!-- lookup parent from repository -->

    </parent>

    <properties>
```

```
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

<project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>

<java.version>1.8</java.version>

</properties>

<dependencies>

    <dependency>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter-jdbc</artifactId>

    </dependency>

    <dependency>

        <groupId>mysql</groupId>

        <artifactId>mysql-connector-java</artifactId>

        <scope>runtime</scope>

    </dependency>

    <!-- https://mvnrepository.com/artifact/javax.inject/javax.inject -->

    <dependency>

        <groupId>javax.inject</groupId>

        <artifactId>javax.inject</artifactId>

        <version>1</version>

    </dependency>

    <!-- https://mvnrepository.com/artifact/org.apache.tomcat/tomcat-dbcp -->

    <dependency>

        <groupId>org.apache.tomcat</groupId>

        <artifactId>tomcat-dbcp</artifactId>
```

```
<version>9.0.0.M15</version>

</dependency>

    <dependency>

        <groupId>com.oracle</groupId>

        <artifactId>ojdbc6</artifactId>

        <version>11.2.0</version>

    </dependency>

    <dependency>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter-test</artifactId>

        <scope>test</scope>

    </dependency>
</dependencies>

<build>

    <plugins>

        <plugin>

            <groupId>org.springframework.boot</groupId>

            <artifactId>spring-boot-maven-plugin</artifactId>

        </plugin>

    </plugins>

</build>

</project>
```

**BootProj5-DAO-NamedParameterJdbcTemplate:****application.properties:**

Same as previous

**EmpDAO.java:**

```
package com.nt.dao;

import java.util.List;

import com.nt.bo.EmployeeBO;

public interface EmpDAO {

    public int getEmpCount();

    public List<EmployeeBO> getEmployeeDetailsByDesg(String desg);

}
```

**EmpDAOImpl.java:**

```
package com.nt.dao;

import java.sql.ResultSet;

import java.sql.SQLException;

import java.util.ArrayList;

import java.util.HashMap;

import java.util.List;

import java.util.Map;

import javax.annotation.Resource;

import javax.inject.Named;

import org.springframework.jdbc.core.RowCallbackHandler;

import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;

import com.nt.bo.EmployeeBO;
```

```
@Named("empDAO")

public class EmpDAOImpl implements EmpDAO {

    private static final String GET_EMP_COUNT="SELECT COUNT(*) FROM EMP";

    private static final String GET_EMP_DETAILS_BY_DESG="SELECT
EMPNO,ENAME,JOB,SAL FROM EMP WHERE JOB=:desg";

@Resource

    private NamedParameterJdbcTemplate njt;

    @Override

    public int getEmpCount() {

        int count=0;

        count=njt.queryForObject(GET_EMP_COUNT, new HashMap(),Integer.class);

        return count;

    }

    @Override

    public List<EmployeeBO> getEmployeeDetailsByDesg(String desg) {

        List<EmployeeBO> listBO=null;

        Map<String,Object> paramMap=null;

        //prepare Map having NamedParams

        paramMap=new HashMap();

        paramMap.put("desg",desg);

        //execute Query

        listBO=new ArrayList<EmployeeBO>();

        njt.query(GET_EMP_DETAILS_BY_DESG,paramMap,new EmpRowCallbackHandler(listBO));

        return listBO;

    }

}
```

```
private static class EmpRowCallbackHandler implements RowCallbackHandler{

    private List<EmployeeBO> listBO=null;

    public EmpRowCallbackHandler(List<EmployeeBO> listBO) {

        this.listBO=listBO;

    }

    @Override

    public void processRow(ResultSet rs) throws SQLException {

        System.out.println("ProcessRow(-)");

        EmployeeBO bo=null;

        //copy ResultSet obj record each record to BO

        bo=new EmployeeBO();

        bo.setEmpno(rs.getInt(1));

        bo.setEname(rs.getString(2));

        bo.setJob(rs.getString(3));

        bo.setSalary(rs.getInt(4));

        listBO.add(bo);

    } //method

} //inner class

} //outer class
```

**EmpService.java:**

```
package com.nt.service;

import java.util.List;

import com.nt.dto.EmployeeDTO;
```

```
public interface EmpService {  
  
    public int findEmpCount();  
  
    public List<EmployeeDTO> findEmployeesByDesg(String desg);  
  
}
```

**EmpServiceImpl.java:**

```
package com.nt.service;  
  
import java.util.ArrayList;  
  
import java.util.List;  
  
import javax.annotation.Resource;  
  
import javax.inject.Named;  
  
import org.springframework.beans.BeanUtils;  
  
import com.nt.bo.EmployeeBO;  
  
import com.nt.dao.EmpDAO;  
  
import com.nt.dto.EmployeeDTO;  
  
@Named("empService")  
  
public class EmpServiceImpl implements EmpService {  
  
    @Resource  
  
    private EmpDAO dao;  
  
    @Override  
  
    public int findEmpCount() {  
  
        int count=0;  
  
        //use dao  
  
        count=dao.getEmpCount();  
  
        return count;  
    }  
}
```

```
}  
  
@Override  
  
public List<EmployeeDTO> findEmployeesByDesg(String desg) {  
  
    List<EmployeeDTO> listDTO=null;  
  
    List<EmployeeBO> listBO=null;  
  
    EmployeeDTO dto=null;  
  
    //use DAO  
  
    listBO=dao.getEmployeeDetailsByDesg(desg);  
  
    //copy ListBO to ListDTO  
  
    listDTO=new ArrayList();  
  
    for(EmployeeBO bo:listBO){  
  
        dto=new EmployeeDTO();  
  
        BeanUtils.copyProperties(bo,dto);  
  
        listDTO.add(dto);  
  
    }  
  
    return listDTO;  
  
} //method  
  
} //class
```

**BootProj4DaoJdbcTemplateApplication.java:**

```
package com.nt.test;  
  
import org.springframework.boot.SpringApplication;  
  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
import org.springframework.context.ApplicationContext;
```



```
import org.springframework.context.ConfigurableApplicationContext;

import org.springframework.context.annotation.ComponentScan;

import com.nt.service.EmpService;

@SpringBootApplication
@ComponentScan(basePackages={"com.nt.service","com.nt.dao"})

public class BootProj4DaoJdbcTemplateApplication {

    public static void main(String[] args) {

        ApplicationContext ctx=null;

        EmpService service=null;

        ctx=SpringApplication.run(BootProj4DaoJdbcTemplateApplication.class, args);

        //get Service bean

        service=ctx.getBean("empService",EmpService.class);

        System.out.println("Emp Count::"+service.findEmpCount());

        System.out.println("CLERK Desg
Employees::"+service.findEmployeesByDesg("CLERK"));

        //close container

        ((ConfigurableApplicationContext) ctx).close();

    }

}
```

**EmployeeBO.java:**

```
package com.nt.bo;

public class EmployeeBO {

    private int empno;

    private String ename;

    private String job;
```

```
private int salary;

public int getEmpno() {

    return empno;

}

public void setEmpno(int empno) {

    this.empno = empno;

}

public String getEname() {

    return ename;

}

public void setEname(String ename) {

    this.ename = ename;

}

public String getJob() {

    return job;

}

public void setJob(String job) {

    this.job = job;

}

public int getSalary() {

    return salary;

}

public void setSalary(int salary) {

    this.salary = salary;

}
```

```
}  
  
}
```

**EmployeeDTO:**

```
package com.nt.dto;  
  
import java.io.Serializable;  
  
public class EmployeeDTO implements Serializable {  
  
    private int empno;  
  
    private String ename;  
  
    private String job;  
  
    private int salary;  
  
    public int getEmpno() {  
  
        return empno;  
  
    }  
  
    public void setEmpno(int empno) {  
  
        this.empno = empno;  
  
    }  
  
    public String getEname() {  
  
        return ename;  
  
    }  
  
    public void setEname(String ename) {  
  
        this.ename = ename;  
  
    }  
  
    public String getJob() {  
  
        return job;  
  
    }  
  
}
```

```
}

    public void setJob(String job) {

        this.job = job;

    }

    public int getSalary() {

        return salary;

    }

    public void setSalary(int salary) {

        this.salary = salary;

    }

    @Override

    public String toString() {

        return "EmployeeDTO [empno=" + empno + ", ename=" + ename + ", job=" + job
+ ", salary=" + salary + "]\n";

    }

}
```

**pom.xml:**

Same as previous

**BootProj6-DAO-SimpleJdbcInsert:****application.properties:**

Same as previous

**StudentDAO.java:**

```
package com.nt.dao;

import com.nt.bo.StudentBO;

public interface StudentDAO {
```

```
public int insert(StudentBO bo);  
  
}
```

**StudentDAOImpl.java:**

```
package com.nt.dao;  
  
import javax.inject.Inject;  
  
import javax.inject.Named;  
  
import javax.sql.DataSource;  
  
import org.springframework.jdbc.core.namedparam.BeanPropertySqlParameterSource;  
  
import org.springframework.jdbc.core.simple.SimpleJdbcInsert;  
  
import com.nt.bo.StudentBO;  
  
@Named("stDAO")  
  
public class StudentDAOImpl implements StudentDAO {  
  
    private SimpleJdbcInsert sji;  
  
    @Inject  
  
    public StudentDAOImpl(DataSource ds){  
  
        sji=new SimpleJdbcInsert(ds);  
  
    }  
  
    @Override  
  
    public int insert(StudentBO bo) {  
  
        BeanPropertySqlParameterSource source=null;  
  
        int count=0;  
  
        //set table name  
  
        sji.setTableName("student");  
  
        //prepare col names and values
```

```
        source=new BeanPropertySqlParameterSource(bo);

        count=sji.execute(source);

        return count;

    }

}
```

**StudentService.java:**

```
package com.nt.service;

import com.nt.dto.StudentDTO;

public interface StudentService {

    public String register(StudentDTO dto);

}
```

**StudentServiceImpl.java:**

```
package com.nt.service;

import javax.annotation.Resource;

import javax.inject.Named;

import org.springframework.beans.BeanUtils;

import com.nt.bo.StudentBO;

import com.nt.dao.StudentDAO;

import com.nt.dto.StudentDTO;

@Named("stService")

public class StudentServiceImpl implements StudentService {

    @Resource

    private StudentDAO dao;

    @Override
```

```
public String register(StudentDTO dto) {  
  
    StudentBO bo=null;  
  
    int count=0;  
  
    //convert DTO to BO  
  
    bo=new StudentBO();  
  
    BeanUtils.copyProperties(dto, bo);  
  
    //use dAO  
  
    count=dao.insert(bo);  
  
    if(count==0)  
  
        return "Registration failed";  
  
    else  
  
        return "Registration succeeded";  
  
} //method  
  
} //class
```

**BootProj4DaoJdbcTemplateApplication.java:**

```
package com.nt.test;  
  
import org.springframework.boot.SpringApplication;  
  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
import org.springframework.context.ApplicationContext;  
  
import org.springframework.context.ConfigurableApplicationContext;  
  
import org.springframework.context.annotation.ComponentScan;  
  
import com.nt.dto.StudentDTO;  
  
import com.nt.service.StudentService;  
  
@SpringBootApplication
```

```
@ComponentScan(basePackages={"com.nt.service","com.nt.dao"})

public class BootProj4DaoJdbcTemplateApplication {

    public static void main(String[] args) {

        ApplicationContext ctx=null;

        StudentService service=null;

        StudentDTO dto=null;

        ctx=SpringApplication.run(BootProj4DaoJdbcTemplateApplication.class, args);

        //get Service Bean

        service=ctx.getBean("stService",StudentService.class);

        //prepare DTO

        dto=new StudentDTO();

        dto.setSno(1123); dto.setSname("raja"); dto.setSadd("hyd");

        //invoke method

        System.out.println(service.register(dto));

        //close container

        ((ConfigurableApplicationContext) ctx).close();

    }

}
```

**StudentBO.java:**

```
package com.nt.bo;

public class StudentBO {

    private int sno;

    private String sname;

    private String sadd;
```



```
public int getSno() {  
    return sno;  
}  
  
public void setSno(int sno) {  
    this.sno = sno;  
}  
  
public String getSname() {  
    return sname;  
}  
  
public void setSname(String sname) {  
    this.sname = sname;  
}  
  
public String getSadd() {  
    return sadd;  
}  
  
public void setSadd(String sadd) {  
    this.sadd = sadd;  
}  
}
```

**StudentDTO.java:**

```
package com.nt.dto;  
  
import java.io.Serializable;  
  
public class StudentDTO implements Serializable {  
    private int sno;
```

```
private String sname;

private String sadd;

public int getSno() {

    return sno;

}

public void setSno(int sno) {

    this.sno = sno;

}

public String getSname() {

    return sname;

}

public void setSname(String sname) {

    this.sname = sname;

}

public String getSadd() {

    return sadd;

}

public void setSadd(String sadd) {

    this.sadd = sadd;

}

}
```

**pom.xml:**

Same as previous

## AOP (Aspect Oriented Programming)

→ AOP is the methodology of programming that makes the programmer to separate primary logics of application from the secondary logics or helper logics.

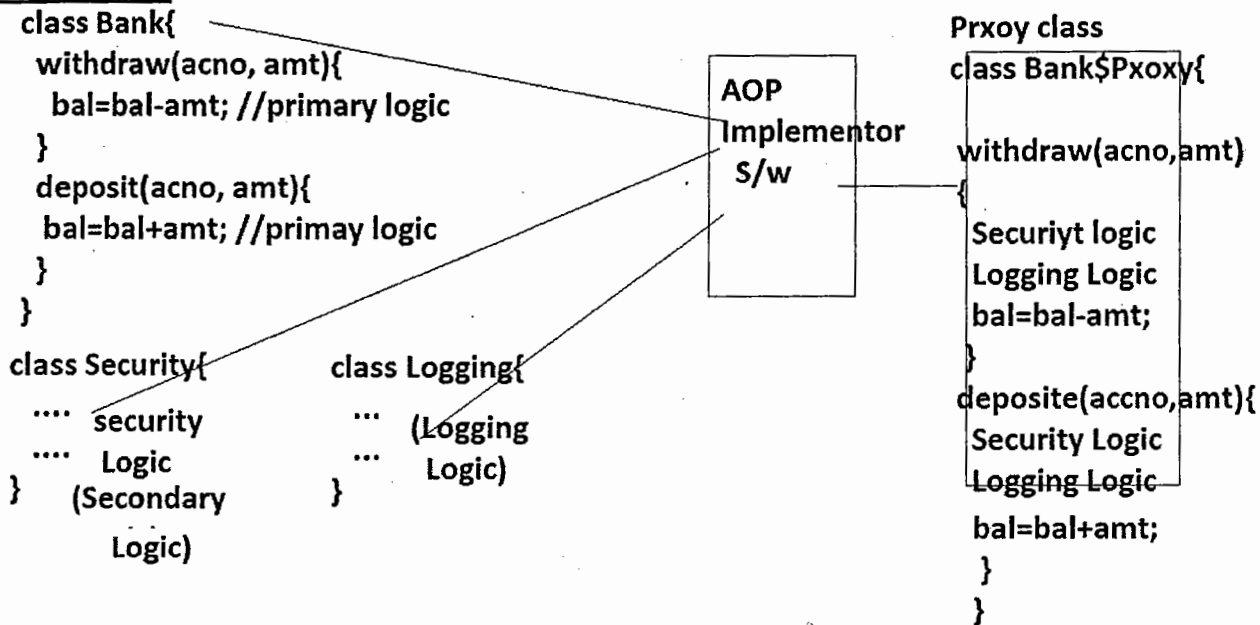
- ⇒ AOP is not replacement for OOP, more ever it complements OOP.
- ⇒ The logic that is minimum to create the task is called the primary logic. Without primary logic task will not be completed.
- ⇒ The logic that supports primary logic and makes primary logic as better logic but optional to have in the application is called secondary logic.
- ⇒ While performing withdraw amount operation deducting amount from balance is called primary logic.
- ⇒ Checking the identity of user, logging, auditing and etc are called secondary logics. Since we need secondary logic in multiple component classes of application or project they can be also called as cross cutting concerns because they need to be applied across the multiple classes.
- ⇒ AOP is not replacement for OOP, moreover it complements OOP. The key component in OOP is class. The key component in AOP Aspect (cross cutting that represents primary logics).
- ⇒ In OOP we develop business methods/service methods in the classes by mixing up both primary, secondary logics. Due to this business methods becomes heavy, the reusability of secondary logics will be killed and we cannot enable or disable secondary logics without touching source code of business method.
- ⇒ To overcome above method use AOP that makes the programmer to write primary logics in separate class and secondary logics in separate class and allows to mix and bind both logics dynamically at runtime by generating new classes.
- ⇒ For example scenario that look at following usecase

### **OOP example:**

```
class Bank{  
    public boolean withdraw (int acctno, int amt)  
    {  
        //security logic  
        //logging logic  
        //Transaction logic  
        bal=bal-amt; //primary logic  
    }  
    public boolean deposit(int acctno, int amt)  
    {  
        //security logic  
        //logging logic  
        //Transaction logic  
        bal=bal+amt; //primary logic  
    }  
}
```

**Problems with oop:**

- 1) kills the reusability of secondary logics
- 2) secondary logics dominate the primary logics
- 3) For enabling or disabling secondary logics we need touch source code

**AOP Model****AOP advantages:**

- 1) Secondary logics are not mixedup with primary logics.
  - 2) Secondary logics become reusable logics.
  - 3) By giving instructions to AOP enabled software we can enable (or) disable secondary logics.
  - 4) We can add secondary logics dynamically at runtime without touching the source code of primary logics (using XML/@Annotation)
- ⇒ The language that is designed based on OOP principle like abstraction, encapsulation, inheritance and etc... is called **OOP enabled languages**.
- ⇒ Similarly to **OOP principles we also have AOP principles**. The software that supports and implements AOP principles is called AOP enabled software. AOP is not just specific to spring.
- ⇒ There are multiple AOP enabled softwares like **spring AOP, AspectJAOP, JBOSS AOP, JAC (java Aspect component), mugzilla AOP** and etc...

**AOP principles/Terminologies:**

1. Aspect
2. JoinPoint
3. Advice

4. Pointcut
5. Target class
6. Weaving/Aspect Weaving
7. Proxy class

**Aspect:**

The logic that has to be applied across the multiple classes of application is called Aspect. This logic is also called as cross-cutting concerns, secondary logics, middleware service logics.

**Eg: Security, logging, transaction and etc...**

**JoinPoint:**

The possible points in the classes on which aspect logics can be applied is called joinpoints. They are like fields, constructors, methods and etc. Spring supports only methods as JoinPoints.

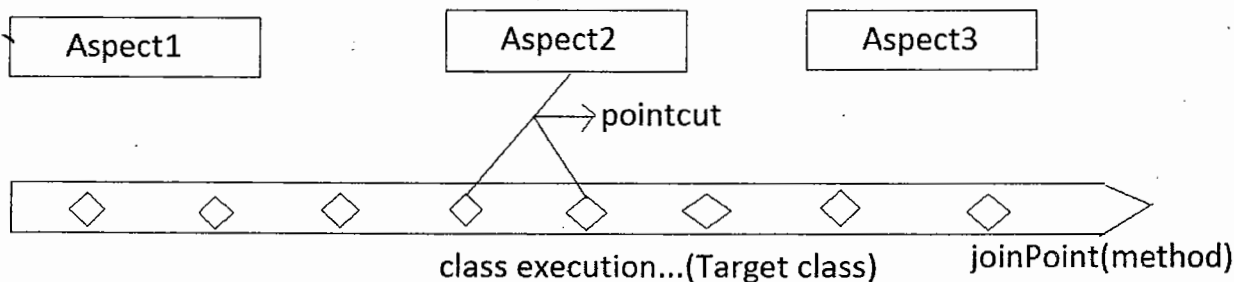
**Advice:**

It specifies or indicates how an aspect should be applied on JoinPoint like applying aspect before executing method or applying at the end of method and etc.....

**Note: Advice indicates that action taken by aspect on join points.**

There are 4 types of advices like:-

- 1) BeforeAdvice
- 2) AfterAdvice
- 3) ThrowsAdvice,
- 4) AroundAdvice.

**PointCut:**

It is a collection of JoinPoints on which aspects are advised (i.e aspects are configured).

**Target Class:**

The class on which we want to advice the aspects. It is **pre-AOP** java class having primary logics in the methods. If we call methods of target class only **primary logics** will be executed.

**Weaving:**

The process of advising aspects on the target class to build proxy class is called **weaving**. It is all about combining multiple logics to build one proxy class.

**Proxy class:**

⇒ The outcome class of weaving process, where primary logics and secondary logics of aspects will be mixed up dynamically at runtime. It is **post-AOP** class. If we call methods on proxy class object the primary logic will be executed along with secondary logics.

⇒ Aspect weaver is an unit of AOP frameworks like spring AOP, AspectAOP and etc.. It uses some

runtime libraries like **JDK libraries** or **CGLIB libraries** to generate proxy classes through weaving process. These proxy classes are dynamically generated **inmemory classes**.

Spring 1.x gives spring AOP to work with AOP. From spring 2.x AspectJAOP has been integrated with spring. So In spring 2.x we can use spring AOP and AspectJAOP to work with AOP concepts.

There are 4 approaches to implement AOP concepts in spring environment.

- Spring AOP Declarative (xml)
- Spring AOP programmatic (java code)
- Spring AOP 100% Code Approach
- Spring integrated AspectJ AOP declarative (xml)
- Spring integrated AspectJ AOP Annotations (@)
- Spring integrated AspectJ AOP 100% code Approach
- Spring integrated AspectJAOP Spring Boot Approach

There are 4 types of advices in AOP style programming.

**1) AroundAdvice:**

Here advice logic executes Around the target class method, i.e before and after target class method (Target method)

**2) BeforeAdvice:**

Here Advice logic executes before executing the target class method. So once target method is executed control does not come back to advice logic/method.

**3) AfterAdvice:**

Here advice logic executes after executing target method and before it returns the control back to caller.

**4) ThrowsAdvice:**

Here the Advice logic executes when exception is raised in target method.

**Spring AOP declarative Approach:**

→ Here we use xml file to configure Aspect, target classes and also to configure the classes that perform weaving. we use **org.springframework.aop.framework.ProxyFactoryBean** class for weaving and to generate the proxy class

**Around Advice:**

→ Advice indicates the action taken by Aspect on JoinPoint. So Around advice makes its aspects to execute around the JoinPoint i.e Aspect/Advice logic executes before entering to target method and after executing the target method.

- This Around advice is useful to implement performance monitoring, caching, around logging and etc usecases.

→ To develop this Advice/Aspect we need to take a class that implements **MethodInterceptor()** should provide **invoke (methodInvocation invocation)** as shown below.

**Sample Code:**

```
public class MyAdvice implements MethodInterceptor{
    public Object invoke (MethodInvocation invocation)
    {
        .....
        ..... //logic that executes before entering into target method.

        Object retval=invocation.proceed(); //calls target method
        .....
        .....
        //logic that executes after finishing the execution of target method.
        .....
        .....
    }
}
```

→ Using MethodInvocation obj we can call **getName()**, **getArguments()** methods to get target method name and its arguments. We can call **proceed()** method to call target method from advice class method.

**Three important points/control points of Around advice**

=> Can access and modify target method arguments.

=> Can control target method execution i.e we can skip or proceed target method execution being from advice class method.

=> Can access and modify the target method return value.

**Procedure to spring AOP Application in Declarative Approach:**

- a) Develop target class having methods with primary logics.
- b) Develop Aspect/Advice class having secondary logics.
- c) Perform the following configurations in springbean configuration file for weaving
  - 1) configure Target class
  - 2) Configure advice class
  - 3) Configure "ProxyFactoryBean" supplying target class, advice class to generate Proxy class
- d) Develop clientApp for testing
  - 1) Create IOC container
  - 2) Get proxy class object by supplying the bean id of "ProxyFactoryBean"
  - 3) Call target class methods on proxy object.

→ In spring JoinPoints are methods of classes where Aspects can be advised. Pointcut specifies which aspect or aspects should be applied on which JoinPoint(s).

- ▲ AOPProj1 (Spring AOp-Decl-AroundAdvice)
  - ▲ src
    - ▲ com.nt.aspect
      - CacheAdvice.java
      - LogAroundAdvice.java
      - PerformanceMonitoringAdvice.java
    - ▲ com.nt.cfgs
      - applicationContext.xml
    - ▲ com.nt.service
      - IntrAmtCalculator.java
    - ▲ com.nt.test
      - ClientApp.java
  - JRE System Library [JavaSE-1.7]
  - ▲ AOPlib
    - spring-aop-4.1.6.RELEASE.jar - C:\Users\de
    - spring-aspects-4.1.6.RELEASE.jar - C:\Users
    - spring-beans-4.1.6.RELEASE.jar - C:\Users\
    - spring-context-4.1.6.RELEASE.jar - C:\Users
    - spring-context-support-4.1.6.RELEASE.jar -
    - spring-core-4.1.6.RELEASE.jar - C:\Users\de
    - spring-expression-4.1.6.RELEASE.jar - C:\U
    - commons-logging.jar - C:\Users\del\Desk
  - com.springsource.org.aopalliance-1.0.0.jar

### CacheAdvice.java (usecase1)

```

package com.nt.aspect;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import org.aopalliance.intercept.MethodInterceptor;
import org.aopalliance.intercept.MethodInvocation;
public class CacheAdvice implements MethodInterceptor {
    Map <String, Object> cache=new HashMap<String, Object>();
    @Override
    public Object invoke(MethodInvocation invocation) throws Throwable {
        String key=invocation.getMethod().getName()+Arrays.toString(invocation.getArguments());

        Object retValue=null;
        if(!cache.containsKey(key)){
            retValue=invocation.proceed();
            cache.put(key, retValue);
            System.out.println("From target method");
            return retValue;
        }
        retValue=cache.get(key);
        System.out.println("From cache");
        return retValue;
    }
}

```



**LogAroundAdvice.java (usecase2)**

```
package com.nt.aspect;
import java.util.Arrays;
import org.aopalliance.intercept.MethodInterceptor;
import org.aopalliance.intercept.MethodInvocation;
public class LogAroundAdvice implements MethodInterceptor{
    @Override
    public Object invoke(MethodInvocation invocation) throws Throwable {
        Object retValue=null;
        System.out.println("Entering into "+invocation.getMethod().getName()+" with
args"+Arrays.toString(invocation.getArguments()));
        // Modify argument values
        Object args[]=invocation.getArguments();
        if(((Float)args[0])<50000){
            args[1]=1.0f;
        }
        //controlling target method
        if( ((Float)args[0])==0.0f || ((Float)args[1])==0.0f || ((Float)args[2])==0.0f){
            return 0.0f;
        }
        else{
            retValue=invocation.proceed();
        }
        System.out.println("Leaving from "+invocation.getMethod().getName()+" with
args"+Arrays.toString(invocation.getArguments()));
        if(((Float)retValue)<=25000)
            return retValue;
        else
            return ((Float)retValue)+(((Float) retValue)*0.1f);
    }
}
```

**PerformanceMonitoringAdvice.java (usecae3)**

```
package com.nt.aspect;
import org.aopalliance.intercept.MethodInterceptor;
import org.aopalliance.intercept.MethodInvocation;
public class PerformanceMonitoringAdvice implements MethodInterceptor{
    long start,end;
    @Override
    public Object invoke(MethodInvocation invocation) throws Throwable {
        Object retValue=null;
        start=System.currentTimeMillis();
        retValue=invocation.proceed();
        end=System.currentTimeMillis();
        System.out.println(invocation.getMethod().getName()+"has taken "+(end-start)+"ms to complete
execution");
        return retValue;
    }
}
```

**IntrAmtCalculator.java**

```
package com.nt.service;
public class IntrAmtCalculator {
    public float calcIntrAmt(float principle,float rate,float time){
        System.out.println("Target class : Method");
        return (principle*rate*time)/100.0f;
    }
}
```

**applicationContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">
    <!--cfg target class -->
    <bean id="target" class="com.nt.service.IntrAmtCalculator"/>
    <!--cfg advice classes -->
    <bean id="logAdvice" class="com.nt.aspect.LogAroundAdvice"/>
    <bean id="pmAdvice" class="com.nt.aspect.PerformanceMonitoringAdvice"/>
    <bean id="cacheAdvice" class="com.nt.aspect.CacheAdvice"/>
    <!--Cfg ProxyFactoryBean -->
    <bean id="pfb" class="org.springframework.aop.framework.ProxyFactoryBean">
        <property name="target" ref="target"/>
        <property name="interceptorNames">
            <list>
                <value>pmAdvice</value>
                <value>logAdvice</value>
                <value>cacheAdvice</value>
            </list>
        </property>
    </bean>
</beans>
```

**ClientApp.java**

```
package test;
import java.util.Arrays;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.IntrAmtCalculator;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get Bean
        IntrAmtCalculator proxy=ctx.getBean("pfb",IntrAmtCalculator.class);
        //Call B.method
        float intrAmt=proxy.calcIntrAmt(80000,2,20);
        System.out.println("IntrAmt =" +intrAmt);
    }
}
```

```

        System.out.println("-----");
        float intrAmt1=proxy.calcIntrAmt(80000,2,20);
        System.out.println("IntrAmt =" +intrAmt1);
        System.out.println("-----");
        float intrAmt2=proxy.calcIntrAmt(90000,2,20);
        System.out.println("IntrAmt =" +intrAmt2);
        System.out.println("-----");
    } //main
} //class

```

### Around Advice Intenal and flow of execution

#### Target class

```

package com.nt.service;
public class IntrAmountCalculator {
    public float calcIntrAmount(float pAmt, float rate, float time){
        return (pAmt*rate*time)/100.0f; (g)
    } //method
} //class

```

#### Advice class

```

private long start,end;
@Override (d)
public Object invoke(MethodInvocation invocation) throws Throwable {
    Object retVal=null;
    start=System.currentTimeMillis();
    //invoke target method
    (h) retVal=invocation.proceed(); (e)
    end=System.currentTimeMillis();
    System.out.println(invocation.getMethod().getName()+" with (i)
args"+Arrays.toString(invocation.getArguments())+" has taken" +(end-start)+" ms for
execution");
    return retVal; (j)
}
}

```

Generated Proxy class(In-Memory Class)

```

public IntrAmountCalculator$Proxy extends IntrAmountCalculator implements
    ApplicationContextAware{
    ApplicationContext ctx;
    public void setApplicationContext(ApplicationContext ctx){
        this.ctx=ctx;
    }
    public float calcIntrAmt(float pAmt,float rate,float time){
        //get Advice class object
        PerformanceMonitoringAdvice advice=
            ctx.getBean("pmAdvice",PerformanceMonitoringAdvice.class);

        //prepare MethodInvocation object
        MethodInvocation invocation=new MethodInvocation();
        invocation.setMethodName("calcIntrAmt");
        invocation.setArguments(new Object[]{pAmt,rate,time});
        //call invoke(-) method
        Object retVal=advice.invoke(invocation);
        return retVal;
    }
}

ClientApp
=====

    IntrAmountCalculator proxy=
        ctx.getBean("pfb",IntrAmountCalculator.class);
    float amt=proxy.calcIntrAmt(2000,2,10);
    (m)          (a)

```

- ⇒ While working with AOP our target class must not be final and target class and methods must not be static, final, because it gives problem towards proxy class while extending from target class and overriding target class methods in proxy class.
- ⇒ It is always recommended to take separate class for every cross cutting (secondary logic). It is better not to mixup multiple logics in a single advice /aspect class.
- ⇒ Aspect Weaver uses either CGLIB libraries (or) JDK libraries to generate Proxy class as Runtime Inmemory class.

The best use cases for around advice are

- i) Performance monitoring
  - ii) Caching
  - iii) Around logging.
- ⇒ Cache/buffer is a temporary memory that holds data for temporary period. In client-server application the cache at clientside holds server App results and uses them across the multiple same requests to reduce network round trips between client and server.
  - ⇒ In AOP programming we store target method results in a cache and we use that results across the multiple same methods calls and with same arguments. We generally take either array or collections (HashMap) as cache.
  - ⇒ While configuring cache Aspect/Advice along with other Aspect/Advice it is recommended to configure after cache advice because if target method is getting it's result from cache then

there is no need of performing Logging, PerformanceMonitoring.

#### Proxy interface:

- ⇒ The interface that implemented by target class is called ProxyInterface. It is actually called as proxy interface because the dynamically generated proxy class implements this Interface. We need to configure this interface while configuring "ProxyFactoryBean" class in xml file.
- ⇒ Once proxy interface is configured in xml file the jdk libraries will be used to generate the proxy class dynamically at runtime and this generated proxy class doesn't extend from target class rather it implements proxy interface. Due to this we can take target class as final class and target methods as final methods.

#### ▲ AOPProj2-SpringAOPDecl-AroundAdvice-ProxyInterfaces

- ▲ src/main/java
  - ▲ com.nt.aspect
    - ▶ CacheAdvice.java
    - ▶ LogAroundAdvice.java
    - ▶ PerformanceMonitoringAdvice.java
  - ▲ com.nt.beans
    - ▶ IntrAmtCalculator.java
    - ▶ IntrAmtCalculatorInterface.java
  - ▲ com.nt.cfigs
    - ▶ applicationContext.xml
  - ▲ com.nt.commons
    - ▶ log.properties
  - ▲ test
    - ▶ AopAroundAdviceTest.java
- ▶ src/test/java
- ▶ JRE System Library [J2SE-1.5]
- ▶ Maven Dependencies
- ▶ src
- ▶ target
- ▶ logAround.html
- ▶ pom.xml

#### CacheAdvice.java

```
package com.nt.aspect;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

import org.aopalliance.intercept.MethodInterceptor;
import org.aopalliance.intercept.MethodInvocation;

public class CacheAdvice implements MethodInterceptor {
    private Map<String,Float> cache=new HashMap<String,Float>();
    public Object invoke(MethodInvocation invocation) throws Throwable {
        String key=null;
        Object retVal=null;
        System.out.println(invocation.getClass());
    }
}
```

```
//prepare target method with args as the key  
key=invocation.getMethod().getName()+Arrays.toString(invocation.getArguments());  
//check for the result in cache  
if(!cache.containsKey(key)){  
    System.out.println("from target method");  
    retVal=invocation.proceed(); //calls target method  
    cache.put(key,((Float)retVal));  
}  
else{  
    System.out.println("from cache");  
    retVal=cache.get(key);  
}  
return retVal;  
} //method  
} //class
```

## LogAroundAdvice.java

```
package com.nt.aspect;

import java.util.Arrays;
import java.util.Date;
import org.aopalliance.intercept.MethodInterceptor;
import org.aopalliance.intercept.MethodInvocation;
import org.apache.log4j.Logger;
import org.apache.log4j.PropertyConfigurator;

public class LogAroundAdvice implements MethodInterceptor {
    private static Logger logger=Logger.getLogger(LogAroundAdvice.class);
    public LogAroundAdvice() {
        PropertyConfigurator.configure("src/main/java/com/nt/commons/log.properties");
    }
    public Object invoke(MethodInvocation invocation) throws Throwable {
        Object retVal=0;
        Object args[]=null;
        logger.warn("Entering into "+invocation.getMethod().getName()+
            " with args"+Arrays.toString(invocation.getArguments())+
            "at "+new Date());
        //modify target method args
        args=invocation.getArguments();

        if((((Float)args[0])<10000)
            args[1]=(Float)args[1]-0.5f;

        if((((Float)args[0])<=0 || ((Float)args[1])<=0 || ((Float)args[2])<=0)
            throw new IllegalArgumentException("inputs must be +ve");

        retVal=invocation.proceed();
    }
}
```

```
logger.warn("Exited from "+invocation.getMethod().getName()+
    " with args"+Arrays.toString(invocation.getArguments())+
    "at "+new Date());

if(((Float)retVal)>=5000)
    retVal=((Float)retVal)+ ((Float)retVal)*0.05f;

return retVal;
}
```

#### PerformanceMonitoringAdvice.java

```
package com.nt.aspect;
import java.util.Arrays;
import org.aopalliance.intercept.MethodInterceptor;
import org.aopalliance.intercept.MethodInvocation;
publicclass PerformanceMonitoringAdvice implements MethodInterceptor {
    private long start=0;
    private long end=0;
    public Object invoke(MethodInvocation invocation) throws Throwable {
        //secondary logic
        Object retVal=null;
        start=System.currentTimeMillis(); //pre logic
        retVal=invocation.proceed(); //calls target method
        end=System.currentTimeMillis(); //post logic
        System.out.println(invocation.getMethod().getName()+"with args
        "+Arrays.toString(invocation.getArguments())+" has taken "+(end-start)+" ms");
        return retVal;
    }
}
```

#### IntrAmtCalculator.java

```
package com.nt.beans;
publicfinalclass IntrAmtCalculator implements IntrAmtCalculatorInterface {
    publicfinalfloat calcIntrAmt(float pAmt,float rate,float time){
        //primary logic
        return (pAmt*rate*time)/100.0f;
    } //method
} //class
```

#### IntrAmtCalculatorInterface.java (Proxy Interface)

```
package com.nt.beans;

publicinterface IntrAmtCalculatorInterface {
    public float calcIntrAmt(float pAmt,float time,float rate);
}
```

**applicationContext.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<!-- Configure Target class -->
<bean id="iac" class="com.nt.beans.IntrAmtCalculator"/>
<!-- Configure Aspect/Advice -->
<bean id="pmAdvice" class="com.nt.aspect.PerformanceMonitoringAdvice"/>
<bean id="logAdvice" class="com.nt.aspect.LogAroundAdvice"/>
<bean id="cacheAdvice" class="com.nt.aspect.CacheAdvice"/>
<!-- Configure ProxyFactory Bean to get PROxy obj -->
<bean id="pfb" class="org.springframework.aop.framework.ProxyFactoryBean">
  <property name="target" ref="iac"/>
  <property name="proxyInterfaces">
    <list>
      <value>com.nt.beans.IntrAmtCalculatorInterface</value>
    </list>
  </property>
  <property name="interceptorNames">
    <list>
      <value>cacheAdvice</value>
      <value>pmAdvice</value>
      <value>logAdvice</value>
    </list>
  </property>
</bean>
</beans>
```

Note: When proxy interface is not configured the CGLib libraries will be used to generate proxy class **extending from target class**.

**Before Advice:**

- ⇒ This advice executes before beginning the execution of target method and once advice method is executed then the control automatically goes to **target method**.
- ⇒ After completing the execution of target method, control doesn't come back to before advice method. To develop Before Advice our class must implement `MethodBeforeAdvice()` and should provide implementation to **before (-,-,-) method**.
- ⇒ The use cases of **BeforeAdvice** are **Auditing, Security check and etc...**

**Method Signature:**

**public void before(Method method, Object args[], Object target)**

Method method :- represents target method.

Object args[] :- represents target method arguments.

Object target :- represents target class object



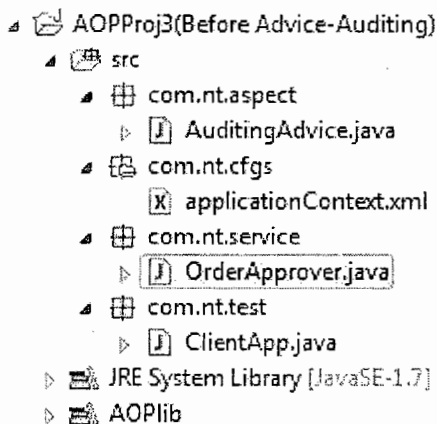
**The important /controlpoints of BeforeAdvice:**

- 1) Can access and modify target method arguments.
- 2) Can not access and modify target method return value because the control doesn't come back to BeforeAdvice, once the target method execution is completed.
- 3) Can not control target method execution being from advice but can stop/abort target method execution by throwing exception in the **before (-,-) method**.

- **Logging keeps track of various components that are involved in the flow of execution of in an Application.**
- Auditing keeps track of various activities that are taken place in the execution of application.
- Generally the audit info will be written to the audit log file or audit table, it's like username, Ordertime, ordered items and etc...

**Note:**

- 1) Keeping track of classes, methods, blocks that are involed in the Application execution is called logging.
- 2) Keeping track of various activities of end user and his operations in the application is called auditing.

**AuditingAdvice.java**

```
package com.nt.aspect;
import java.io.FileWriter;
import java.lang.reflect.Method;
import java.util.Arrays;
import java.util.Date;
import org.springframework.aop.MethodBeforeAdvice;
public class AuditingAdvice implements MethodBeforeAdvice {
    @Override
    public void before(Method method, Object[] args, Object target)
        throws Throwable {
        System.out.println("Target class name"+target.getClass());
        System.out.println("Target method name"+method.getName());
    }
}
```

```
System.out.println("Target method args"+Arrays.toString(args));
if((Integer)args[0]==0)
    throw new IllegalArgumentException("Wrong Order id ");

if((Integer) args[0]<0)
    args[0]=((Integer)args[0])*-1;
//Auditng
FileWriter fw=new FileWriter("E:\\Spring\\AuditLog.txt",true);
fw.write(args[0]+" order has come for approval at"+new Date()+"\n");
fw.flush();
fw.close();
}
}
```

#### OrderApprover.java

```
package com.nt.service;
import java.util.Calendar;
public class OrderApprover {
    public String approveOrder(int orderid){
        // get system date
        Calendar cl=Calendar.getInstance();
        // get date current month
        int month=cl.get(Calendar.MONTH);
        if(month>=7 && month<=10)
            return orderid+" order is not approved";
        else
            return orderid+" order is Approved";
    }
    //method
}
//class
```

#### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">
<!-- Configure target class -->
<bean id="approver" class="com.nt.service.OrderApprover"/>
<!-- Configure Audit advice -->
<bean id="auditingAdvice" class="com.nt.aspect.AuditingAdvice"/>
<!-- Configure ProxyFactoryBean -->
<bean id="pfb" class="org.springframework.aop.framework.ProxyFactoryBean">
    <property name="target" ref="targetBean"/>
    <property name="interceptorNames">
        <list>
            <value>auditingAdvice</value>
        </list>
    </property>
</bean>
</beans>
```

**ClientApp.java**

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.OrderApprover;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext ctx=new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get Bean
        OrderApprover proxy=ctx.getBean("pfb",OrderApprover.class);
        //Call B.methods
        System.out.println("Order Approved?" + proxy.approveOrder(1234));
        System.out.println("-----");
        System.out.println("Order Approved?" + proxy.approveOrder(-4567));
        System.out.println("-----");
        System.out.println("Order Approved?" + proxy.approveOrder(0));
        System.out.println("-----");
    } //main
} //class

```

**BeforeAdvice internals and Flow of execution****OrderApprover.java**

```

public class OrderApprover { (f)
    public String approveOrder(int orderId){
        Calendar calendar=null;
        int month=0;
        //get System Date
        calendar=Calendar.getInstance();
        //get current month
        month=calendar.get(Calendar.MONTH);
        if(month>=7 && month<=10)
            (g) return orderId+" order is rejected";
        else
            (g) return orderId+" order is approved ";
    }
}

```

**OrderAuditingAdvice.java**

```

public class OrderAuditingAdvice implements MethodBeforeAdvice {
    public void before(Method method, Object[] args, Object target) throws Throwable {
        FileWriter fw=null;
        System.out.println("target method name"+method.getName());
        System.out.println("target class name"+target.getClass());
        System.out.println("target method args"+Arrays.toString(args));
        //access and modify target method argument values
        if(((Integer)args[0])<0)
            args[0]=((Integer)args[0])*-1;
        //write audit info to Audit Log file...
        try{
            fw=new FileWriter("e:/AuditLog.txt");
            fw.write(args[0]+" order has come for approval at "+new Date());
            fw.flush();
            fw.close();
        } //try
        catch(Exception e){
            e.printStackTrace();
        }
    } //method
} //class

```

**ClientApp**

```

(i) (a)
String result= proxy.approveOrder(1001);

```

Generated In Memory Proxy class

```

public class OrderApprover$Proxy extends OrderApprover implements ApplicationContextAware{
    private ApplicationContext ctx=null;
    public setApplicationContext(ApplicationContext ctx){
        this.ctx=ctx;
    }
    public String approverOrder(int orderId){
        //get Advice object
        OrderAuditingAdvice advice=ctx.getBean("auditAdvice",OrderAuditingAdvice.class);
        //get Target object
        OrderApprover target=ctx.getBean("approver",OrderApprover.class);
        //prepare args that required to call before(-,-,-)
        Method method=target.getClass().getDeclaredMethods()[0];
        Object args[]=new Object[]{orderId};

        //invoke before method (c)
        advice.before(method,args,target);

        Object retVal=super.approveOrder(orderId);
        return retVal; (h)
    }
}

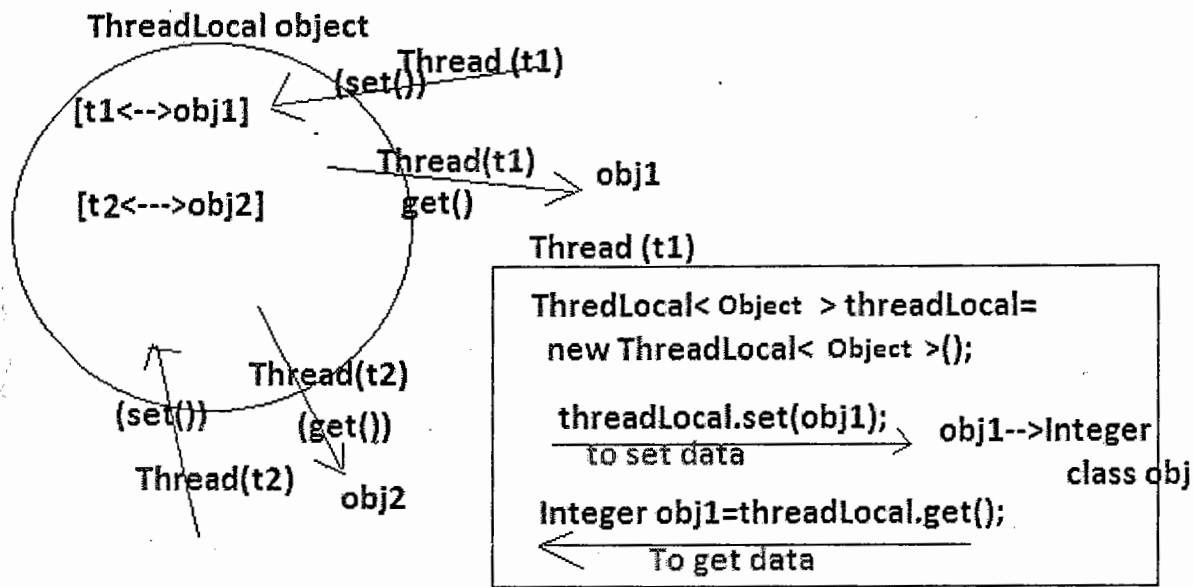
```

### Before Advice(UseCase2)

- ⇒ Security means authentication and authorization. Checking identity of a user is called authentication. Checking the access permissions of users is called authorization. Security is mainly about **authentication**
- ⇒ We need to authentication the user before entering into target method/business method. So it is recommended to develop **security logic as before advice logic**.
- ⇒ Since target method call can not supply username, password everytime. We need to make client performing login/signin operation, submitting username, password before calling target method, so these username, password must be saved temporarily and should use them when Real B.Method/ target method is called ,For this we can use **ThreadLocal object**.
- ⇒ If we want to make data specific to each thread having global visibility without using local variables and synchronization concepts then we can keep that data in **ThreadLocal object that is taken as instance variable**.
- ⇒ This ThreadLocal object allows each thread to keep one object at a time and doesn't allow other threads to access and use that object. So we can say ThreadLocal object maintaining data in Thread scope having thread safety.

In java we can keep data in 4 scopes.

- 1) Local
- 2) Instance
- 3) class
- 4) Thread scope

**Example**

UserList (db table in Oracle)

| <u>Uname(VC2)</u> | <u>pwd(VC2)</u> |
|-------------------|-----------------|
| Raja              | Rani            |
| King              | Kingdom         |

→ Without using Synchronization Concepts, if you want to maintain thread safe data having global visibility use ThreadLocal object. The data placed by one thread in ThreadLocal object can be accessed and modified only by that thread.

→ ThreadLocal object is internally a Map collection having thread names as keys, objects as the values.

→ While Operating ATM we give CardNo, PIN but they will not be used for authentication immediately, they will be actually used for authentication before performing Serious business operations like withdraw, deposit and etc...

- ▲ AOPProj4(SpringAOP-Decl BeforeAdvice-Security)
  - ▲ src
    - ▲ com.nt.aspect
      - ▶ AuthenticationManager.java
      - ▶ SecurityAdvice.java
      - ▶ UserDetails.java
    - ▲ com.nt.cfgs
      - ▶ applicationContext.xml
    - ▲ com.nt.dao
      - ▶ AuthenticateDAO.java
    - ▲ com.nt.service
      - ▶ IntrAmtCalculator.java
    - ▲ com.nt.test
      - ▶ ClientApp.java
  - ▶ JRE System Library [JavaSE-1.7]
  - ▶ AOPlib
  - ▲ Referenced Libraries
    - ▶ spring-jdbc-4.1.6.RELEASE.jar - C:\Users\de
    - ▶ spring-tx-4.1.6.RELEASE.jar - C:\Users\del\I
    - ▶ ojdbc14.jar - D:\oracle\product\10.2.0\db\_2

#### AuthenticateDAO.java

```
package com.nt.dao;
import org.springframework.jdbc.core.JdbcTemplate;
import com.nt.aspect.UserDetails;
public class AuthenticateDAO {
    private static final String AUTH_QRY="SELECT COUNT(*) FROM USERLIST WHERE UNMAE=?
AND PWD=?";
    private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    public boolean authenticate(UserDetails ud){
        int cnt=jt.queryForInt(AUTH_QRY, ud.getUser(),ud.getPwd());
        if(cnt==0)
            return false;
        else
            return true;
    }
}
```

#### AuthenticationManager.java

```
package com.nt.aspect;
import com.nt.dao.AuthenticateDAO;
public class AuthenticationManager {
    private ThreadLocal<UserDetails> tl=new ThreadLocal<UserDetails>();
    private AuthenticateDAO dao;
    public void setDao(AuthenticateDAO dao) {
        this.dao = dao;
    }
}
```

```
public void signIn(String user,String pwd){
    UserDetails ud=new UserDetails(user,pwd);
    tl.set(ud);
}
public void signOut(){
    tl.remove();
}
public boolean isAuthenticated(){
    //get UserDetails obj
    UserDetails ud=tl.get();
    boolean flag=dao.authenticate(ud);
    return flag;
}
}
```

#### SecurityAdvice.java

```
package com.nt.aspect;
import java.lang.reflect.Method;
import org.springframework.aop.MethodBeforeAdvice;
public class SecurityAdvice implements MethodBeforeAdvice{
    private AuthenticationManager am;
    public void setAm(AuthenticationManager am) {
        this.am = am;
    }
    @Override
    public void before(Method method, Object[] args, Object target)
        throws Throwable {
        //perform Authentication
        boolean flag=am.isAuthenticated();
        if(flag==false)
            throw new IllegalArgumentException("In Valid Credentials");
    }
}
```

#### UserDetails.java

```
package com.nt.aspect;
public class UserDetails {
    private String user,pwd;
    public String getUser() {
        return user;
    }
    public void setUser(String user) {
        this.user = user;
    }
    public String getPwd() {
        return pwd;
    }
}
```

```
public void setPwd(String pwd) {
    this.pwd = pwd;
}
public UserDetails() {
}
public UserDetails(String user, String pwd) {
    this.user = user;
    this.pwd = pwd;
}
}
```

#### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">

<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
    <property name="username" value="scott"/>
    <property name="password" value="tiger"/>
</bean>

<bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="dataSource"/>
</bean>

<bean id="authDAO" class="com.nt.dao.AuthenticateDAO">
    <property name="jt" ref="template"/>
</bean>

<bean id="authManager" class="com.nt.aspect.AuthenticationManager">
    <property name="dao" ref="authDAO"/>
</bean>

<!--target class -->
<bean id="targetBean" class="com.nt.service.IntrAmtCalculator"/>

<!--Cfg advice class -->
<bean id="securityAdvice" class="com.nt.aspect.SecurityAdvice">
    <property name="am" ref="authManager"/>
</bean>

<bean id="pfb" class="org.springframework.aop.framework.ProxyFactoryBean">
    <property name="target" ref="targetBean"/>
    <property name="interceptorNames">
        <list>
            <value>securityAdvice</value>
        </list>
    </property>
</bean>
```



```
</property>
</bean>
</beans>
```

**IntrAmtCalculator.java**

```
package com.nt.service;
public class IntrAmtCalculator {
    public float calcIntrAmt(float p,float t,float r){
        return p*t*r/100.0f;
    }
}
```

**ClientApp.java**

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.aspect.AuthenticationManager;
import com.nt.service.IntrAmtCalculator;
public class ClientApp {
    public static void main(String[] args) throws Exception{
        //Create IOC Container
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        // get AuthnticationManager obj
        AuthenticationManager
        authManager=ctx.getBean("authManager",AuthenticationManager.class);
        //signIn
        authManager.signIn("raja","rani");
        //call b.method
        IntrAmtCalculator proxy=ctx.getBean("pfb",IntrAmtCalculator.class);
        float amt=proxy.calcIntrAmt(20000,10,2);
        System.out.println("Intr Amt="+amt);
        //signOUT
        authManager.signOut();
    } //main
} //class
```

Jar: AOP+DAO+IOC

**AfterAdvice/AfterReturningAdvice:**

- ⇒ This advice method/Logic executes once target method finishes the execution and **before** it returns the control to caller method.
- ⇒ This advice can catch the return value but cannot modify the return value. To develop this advice we need to take a class implementing **AfterReturningAdvice(i)** and should provide implementation for **afterReturning(-,-,-)** method.
- ⇒ **Signature :**  
public void afterReturing(Object retVal,Method method,Object args[],Object target)

**Useases:**

- i) Generating discount voucher for next purchase based on current purchase bill

amount.

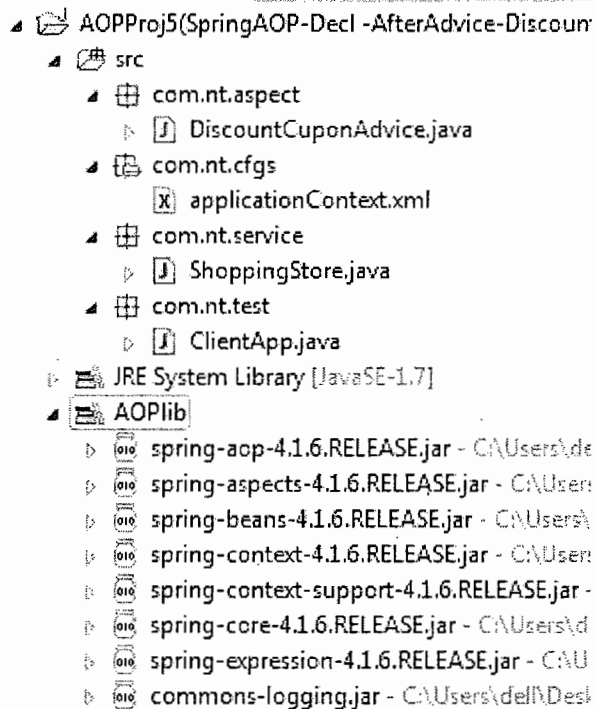
ii) ATM pin generation.

**Important/Control Points:**

- a) We can access target method arguments, but there is no use of modifying them because control comes to target method only after finishing execution of target method.
- b) Cannot control target method execution because control comes to advice method only after executing target method. But by throwing exception, we can stop target method sending return value to caller method and verification.
- c) Can access the return value, but cannot modify return value. if modified that will not be reflected.

→note: AOP is designed based on Decorator, Proxy design patterns.

Example application on AfterAdvice to generate Discount Coupon based OnBillAmount.



**DiscountCouponAdvice.java**

```
package com.nt.aspect;
import java.io.FileWriter;
import java.lang.reflect.Method;
import org.springframework.aop.AfterReturningAdvice;
public class DiscountCouponAdvice implements AfterReturningAdvice {
    @Override
    public void afterReturning(Object returnValue, Method method, Object[] args,
        Object target) throws Throwable {
        float billAmt=(Float)returnValue;
        //we can not modify return Value.
        billAmt=billAmt+100;
        returnValue=billAmt;
    }
}
```

```
String msg=null;
if(billAmt>=50000)
    msg="Avail 30% Discount on next purchase";
else if(billAmt>=30000)
    msg="Avail 20% Discount on next purchase";
else
    msg="Avail 10% Discount on next purchase";

//store discount coupon in a file
FileWriter fw=new FileWriter("D:\\cupon.Tt");
fw.write(msg);
fw.flush();
fw.close();
} //afterReturning
} //class
```

#### ShoppingStore.java

```
package com.nt.service;
public class ShoppingStore {
    public float calcBillAmt(String item,float price,int qty){
        return price*qty;
    }
}
```

#### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
<!--cfg target bean -->
<bean id="store" class="com.nt.service.ShoppingStore"/>
<!--cfg advice bean -->
<bean id="discountAdvice" class="com.nt.aspect.DiscountCouponAdvice"/>
<!--cfg Proxy Factory Bean -->
<bean id="pfb" class="org.springframework.aop.framework.ProxyFactoryBean">
    <property name="target" ref="store"/>
    <property name="interceptorNames">
        <list>
            <value>discountAdvice</value>
        </list>
    </property>
</bean>
</beans>
```

#### ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.ShoppingStore;
public class ClientApp {
    public static void main(String[] args) {
```

```

        // create IOC Container
        ApplicationContext ctx=new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get Proxy obj
        ShoppingStore proxy=ctx.getBean("pfb",ShoppingStore.class);
        //call target method
        float billAmt=proxy.calcBillAmt("table", 3000,20);
        System.out.println("Bill Amount="+billAmt);
    }
} //main & Class

```

→ We can use **AfterAdvice** to check wheather the return value generated by target method is valid value or not. If not valid we can throw exception in Afteradvice and we can stop that return value being returned to ClientApp (caller).

**Usecase:** If target method returns ATM pin we can check weather that generated pin is weak pin or strong pin in Afteradvice. If it is a weak pin then we throw exception

- ▲  AOPProj6(SpringAOP-Decl AfterAdvice-PinVerifier)
  - ▲  src
    - ▲  com.nt.aspect
      - ▷  CheckPinAdvice.java
    - ▲  com.nt.cfgs
      - ▷  applicationContext.xml
    - ▲  com.nt.service
      - ▷  PinGenerator.java
    - ▲  com.nt.test
      - ▷  ClientApp.java
  - ▷  JRE System Library [JavaSE-1.7]
  - ▷  AOPlib
  - ▷  Referenced Libraries

### CheckPinAdvice.java

```

package com.nt.aspect;
import java.lang.reflect.Method;
import java.util.Arrays;
import org.springframework.aop.AfterReturningAdvice;
public class CheckPinAdvice implements AfterReturningAdvice {
    @Override
    public void afterReturning(Object retValue, Method method, Object[] args,
        Object target) throws Throwable {
        System.out.println("Generated pin"+retValue);
        System.out.println("target class"+target.getClass());
        System.out.println("target method"+method.getName());
        System.out.println("target method args"+Arrays.toString(args));
        int cardNo=(Integer)args[0]+100;
        //get return value
        int pin=(Integer)retValue;
        //check the pin
        if(pin<=999)

```

```
        throw new IllegalArgumentException(" Weak Pin generated");
    } //afterReturning
} //class
```

#### PinGenerator.java

```
package com.nt.service;
import java.util.Random;
public class PinGenerator {
    public int generatePin(int cardNo){
        //generate ATM Pin
        Random rad=new Random();
        int pin =rad.nextInt(9999);
        return pin;
    }
}
```

#### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <!--target bean -->
    <bean id="pin" class="com.nt.service.PinGenerator"/>
    <!--service bean -->
    <bean id="pinAdvice" class="com.nt.aspect.CheckPinAdvice"/>

    <bean id="pfb" class="org.springframework.aop.framework.ProxyFactoryBean">
        <property name="target" ref="pin"/>
        <property name="interceptorNames">
            <list>
                <value>pinAdvice</value>
            </list>
        </property>
    </bean>
</beans>
```

#### ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.PinGenerator;
public class ClientApp {
    public static void main(String[] args) {
        // create IOC Container
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get Proxy obj
        PinGenerator proxy=ctx.getBean("pfb",PinGenerator.class);
    }
}
```

```
//call target method
int pin=proxy.generatePin(88888);
System.out.println("Generated Pin="+pin);
//main
} //class
```

### ThrowsAdvice:

- ⇒ This advice will execute when exception is raised in target method. Using this advice we cannot catch and **handle the exception, but we can see the details of exception that is raised.**
- ⇒ We can use this advice to develop **common/central Exception logger** that logs all the exception related messages that are raised in various target methods of target classes.
- ⇒ We can catch and handle the exception raised by target method in client App where target method will be called.
- ⇒ To develop this advice we need to take class that implements "ThrowsAdvice (i)" (marker interface--->Empty interface) and we need to place **one or more afterThrowing methods having the following signatures.**
- ⇒ **public void afterThrowing (<sub class of Throwable>)**  
Or
- ⇒ **public void afterThrowing (Method method, Object args[], Object target <subclass of throwable> )**

### **Control points/important points:**

- a) We can access Target Method arguments, but there is no use of modifying them.
- b) We can not control target method execution ,because control comes to throwsAdvice after executing target method , that to for exception raising.
- c) We can not catch the return value of target method.

### **NOTE:**

Throws advice logic not only executes when exception is raised in target method. It also responds for the exception raised in other advices that are configured on the target method...

### **Uses:**

- 1) To develop common/central exception logger that logs all exception related messages(useful in incident management)
- 2) To cascade/translate the technology specific exceptions to project specific custom exceptions (to achieve loose coupling between Presentation tier and Business tier components)

Note: servlet, jsp, html etc...files are called view, controller components(**presentation tier components**). DAO and service classes are called **business-tier components**. If business tier components propagates the technology specific exceptions to presentation-tier components then tight coupling problem will be raised. **To solve this problem use ThrowsAdvice to translate technology exceptions to project specific custom exceptions.**

- ▲ AOPProj7(Spring AOP Decl-ThrowAdvice-Exception)
  - ▲ src
    - ▲ com.nt.aspect
      - ▶ ExceptionLoggerAdvice.java
    - ▲ com.nt.cfgs
      - ▶ applicationContext.xml
    - ▲ com.nt.service
      - ▶ PriceCalculator.java
    - ▲ com.nt.test
      - ▶ ClientApp.java
  - ▶ JRE System Library [JavaSE-1.7]
  - ▶ AOPlib

### ExceptionLoggerAdvice.java

```
package com.nt.aspect;
import java.lang.reflect.Method;
import java.util.Arrays;
import org.springframework.aop.ThrowsAdvice;
public class ExceptionLoggerAdvice implements ThrowsAdvice {
    public void afterThrowing(IllegalArgumentException ie){
        throw new NumberFormatException("Changed exception");
    }
    public void afterThrowing(Method method, Object args[], Object target, Throwable ie){
        System.out.println(ie.getClass()+" Exception is raised in "+method.getName()+" with args
        "+Arrays.toString(args)+" in "+target.getClass());
    }
}
```

### PriceCalculator.java

```
package com.nt.service;
public class PriceCalculator {
    public float calcPrice(float price, float qty){
        if(price==0.0f || qty==0.0f)
            throw new IllegalArgumentException(" price and qty must not be zero..");
        return price*qty; }
}
```

### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

<!--target bean -->
<bean id="price" class="com.nt.service.PriceCalculator"/>
<!--advice bean -->
<bean id="throwsAdvice" class="com.nt.aspect.ExceptionLoggerAdvice"/>

<bean id="pfb" class="org.springframework.aop.framework.ProxyFactoryBean">
    <property name="target" ref="price"/>
</bean>
```

```
<property name="interceptorNames">
  <list>
    <value>throwsAdvice</value>
  </list>
</property>
</bean>
</beans>
```

#### ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.PriceCalculator;
public class ClientApp {
    public static void main(String[] args) {
        // create IOC Container
        ApplicationContext ctx = new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get Proxy obj
        PriceCalculator proxy=ctx.getBean("pfb",PriceCalculator.class);
        //call target method
        try{
            float price=proxy.calcPrice(45.65f,0);
            System.out.println("Price="+price);
        }
        catch(Exception ie){
            System.out.println("Internal Problem"+ie.getClass()+" "+ie.getMessage());
        }
    }
}
//main
}
//class
```

⇒ If exception is raised in target method, control comes to ThrowsAdvice and looks for afterThrowing() method in the following order.

- 1) Exact Exception type
- 2) Nearest super class of Raised Exception
- 3) Super class of Raised Exception.

Note: If multiple afterThrowing methods contain exact match then less params(one param) method executes.

Note: If multiple afterThrowing methods contain super class of raised exception then less param method(one) executes.

Note: If one afterThrowing() methods contains exact match and another afterThrowing() contains super type then the exact match afterThrowing() will execute. If both are having different types of classes then the method that is having nearest super class param will execute.



- ⇒ We can also use ThrowsAdvice to graph/cascade/translate one form of Exception to another form of Exception. Like Translating the business tier components generated technology specific exception to application/project specific custom exceptions.

| <u>Advice Type</u>   | <u>When advice logic executes</u>         | <u>Access to target method args</u> | <u>Access to target method return value</u> | <u>Control on target method execution</u>                                     |
|----------------------|-------------------------------------------|-------------------------------------|---------------------------------------------|-------------------------------------------------------------------------------|
| <b>Around Advice</b> | Around target method execution            | Can see and modify                  | Can see and modify                          | Yes<br><br>(UseCases:Caching,Around logging,Performance monitoring)           |
| <b>Before Advice</b> | Before the target method Execution        | Can see and modify                  | NA                                          | NA (can stop by throwing Exception)<br><br>(UseCases:Auditing,Security Check) |
| <b>After Advice</b>  | After target method execution             | Can see but can not modify          | Can see but can not modify                  | NA<br><br>(UseCases:GiftVoucher generator,ATM Pin generator)                  |
| <b>Throws Advice</b> | When exception is raised in target method | Can see but can not modify          | NA                                          | NA<br><br>(UseCases:CommonException on logger ,Exception Translator)          |

### Programatic Spring AOP:

- ⇒ Here no xml file support or bean configuration file support will be taken to generate proxy class by supplying Target class, advice classes. All inputs will be given directly in client application through java statements. This is not recommended approach. Here we use **ProxyFactory** class directly in client app to generate the proxy class.

```
ProxyFactory factory= new ProxyFactory();  
factory.setTarget (--);  
factory.addAdvice(--);  
proxy=factory.getProxy();  
proxy.bm1();  
proxy.bm2(); //call B.methods
```

- ⇒ Programatic SpringAO is not recommended to use, because it makes the programmer to modify java code to enable or disable Aspects and doesnot allow programmer to take the benefits of **spring's IOC Container**.

#### PointCut:

- ⇒ So far the aspects that we are developing will be applied on all the target methods of target class. In some situations we don't want to apply aspects on **certain joinpoints or methods of target class**.
- ⇒ For this we can write additional conditional logic in advice class to execute advice logic only for specific joinpoint methods (Target methods)

```
public class PerformanceMonitoringAdvice implements MethodInterceptor{  
  
    public Object invoke(MethodInvocation invocation) throws Throwable {  
  
        String methodName=null;  
  
        methodName=invocation.getMethod().getName();  
  
        if(methodName.equals("add")){  
  
            //performance monitoring logic  
  
        }  
        else{  
            return invocation.proceed();  
        }  
    }  
}
```

- ⇒ In the above approach the generated proxy class overrides all the methods of target class having logic to apply aspects on target methods. But advice logic will be executed only for specific method because of the condition that is specified. This indicates proxy class is not generated in optimized manner because it is also overriding those target methods on which we don't want to apply aspects, moreover those methods of target class should be executed directly without entering into proxy class when they are called from clients.

- ⇒ To overcome the above problem use pointcuts. Pointcut is a collection of **joinpoints** on whom you want to advice the aspects. It specifies which aspects have to be applied on which **JoinPoints**.

There are 2 types of pointcuts.

**1) static Pointcut:**

- ⇒ Allows to specify target class name, method names while preparing the Pointcut. But we can't specify target method argument values. This indicates static Pointcuts will be processed during weaving process that generates proxy class.

**2) Dynamic Pointcut:**

- ⇒ This allows to specify target class, target method names and their arguments values. That means this pointcut will be processed after target method call on proxy object dynamically at runtime.

Eg: Apply aspect on "add" method is static pointcut.

Eg: Apply aspect on "add" method when argument values are <100 is dynamic pointcut.

- ⇒ While working with pointcuts the generated proxy class overrides only those methods that are specified in pointcut to apply aspects. **Due to this proxy class code will be optimized to improve performance.**

- ⇒ To develop userdefined pointcut we need to take a class implementing pointcut Interface. Spring api has supplied some predefined classes implementing this interface. They are

Static pointcut classes

- **StaticMethodMatcherPointcut (AC)**
- **NameMatchMethodPointcut (C)**
- **JdkRegexpMethodPointcut (C)**

Dynamic pointcut classes

- **DynamicMethodMatcherPointcut (AC)**

- ⇒ To develop custom pointcut classes we can extend from above classes instead of implementing Pointcut Interface.

- ⇒ **Advisor is the combination of Pointcut+Aspect/advice.**

- ⇒ In order to apply aspects specifying pointcuts we take the support of Advisor. Every advisor class is the implementation class of Advisor (i)

- ⇒ The spring API supplied predefined Advisors are

- **DefaultPointcutAdvisor**
- **NameMatchMethodPointcutAdvisor and etc..**

- ⇒ While working with static pointcut classes we need to override matches(-,-) method that gives access to target class and target method. If this method returns true for certain target method then aspect will be applied for that method, otherwise aspect will not be applied.

- ⇒ In order to develop **Dynamic pointcut**, we need to take a class extending from **DynamicMethodMatcherPointcut** class and implement **matches(Method method, Object [] args, class<?> target)** method. If this method returns true for the certain target method then aspect will be applied on that method otherwise aspect will not be applied.

Eg:

```
graph TD
    AOPProj9[AOPProj9(Spring AOP Programatic)] --> src[src]
    src --> com_nt_aspect[com.nt.aspect]
    src --> com_nt_service[com.nt.service]
    src --> com_nt_test[com.nt.test]
    com_nt_aspect --> CacheAdvice[CacheAdvice.java]
    com_nt_aspect --> LogAroundAdvice[LogAroundAdvice.java]
    com_nt_aspect --> PerformanceMonitoringAdvice[PerformanceMonitoringAdvice.java]
    com_nt_service --> ArithmeticService[ArithmeticService.java]
    com_nt_test --> ClientApp[ClientApp.java]
    AOPProj9 --> JRE[JRE System Library [JavaSE-1.7]]
    AOPProj9 --> AOPlib[AOPlib]
```

### CacheAdvice.java

```
package com.nt.aspect;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import org.aopalliance.intercept.MethodInterceptor;
import org.aopalliance.intercept.MethodInvocation;
public class CacheAdvice implements MethodInterceptor {
    Map <String,Object> cache=new HashMap<String,Object>();
    @Override
    public Object invoke(MethodInvocation invocation) throws Throwable {
        String key=invocation.getMethod().getName()+Arrays.toString(invocation.getArguments());
        Object retValue=null;
        if(!cache.containsKey(key)){
            retValue=invocation.proceed();
            cache.put(key, retValue);
            System.out.println("From target method");
            return retValue;
        }
        retValue=cache.get(key);
        System.out.println("From cache");
        return retValue;
    }
}
```

### PerformanceMonitoringAdvice.java

```
package com.nt.aspect;
import java.util.Arrays;
import org.aopalliance.intercept.MethodInterceptor;
import org.aopalliance.intercept.MethodInvocation;
public class PerformanceMonitoringAdvice implements MethodInterceptor {
    long start,end;
```

```
@Override
public Object invoke(MethodInvocation invocation) throws Throwable {
    start=System.currentTimeMillis();
    Object retVal=invocation.proceed();
    end=System.currentTimeMillis();
    System.out.println(invocation.getMethod().getName()+" with
args"+Arrays.toString(invocation.getArguments())+" has taken "+(end-start)+"ms");
    return retVal;
}
}
```

**CommonDynaPointcut.java**

```
package com.nt.poincut;
import java.lang.reflect.Method;
import org.springframework.aop.support.DynamicMethodMatcherPointcut;
import com.nt.service.ArithmeticService;
public class CommonDynaPointcut extends DynamicMethodMatcherPointcut {
    @Override
    public boolean matches(Method method, Class<?> target, Object[] args) {
        if(target==ArithmeticService.class && method.getName().equals("mul") &&
(Integer)args[0]>=1000 &&(Integer)args[1]>=1000)
            return true;
        else
            return false;
    }
}
```

**PerformanceMonitoringPointcut.java [Static pointcut]**

```
package com.nt.poincut;
import java.lang.reflect.Method;
import org.springframework.aop.support.StaticMethodMatcherPointcut;
import com.nt.service.ArithmeticService;
public class PerformanceMonitoringPointcut extends StaticMethodMatcherPointcut {
    @Override
    public boolean matches(Method method, Class<?> target) {
        if(target==ArithmeticService.class && method.getName().equals("add"))
            return true;
        else
            return false;
    }
}
```

**ArithmeticService.java [Target class]**

```
package com.nt.service;
public class ArithmeticService {
    public int add(int x,int y){
        return x+y;
    }
    public int sub(int x,int y){
        return x-y;
    }
}
```

```
}  
public int mul(int x,int y){  
    return x*y;  
}  
}
```

**ClientApp.java [ programmatic/applying pointcut/static/dynamic]**

```
package test;  
import org.springframework.aop.framework.ProxyFactory;  
import org.springframework.aop.support.DefaultPointcutAdvisor;  
import org.springframework.aop.support.NameMatchMethodPointcut;  
import com.nt.aspect.CacheAdvice;  
import com.nt.service.ArithmeticService;  
public class ClientApp {  
    public static void main(String[] args) {  
        //Get ProxyFactory class obj  
        ProxyFactory pf=new ProxyFactory();  
        //set Target class obj  
        pf.setTarget(new ArithmeticService());  
        /*  
        //set Advice class obj  
        pf.addAdvice(new PerformanceMonitoringAdvice()); */  
        /*  
        //prepre advisor (poincut+ advice/aspect)  
        DefaultPointcutAdvisor advisor=  
            new DefaultPointcutAdvisor(new PerformanceMonitoringPointcut(),  
                new PerformanceMonitoringAdvice());  
        pf.addAdvisor(advisor); */  
        /*  
        DefaultPointcutAdvisor PMAdivisor=  
            new DefaultPointcutAdvisor(new CommonDynaPointcut()  
                ,new PerformanceMonitoringAdvice());  
        DefaultPointcutAdvisor Cacheadvisor=  
            new DefaultPointcutAdvisor(new CommonDynaPointcut(), new CacheAdvice());  
        pf.addAdvisor(0,Cacheadvisor);  
        pf.addAdvisor(1, PMAdivisor); */  
        /*DefaultPointcutAdvisor PMAdivisor=  
            new DefaultPointcutAdvisor(new PerformanceMonitoringPointcut(),  
                new PerformanceMonitoringAdvice());  
        DefaultPointcutAdvisor Cacheadvisor=  
            new DefaultPointcutAdvisor(new CommonDynaPointcut(), new CacheAdvice());  
        pf.addAdvisor(0,PMAdivisor);  
        pf.addAdvisor(1,Cacheadvisor);  
        /*  
        NameMatchMethodPointcut nmp=new NameMatchMethodPointcut();
```

```
nmp.setMappedNames("add","mul");
DefaultPointcutAdvisor advisor=
    new DefaultPointcutAdvisor(nmp,new CacheAdvice());
pf.addAdvisor(advisor);

//get Proxy obj
ArithmeticService proxy=(ArithmeticService)pf.getProxy();

// Call target methods
System.out.println("Sum="+proxy.add(1000,2000));
System.out.println("-----");
System.out.println("Sub="+proxy.sub(100000,20000));
System.out.println("-----");
System.out.println("Mul="+proxy.mul(10000,20000));
System.out.println("-----");
System.out.println("Mul="+proxy.mul(10000,20000));
System.out.println("-----");
System.out.println("Mul="+proxy.mul(100,200));
System.out.println("-----");
} //main
} //class
```

⇒ Instead of developing separate userdefined class by extending from abstract class **StaticMethodMatcherPointcut** or **DynamicMethodMatcherPointcut** we can use spring API supplied concrete static pointcut class **NameMatchMethodPointcut** as shown below:

```
NameMatchMethodPointcut nmp=new NameMatchMethodPointcut();
nmp.setMappedNames("sum", "mul");
DefaultPointcutAdvisor advisor=
    new DefaultPointcutAdvisor(nmp, new PerformanceMonitoringAdvice());
factory.addAdvisor(advisor);
```

**Standard procedure to work with Poincuts(programmatic):**

a) Create object for userdefined or predefined poincuts

```
NameMatchMethodPointcut nmp=new NameMatchMethodPointcut();

nmp.setMappedNames("add","mul");
```

b) Create Advice class object

```
PerformanceMonitoringAdvice pmAdvice= new PerformanceMonitoringAdvice();
```

c) Create Advisor class object

```
DefaultPointcutAdvisor advisor=new DefaultPointcutAdvisor(nmp, pmAdvice);
```

Note: we can directly use "NameMatchMethodPointcutAdvisor" to avoid step (a).

step d) prepare Advisor having having pointcut and aspect

```
NameMatchMethodPointcutAdvisor advisor=new NameMatchMethodPointcutAdvisor();  
  
advisor.setMethodName("add");  
  
advisor.addAdvice(pmAdvice);
```

Step e) add Advisor ProxyFactory object

```
ProxyFactory factory= new ProxyFactory();  
  
factory.setTarget(----);  
  
factory.addAdvisor(advisor);
```

Step f) Get proxy class

```
ArithmeticService proxy=factory.getProxy();
```

### Working with Pointcut in declarative spring AOP:

→ In this approach we can configure pointcut classes, advisor classes, Advice classes as beans in Spring bean configuration file and we can specify advisors directly as the values of "interceptorNames" property while configuring ProxyFactoryBean.

→ Instead of working with "NameMatchMethodPointcut", "DefaultPointcutAdvisor" classes separately we can use "NameMatchMethodPointcutAdvisor" class directly to bring the same effect.

### Standard procedure to work with pointcuts(declarative)

a) Configure user defined or predefined pointcut class as spring bean

```
<bean id="nmp" class="pkg.NamedMatchMethodPointcut">  
    <property name="mappedName" value="add"/>  
</bean>
```

b) Configure Advisor class as spring bean

```
<bean id="advisor" class="pkg.DefaultPointcutAdvisor">  
    <property name="advice" value="pmAdvice"/>  
  
    <property name="pointcut" value="nmp"/>  
  
</bean>
```

Note: we can directly configure "NamedMatchMethodPointcutAdvisor" to avoid step(a).

c) Configure ProxyFactoryBean

```
<bean id="pfb" class="pkg.ProxyFactoryBean">  
  
    <property name="target" value="-----"/>  
  
    <property name="interceptorNames">  
  
</list>
```



```

<value>advisor</value>

</list>

</property>

</bean>

```

- Use spring AOP programmatic/declarative environment we can work with pointcuts in 3 ways
  - 1) Develop user defined pointcut class and inject to Advisor.
  - 2) Use spring API supplied predefined concrete class and inject to Advisor
  - 3) Use spring API supplied predefined Advisor class. Which also internally maintains pointcut effect.
- ⇒ With respect to SpringAOP declarative pointcuts We can develop static Pointcut either as user defined class or by extending from StaticMethodMatcherPointcut or we can use spring specific direct concrete classes like **NameMatchMethodPointcut**.
- ⇒ Inorder to work with DynamicMethodPointcut we must take user defined class extending from **DynamicMethodMatcherPointcut** and there is no direct concrete classes supplied by spring API to work with dynamic pointcuts.

### AspectJAOP

- ⇒ A third party AOP framework, where we can develop aspect classes as pojo classes without having tight coupling with spring API. i.e **aspect class need not implement spring api interface and need not to extend spring API class**, but we need to use AspectJ api inside the aspects classes.

**Difference between spring AOP and AspectJ AOP.**

#### Spring AOP

#### Plain Aspectj AOP

|                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>⇒ Performs runtime weaving</li> <li>⇒ Generates proxy class at runtime</li> <li>⇒ Supports only methods as JoinPoints.</li> <li>⇒ Support both static and dynamic pointcuts</li> <li>⇒ Doesn't allow annotations</li> <li>⇒ Can be used in programmatic, Decalarative xml approach.</li> <li>⇒ Advice classes are non POJO classes.</li> </ul> | <ul style="list-style-type: none"> <li>=&gt; Performs compile time weaving</li> <li>=&gt; Generates proxy class at compile time. Uses AspectJ compiler.</li> <li>=&gt; Supports fields, constructors methods as Joinpoints.</li> <li>=&gt; Supports only static Pointcuts.</li> <li>=&gt; Allows annotations.</li> <li>=&gt; can be used in declarative (xml) and annotations.</li> <li>=&gt; Advice classes are POJO classes.</li> </ul> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

- ⇒ The limitations with Spring AOP is the Advice classes are invasive classes (should implement or extend from Spring API interface or class).
- ⇒ From spring 2.0 the AspectJ AOP is integrated with spring AOP due to this some limitations are applied on AspectJAOP

- Supports only method level joinpoints.
- Performs runtime weaving i.e proxy class will be generated at runtime.
- Still supports only static pointcuts.
- No AspectJ compiler is required.

We can use spring integrated AspectJ AOP in 2 ways

- a) Declarative AspectJ AOP (using xml)
- b) Annotations AspectJ AOP

**Note: AspectsJAOP also supports 4 types of Advices**

**Declarative AspectsJAOP:**

- ⇒ Here we develop aspect classes as pojo classes by using AspectJ api internally and we configure them in xml file by using tags as aspect classes. We need to import AOP namespace for this configuration.
- ⇒ The important tags are

<aop: config>---->to enable AspectJ AOP from spring container.  
<aop: aspect>---->to config spring bean as Aspect class.  
<aop: around>----->to configure aspect class method as Around advice.  
<aop: before>----->to configure aspect class method as before advice  
<aop: after-returning>----> to configure aspect class method as after advice.  
<aop: after-throwing>----->to configure aspect class method s throws advice  
<aop: pointcut>----->to configure pointcut

We use OGNL for pointcut expression

Syntax:

execution (<return type> <package name>.<class name>.<method name> (..))

**Around Advice:**

- ⇒ This advice executes around target methods.
- ⇒ The usecases are **caching, logging, PperformanceMonitoring and etc....**
- ⇒ To develop this advice we can use any arbitrary class having any arbitrary method with following signature.

**public Object <method>(ProceedingJoinPoint pjp) throws Throwable**

→ ProceedingJoinPoint **pjp** can be used to get target class, method info and to control target method invocation.

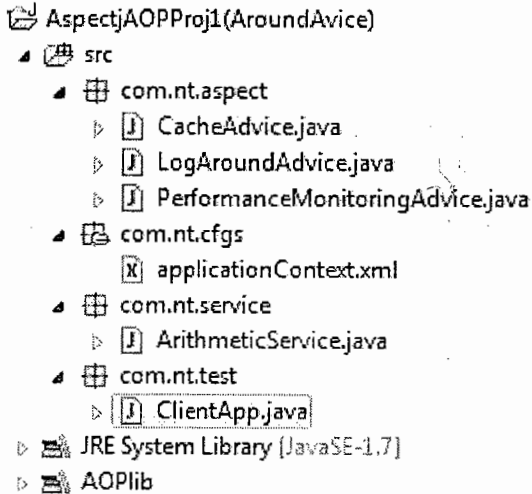
Note:- In spring bean cfg file, It is always recommended to have separate pointcut for every Aspect/Advice without having dependency with other Aspects/Advices.

**Steps to work with AspectJ AOP:**

- 1) Develop and configure target class
- 2) Develop and configure aspect class
- 3) Enable AspectJ AOP using <aop:config> tag

- 4) Prepare pointcut expression using <aop:pointcut> tag
- 5) Configure Aspect/Advice spring bean as AspectJ aspect class using <aop: aspect> tag and also configure its methods as advice methods.

Ex:



+ aspectjrt-1.5.3.jar,aspectjweaver-1.5.3.jar (use 1.8 <version>)

#### CacheAdvice.java

```
package com.nt.aspect;
import java.util.Arrays;
import java.util.HashMap;
import org.aspectj.lang.ProceedingJoinPoint;
public class CacheAdvice {
    HashMap<String,Object> cache=new HashMap<String,Object>();
    public Object caching(ProceedingJoinPoint pjp)throws Throwable{

        String key=pjp.getSignature()+Arrays.toString(pjp.getArgs());
        if(!cache.containsKey(key)){
            Object retVal=pjp.proceed();
            cache.put(key,retVal);
            System.out.println("From target method");
            return retVal;
        }
        else{
            System.out.println("From cache");
            return cache.get(key);
        }
    }
}
//caching
}
//class
```

#### LogAroundAdvice.java

```
package com.nt.aspect;
import java.util.Arrays;
import org.aspectj.lang.ProceedingJoinPoint;
```

```
public class LogAroundAdvice {
    public Object logging(ProceedingJoinPoint pjp)throws Throwable{
        System.out.println("Entering into "+pjp.getSignature()+" with
args"+Arrays.toString(pjp.getArgs()));
        //Get args and modify args
        Object args[]=pjp.getArgs();
        if((Float)args[0]<=50000){
            args[2]=(Float)args[2]-1;
        }
        Object retVal=null;
        if((Float)args[0]==0 || (Float)args[1]==0 || (Float)args[2]==0 )
            throw new IllegalArgumentException("Provide +ve values as the p,t,r
values");
        else
            retVal=pjp.proceed(args);
        // add Service tax on intr amt
        retVal=(Float)retVal+(Float)retVal*0.05f;
        System.out.println("Exiting from "+pjp.getSignature()+" with
args"+Arrays.toString(pjp.getArgs()));
        return retVal;
    } //method
} //class
```

#### PerformanceMonitoringAdvice.java

```
package com.nt.aspect;
import java.util.Arrays;
import org.aspectj.lang.ProceedingJoinPoint;
public class PerformanceMonitoringAdvice {
    private long start,end;
    public Object pMonitor(ProceedingJoinPoint pjp)throws Throwable{
        start=System.currentTimeMillis();
        Object retVal=pjp.proceed();
        end=System.currentTimeMillis();
        System.out.println(pjp.getSignature()+" with" +Arrays.toString(pjp.getArgs())+" has taken
"+(end-start)+"ms");
        return retVal;
    }
}
```

#### IntrAmtCalculator.java

```
package com.nt.service;
public class IntrAmtCalculator {
    public float calcSimpleIntrAmt(float p,float t,float r){
        return (p*t*r)/100.0f;
    }
    public float calcCompoundIntrAmt(float p,float t,float r){
        return (float) (p*Math.pow((1+r/100.0f),t)-p);
    }
}
```

**applicationContext.xml**

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
  <bean id="target" class="com.nt.service.IntrAmtCalculator"/>
  <bean id="logAdvice" class="com.nt.aspect.LogAroundAdvice"/>
  <bean id="cacheAdvice" class="com.nt.aspect.CacheAdvice"/>
  <bean id="PMAAdvice" class="com.nt.aspect.PerformanceMonitoringAdvice"/>
  <aop:config>
    <aop:pointcut id="ptc1" expression="execution(* com.nt.service.*.*(..))"/>
    <aop:pointcut id="ptc2" expression="execution(
*com.nt.service.IntrAmtCalculatorInterface.calcCompoundIntrAmt(..))"/>
    <aop:aspect id="logAspect" order="3" ref="logAdvice" >
      <aop:around pointcut-ref="ptc1" method="logging" />
    </aop:aspect>
    <aop:aspect id="cacheAspect" order="1" ref="cacheAdvice">
      <aop:around pointcut-ref="ptc2" method="caching"/>
    </aop:aspect>
    <aop:aspect id="PMAAspect" order="2" ref="PMAAdvice">
      <aop:around pointcut-ref="ptc1" method="pMonitor"/>
    </aop:aspect>
  </aop:config>
</beans>

```

**ClientApp.java**

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.IntrAmtCalculator;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext ctx=new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //GEt Proxy obj
        IntrAmtCalculator proxy=ctx.getBean("target",IntrAmtCalculator.class);
        System.out.println(proxy.getClass());
        //Call target method
        System.out.println("Simple Intr Amt"+proxy.calcSimpleIntrAmt(10000,20,2));
        System.out.println("-----");
        System.out.println("Compound Intr Amt"+proxy.calcCompoundIntrAmt(60000, 20, 2));
        System.out.println("-----");
        System.out.println("Compound Intr Amt"+proxy.calcCompoundIntrAmt(60000, 20, 2));
    }
}

```

**Jar: AOPLib+aspectjrt1.5.3.jar+aspectweaver-1.5.3.jar(use 1.8 <version>)**

**Note:**

- ⇒ While working with AspectJ AOP AroundAdvice the modification done in target method args in Around advice will be reflected only when we call **pjp.proceed (args)** having that object[] as argument. Here Object[] holds the modified argument values.
- ⇒ When we run the ClientApp of **spring integrateted AspectJAOP**, the internally created IOC container maintains inmemory metadata along with **AOP configurations based on <aop:config> and its sub tags.**
- ⇒ When ctx.getBean(-,-) method is called having target class bean id, the IOC container doesn't create target class object because of <aop:config> tag based configurations. It generates proxy class extending form target class and returns that proxy class object to the **client Application.**
- ⇒ Like spring AOP if we start working with proxyInterface (The interface that is implemented by target class) in AspectJAOP then the generated proxy class will come because of jdk libraries. For this we need to supply proxy interface name in OGNL expression of pointcut. In this AspectJAOP if proxy interface is not specified (if target class is specified) then CGLIB libraries will be utilized to generate the proxy class.
- ⇒ While working with AspectJAOP the already configured pointcut expression is mandatory even though you want to apply the advices on all the target methods.

**Before Advice:**

- ⇒ This advice executes before executing target method.
- ⇒ Using this advice we can access target method parameters but we can't control target method execution and we can't catch the return value.
- ⇒ Using this advice we can stop target method execution by throwing exception.
- ⇒ To develop this advice we can take any class having any method with following signature.

```
public void <method> (JoinPoint jp)
or
public void <method> (JoinPoint jp, param1, param2,.....)
```

- param1,param2,param3,---to hold the target method argument values
- JoinPoint object just gives target method method details where as ProceedingJoinPoint object gives target method details, control on target method invocation.
- BeforeAdvice use cases are security check, auditing and etc

- AspectJ AOP3 (Before Advice- Auditing)
  - src
    - com.nt.aspect
      - AuditingAdvice.java
    - com.nt.cfgs
      - applicationContext.xml
    - com.nt.service
      - OrderApprover.java
    - com.nt.test
      - ClientApp.java
  - JRE System Library [JavaSE-1.7]
  - AOPlib

### AuditingAdvice.java

```
package com.nt.aspect;
import java.io.FileWriter;
import java.util.Arrays;
import java.util.Date;
import org.aspectj.lang.JoinPoint;
public class AuditingAdvice {
    public void audit(JoinPoint jp, int oid)throws Throwable{
        System.out.println("target method:"+jp.getSignature());
        System.out.println("target class "+jp.getTarget().getClass());
        System.out.println("target method args"+Arrays.toString(jp.getArgs()));
        System.out.println("Order Id is"+oid);
        Object args[]=jp.getArgs();
        args[0]=(Integer)args[0]+1;
        jp.getArgs()[0]=args[0];

        //perform auditing
        FileWriter fw=new FileWriter("e:\\audit.log",true);
        fw.write("order with order id "+oid+" sumited for approval on Date"+new Date());

        fw.flush();
        fw.close();
    }
}

```

### OrderApprover.java

```
package com.nt.service;
import java.util.Calendar;
public class OrderApprover {
    public String approve(int orderid){
        // get Current month
        Calendar cl=Calendar.getInstance();
        int h=cl.get(Calendar.MONTH);
        if(h>=7 && h<=10)
            return "We do not accept this "+orderid+" order in rainy seasno";
        else
            return orderid+" order is approved";
    }
}

```

```

    }//approve
} //class

```

#### applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
<bean id="orderApprover" class="com.nt.service.OrderApprover"/>
<bean id="auditAdvice" class="com.nt.aspect.AuditingAdvice"/>
<aop:config>
    <aop:pointcut id="ptc1" expression="execution(* com.nt.service.*(..)) and args(oid)"/>
    <aop:aspect id="auditAspect" ref="auditAdvice">
        <aop:before method="audit" pointcut-ref="ptc1"/>
    </aop:aspect>
</aop:config>
</beans>

```

#### ClientApp.java

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.OrderApprover;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext ctx=new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get Proxy
        OrderApprover proxy=ctx.getBean("orderApprover",OrderApprover.class);
        System.out.println("status:"+proxy.approve(1001));
    }
}

```

- ⇒ In AspectJAOP the **BeforeAdvice** can access TargetMethod arguments but cannot modify their values. It is not similar to spring AOP before Advice.
- ⇒ While working with beforeAdvice we can even get Targetmethod arguments without making the support of JoinPoint object. We need to take additional parameters support in advice method to store target method arguments and we must specify those parameters in pointcut expression.

In applicationContext.xml:

```

<aop: pointcut id="ptel" expression="execution(* com.nt.serivce.*(..)) and args (oid)"/>

```

→ While working with AspectJ AOP if we call **ctx.getBean(-,-)** method by using the bean id whose class name is specified in pointcut OGNL expression then we get proxy class object. Otherwise we



will get the given bean id related object.

#### AfterAdvice/After Returning Advice:

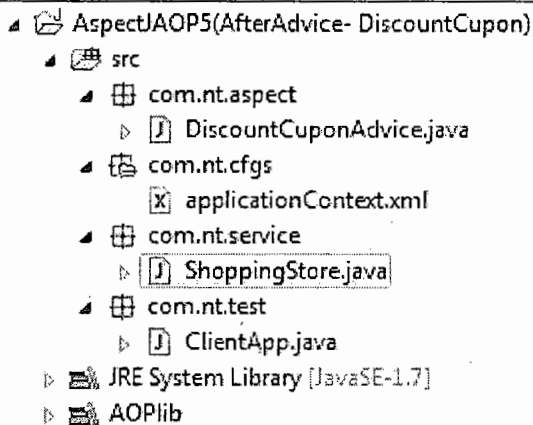
- ⇒ This advice method executes when target method completes execution and before it returns the **control/value to caller**.
- ⇒ To develop this advice we can take any class having the following signature.

```
public void <method>(JoinPoint JP, <param to hold return value>)
```

#### Usecases:

- 1) Generating DiscountCoupon for next purchase.
  - 2) ATM pin verifier.
- ⇒ This advice allows to access target method args values, but doesn't allow to modify.
  - ⇒ This advice allows to access return value but doesn't allow to modify. By throwing exception in this advice we can stop return value being returned to caller method.
  - ⇒ The parameter taken in advice method to hold return value of target method and must be configured in `<aop:after-returning>` by using "returning" attribute.

```
<aop:after-returning method="coupon" pointcut-ref="ptc" returning="billAmt"/>
```



#### DiscountCouponAdvice.java

```
package com.nt.aspect;
import java.io.FileWriter;
import org.aspectj.lang.JoinPoint;
public class DiscountCouponAdvice {
    public void generateCoupon(JoinPoint jp, float billAmt) throws Throwable {
        String couponMsg = null;
        if (billAmt >= 50000)
            couponMsg = "Avail 30% Discount on next purchase";
        else if (billAmt >= 30000)
            couponMsg = "Avail 20% Discount on next purchase";
        else
            couponMsg = "Avail 10% Discount on next purchase";
        // Save Coupon in a file
        FileWriter fw = new FileWriter("e:\\cupont.Tt");
        fw.write(couponMsg);
    }
}
```

```
        fw.flush();
        fw.close();
    } //generateCupon
} //class
```

**ShoppingStore.java**

```
package com.nt.service;
public class ShoppingStore {
    public float generateBillAmt(String item,float price,float qty ){
        return price*qty;
    }
}
```

**applicationContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
<!-- target class -->
    <bean id="shopping" class="com.nt.service.ShoppingStore"/>
<!-- advice class -->
    <bean id="discountAdvice" class="com.nt.aspect.DiscountCuponAdvice"/>

    <aop:config>
        <aop:pointcut id="ptc1" expression="execution(* com.nt.service.*.*(..))"/>
        <aop:aspect id="discountAspect" ref="discountAdvice">
            <aop:after-returning method="generateCupon" pointcut-ref="ptc1" returning="billAmt"/>
        </aop:aspect>
    </aop:config>

</beans>
```

**ClientApp.java**

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.ShoppingStore;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //GEt Proxy obj
        ShoppingStore proxy=ctx.getBean("shopping",ShoppingStore.class);
        //call B.method
        System.out.println("Bill Amt"+proxy.generateBillAmt("shirt",900,2));
    }
}
```

ThrowsAdvice:

- ⇒ This advice method executes when exception is raised in target method. This advice is useful to develop central, common exception logger.
- ⇒ This advice doesn't catch and handle the exceptions, but allows to see the details of exception and translate/graph them from one form of exception to another form of exception.
- ⇒ To develop this advice we can take a class having method with following signature.

```
public void <method name>(JoinPoint JP, <param of type subclass of Throwable>);
```

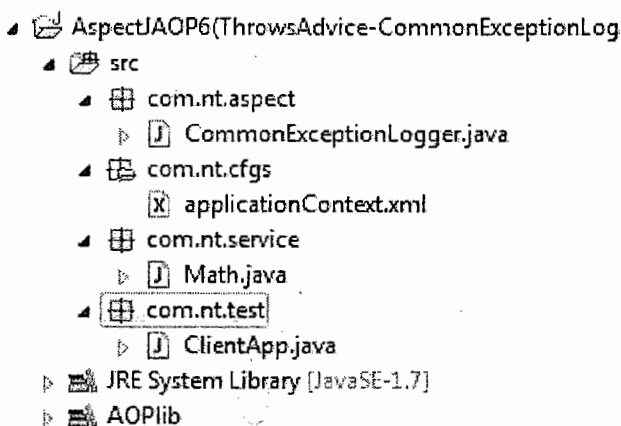
- ⇒ The additional param of this advice method is capable of referring Raised exception and must be specified in <aop:after-throwing> tag by using "throwing" attribute.

```
eg:<aop:after-throwing method="....." pointcut-ref="....." throwing="....."/>
```

- ⇒ Throws advice also responds for the exceptions that are raised in other advices that are applied on the target method. Throws advice is really useful in real time to graph or translate the technology specific exceptions of service, DAO classes into application specific Exception. This allows to keep presentation tier components untouchable even though service, DAO classes technology are changed...

AspectJAOP (ThrowsAdvice-CommonExceptionLogger)

- ⇒ ThrowsAdvice even useful for exception graphing, which is nothing but transforming of one form of exception to another form.
- ⇒ While working with AspectJAOP specifying pointcut expression is mandatory even though you want to apply all the advices on all the target methods of target class, because if target class name or if implemented proxy interface is not there in the OGNL Expression then the proxy class will not be generated when ctx.getBean(-,-) is called...

CommonExceptionLogger.java

```
package com.nt.aspect;
import java.util.Arrays;
import org.aspectj.lang.JoinPoint;
public class CommonExceptionLogger {
    public void exceptionLogging(JoinPoint jp, ArithmeticException ae){
        System.out.println(ae.getClass()+"Exception raised With Msg"
            +ae.getMessage()+" in "+jp.getSignature()
            +"with args"+Arrays.toString(jp.getArgs()));
    }
}
```

```
        throw new IllegalArgumentException("Provide valid inputs");
    }
}
```

### Math.java

```
package com.nt.service;
public class Math {
    public float div(int x,int y){
        if(y==0)
            throw new ArithmeticException(" Divider can not Zero ");
        else
            return (float)x/y;
    }
}
```

### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
<bean id="math" class="com.nt.service.Math"/>
<bean id="exLogAdvice" class="com.nt.aspect.CommonExceptionLogger"/>

<aop:config>
    <aop:pointcut id="ptc1" expression="execution(* com.nt.service.*(..))"/>

    <aop:aspect id="exLogAspect" ref="exLogAdvice">

        <aop:after-throwing method="exceptionLogging"
            pointcut-ref="ptc1" throwing="ae"/>
    </aop:aspect>
</aop:config>
</beans>
```

### ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.Math;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //Get Proxy obj
    }
}
```

```
Math proxy=ctx.getBean("math",Math.class);

//call B.method
try{
    System.out.println("Result="+proxy.div(10,20));
    System.out.println("-----");
    System.out.println("Result="+proxy.div(10,0));
}
catch(ArithmeticException ae){
    System.out.println("Internal Problem");
}
}
```

### Annotations driven AspectJ AOP/Annotation based AspectJAOP/@AspectJ approach AOP:

→ Here we don't use xml file to configure advice classes and advice methods, rather we use list of annotations given by AspectjAOP.

**@Aspect**-----> To configure spring bean as AspectClass. (\* Should be used along with @component & @ Service)

**@Before**-----> To configure method as before advice method.

**@Around**-----> To configure method as around advice method.

**@AfterReturning**-----> to configure method as AfterAdvice

**@AfterThrowing**-----> To configure method as throw advice

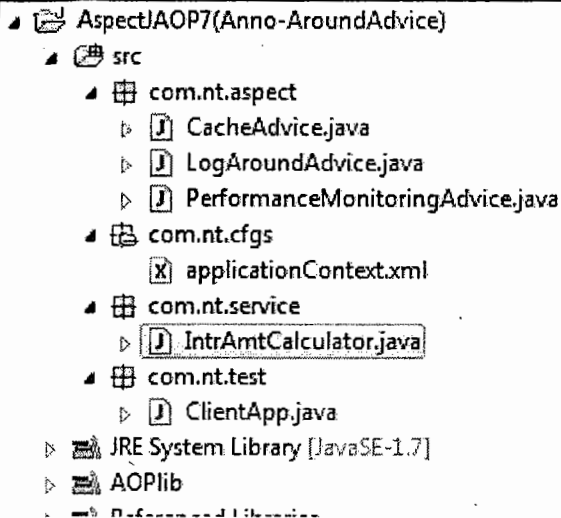
**@Pointcut** -----> To specify pointcut expression with reusability

**@Order**-----> To specify order

- To make IOC container recognizing all these annotations, we need to place **<aop:aspectj-autoproxy>** tag in xml file. It is equal to **<aop:config>** tag.
- While working with AspectJAOP we can take Advice classes as pojo classes and we can place multiple advice logics in a single class.

**@Around, @AfterReturning, @Before, @AfterThrowing** annotations allow to place pointcut expressions directly.

- To place pointcut expression with reusability we can use **@Pointcut**.



### CacheAdvice.java

```
package com.nt.aspect;
import java.util.Arrays;
import java.util.HashMap;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Aspect
@Order(1) Useful to specify Order of applying advice when they are configured through
annotations. Highvalue indicates low priority and low value indicates high priority among
The numbers that are given )
@Component
public class CacheAdvice {
    HashMap<String,Object> cache=new HashMap<String,Object>();

    @Around("execution(* com.nt.service.*(..))")
    public Object caching(ProceedingJoinPoint pjp)throws Throwable{
        String key=pjp.getSignature()+Arrays.toString(pjp.getArgs());

        if(!cache.containsKey(key)){
            Object retVal=pjp.proceed();
            cache.put(key,retVal);
            System.out.println("From target method");
            return retVal;
        }
        else{
            System.out.println("From cache");
            return cache.get(key);
        }
    }
    //caching
}
//class
```

LogAroundAdvice.java

```
package com.nt.aspect;
import java.util.Arrays;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Aspect
@Order(2)
@Component
public class LogAroundAdvice {

    @Around("execution(* com.nt.service.*(..))")
    public Object logging(ProceedingJoinPoint pjp)throws Throwable{
        System.out.println("Entering into "+pjp.getSignature()+" with
args"+Arrays.toString(pjp.getArgs()));
        //Get args and modify args
        Object args[]=pjp.getArgs();
        if((Float)args[0]<=50000){
            args[2]=(Float)args[2]-1;
        }

        Object retVal=null;
        if((Float)args[0]==0 || (Float)args[1]==0 || (Float)args[2]==0 )
            throw new IllegalArgumentException("Provide +ve values as the p,t,r
values");
        else
            retVal=pjp.proceed(args);
        // add Service tax on intr amt
        retVal=(Float)retVal+(Float)retVal*0.05f;
        System.out.println("Exiting from "+pjp.getSignature()+" with
args"+Arrays.toString(pjp.getArgs()));
        return retVal;
    } //method
} //class
```

PerformanceMonitoringAdvice.java

```
package com.nt.aspect;
import java.util.Arrays;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Aspect
@Order(3)
@Component
public class PerformanceMonitoringAdvice {
    long start,end;
```

```
@Around("execution(* com.nt.service.*.*(..))")
public Object pMonitor(ProceedingJoinPoint pjp)throws Throwable{
    start=System.currentTimeMillis();
    Object retVal=pjp.proceed();
    end=System.currentTimeMillis();
    System.out.println(pjp.getSignature()+" with" +Arrays.toString(pjp.getArgs())+" has
taken "+(end-start)+"ms");
    return retVal;
}
```

#### IntrAmtCalculator.java

```
package com.nt.service;
import org.springframework.stereotype.Component;
@Component("iac")
public class IntrAmtCalculator {
    public float calcSimpleIntrAmt(float p,float t,float r){
        return (p*t*r)/100.0f;
    }
    public float calcCompoundIntrAmt(float p,float t,float r){
        return (float) (p*Math.pow((1+r/100.0f),t)-p);
    }
}
```

#### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xmlns:aop="http://www.springframework.org/schema/aop"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
    <context:component-scan base-package="com.nt.aspect,com.nt.service"/>
</beans>
```

#### ClientApp.java

```
package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.IntrAmtCalculator;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext ctx =new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //GEt Proxy obj
        IntrAmtCalculator proxy=ctx.getBean("targetBean",IntrAmtCalculator.class);
```



```
//Call target method
System.out.println("Simple Intr Amt"+proxy.calcSimpleIntrAmt(10000,20,2));
System.out.println("-----");
System.out.println("Compound Intr Amt"+proxy.calcCompoundIntrAmt(60000, 20, 2));
System.out.println("-----");
System.out.println("Compound Intr Amt"+proxy.calcCompoundIntrAmt(60000, 20, 2));
}
}
```

- ⇒ While working with AspectJAOP with xml the order they follow to apply advices on target method will be decided based on the order that is used to configure advice classes as **spring beans in applicationContext.xml**
- ⇒ Here advice/aspect classes are not configured in xml file and they are configured through annotations like
- 1) @Component , @Aspect then we can define the order by using
  - 2) @Order(-) annotation
  - 3) @Component or @Service can be used to configure ordinary java class as spring bean.
  - 4) @Aspect can't be used to configure ordinary java class as aspect class. But it can be used to configure spring bean as aspect class.

#### While working with annotation based AspectJAOP

<aop:aspectj-autoproxy> is alternate to <aop:config> tag to enable AspectJAOP based on the annotations.

ThrowsAdvice not only executes for the exception that is raised in target method, it also executes for the exception that is raised in advices that are related to target method.

- ⇒ When we define multiple advice methods in single aspect class having same pointcut expression then instead of defining that pointcut expression separately for every method it is recommended to take dummy method in same Aspect class having pointcut expression defined through @Pointcut and specify that dummy method name in every advice method as pointcut expression.

```
@Aspect
public class MyAspect{
    @Pointcut("execution(*com.nt.service,*.*(..))")
    public void myPointcut(){ //Dummy method representing Pointcut expression
    }
    @Around(value="myPointcut()")
    public Object log(ProceedingJoinPoint pjp){
    ....
    ....}
    @Before(value="myPontcut()")
    public void audit(JoinPoint jp){
    ....
    ....}
}
```

- ▲  AspectJAOP8(Anno-All advices)
  - ▲  src
    - ▲  com.nt.aspect
      - ▷  LoggingAdvice.java
    - ▲  com.nt.cfgs
      - ▷  applicationContext.xml
    - ▲  com.nt.service
      - ▷  Math.java
    - ▲  com.nt.test
      - ▷  ClientApp.java
  - ▷  JRE System Library [JavaSE-1.7]
  - ▷  AOPlib

eg:

#### LoggingAdvice.java

```
package com.nt.aspect;
import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.AfterReturning;
import org.aspectj.lang.annotation.AfterThrowing;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;
import org.springframework.stereotype.Component;

@Component
@Aspect
public class LoggingAdvice {
    @Pointcut("execution(* com.nt.service.Math.*(..))")
    public void myPointcut(){
    }

    @Around("myPointcut()")
    public Object log(ProceedingJoinPoint pjp)throws Throwable{
        System.out.println("Entering into "+pjp.getSignature());
        Object retVal=pjp.proceed();
        System.out.println("Leaving from "+pjp.getSignature());
        return retVal;
    }

    @Before("myPointcut()")
    public void checkInputs(JoinPoint jp){
        Object args[]=jp.getArgs();
        if((Integer)args[0]<=0 || (Integer)args[1]<=0){
            throw new IllegalArgumentException("Provide proper inputs");
        }
    }

    @AfterReturning(value="myPointcut()",returning="result")
    public void checkResult(JoinPoint jp,int result){
        if(result<=0)
            throw new IllegalArgumentException("Invalid result");
    }
}
```

```
}  
@AfterThrowing(value="myPointcut()",throwing="e")  
public void exceptionLogger(JoinPoint jp,Exception e){  
    System.out.println(jp.getSignature()+" has raised"+e.getClass()+" Exception");  
}  
}
```

### Math.java

```
package com.nt.service;  
import org.springframework.stereotype.Component;  
@Component("math")  
public class Math {  
    public int sum(int x,int y){  
        return x+y;  
    }  
    public int sub(int x,int y){  
        return x-y;  
    }  
    public int mul(int x,int y){  
        return x*y;  
    }  
    public int div(int x,int y){  
        return (int)x/y;  
    }  
}
```

### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xmlns:context="http://www.springframework.org/schema/context"  
xmlns:aop="http://www.springframework.org/schema/aop"  
xsi:schemaLocation="http://www.springframework.org/schema/beans  
http://www.springframework.org/schema/beans/spring-beans.xsd  
http://www.springframework.org/schema/aop  
http://www.springframework.org/schema/aop/spring-aop.xsd  
http://www.springframework.org/schema/context  
http://www.springframework.org/schema/context/spring-context.xsd">  
    <aop:aspectj-autoproxy/>  
    <context:component-scan base-package="com.nt.aspect,com.nt.service"/>  
</beans>
```

### ClientApp.java

```
package test;  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.FileSystemXmlApplicationContext;  
import com.nt.service.Math;  
public class ClientApp {
```

```
public static void main(String[] args) {
    //create IOC container
    ApplicationContext ctx =new
FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
    //GEt Proxy obj
    Math proxy=ctx.getBean("math",Math.class);
    //Call target method
    System.out.println(proxy.sum(10,20));
    System.out.println("-----");
    System.out.println(proxy.sub(10,30));
    System.out.println("-----");
    System.out.println(proxy.mul(20,30));
    System.out.println("-----");
    System.out.println(proxy.div(20,10));
}
```

Note: in real time most reguraly used AOP is annotation driven aspect AOP.

**AspectJAOP8-100Pcode-AroundAdvice Example:**

=====

**CacheAdvice.java**

```
package com.nt.aspect;

import java.util.Arrays;
import java.util.HashMap;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Aspect
@Component("cacheAdvice")
@Order(1)
public class CacheAdvice {
    private HashMap<String, Object> cache=new HashMap();

    @Around("execution(* com.nt.service.IntrAmountCalculator.*(..))")
    public Object caching(ProceedingJoinPoint pjp)throws Throwable{
        Object retVal=null;
        String key=null;
        //prepare methodname with args as key
        key=pjp.getSignature()+Arrays.toString(pjp.getArgs());
        if(!cache.containsKey(key)){
            retVal=pjp.proceed();
            System.out.println("From target method....");
            cache.put(key,retVal);
        }
    }
}
```

```
        else{
            retVal=cache.get(key);
            System.out.println("from cache...");
        }
        return retVal;
    } //method
} //class
```

**LogAroundAdvice.java**

```
package com.nt.aspect;

import java.util.Arrays;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Aspect
@Component("logAdvice")
@Order(2)
public class LogAroundAdvice {
    @Around("execution(* com.nt.service.IntrAmountCalculator.*(..))")
    public Object logging(ProceedingJoinPoint pjp)throws Throwable{
        Object retVal=null;
        Object args[]=null;
        System.out.println("Entering into"+pjp.getSignature()+" with args"+Arrays.toString(pjp.getArgs()));

        //Access & modify argument values
        args=pjp.getArgs();
        if(((Float)args[0])<50000)
            args[1]=((Float)args[1])-0.5f;

        //call target method
        retVal=pjp.proceed(args);
        System.out.println("Leaving from"+pjp.getSignature()+" with
args"+Arrays.toString(pjp.getArgs()));
        retVal=((Float)retVal)+(((Float)retVal)*0.01f);
        return retVal;
    } //method
} //class
```

**PerformanceMonitoringAdvice.java**

```
package com.nt.aspect;

import java.util.Arrays;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
```

```
import org.aspectj.lang.annotation.Aspect;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;
@Aspect
@Component("pmAdvice")
@Order(3)
public class PerformanceMonitoringAdvice {
    private long start, end;

    @Around("execution(* com.nt.service.IntrAmountCalculator.*(..))")
    public Object monitor(ProceedingJoinPoint pjp) throws Throwable {
        Object retVal = null;
        start = System.currentTimeMillis();
        retVal = pjp.proceed();
        end = System.currentTimeMillis();
        System.out.println(" Method "+pjp.getSignature()+"with args"+Arrays.toString(pjp.getArgs())+" has
        taken "+(end-start)+" ms");
        return retVal;
    } // monitor
} // class
```

#### IntrAmountCalculator.java

```
package com.nt.service;
import org.springframework.stereotype.Component;

@Component("iac")
public class IntrAmountCalculator {
    public float calcIntrAmount(float pAmt, float rate, float time) {
        return (pAmt * rate * time) / 100.0f;
    } // method

    public float calcCompoundIntrAmount(float pAmt, float rate, float time) {
        return (float) (pAmt*Math.pow((1+rate/100),time)-pAmt);
    }
}
```

#### AspectJAroundAdviceTest.java

```
package com.nt.test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.support.AbstractApplicationContext;
import com.nt.cfigs.AppConfig;
import com.nt.service.IntrAmountCalculator;

public class AspectJAroundAdviceTest {
    public static void main(String[] args) {
        ApplicationContext ctx=null;
        IntrAmountCalculator proxy=null;

        //create IOC container
```

```
        ctx=new AnnotationConfigApplicationContext(AppConfig.class);
        //get Prxoy object
        proxy=ctx.getBean("iac",IntrAmountCalculator.class);

        //invoke method
        System.out.println("simple Intrest Amount:::"+proxy.calcIntrAmount(10000, 2,
10));
        System.out.println("-----");
        System.out.println("compound Intrest
Amount:::"+proxy.calcCompoundIntrAmount(10000, 2, 10));
        System.out.println("-----");
        System.out.println("compound Intrest
Amount:::"+proxy.calcCompoundIntrAmount(10000, 2, 10));

        //close IOC
        ((AbstractApplicationContext) ctx).close();
    } //main
} //class
```

**AppConfig.java**

```
package com.nt.cfgs;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.EnableAspectJAutoProxy;

@Configuration
@ComponentScan(basePackages="com.nt.service,com.nt.aspect")
@EnableAspectJAutoProxy
public class AppConfig {

}
```

## Transaction Management

The process of combining related operations into single unit and executing then by applying do everything or nothing principle called "Transaction Management".

- 1) The withdraw amount from source account, deposite amount into dest amount must happen in Transaction envinorment.
- 2) In employee registration employee details insertion into DB and insertion into payroll DB most happen in Transactional envinorment.
- 3) In online shopping website order booking and payment must happen in transactional environment.

➔ Transaction management allows to implement ACID properties.

A-->Atomocity

C--->Consistency

I----->Isolation

D---->Durability

- ⇒ The process of combining indivisible operations into single unit is called Atomocity. **Atomocity word is derived from the term called Atom.**
- ⇒ Even though rules kept on DB software (like balance must not be negative) are violated during the coarse of Transaction management , but if there is a guarrentee that no rule will be vailoated at the end of **Transaction management** , then we can say **DB is consistent**.
- ⇒ Applying locks on DB software and allowing one user or one application at a time to manipulate db data is called **isolation**.
- ⇒ Brining data back to normal state by using log files, backup files though it is crashed indicates **durability of DB**

**Every Transaction contains 3 operations:-**

**1). Begin Transaction**

.....

..... **2). //continue Transaction (Execute logics)**

.....

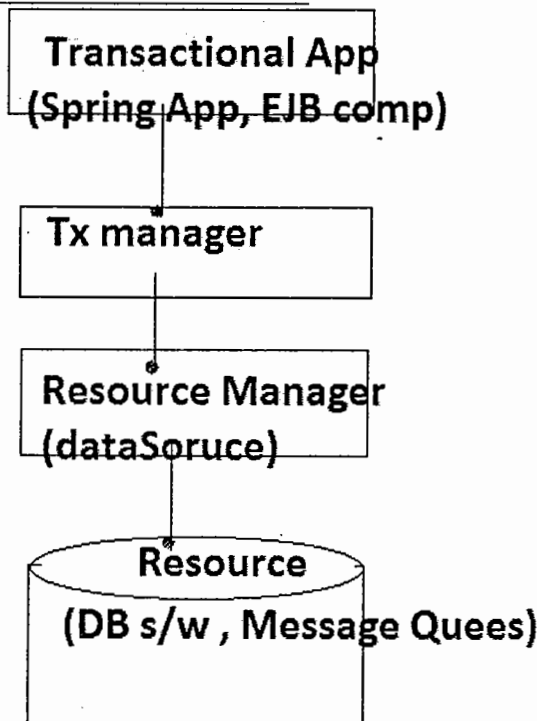
**3).Commit/Rollback Transaction**

Transaction management resource must allow data commit or rollback operations.

File can't be taken as Transaction resource. DB software, message Queues can be taken as Transaction resource. Because file doesn,t support commit and roll ball operating.



## Transaction Architecture:



⇒ Based on the **Transaction management** applications instruction the Transaction manager connects to resource through resource manager for committing or rollbacking the Transaction

⇒ Based on number of resources that are involved in a transaction there are 2 types of transaction.

### (1) Local Transaction

→ Single resource will participate in transaction boundary i.e all the operations of Transaction Takes place in **Single Resource**.

eg: Transfer money operation between 2 accounts of same bank.

### (2) Global Transaction / Distributed Transaction / XA Transaction

→ Multiple resources will participate in a single transaction boundary.

eg: Transfer money operation between two accounts of two different banks

⇒ Global Transaction runs based on **2pc protocol (2 phase commit)**

### Phase-1 :

→ Transaction manager talks with all the resources through **Resource manager** seeking permission to commit the Transaction by checking the data consistency.

### Phase-2:

→ If all the resource managers respond and says **"OK"** then the data of multiple resources will be committed. If any Resource manager says **"no"** then data of multiple database **will be rollbacked**.

→ JDBC, Hibernate supports only local Transaction management . Spring, EJB supports **both Local and Global Transaction management** .To work with Distributed Transaction we need to **XA JDBC driver and XA DataSource(Like ojdbc-xa.jar)**

### Note:

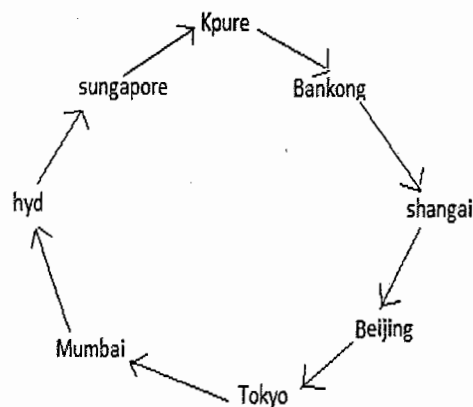
→ Using jdbc, hibernate we can bring the effect of Distributed Transaction management by calling

commit()/rollback() on multiple connection objects (or) session objects but that is not a good practice because, if exception raises between two commit method calls then the transaction management will fail completely.

### Transaction Models:

#### a) Flat Trasaction b) Nested Transaction

#### World tour plan:



- ⇒ When the tour plan is given to Flat Transaction management model applicaton, if one or another journey tickets are not available then it will cancel the remaining journey tikets. cecause all the journey ticket bookings will be taken as the direct operations of **main transaction**.
- ⇒ If the above tour plan is given to nested Transaction management model application, it will confirm remaining journey tickets even though one or another journey ticket are not available . Becase here each journey ticket booking will be taken as sub Transaction management of main Transaction management, so the success or failure of one transaction doesn't affect other sub Transactions.

⇒ 1.Flat Transaction Model

```
Main Transaction{  
    journey1 tkt booking  
    Journey2 tkt booking  
    .....  
    .....  
    Journeryn tkt booking  
}
```

⇒ 2.Nested Transaction Model

```
Main tx{  
    Sub tx1{  
        ....  
        ....  
        } //Journey 1 ticket booking  
    ....  
    Sub tx2{  
        .....  
        .....//journey ticket booking  
    }  
    Sub txn{  
        .....  
        .....//journey n ticket booking  
    }  
}
```

⇒ JDBC, Hibernate, EJB supports only Flat transactions, spring supports both Flat Transaction management and Nested Transaction management .

Need of Spring Trasaction managenent

⇒ Generally we use JDBC code for Local Transcation Management as shown below

Local Transaction management sample code using jdbc:

```
try{  
con.setAutoCommit(false); // Transaction begins  
.....  
..... //Transaction management .logic  
.....  
con.commit ();  
}catch (Exception e){  
    con.rollback();  
}
```

⇒ To perform distributed Transaction management we need either application server (JEE server) or Third party api like atomokis that gives **userTransaction Object as Transaction manager**. We also need XA jdbc drivers, XA Datasource to get Transaction manager required for distributed Transaction management .

- ⇒ Here we never directly call `con.commit()` or `con.rollback()`, we ask `TransactionManager` to perform these operations by getting connection through `Resource Manager (Datasource)`. The server like `weblogic` supply "`userTransaction`" object in its `jndi` registry having fixed `jndi` name like "`javax/transaction/userTransaction`".
  - ⇒ We use `JTA` support to perform **global Transaction management in programmatic approach**. We use `EJB` to perform **Global Transaction management in declarative approach (xml configuration/ annotation configuration)**.
  - ⇒ In programmatic approach we write `Transaction management` code inside the `B.methods`. So **disabling Transaction management service is very complex**.
  - ⇒ In declarative approach, the `b.method` contains only `b.logic` but the `Tx Transaction configuration` will be placed in `xml` file. The container/server/framework enables **Transaction management service on b.methods based on xml file or annotation cfigs**.
- `JDBC, JTA` supports only programmatic `Transaction management`.
- `EJB` supports both `Programmatic` and `Declarative management`.

**Global Transaction management in the component that is deployable in the server using JTA (programmatic approach)**

```
InitialContext ic=new InitialContext();
UserTransaction ut=( UserTransaction)ic.lookup ("javax/transaction/userTransaction");
try{
    ut.begin(); //begin Transaction management
    .....
    .....      Transaction management .logic
    .....
    ut.commit();
}catch (Exception e){
    ut.rollback();
}
}
```

**Declarative Transaction management Management Using EJB:**  
**EJB component**

```
public class Mycomp implements SessionBean
{
    public void bm1()
    {
        .....
        ..... // only B.logic or no Transaction management logic
        .....
    }
}
```

**ejb-jar.xml**

- ⇒ Bean class configuration
- ⇒ Transaction management configuration

**Note:** Based on xml configurations, the EJB container applies Transaction management services on to b.methods of EJB component.

→ The EJB's declarative Transaction management service is really nice, **But it is heavy weight.**

**Spring Transaction:**

- ⇒ In the above discussion we need to use different java, JEE apis or 3<sup>rd</sup> party apis like JPA, hibernate and etc ..... **To perform different modes of Transaction management.**
- ⇒ Due to this switching from local Transaction management to Distributed or switching from one Transaction management implementation to another Transaction management implementation becomes very complex for the programmer.
- ⇒ Spring provides unified environment to work with any Transaction management because it is a framework providing abstraction layer on **multiple technologies**. Due to this we can make spring to use internally any implementation of Transaction management . But we can write the **unified code for Transaction management**. Due to this switching to distributed Transaction management from local Transaction management and switching from one Transaction management implementation to another Transaction management implementation doesn't make to change spring code much.

Spring supports 4 ways of transaction management:-

- 1) Programatic Transaction management
  - 2) Declarative Transaction management using spring AOP
  - 3) Declarative Transaction management using AspectJAOP
  - 4) Annotations based Transaction management using AspectJAOP
  - 5) TransactionManagement using AspectJAOP using 100% codep
  - 6) TransactionManagement using Spring BOOT with AspectJAOP
- ⇒ In spring Transaction service is secondary logic/cross cutting concern. It is given as a pre-defined Aspect logic as the combination of Around advice and throwsAdvice type.
  - Transaction management will begin while entering into method and will be committed when method returns a value or will be rolledback when methods **throw exceptions**. So the Transaction management service is combination of **AroundAdvice and throwsAdvice**.
  - ⇒ In order to perform Transaction management we need to choose the appropriate **Transaction manager/ Transaction management coordinator**
    - a) **DataSourceTransactionManager**---->For underlying jdbc code (Local Transaction management )
    - b) **HibernateTransactionManager**----->for hibernate code(Local Transaction management )
    - c) **JDOTransactionManager**-----> for the JDO code (Local Transaction management )
    - d) **JtaTransactionManager**----->for Distributed Transaction management and etc.....

**Declarative Transaction management using spring AOP:**

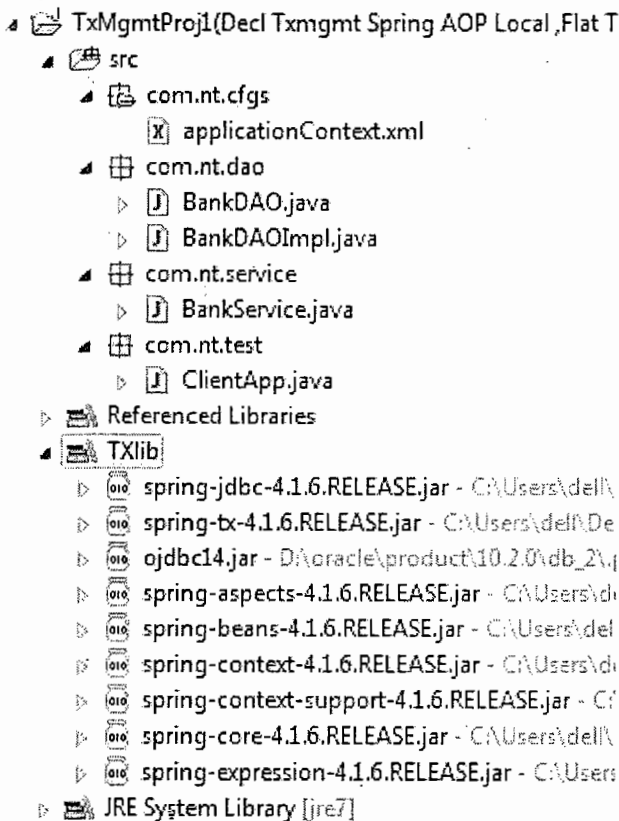
- ⇒ **TransactionProxyFactoryBean**----->Generates proxy class applying Transaction service on Target methods.
- ⇒ **NameMatchTransactionAttributeSource**----->Allows to specify Transaction management attributes on target methods it is like pointcut

**Note:**

- ⇒ To create DataSourceTransactionManager, the “**dataSource**” object is dependent.
- ⇒ In programmatic Transaction management, we write Transaction management code inside the b.method which is against of **AOP**, so don't use programmatic Transaction management management.
- ⇒ In declarative Transaction management we write Transaction management related configurations in xml file, so target method just contain business logic. But Transaction management service will be applied on business method because of proxy class
- ⇒ While working with any mode of Declarative transaction management we never begin (or) commit (or) rollback transaction explicitly. All these operations will be taken care by Transaction manager with the support of proxy class. The transaction manager begins the transaction while entering into the business method and commits the transaction at the end of the business method if no unchecked exception is raised. Otherwise rollbacks the transaction .

**Steps to work with spring AOP declarative transaction manager**

- 1) Configure the target class/service class.
- 2) Configure Transaction manager as transaction advice.
- 3) Configure TransactionProxyFactoryBean injecting **transactionmanager,target,specifying transaction attribute** to generate proxy class having transaction service.



### BankDAO.java

```
package com.nt.dao;
public interface BankDAO {
    public int withdraw(int acno,int amt);
    public int deposit(int acno,int amt);
}
```

### BankDAOImpl.java

```
package com.nt.dao;
import org.springframework.jdbc.core.JdbcTemplate;
public class BankDAOImpl implements BankDAO{
    private JdbcTemplate jt;
    private static final String WITHDRAW_QRY="update tx_account set balance=balance-?
where accno=?";
    private static final String DEPOSITE_QRY="update tx_account set balance=balance+? where
accno=?";
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    @Override
    public int withdraw(int acno, int amt) {
        int cnt=jt.update(WITHDRAW_QRY,amt,acno);
        return cnt;
    }
    @Override
    public int deposit(int acno, int amt) {
```

```
int cnt=jt.update(DEPOSITE_QRY,amt,acno);
return cnt;
```

```
}
```

### BankService.java

```
package com.nt.service;
import com.nt.dao.BankDAO;
public class BankService {
    private BankDAO dao;
    public void setDao(BankDAO dao) {
        this.dao = dao;
    }
    public boolean transferMoney(int srcNo,int destNo,int amt){
        int result1=dao.withdraw(srcNo, amt);
        int result2=dao.deposit(destNo, amt);

        if(result1==0 || result2==0){
            throw new RuntimeException("Money not Transferred(Tx rolled back)");
        }
        else{
            System.out.println("Money Transferred(Tx committed)");
            return true;
        }
    }
}
```

### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.0.xsd">
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
    <property name="username" value="scott"/>
    <property name="password" value="tiger"/>
</bean>
<bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="dataSource"/>
</bean>
<bean id="bankDAO" class="com.nt.dao.BankDAOImpl">
    <property name="jt" ref="template"/>
</bean>
<bean id="txMgr" class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
    <property name="dataSource" ref="dataSource"/>
</bean>
<bean id="bankService" class="com.nt.service.BankService">
```



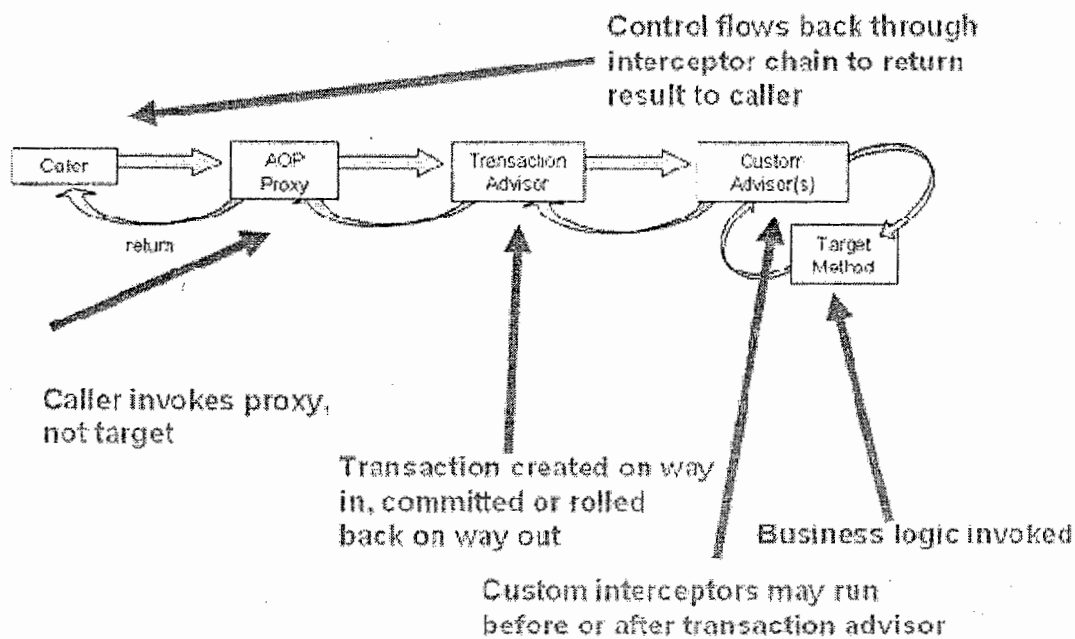
```
<property name="dao" ref="bankDAO"/>
</bean>
<bean id="nmtas"
class="org.springframework.transaction.interceptor.NameMatchTransactionAttributeSource">
  <property name="properties">
    <props>
      <prop key="transferMoney">PROPAGATION_REQUIRED</prop>
    </props>
  </property>
</bean>
<bean id="tpfb"
class="org.springframework.transaction.interceptor.TransactionProxyFactoryBean">
  <property name="transactionManager" ref="txMgr"/>
  <property name="target" ref="bankService"/>
  <property name="transactionAttributeSource" ref="nmtas"/>
</bean>
</beans>
```

#### ClientApp.java

```
package com.nt.test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.BankService;
public class ClientApp {
public static void main(String[] args) {
    //create IOC container
    ApplicationContext ctx=new
    FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
    //get Proxy obj
    BankService proxy=ctx.getBean("tpfb",BankService.class);
    //call b.method
    System.out.println("Money Transferred?" + proxy.transferMoney(1002, 1001,500));
}
}
```

Jar: AOPLib+springjdbcLib.

Conceptually, calling a method on a transactional proxy looks like this...

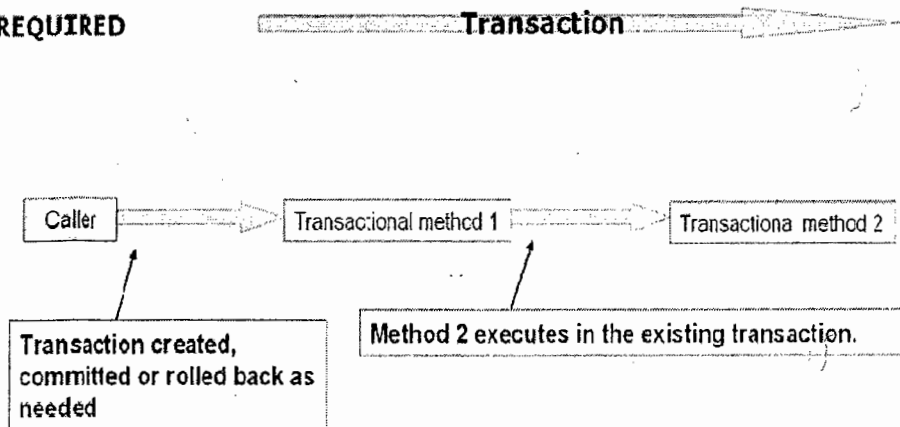


⇒ In Declarative Transaction management the Transaction management manager rolls back the Transaction management automatically if unchecked exception is raised in the target method (RuntimeException and its subclass) otherwise the Transaction management will be committed.

→ In declarative Transaction management, we can specify Transaction management attribute on the target method to make **target method running with no transaction or new transaction or in client supplied Transaction.**

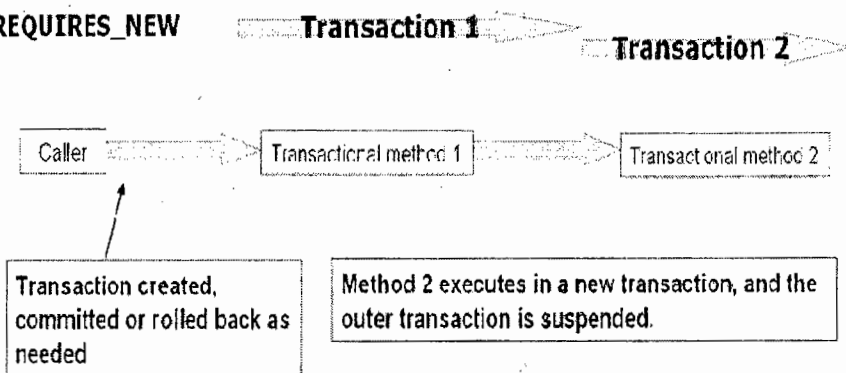
Transaction Attribute:

|              |                                                 |
|--------------|-------------------------------------------------|
| Required     | (PROOAGATION_REQUIRED) (//Most Regularly used ) |
| Requirednew  | (PROOAGATION_REQUIRES_NEW)                      |
| Supports     | (PROOAGATION_SUPPORTS) (//default)              |
| Never        | (PROOAGATION_NEVER)                             |
| NotSupported | (PROOAGATION_NOT_SUPPORTED)                     |
| Mandatory    | (PROOAGATION_MANDATORY)                         |

**Required:****REQUIRED**

- ⇒ If clientApp calls the B.method in a Transaction management then b.method runs that Transaction management .
- ⇒ If clientApp calls the B.method without Transaction management then b.method runs in a new Transaction management .

Note: It is most popularly used **Transactionmanagement** attribute.

**RequiresNew:****REQUIRES\_NEW**

- ⇒ B.method/target method always runs in a new Transaction management irrespective of wheather clientApp calls b.method in a **Transaction management** or not.

**Supports:**

⇒ B.method runs in client Application Transaction management , if client appplication calls B.method in a **Transaction management**

⇒ B.method runs without Transaction management , if client application calls the b.method without Transaction management .It is default attribute/transaction attribute.

**Never:**

ClientApp must not call b.method ina Transaction management , if called Exception will be raised

**Not Supported:**

B.method/Target method always run in No Transaction irrespective of wheather client application calls B.method with or without transaction.

**Manadatory:**

→ ClientApp must call B.method in a transaction otherwise Exception will be raised.

**Transaction attributes table**

| Transaction attribute | clients Transaction | Bean's Transaction |
|-----------------------|---------------------|--------------------|
| <b>Required</b>       | none                | T2 (new Tx)        |
|                       | T1                  | T1 (same Tx)       |
| <b>RequiresNew</b>    | None                | T2                 |
|                       | T1                  | T2                 |
| <b>Supports</b>       | None                | None               |
|                       | T1                  | T1                 |
| <b>Manadatory</b>     | None                | Error              |
|                       | T1                  | T1                 |
| <b>Not Supported</b>  | None                | None               |
|                       | T1                  | None               |
| <b>Never</b>          | None                | None               |
|                       | T1                  | error              |

**Declarative Transaction management Using AspectJ AOP:**

⇒ It is most regular used Transaction management mechanism. To work with this we need to import AOP, tx namespaces. In order to activate Transaction management service as as Advice having bean id and to specify Transaction management attribute on target methods we can use `<tx:advice>`, `<tx:attributes>`, `<tx:method>` tags. The Transaction management advice performs commit or rollback operations through Transaction manager so we need to specify Transaction manager while configuring Transaction Advice.

⇒ In order to apply Transaction advice on the specific methods of Target/service class use `<aop:config>`, `<aop:advisor>`, `<aop:pointcut>` tag as shown below.

▲ TxMgmtProj2(Decl Txmgmnt AspectJAOP Local,Flat

▲ src

▲ com.nt.cfgs

▲ applicationContext.xml

▲ com.nt.dao

▶ BankDAO.java

▶ BankDAOImpl.java

▲ com.nt.service

▶ BankService.java

▲ com.nt.test

▶ ClientApp.java

▶ Referenced Libraries

▶ TXlib

▶ JRE System Library [jre7]

Add AspectJ to jar files.

#### BankDAO.java

```
package com.nt.dao;  
public interface BankDAO {  
    public int withdraw(int acno,int amt);  
    public int deposit(int acno,int amt);  
}
```

#### BankDAOImpl.java

```
package com.nt.dao;  
import org.springframework.jdbc.core.JdbcTemplate;  
public class BankDAOImpl implements BankDAO{  
    private JdbcTemplate jt;  
    private static final String WITHDRAW_QRY="UPDATE TX_ACCOUNT SET  
BALANCE=BALANCE-? WHERE ACCNO=?";  
    private static final String DEPOSITE_QRY="UPDATE TX_ACCOUNT SET  
BALANCE=BALANCE+? WHERE ACCNO=?";  
    public void setJt(JdbcTemplate jt) {  
        this.jt = jt;  
    }  
    @Override  
    public int withdraw(int acno, int amt) {  
        int cnt=jt.update(WITHDRAW_QRY,amt,acno);  
        return cnt;  
    }  
    @Override  
    public int deposit(int acno, int amt) {  
        int cnt=jt.update(DEPOSITE_QRY,amt,acno);  
        return cnt;  
    }  
}
```

#### BankService.java

```
package com.nt.service;  
import com.nt.dao.BankDAO;  
public class BankService {  
    private BankDAO dao;  
    public void setDao(BankDAO dao) {  
        this.dao = dao;  
    }  
    public boolean transferMoney(int srcNo,int destNo,int amt){  
        int result1=dao.withdraw(srcNo, amt);  
        int result2=dao.deposit(destNo, amt);  
  
        if(result1==0 || result2==0){  
            throw new RuntimeException("Money not Transferd(Tx rolled back)");  
        }  
        else{  

```

```
        System.out.println("Money Transferred(Tx committed)");
    }
    return true;
}
}
```

#### applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xmlns:tx="http://www.springframework.org/schema/tx"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
    <property name="username" value="scott"/>
    <property name="password" value="tiger"/>
</bean>

<bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="dataSource"/>
</bean>

<bean id="bankDAO" class="com.nt.dao.BankDAOImpl">
    <property name="jt" ref="template"/>
</bean>

<bean id="txMgr" class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
    <property name="dataSource" ref="dataSource"/>
</bean>

<bean id="bankService" class="com.nt.service.BankService">
    <property name="dao" ref="bankDAO"/>
</bean>

<tx:advice id="txAdvice" transaction-manager="txMgr">
    <tx:attributes>
        <tx:method name="transferMoney" propagation="REQUIRED"
                    read-only="false"/>
    </tx:attributes>
</tx:advice>
<aop:config>
    <aop:pointcut id="ptc1"
        expression="execution(* com.nt.service.BankService.transferMoney(..))"/>
    <aop:advisor advice-ref="txAdvice" pointcut-ref="ptc1"/>
</aop:config>
</beans>
```

```
</aop:config>
</beans>
```

### ClientApp.java

```
package com.nt.test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.BankService;
public class ClientApp {
    public static void main(String[] args) {
        //create IOC container
        ApplicationContext ctx=new
        FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
        //get Proxy obj
        BankService proxy=ctx.getBean("bankService",BankService.class);
        try{
            //call b.method
            System.out.println("Money TransferMoney?" + proxy.transferMoney(1006, 1001,500));
        }
        catch(Exception e){
            e.printStackTrace();
        }
    }
}
```

- ⇒ We can repeat <tx:method> tag for multiple times to apply different Transaction management attributes on different target methods. If we specify \* then it represents all the methods of target class.

```
<tx:method name="*" propagation="REQUIRED"/>
```

- ⇒ In declarative, annotation based Transaction management management the Transaction manager will rollback the transaction only when target method throws unchecked exception (RuntimeException and it's subclass) doesn't rollback for checked exception.
- ⇒ We Generally design DAO class one on per table strategy. Instead of applying Transaction management service on DAO class methods directly use DAO classes in service class methods and apply Transaction management service on that methods of service class.

### Annotations driven AspectJAOP Transaction management management:

- ⇒ For this Transaction, the Transaction management configurations will not be done in xml file. They will be configured directly on target class methods by using @Transactional.
- ⇒ To make IOC container detecting and using these annotations we can use <tx:annotation-driven transaction-manager="..." />

▲ TxMgmtProj3 (Anno Tx mgmt Local Tx, Flat Tx)

▲ src

▲ com.nt.cfgs

    [X] applicationContext.xml

▲ com.nt.dao

    ▶ [J] BankDAO.java

    ▶ [J] BankDAOImpl.java

▲ com.nt.service

    ▶ [J] BankService.java

▲ com.nt.test

    ▶ [J] ClientApp.java

▶ Referenced Libraries

▶ TXlib

▶ JRE System Library [jre7]

### BankDAO.java

```
package com.nt.dao;

public interface BankDAO {
    public int withdraw(int acno,int amt);
    public int deposit(int acno,int amt);
}
```

### BankDAOImpl.java

```
package com.nt.dao;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
@Repository("bankDAO")
public class BankDAOImpl implements BankDAO{
    private JdbcTemplate jt;
    private static final String WITHDRAW_QRY="UPDATE TX_ACCOUNT SET
BALANCE=BALANCE-? WHERE ACCNO=?";
    private static final String DEPOSITE_QRY="UPDATE TX_ACCOUNT SET
BALANCE=BALANCE+? WHERE ACCNO=?";
    @Autowired
    public void setJt(JdbcTemplate jt) {
        this.jt = jt;
    }
    @Override
    public int withdraw(int acno, int amt) {
        int cnt=jt.update(WITHDRAW_QRY,amt,acno);
        return cnt;
    }
    @Override
    public int deposit(int acno, int amt) {
        int cnt=jt.update(DEPOSITE_QRY,amt,acno);
        return cnt;
    }
}
```



**BankService.java**

```
package com.nt.service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;
import com.nt.dao.BankDAO;
@Component("bankService")
public class BankService {
    private BankDAO dao;
    @Autowired
    public void setDao(BankDAO dao) {
        this.dao = dao;
    }
    @Transactional(propagation=Propagation.REQUIRED)
    public boolean transferMoney(int srcNo,int destNo,int amt){
        int result1=dao.withdraw(srcNo, amt);
        int result2=dao.deposit(destNo, amt);
        if(result1==0 || result2==0){
            throw new RuntimeException("Money not Transferred(Tx rolled back)");
        }
        else{
            System.out.println("Money Transferred (Tx committed)");
            return true;
        }
    }
}
```

**applicationContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xmlns:tx="http://www.springframework.org/schema/tx"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
    <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
    <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
    <property name="username" value="scott"/>
    <property name="password" value="tiger"/>
</bean>
```

```
</bean>

<bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
  <property name="dataSource" ref="dataSource"/>
</bean>
<bean id="txMgr"
class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
  <property name="dataSource" ref="dataSource"/>
</bean>

<context:component-scan base-package="com.nt.service,com.nt.dao"/>
  <tx:annotation-driven transaction-manager="txMgr"/>
</beans>
```

### ClientApp.java

```
package com.nt.test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.BankService;
public class ClientApp {
public static void main(String[] args) {
    //create IOC container
    ApplicationContext ctx=new FileSystemXmlApplicationContext ("src/com/ nt/cfgs/
                                                                    applicationContext.xml");

    //get Proxy obj
    BankService proxy=ctx.getBean("bankService",BankService.class);
    try{
    //call b.method
    System.out.println("MoneyTransferred?" + proxy.transferMoney(1001, 1008,500));
    }
    catch(Exception e){
        e.printStackTrace();
    }
}
}
```

### Converting AnnotationDriven TransactionManagement application to 100% code based application

- a) Remove applicationContext.xml
- b) Develop applicationConfiguration class (like AppConfig.java)

**@configuration**

**@ComponentScan(basePackages={"com.nt.dao", "com.nt.service"})**

**@EnableTransactionManagement**

```
public class AppConfig{

    @Bean(name="dbcpds")

    public DataSource createDataSource{

        BasicDataSource ds= new BasicDataSource();

        ds.setDriverCalssName("oracle.jdbc.driver.OracleDriver");

        =====

        =====

        return ds;

    }

    @Bean(name="jt")

    public JdbcTemplate createJdbcTemplate{

        JdbcTemplate jt=null;

        Jt=new JdbcTemplate();

        jt.setDataSource(createDataSource());

        return jt;

    }

    @Bean(name="dsTxMgmr")

    public FlatformTransactionManager createTxManager{

        DataSourceTransactionManager txMgr =null;

        Jt=new DataSourceTransactionManager ();

        jt.setDataSource(createDataSource());

        return txMgr;

    }

}
```

```
}//class
```

- c) Add @Transactional on the top of target method of target class.

```
@Transactional(Propagation=Propagation.REQUIRED,transactionManager="dsTxMgmr")

public Boolean transformmoney(-,-){

==

}
```

- d) Use AnnotationConfigApplicationContext in client application.

```
ApplicationContext ctx=new AnnotationConfigApplicationContext(AppConfig.class);
```

### @Transactional properties

| Property               | Type                                                               | Description                                                                |
|------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------------|
| <u>value</u>           | String                                                             | Optional qualifier specifying the transaction manager to be used.          |
| <u>propagation</u>     | enum: Propagation                                                  | Optional propagation setting.                                              |
| Isolation              | enum: Isolation                                                    | Optional isolation level.                                                  |
| readOnly               | Boolean                                                            | Read/write vs. read-only transaction                                       |
| Timeout                | int (in seconds granularity)                                       | Transaction timeout.                                                       |
| rollbackFor            | Array of Class objects, which must be derived from Throwable.      | Optional array of exception classes that must cause rollback.              |
| rollbackForClassname   | Array of class names. Classes must be derived from Throwable.      | Optional array of names of exception classes that must cause rollback.     |
| noRollbackFor          | Array of Class objects, which must be derived from Throwable.      | Optional array of exception classes that must not cause rollback.          |
| noRollbackForClassname | Array of String class names, which must be derived from Throwable. | Optional array of names of exception classes that must not cause rollback. |

- ⇒ While working with @Transactional we can specify readOnly param and propagation param for transaction attributes.
- ⇒ In order to apply same Transaction management attributes on all methods of target class we can place @Transactional on top of class.

### Other use cases of transactionmanagement:

In employee region its details should be saved in hrDB and in finance DB.

In student register its details should be save in library db and admission DB.

In online shopping goods delivery and payment must be executed in a transaction.

**TxMgmtProj4-LocalTx-FlatTx-SpringJDBC-AspectJAOP-XmlAnnotations:**

=====

**Example:****BankService.java**

```
package com.nt.service;

import java.sql.SQLException;

public interface BankService {

    public boolean transferMoney(int srcAcno,int destAcno,float amt);

}
```

**BankServiceImpl.java**

```
package com.nt.service;

import java.sql.SQLException;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

import com.nt.dao.BankDAO;

@Service("bankService")
@Transactional(transactionManager="dsTxMgmr",readOnly=false,
propagation=Propagation.REQUIRED,noRollbackFor=NullPointerException.class,
rollbackFor=SQLException.class,timeout=20)
public class BankServiceImpl implements BankService {

    @Autowired
    private BankDAO dao;

    public boolean transferMoney(int srcAcno, int destAcno, float amt){
        int count1=0,count2=0;
        //write b.logic
        /*
        try{
            Thread.sleep(30000);
        }
        catch(Exception e){
            e.printStackTrace();
        }
        */
        count1=dao.withdraw(srcAcno, amt);
        count2=dao.deposit(destAcno, amt);
        if(count1==0 || count2==0){
            throw new IllegalStateException();
        }
        else{
            return true;
        }
    }
}
```

```
    }  
    }  
} //method  
} //class
```

**BankDAO.java**

```
package com.nt.dao;  
  
public interface BankDAO {  
    public int withdraw(int srcAcno, float amt);  
    public int deposit(int destAcno, float amt);  
}
```

**BankDAOImpl.java**

```
package com.nt.dao;  
  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.jdbc.core.JdbcTemplate;  
import org.springframework.stereotype.Repository;  
  
@Repository("bankDAO")  
public class BankDAOImpl implements BankDAO {  
    private static final String WITHDRAW_QUERY="UPDATE TX_ACCOUNT SET BALANCE=BALANCE-  
? WHERE ACCNO=?";  
    private static final String DEPOSITE_QUERY="UPDATE TX_ACCOUNT SET BALANCE=BALANCE+?  
WHERE ACCNO=?";  
    @Autowired  
    private JdbcTemplate jt;  
    public int withdraw(int srcAcno, float amt) {  
        int count=0;  
        count=jt.update(WITHDRAW_QUERY, amt,srcAcno);  
        return count;  
    }  
  
    public int deposit(int destAcno, float amt) {  
        int count=0;  
        count=jt.update(DEPOSITE_QUERY, amt,destAcno);  
        return count;  
    }  
}
```

**TxMgmtTest.java**

```
package com.nt.test;  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.AbstractApplicationContext;  
import org.springframework.context.support.FileSystemXmlApplicationContext;  
import com.nt.service.BankService;
```

```
public class TxMgmtTest {  
  
    public static void main(String[] args) {  
        ApplicationContext ctx=null;  
        BankService proxy=null;  
        //create IOC container  
        ctx=new  
FileSystemXmlApplicationContext("src/main/java/com/nt/cfgs/applicationContext.xml");  
        //get Bean  
        proxy=ctx.getBean("bankService",BankService.class);  
        try{  
            //invoke methods  
            System.out.println("Money Transffered??"+proxy.transferMoney(1001,1002,500));  
        }  
        catch(Exception e){  
            System.out.println("Money not Transffered");  
            e.printStackTrace();  
        }  
  
        //close IOC container  
        ((AbstractApplicationContext) ctx).close();  
  
    }  
  
} //main  
}  
}
```

## aop-beans.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:aop="http://www.springframework.org/schema/aop"
        xmlns:tx="http://www.springframework.org/schema/tx"
        xsi:schemaLocation="
            http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans.xsd
            http://www.springframework.org/schema/tx
            http://www.springframework.org/schema/tx/spring-tx.xsd
            http://www.springframework.org/schema/aop
            http://www.springframework.org/schema/aop/spring-aop.xsd">

    <bean id="dsTxMgr"
        class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
        <property name="dataSource" ref="bds" />
    </bean>

    <tx:annotation-driven/>

</beans>
```

**service-beans.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">

    <context:component-scan base-package="com.nt.service"/>

</beans>
```

**persistence-beans.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">

    <bean id="bds" class="org.apache.commons.dbcp2.BasicDataSource">
        <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"/>
        <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"/>
        <property name="username" value="system"/>
        <property name="password" value="manager"/>
        <property name="initialSize" value="8"/>
    </bean>

    <bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
        <property name="dataSource" ref="bds"/>
    </bean>

    <context:component-scan base-package="com.nt.dao"/>

</beans>
```

**applicationContext.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
```



```
http://www.springframework.org/schema/context/spring-context.xsd">
```

```
<import resource="persistence-beans.xml"/>
<import resource="service-beans.xml"/>
<import resource="aop-beans.xml"/>
</beans>
```

**TxMgmtProj5-LocalTx-FlatTx-SpringJDBC-AspectJAOP-100p:**

### Example

#### **BankService.java**

```
package com.nt.service;

import java.sql.SQLException;

public interface BankService {

    public boolean transferMoney(int srcAcno,int destAcno,float amt);

}
```

#### **BankServiceImpl.java**

```
package com.nt.service;

import java.sql.SQLException;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

import com.nt.dao.BankDAO;

@Service("bankService")
@Transactional(transactionManager="dsTxMgmt",readOnly=false,
propagation=Propagation.REQUIRED,noRollbackFor=NullPointerException.class,
rollbackFor=SQLException.class,timeout=20)
public class BankServiceImpl implements BankService {

    @Autowired
    private BankDAO dao;

    public boolean transferMoney(int srcAcno, int destAcno, float amt){
        int count1=0,count2=0;
        //write b.logic
        /* try{
```

```
        Thread.sleep(30000);
    }
    catch(Exception e){
        e.printStackTrace();
    }*/
    count1=dao.withdraw(srcAcno, amt);
    count2=dao.deposite(destAcno, amt);
    if(count1==0 || count2==0){
        throw new IllegalStateException();
    }
    else{
        return true;
    }
} //method
} //class
```

### BankDAO.java

```
package com.nt.dao;

public interface BankDAO {
    public int withdraw(int srcAcno, float amt);
    public int deposite(int destAcno, float amt);
}
```

### BankDAOImpl.java

```
package com.nt.dao;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;

@Repository("bankDAO")
public class BankDAOImpl implements BankDAO {
    private static final String WITHDRAW_QUERY="UPDATE TX_ACCOUNT SET BALANCE=BALANCE-
? WHERE ACCNO=?";
    private static final String DEPOSITE_QUERY="UPDATE TX_ACCOUNT SET BALANCE=BALANCE+?
WHERE ACCNO=?";
    @Autowired
    private JdbcTemplate jt;

    public int withdraw(int srcAcno, float amt) {
        int count=0;
        count=jt.update(WITHDRAW_QUERY, amt,srcAcno);
        return count;
    }

    public int deposite(int destAcno, float amt) {
        int count=0;
        count=jt.update(DEPOSITE_QUERY, amt,destAcno);
    }
}
```

```
        return count;
    }
}
```

**TxMgmtTest.java**

```
package com.nt.test;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.support.AbstractApplicationContext;

import com.nt.config.AppConfig;
import com.nt.service.BankService;

public class TxMgmtTest {

    public static void main(String[] args) {
        ApplicationContext ctx=null;
        BankService proxy=null;
        //create IOC container
        ctx=new AnnotationConfigApplicationContext(AppConfig.class);
        //get Bean
        proxy=ctx.getBean("bankService",BankService.class);
        try{
            //invoke methods
            System.out.println("Money Transferred?" + proxy.transferMoney(10012,1002,500));
        }
        catch(Exception e){
            System.out.println("Money not Transferred");
            e.printStackTrace();
        }

        //close IOC container
        ((AbstractApplicationContext) ctx).close();

    }

}

//main
}
//class
```

**AopConfig.java**

```
package com.nt.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.EnableAspectJAutoProxy;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

@Configuration
```

```
@EnableTransactionManagement
public class AopConfig {

    @Bean(name="dsTxMgmr")
    public DataSourceTransactionManager createTxManager(){
        return new
        DataSourceTransactionManager(PersistenceConfig.createDataSource());
    }
}
```

**AppConfig.java**

```
package com.nt.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Import;

@Configuration
@Import(value={PersistenceConfig.class,ServiceConfig.class,AopConfig.class})
public class AppConfig {

}
```

**ServiceConfig.java**

```
package com.nt.config;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages="com.nt.service")
public class ServiceConfig {

}
```

**PersistenceConfig.java**

```
package com.nt.config;
import javax.sql.DataSource;
import org.apache.commons.dbcp2.BasicDataSource;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;

@Configuration
@ComponentScan(basePackages="com.nt.dao")
public class PersistenceConfig {
```

```

@Bean
public static DataSource createDataSource(){
    BasicDataSource bds=null;
    bds=new BasicDataSource();
    bds.setDriverClassName("oracle.jdbc.driver.OracleDriver");
    bds.setUrl("jdbc:oracle:thin:@localhost:1521:xe");
    bds.setUsername("system");
    bds.setPassword("manager");
    return bds;
}

@Bean
public JdbcTemplate createJdbcTemplate(){
    return new JdbcTemplate(createDataSource());
}
}

```

### Distributed Transaction management:

- ⇒ It uses 2pc principle, by involving multiple resources in a single Tx boundary.
- ⇒ For this we need to use XA jdbc drivers, XA dataSources and XA databases.
- ⇒ Oracle all versions, mysql 5.x (or) XA database.
- ⇒ To perform distributed Tx management , we use JTA Tx Manager by supplying distributed TX manager Object and TXService Object.
- ⇒ We can get this 2 object either from jndi registry of application server or by using third party api like –“atomikos”, narayana.

Sample code:(atomikos)

Distributed TX Management:

```

<bean id="atomikos TX Mgr" class="com.atomikos.icatch.jta.UserTransactionManager"/>
<bean id="atomikosUT" class="com.atomikos.icatch.jta.UserTransactionImp"/>
<bean id="dtXmgr" class="org.sf.transaction.jta.JtaTransactionManager">
    <property name="transactionManager" ref="atomikosTxMgr"/>
    <property name="userTransaction" ref="atomikosUT"/>
</bean>

```

For web logic:

```

<bean id="jofb1" class="org.springframework.jndi.JndiObjectFactoryBean"/> //tx service
    <property name="jndiName" value="javax/transaction/UserTransaction"/>
</bean>
<bean id="jofb2" class="org.springframework.jndi.JndiObjectFactoryBean"/> //tx manager
    <property name="jndiName" value="javax/transaction/TransactionManager"/>
</bean>
<bean id="dtxMgr" class="pkg.JTATransactionManager"/> //tx service
    <property name="UserTransaction" ref="jofb1"/>
    <property name="TransactionManager" ref="jofb2"/>
</bean>

```

EX:

- ▲ TxMgmtProj4(Anno-Distributed Tx)
  - ▲ src
    - ▲ com.nt.cfgs
      - applicationContext.xml
    - ▲ com.nt.dao
      - ▷ DepositDAO.java
      - ▷ DepositDAOImpl.java
      - ▷ WithdrawDAO.java
      - ▷ WithdrawDAOImpl.java
    - ▲ com.nt.service
      - ▷ AccountService.java
    - ▲ com.nt.test
      - ▷ ClientApp.java
  - ▷ Referenced Libraries
  - ▷ TXlib
  - ▷ JRE System Library [jre7]

**DepositDAO.java**

```
package com.nt.dao;  
public interface DepositDAO {  
    public boolean deposit(int accNo,int amt);  
}
```

**DepositDAOImpl.java**

```
package com.nt.dao;  
import org.springframework.jdbc.core.JdbcTemplate;  
public class DepositDAOImpl implements DepositDAO {  
    private static final String DEPOSITE_QRY="UPDATE TX_ACCOUNT set BALANCE=BALANCE  
+? WHERE ACCNO=?";  
    private JdbcTemplate jt;  
  
    public void setJt(JdbcTemplate jt){  
        this.jt=jt;  
    }  
    @Override  
    public boolean deposit(int accNo, int amt) {  
        int cnt=jt.update(DEPOSITE_QRY,amt,accNo);  
        if(cnt<=0)  
            return false;  
        else  
            return true;  
    }  
}
```

**WithdrawDAO.java**

```
package com.nt.dao;  
public interface WithdrawDAO {  
    public boolean withdraw(int accNo,int amt);  
}
```

**WithdrawDAOImpl.java**

```
package com.nt.dao;
import org.springframework.jdbc.core.JdbcTemplate;
public class WithdrawDAOImpl implements WithdrawDAO {
    private static final String DEPOSITE_QRY="UPDATE TX_ACCOUNT set BALANCE=BALANCE
-? WHERE ACCNO=?";

    where private JdbcTemplate jt;
    public void setJt(JdbcTemplate jt){
        this.jt=jt;
    }

    @Override
    public boolean withdraw(int accNo, int amt) {
        int cnt=jt.update(WITHDRAW_QRY,amt,accNo);
        if(cnt<=0)
            return false;
        else
            return true;
    }
}
```

**AccountService.java**

```
package com.nt.service;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;
import com.nt.dao.DepositDAO;
import com.nt.dao.WithdrawDAO;
public class AccountService {
    private DepositDAO depositDAO;
    private WithdrawDAO withdrawDAO;
    public void setDepositDAO(DepositDAO depositDAO) {
        this.depositDAO = depositDAO;
    }
    public void setWithdrawDAO(WithdrawDAO withdrawDAO) {
        this.withdrawDAO = withdrawDAO;
    }
    @Transactional(propagation=Propagation.REQUIRED)
    public boolean transferMoney(int srcAcno,int destAcno,int amt){
        boolean result1=withdrawDAO.withdraw(srcAcno, amt);
        boolean result2=depositDAO.deposit(destAcno, amt);
        if(result1==false || result2==false){
            throw new RuntimeException(" Money not transfered");
        }
        return true;
    }
    }//transferMoney(-,-)
} //class
```

**appliactionContext.xml**

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xmlns:tx="http://www.springframework.org/schema/tx"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
<bean id="orads" class="com.atomikos.jdbc.AtomikosDataSourceBean" init-method="init"
destroy-method="close">
  <property name="uniqueResourceName" value="XAOrs"/>
  <property name="xaDataSourceClassName" value="oracle.jdbc.xa.client.OracleXADataSource"/>
  <property name="xaProperties">
<props>
  <prop key="databaseName">xe</prop>
  <prop key="user">scott</prop>
  <prop key="password">tiger</prop>
  <prop key="URL">jdbc:oracle:thin:@localhost:1521:xe</prop>
</props>
</property>
  <property name="poolSize" value="10"/>
</bean>
<bean id="mysqlDs" class="com.atomikos.jdbc.AtomikosDataSourceBean" init-method="init"
destroy-method="close">
  <property name="uniqueResourceName" value="XAMysql"/>
  <property name="xaDataSourceClassName"
value="com.mysql.jdbc.jdbc2.optional.MysqlXADataSource"/>
  <property name="xaProperties">
<props>
  <prop key="databaseName">ntsp7</prop>
  <prop key="user">root</prop>
  <prop key="password">root</prop>
  <prop key="URL">jdbc:mysql:///ntsp7</prop>
</props>
</property>
  <property name="poolSize" value="10"/>
</bean>
<bean id="template1" class="org.springframework.jdbc.core.JdbcTemplate">
  <property name="dataSource" ref="orads"/>
</bean>
<bean id="template2" class="org.springframework.jdbc.core.JdbcTemplate">
```



```

<property name="dataSource" ref="mysqlDS"/>
</bean>
<bean id="wDao" class="com.nt.dao.WithdrawDAOImpl">
  <property name="jt" ref="template1"/>
</bean>
<bean id="dDao" class="com.nt.dao.DepositeDAOImpl">
  <property name="jt" ref="template2"/>
</bean>
<bean id="service" class="com.nt.service.AccountService">
  <property name="depositDAO" ref="dDao"/>
  <property name="withdrawDAO" ref="wDao"/>
</bean>
<!-- Construct Atomikos UserTransactionManager, needed to configure Spring -->
<bean id="atomikosTxMgmr" class="com.atomikos.icatch.jta.UserTransactionManager"
  init-method="init" destroy-method="close">
  <!-- when close is called, should we force transactions to terminate or not? -->
  <property name="forceShutdown" value="true"/>
</bean>
<!-- Also use Atomikos UserTransactionImp, needed to configure Spring -->
<bean id="atomikosUT" class="com.atomikos.icatch.jta.UserTransactionImp">
  <property name="transactionTimeout" value="300"/>
</bean>
<!-- Configure the Spring framework to use JTA transactions from Atomikos -->
<bean id="dtxmgr" class="org.springframework.transaction.jta.JtaTransactionManager">
  <property name="transactionManager" ref="atomikosTxMgmr"/>
  <property name="userTransaction" ref="atomikosUT"/>
</bean>
<tx:annotation-driven transaction-manager="dtxmgr"/>
</beans>

```

#### ClientApp.java

```

package test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;
import com.nt.service.AccountService;
public class ClientApp {
  public static void main(String[] args) {
    //create IOC container
    ApplicationContext ctx=new
    FileSystemXmlApplicationContext("src/com/nt/cfgs/applicationContext.xml");
    //get Proxy obj
    AccountService proxy=ctx.getBean("service",AccountService.class);
    try{
      //call b.method
      System.out.println("Money Transferred?" + proxy.transferMoney(1201, 1002, 500));
    }
    catch(Exception e){
      e.printStackTrace();
    }
  }
}

```

```
}  
}  
}
```

Jars: Spring jdbc-lib, AOP library, Aspectjrt.jar, AspectJweaver.jar

Atomikos-transaction-api.jar, atomikos-transactions-jta.jar, atomikos-util.3.7.0.jar, com.mysql.jdbc-5.1.5.jar, jta.jar ojdbc.jar, transactions-3.7.0.jar, Transactions-jdbc-3.7.0.jar.

DB table in Oracle:

**Tx account**

| Accno | holder | balance |
|-------|--------|---------|
| 101   | Raja   | 9000    |

DB table in mysql 5.x (logical db:ntsp27)

**Tx account**

| Accno | holder | balance |
|-------|--------|---------|
| 102   | Ravi   | 8000    |

- ⇒ While working with distributed Transaction management the UserTransaction object represent Transaction service and TransactionManager object represents the Transaction manager that performs commit or rollback by using Transaction service.
- ⇒ JtaTransactionManager can use either server supplied or 3<sup>rd</sup> party API supplied Tx manager, UserTransaction objects to perform Tx management.

## SPRING FAQs

### Q. Explain DI or IOC pattern.

A: Dependency injection (DI) is a programming design pattern and architectural model, sometimes also referred to as inversion of control or IOC, although technically speaking, dependency injection specifically refers to an implementation of a particular form of IOC. Dependency Injection describes the situation where one object uses a second object to provide a particular capacity. For example, being passed a database connection as an argument to the constructor instead of creating one internally. The term "Dependency injection" is a misnomer, since it is not a dependency that is injected, rather it is a provider of some capability or resource that is injected. There are three common forms of dependency injection: setter-, constructor- and interface-based injection. Dependency injection is a way to achieve loose coupling. Inversion of control (IOC) relates to the way in which an object obtains references to its dependencies. This is often done by a lookup method. The advantage of inversion of control is that it decouples objects from specific lookup mechanisms and implementations of the objects it depends on. As a result, more flexibility is obtained for production applications as well as for testing.

### Q. What are the different IOC containers available?

A. Spring is an IOC container. Other IOC containers are HiveMind, Avalon, PicoContainer.

### Q. What are the different types of dependency injection. Explain with examples.

A: There are two types of dependency injection: setter injection and constructor injection. Setter Injection: Normally in all the java beans, we will use setter and getter method to set and get the value of property as follows:

```
public class namebean {  
    String name;  
    public void setName(String a) {  
        name = a; }  
    public String getName() {  
        return name; }  
}
```

We will create an instance of the bean 'namebean' (say bean1) and set property as bean1.setName("tom"); Here in setter injection, we will set the property 'name' in spring configuration file as shown below:

```
<bean id="bean1" class="namebean">  
    <property name="name" >  
        <value>tom</value>  
    </property>  
</bean>
```

The subelement <value> sets the 'name' property by calling the set method as setName("tom"); This process is called setter injection.

To set properties that reference other beans <ref>, subelement of <property> is used as shown below,

```
<bean id="bean1" class="bean1impl">
  <property name="game">
    <ref bean="bean2"/>
  </property>
</bean>
<bean id="bean2" class="bean2impl" />
```

**Constructor Injection:** For constructor injection, we use constructor with parameters as shown below,

```
public class namebean {
    String name;
    public namebean(String a) {
        name = a;
    }
}
```

We will set the property 'name' while creating an instance of the bean 'namebean' as namebean bean1 = new namebean("tom");

Here we use the <constructor-arg> element to set the the property by constructor injection as

```
<bean id="bean1" class="namebean">
  <constructor-arg>
    <value>My Bean Value</value>
  </constructor-arg>
</bean>
```

**Q. What is spring? What are the various parts of spring framework? What are the different persistence frameworks which could be used with spring?**

A. Spring is an open source framework created to address the complexity of enterprise application development. One of the chief advantages of the Spring framework is its layered architecture, which allows you to be selective about which of its components you use while also providing a cohesive framework for J2EE application development. The Spring modules are built on top of the core container, which defines how beans are created, configured, and managed, as shown in the following figure. Each of the modules (or components) that comprise the Spring framework can stand on its own or be implemented jointly with one or more of the others. The functionality of each component is as follows:

**The Core Container:** The core container provides the essential functionality of the Spring framework. A primary component of the core container is the BeanFactory, an implementation of the Factory pattern. The BeanFactory applies the Inversion of Control (IOC) pattern to separate an application's configuration and dependency specification from the actual application code.

**Spring Context:** The Spring context is a configuration file that provides context information to the Spring framework. The Spring context includes enterprise services such as JNDI, EJB, e-mail, internalization, validation, and scheduling functionality.

**Spring AOP:** The Spring AOP module integrates aspect-oriented programming functionality directly into the Spring framework, through its configuration management feature. As a result you can easily AOP-enable any object managed by the Spring framework. The Spring AOP module provides transaction management services for objects in any Spring-based application. With Spring AOP you can incorporate declarative transaction management into your applications without relying on EJB components.

**Spring DAO:** The Spring JDBC DAO abstraction layer offers a meaningful exception hierarchy for managing the exception handling and error messages thrown by different database vendors. The exception hierarchy simplifies error handling and greatly reduces the amount of exception code you need to write, such as opening and closing connections. Spring DAO's JDBC-oriented exceptions comply to its generic DAO exception hierarchy.

**Spring ORM:** The Spring framework plugs into several ORM frameworks to provide its Object Relational tool, including JDO, Hibernate, and iBatis SQL Maps. All of these comply to Spring's generic transaction and DAO exception hierarchies.

**Spring Web Module:** The Web context module builds on top of the application context module, providing contexts for Web-based applications. As a result, the Spring framework supports integration with Jakarta Struts. The Web module also eases the tasks of handling multi-part requests and binding request parameters to domain objects.

**Spring MVC Framework:** The Model-View-Controller (MVC) framework is a full-featured MVC implementation for building Web applications. The MVC framework is highly configurable via strategy interfaces and accommodates numerous view technologies including JSP, Velocity, Tiles, iText, and POI.

**Q. What is AOP? How does it relate with IOC? What are different tools to utilize AOP?**

**A:** Aspect-oriented programming, or AOP, is a programming technique that allows programmers to modularize crosscutting concerns, or behavior that cuts across the typical divisions of responsibility, such as logging and transaction management. The core construct of AOP is the aspect, which encapsulates behaviors affecting multiple classes into reusable modules. AOP and IOC are complementary technologies in that both apply a modular approach to complex problems in enterprise application development. In a typical object-oriented development approach you might implement logging functionality by putting logger statements in all your methods and Java classes. In an AOP approach you would instead modularize the logging services and apply them declaratively to the components that required logging. The advantage, of course, is that the Java class doesn't need to know about the existence of the logging service or concern itself with any related code. As a result, application code written using Spring AOP is loosely coupled. The best tool to utilize AOP to its capability is AspectJ. However AspectJ works at the byte code level and you need to use AspectJ compiler to get the aop features built into your compiled code. Nevertheless AOP functionality is fully integrated into the Spring context for transaction management, logging, and various other features. In general any AOP framework control aspects in three possible ways:

**Joinpoints:** Points in a program's execution. For example, joinpoints could define calls to specific methods in a class  
**Pointcuts:** Program constructs to designate joinpoints and collect specific context at those points  
**Advices:** Code that runs upon meeting certain conditions. For example, an advice could log a message before executing a joinpoint

**Q. What are the advantages of spring framework?**

**A.**

1. Spring has layered architecture. Use what you need and leave you don't need now.
2. Spring Enables POJO Programming. There is no behind the scene magic here. POJO programming enables continuous integration and testability.
3. Dependency Injection and Inversion of Control Simplifies JDBC (Read the first question.)
4. Open source and no vendor lock-in.

**Q. Can you name a tool which could provide the initial ant files and directory structure for a new spring project.**

**A:** Appfuse or equinox.

**Q. Explain BeanFactory in spring.**

A: Bean factory is an implementation of the factory design pattern and its function is to create and dispense beans. As the bean factory knows about many objects within an application, it is able to create association between collaborating objects as they are instantiated. This removes the burden of configuration from the bean and the client. There are several implementation of BeanFactory. The most useful one is "org.springframework.beans.factory.xml.XmlBeanFactory" It loads its beans based on the definition contained in an XML file. To create an XmlBeanFactory, pass a InputStream to the constructor. The resource will provide the XML to the factory. BeanFactory factory = new XmlBeanFactory(new FileInputStream("myBean.xml"));

This line tells the bean factory to read the bean definition from the XML file. The bean definition includes the description of beans and their properties. But the bean factory doesn't instantiate the bean yet. To retrieve a bean from a 'BeanFactory', the getBean() method is called. When getBean() method is called, factory will instantiate the bean and begin setting the bean's properties using dependency injection. myBean bean1 = (myBean)factory.getBean("myBean");

**Q. Explain the role of ApplicationContext in spring.**

A. While Bean Factory is used for simple applications, the Application Context is spring's more advanced container. Like 'BeanFactory' it can be used to load bean definitions, wire beans together and dispense beans upon request. It also provide

- 1) a means for resolving text messages, including support for internationalization.
- 2) a generic way to load file resources.
- 3) events to beans that are registered as listeners.

Because of additional functionality, 'Application Context' is preferred over a BeanFactory. Only when the resource is scarce like mobile devices, 'BeanFactory' is used. The three commonly used implementation of 'Application Context' are

**1. ClassPathXmlApplicationContext :** It Loads context definition from an XML file located in the classpath, treating context definitions as classpath resources. The application context is loaded from the application's classpath by using the code

```
ApplicationContext context = new ClassPathXmlApplicationContext("bean.xml");
```

**2. FileSystemXmlApplicationContext :** It loads context definition from an XML file in the filesystem. The application context is loaded from the file system by using the code

```
ApplicationContext context = new FileSystemXmlApplicationContext("bean.xml");
```

**3. XmlWebApplicationContext :** It loads context definition from an XML file contained within a web application.

**Q. How does Spring supports DAO in hibernate?**

A. Spring's HibernateDaoSupport class is a convenient super class for Hibernate DAOs. It has handy methods you can call to get a Hibernate Session, or a SessionFactory. The most convenient method is getHibernateTemplate(), which returns a HibernateTemplate. This template wraps Hibernate checked exceptions with runtime exceptions, allowing your DAO interfaces to be Hibernate exception-free.

**Example:**

```
public class UserDAOHibernate extends HibernateDaoSupport {
```

```
    public User getUser(Long id) {  
        return (User) getHibernateTemplate().get(User.class, id);  
    }  
}
```

```
public void saveUser(User user) {  
    getHibernateTemplate().saveOrUpdate(user);  
    if (log.isDebugEnabled()) {  
        log.debug("userId set to: " + user.getID());  
    }  
}  
  
public void removeUser(Long id) {  
    Object user = getHibernateTemplate().load(User.class, id);  
    getHibernateTemplate().delete(user);  
}  
  
}
```

**Q. What are the id generator classes in hibernate?**

A: increment: It generates identifiers of type long, short or int that are unique only when no other process is inserting data into the same table. It should not be used in the clustered environment.

identity: It supports identity columns in DB2, MySQL, MS SQL Server, Sybase and HypersonicSQL. The returned identifier is of type long, short or int.

sequence: The sequence generator uses a sequence in DB2, PostgreSQL, Oracle, SAP DB, McKoi or a generator in Interbase. The returned identifier is of type long, short or int.

hilo: The hilo generator uses a hi/lo algorithm to efficiently generate identifiers of type long, short or int, given a table and column (by default hibernate\_unique\_key and next\_hi respectively) as a source of hi values. The hi/lo algorithm generates identifiers that are unique only for a particular database. Do not use this generator with connections enlisted with JTA or with a user-supplied connection.

seqhilo: The seqhilo generator uses a hi/lo algorithm to efficiently generate identifiers of type long, short or int, given a named database sequence.

uuid: The uuid generator uses a 128-bit UUID algorithm to generate identifiers of type String, unique within a network (the IP address is used). The UUID is encoded as a String of hexadecimal digits of length 32.

guid: It uses a database-generated GUID String on MS SQL Server and MySQL.

native: It picks identity, sequence or hilo depending upon the capabilities of the underlying database.

assigned: lets the application to assign an identifier to the object before save() is called. This is the default strategy if no <generator> element is specified.

select: retrieves a primary key assigned by a database trigger by selecting the row by some unique key and retrieving the primary key value.

foreign: uses the identifier of another associated object. Usually used in conjunction with a <one-to-one> primary key association.

**Q. How is a typical spring implementation look like?**

A. For a typical Spring Application we need the following files

1. An interface that defines the functions.
2. An Implementation that contains properties, its setter and getter methods, functions etc.,
3. A XML file called Spring configuration file.
4. Client program that uses the function.

**Q. How do you define hibernate mapping file in spring?**

A. Add the hibernate mapping file entry in mapping resource inside Spring's applicationContext.xml file in the web/WEB-INF directory.

```
<property name="mappingResources">
  <list>
    <value>org/appfuse/model/User.hbm.xml</value>
  </list>
</property>
```

**Q. How do you configure spring in a web application?**

A. It is very easy to configure any J2EE-based web application to use Spring. At the very least, you can simply add Spring's ContextLoaderListener to your web.xml file:

```
<listener>
  <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
</listener>
```

**Q. Can you have xyz.xml file instead of applicationContext.xml?**

A. ContextLoaderListener is a ServletContextListener that initializes when your webapp starts up. By default, it looks for Spring's configuration file at WEB-INF/applicationContext.xml. You can change this default value by specifying a <context-param> element named "contextConfigLocation." Example:

```
<listener>
  <listener-class>org.springframework.web.context.ContextLoaderListener

  <context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/xyz.xml</param-value>
  </context-param>
</listener-class>
</listener>
```

**Q. How do you configure your database driver in spring?**

A. Using datasource "org.springframework.jdbc.datasource.DriverManagerDataSource".

Example:

```
<bean id="dataSource" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
  <property name="driverClassName">
    <value>org.hsqldb.jdbcDriver</value>
  </property>
  <property name="url">
    <value>jdbc:hsqldb:db/appfuse</value>
  </property>
  <property name="username"><value>sa</value></property>
  <property name="password"><value></value></property>
</bean>
```

**Q. How can you configure JNDI instead of datasource in spring applicationContext.xml?**

A. Using "org.springframework.jndi.JndiObjectFactoryBean". Example:

```
<bean id="dataSource" class="org.springframework.jndi.JndiObjectFactoryBean">
```



```
<property name="jndiName">  
  <value>java:comp/env/jdbc/appfuse</value>  
</property>  
</bean>
```

### **What is IOC (or Dependency Injection)?**

The basic concept of the Inversion of Control pattern (also known as dependency injection) is that you do not create your objects but describe how they should be created. You don't directly connect your components and services together in code but describe which services are needed by which components in a configuration file. A container (in the case of the Spring framework, the IOC container) is then responsible for hooking it all up.

i.e., Applying IoC, objects are given their dependencies at creation time by some external entity that coordinates each object in the system. That is, dependencies are injected into objects. So, IoC means an inversion of responsibility with regard to how an object obtains references to collaborating objects.

### **What are the different types of IOC (dependency injection) ?**

There are three types of dependency injection:

- Constructor Injection (e.g. Pico container, Spring etc): Dependencies are provided as constructor parameters.
- Setter Injection (e.g. Spring): Dependencies are assigned through JavaBeans properties (ex: setter methods).
- Interface Injection (e.g. Avalon): Injection is done through an interface.

Note: Spring supports only Constructor and Setter Injection

### **What are the benefits of IOC (Dependency Injection)?**

Benefits of IOC (Dependency Injection) are as follows:

- Minimizes the amount of code in your application. With IOC containers you do not care about how services are created and how you get references to the ones you need. You can also easily add additional services by adding a new constructor or a setter method with little or no extra configuration.
- Make your application more testable by not requiring any singletons or JNDI lookup mechanisms in your unit test cases. IOC containers make unit testing and switching implementations very easy by manually allowing you to inject your own objects into the object under test.
- Loose coupling is promoted with minimal effort and least intrusive mechanism. The factory design pattern is more intrusive because components or services need to be requested explicitly whereas in IOC the dependency is injected into requesting piece of code. Also some containers promote the design to interfaces not to implementations design concept by encouraging managed objects to implement a well-defined service interface of your own.
- IOC containers support eager instantiation and lazy loading of services. Containers also provide support for instantiation of managed objects, cyclical dependencies, life cycles management, and dependency resolution between managed objects etc.

## **4. What is Spring ?**

Spring is an open source framework created to address the complexity of enterprise application development. One of the chief advantages of the Spring framework is its layered architecture, which allows you to be selective about which of its components you use while also providing a

cohesive framework for J2EE application development.

**Q) What are the advantages of Spring framework?**

The advantages of Spring are as follows:

- Spring has layered architecture. Use what you need and leave you don't need now.
- Spring Enables POJO Programming. There is no behind the scene magic here. POJO programming enables continuous integration and testability.
- Dependency Injection and Inversion of Control Simplifies JDBC
- Open source and no vendor lock-in.

**Q) What are features of Spring ?**

**Lightweight:**

spring is lightweight when it comes to size and transparency. The basic version of spring framework is around 1MB. And the processing overhead is also very negligible.

**Inversion of control (IOC):**

Loose coupling is achieved in spring using the technique Inversion of Control. The objects give their dependencies instead of creating or looking for dependent objects.

**Aspect oriented (AOP):**

Spring supports Aspect oriented programming and enables cohesive development by separating application business logic from system services.

**Container:**

Spring contains and manages the life cycle and configuration of application objects.

**MVC Framework:**

Spring comes with MVC web application framework, built on core Spring functionality. This framework is highly configurable via strategy interfaces, and accommodates multiple view technologies like JSP, Velocity, Tiles, iText, and POI. But other frameworks can be easily used instead of Spring MVC Framework.

**Transaction Management:**

Spring framework provides a generic abstraction layer for transaction management. This allowing the developer to add the pluggable transaction managers, and making it easy to demarcate transactions without dealing with low-level issues. Spring's transaction support is not tied to J2EE environments and it can be also used in container less environments.

**JDBC Exception Handling:**

The JDBC abstraction layer of the Spring offers a meaningful exception hierarchy, which simplifies the error handling strategy. Integration with Hibernate, JDO, and iBATIS: Spring provides best Integration services with Hibernate, JDO and iBATIS contexts for Web-based applications. As a result, the Spring framework supports integration with Jakarta Struts. The Web module also eases the tasks of handling multi-part requests and binding request parameters to domain objects.

**Q). What is web module?**

This module is built on the application context module, providing a context that is appropriate for web-based applications. This module also contains support for several web-oriented tasks such as transparently handling multipart requests for file uploads and programmatic binding of request

parameters to your business objects. It also contains integration support with Jakarta Struts.

8. What are the types of Dependency Injection Spring supports?

Setter Injection:

Setter-based DI is realized by calling setter methods on your beans after invoking a no-argument constructor or no-argument static factory method to instantiate your bean.

Constructor Injection:

Constructor-based DI is realized by invoking a constructor with a number of arguments, each representing a collaborator.

### **Q)What is Bean Factory ?**

A BeanFactory is like a factory class that contains a collection of beans. The BeanFactory holds Bean Definitions of multiple beans within itself and then instantiates the bean whenever asked for by clients.

- BeanFactory is able to create associations between collaborating objects as they are instantiated. This removes the burden of configuration from bean itself and the beans client.
- BeanFactory also takes part in the life cycle of a bean, making calls to custom initialization and destruction methods.

### **Q) What is Application Context?**

A bean factory is fine to simple applications, but to take advantage of the full power of the Spring framework, you may want to move up to Springs more advanced container, the application context. On the surface, an application context is same as a bean factory. Both load bean definitions, wire beans together, and dispense beans upon request. But it also provides:

- A means for resolving text messages, including support for internationalization.
- A generic way to load file resources.
- Events to beans that are registered as listeners

### **Q) What is the difference between Bean Factory and Application Context ?**

On the surface, an application context is same as a bean factory. But application context offers much more..

- Application contexts provide a means for resolving text messages, including support for i18n of those messages.
- Application contexts provide a generic way to load file resources, such as images.
- Application contexts can publish events to beans that are registered as listeners.
- Certain operations on the container or beans in the container, which have to be handled in a programmatic fashion with a bean factory, can be handled declaratively in an application context.
- ResourceLoader support: Spring's Resource interface us a flexible generic abstraction for handling low-level resources. An application context itself is a ResourceLoader, Hence provides an application with access to deployment-specific Resource instances.
- MessageSource support: The application context implements MessageSource, an interface used to obtain localized messages, with the actual implementation being pluggable

### **Q)what are the common implementations of the Application Context ?**

The three commonly used implementation of 'Application Context' are

- ClassPathXmlApplicationContext : It Loads context definition from an XML file located in the

classpath, treating context definitions as classpath resources. The application context is loaded from the application's classpath by using the code .

- `ApplicationContext context = new ClassPathXmlApplicationContext("bean.xml");`
- `FileSystemXmlApplicationContext` : It loads context definition from an XML file in the filesystem. The application context is loaded from the file system by using the code .
- `ApplicationContext context = new FileSystemXmlApplicationContext("bean.xml");`
- `XmlWebApplicationContext` : It loads context definition from an XML file contained within a web application.

### Q) How is a typical spring implementation look like ?

For a typical Spring Application we need the following files:

- An interface that defines the functions.
- An Implementation that contains properties, its setter and getter methods, functions etc.,
- Spring AOP (Aspect Oriented Programming)
- A XML file called Spring configuration file.
- Client program that uses the function.

### Q) What is the typical Bean life cycle in Spring Bean Factory Container ?

Bean life cycle in Spring Bean Factory Container is as follows:

- The spring container finds the bean's definition from the XML file and instantiates the bean.
- Using the dependency injection, spring populates all of the properties as specified in the bean definition
- If the bean implements the `BeanNameAware` interface, the factory calls `setBeanName()` passing the bean's ID.
- If the bean implements the `BeanFactoryAware` interface, the factory calls `setBeanFactory()`, passing an instance of itself.
- If there are any `BeanPostProcessors` associated with the bean, their `postProcessBeforeInitialization()` methods will be called.
- If an `init-method` is specified for the bean, it will be called.
- Finally, if there are any `BeanPostProcessors` associated with the bean, their `postProcessAfterInitialization()` methods will be called.

### Q) What do you mean by Bean wiring ?

The act of creating associations between application components (beans) within the Spring container is referred to as Bean wiring.

### Q). What do you mean by Auto Wiring?

The Spring container is able to autowire relationships between collaborating beans. This means that it is possible to automatically let Spring resolve collaborators (other beans) for your bean by inspecting the contents of the `BeanFactory`. The autowiring functionality has five modes.

- no
- `byName`
- `byType`
- constructor
- `autodirect`

**Q) What is DelegatingVariableResolver?**

Spring provides a custom JavaServer Faces VariableResolver implementation that extends the standard Java Server Faces managed beans mechanism which lets you use JSR169 and Spring together. This variable resolver is called as DelegatingVariableResolver

**Q). What is AOP?**

Aspect-oriented programming, or AOP, is a programming technique that allows programmers to modularize crosscutting concerns, or behavior that cuts across the typical divisions of responsibility, such as logging and transaction management. The core construct of AOP is the aspect, which encapsulates behaviors affecting multiple classes into reusable modules.

**Q) How the AOP used in Spring?**

AOP is used in the Spring Framework: To provide declarative enterprise services, especially as a replacement for EJB declarative services. The most important such service is declarative transaction management, which builds on the Spring Framework's transaction abstraction. To allow users to implement custom aspects, complementing their use of OOP with AOP.

**Q) What do you mean by Aspect ?**

A modularization of a concern that cuts across multiple objects. Transaction management is a good example of a crosscutting concern in J2EE applications. In Spring AOP, aspects are implemented using regular classes (the schema-based approach) or regular classes annotated with the @Aspect annotation (@AspectJ style).

**Q) What do you mean by JointPoint?**

A point during the execution of a program, such as the execution of a method or the handling of an exception. In Spring AOP, a join point always represents a method execution.

**Q) What do you mean by Advice?**

Action taken by an aspect at a particular join point. Different types of advice include "around," "before" and "after" advice. Many AOP frameworks, including Spring, model an advice as an interceptor, maintaining a chain of interceptors "around" the join point.

**Q) What are the types of Advice?**

Types of advice:

- Before advice: Advice that executes before a join point, but which does not have the ability to prevent execution flow proceeding to the join point (unless it throws an exception).
- After returning advice: Advice to be executed after a join point completes normally: for example, if a method returns without throwing an exception.
- After throwing advice: Advice to be executed if a method exits by throwing an exception.
- After (finally) advice: Advice to be executed regardless of the means by which a join point exits (normal or exceptional return).
- Around advice: Advice that surrounds a join point such as a method invocation. This is the most powerful kind of advice. Around advice can perform custom behavior before and after the method invocation. It is also responsible for choosing whether to proceed to the join point or to shortcut the advised method execution by returning its own return value or throwing an exception

**Q). What are the types of the transaction management Spring supports ?**

Spring Framework supports:

- Programmatic transaction management.
- Declarative transaction management.

**Q) What are the benefits of the Spring Framework transaction management ?**

The Spring Framework provides a consistent abstraction for transaction management that delivers the following benefits:

- Provides a consistent programming model across different transaction APIs such as JTA, JDBC, Hibernate, JPA, and JDO.
- Supports declarative transaction management.
- Provides a simpler API for programmatic transaction management than a number of complex transaction APIs such as JTA.
- Integrates very well with Spring's various data access abstractions.

**Q) Why most users of the Spring Framework choose declarative transaction management ?**

Most users of the Spring Framework choose declarative transaction management because it is the option with the least impact on application code, and hence is most consistent with the ideals of a non-invasive lightweight container.

**Q). What are object/relational mapping integration module?**

Spring also supports for using of an object/relational mapping (ORM) tool over straight JDBC by providing the ORM module. Spring provide support to tie into several popular ORM frameworks, including Hibernate, JDO, and iBATIS SQL Maps. Spring's transaction management supports each of these ORM frameworks as well as JDBC.

**Q) When to use programmatic and declarative transaction management ?**

Programmatic transaction management is usually a good idea only if you have a small number of transactional operations. On the other hand, if your application has numerous transactional operations, declarative transaction management is usually worthwhile. It keeps transaction management out of business logic, and is not difficult to configure.

**Q) Explain about the Spring DAO support ?**

The Data Access Object (DAO) support in Spring is aimed at making it easy to work with data access technologies like JDBC, Hibernate or JDO in a consistent way. This allows one to switch between the persistence technologies fairly easily and it also allows one to code without worrying about catching exceptions that are specific to each technology.

**Q) What are the exceptions thrown by the Spring DAO classes ?**

Spring DAO classes throw exceptions which are subclasses of `DataAccessException(org.springframework.dao.DataAccessException)`. Spring provides a convenient translation from technology-specific exceptions like `SQLException` to its own exception class hierarchy with the `DataAccessException` as the root exception. These exceptions wrap the original exception.

**Q) What is `SQLExceptionTranslator` ?**

`SQLExceptionTranslator`, is an interface to be implemented by classes that can translate between `SQLExceptions` and Spring's own data-access-strategy-agnostic `org.springframework.dao.DataAccessException`.

**Q) What is Spring's `JdbcTemplate` ?**

Spring's `JdbcTemplate` is central class to interact with a database through JDBC. `JdbcTemplate` provides many convenience methods for doing things such as converting database data into primitives or objects, executing prepared and callable statements, and providing custom database

error handling.

```
JdbcTemplate template = new JdbcTemplate(myDataSource);
```

### Q) What is PreparedStatementCreator ?

PreparedStatementCreator:

- Is one of the most common used interfaces for writing data to database.
- Has one method – createPreparedStatement(Connection)
- Responsible for creating a PreparedStatement.
- Does not need to handle SQLExceptions.

### Q) What is SQLProvider ?

SQLProvider:

- Has one method – getSql()
- Typically implemented by PreparedStatementCreator implementers.
- Useful for debugging.

### Q) What is RowCallbackHandler ?

The RowCallbackHandler interface extracts values from each row of a ResultSet.

- Has one method – processRow(ResultSet)
- Called for each row in ResultSet.
- Typically stateful.

### Q) What are the differences between EJB and Spring ?

Transaction management

EJB:

- Must use a JTA transaction manager.
- Supports transactions that span remote method calls.

Spring:

- Supports multiple transaction environments through its PlatformTransactionManager interface, including JTA, Hibernate, JDO, and JDBC.
- Does not natively support distributed transactions—it must be used with a JTA transaction manager.

Declarative transaction support

EJB:

- Can define transactions declaratively through the deployment descriptor.
- Can define transaction behavior per method or per class by using the wildcard character \*.
- Cannot declaratively define rollback behavior—this must be done programmatically.

Spring:

- Can define transactions declaratively through the Spring configuration file or through class metadata.
- Can define which methods to apply transaction behavior explicitly or by using regular expressions.
- Can declaratively define rollback behavior per method and per exception type.

Persistence

EJB:

Supports programmatic bean-managed persistence and declarative container managed persistence.



**Spring:**

Provides a framework for integrating with several persistence technologies, including JDBC, Hibernate, JDO, and iBATIS.

**Declarative security****EJB:**

- Supports declarative security through users and roles. The management and implementation of users and roles is container specific.
- Declarative security is configured in the deployment descriptor.

**Spring:**

- No security implementation out-of-the box.
- Acegi, an open source security framework built on top of Spring, provides declarative security through the Spring configuration file or class metadata.

**Distributed computing****EJB:**

Provides container-managed remote method calls.

**Spring:**

Provides proxying for remote calls via RMI, JAX-RPC, and web services.

**Q) Explain Bean lifecycle in Spring framework?**

- The spring container finds the bean's definition from the XML file and instantiates the bean.
- Using the dependency injection, spring populates all of the properties as specified in the bean definition.
- If the bean implements the BeanNameAware interface, the factory calls setBeanName() passing the bean's ID.
- If the bean implements the BeanFactoryAware interface, the factory calls setBeanFactory(), passing an instance of itself.
- If there are any BeanPostProcessors associated with the bean, their postProcessBeforeInitialization() methods will be called.
- If an init-method is specified for the bean, it will be called.
- Finally, if there are any BeanPostProcessors associated with the bean, their postProcessAfterInitialization() methods will be called.

**Q). What are Inner Beans?**

When wiring beans, if a bean element is embedded to a property tag directly, then that bean is said to be an Inner Bean. The drawback of this bean is that it cannot be reused anywhere else.

**Q) What are the different types of events related to Listeners?**

There are a lot of events related to ApplicationContext of spring framework. All the events are subclasses of org.springframework.context.ApplicationEvent. They are

- ContextClosedEvent – This is fired when the context is closed.
- ContextRefreshedEvent – This is fired when the context is initialized or refreshed.
- RequestHandledEvent – This is fired when the web context handles any request.

**Q) Explain about BatchPreparedStatementSetter?**

If the user wants to update more than one row at a shot then he can go for BatchPreparedStatementSetter. This interface provides two methods

```
setValues(PreparedStatement ps, int i) throws SQLException;  
int getBatchSize();
```



The `getBatchSize()` tells the `JdbcTemplate` class how many statements to create. And this also determines how many times `setValues()` will be called.

**Q). Explain about RowCallbackHandler and why it is used?**

In order to navigate through the records we generally go for `ResultSet`. But spring provides an interface that handles this entire burden and leaves the user to decide what to do with each row. The interface provided by spring is `RowCallbackHandler`. There is a method `processRow()` which needs to be implemented so that it is applicable for each and every row.

```
void processRow(java.sql.ResultSet rs);
```

**Q) What is JDBC abstraction and DAO module?**

Using this module we can keep up the database code clean and simple, and prevent problems that result from a failure to close database resources. A new layer of meaningful exceptions on top of the error messages given by several database servers is brought in this module. In addition, this module uses Spring's AOP module to provide transaction management services for objects in a Spring application.

